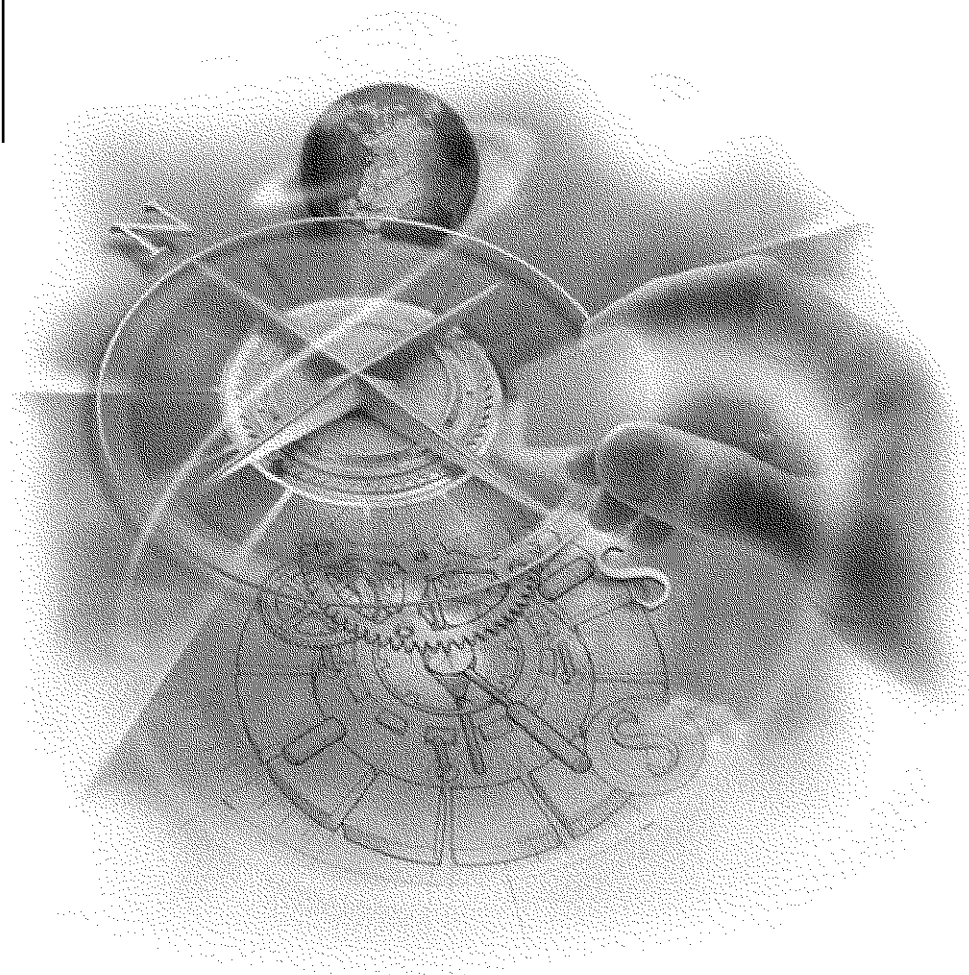


LDAP Libraries for C



Novell Developer Kit

Novell®

Legal Notices

Novell, Inc. makes no representations or warranties with respect to the contents or use of this documentation, and specifically disclaims any express or implied warranties of merchantability or fitness for any particular purpose. Further, Novell, Inc. reserves the right to revise this publication and to make changes to its content, at any time, without obligation to notify any person or entity of such revisions or changes.

Further, Novell, Inc. makes no representations or warranties with respect to any software, and specifically disclaims any express or implied warranties of merchantability or fitness for any particular purpose. Further, Novell, Inc. reserves the right to make changes to any and all parts of Novell software, at any time, without any obligation to notify any person or entity of such changes.

This product may require export authorization from the U.S. Department of Commerce prior to exporting from the U.S. or Canada.

Copyright © 1993-2001 Novell, Inc. All rights reserved. No part of this publication may be reproduced, photocopied, stored on a retrieval system, or transmitted without the express written consent of the publisher.

U.S. Patent Nos 5,553,139; 5,553,143; 5,677,851; 5,758,069; 5,784,560; 5,818,936; 5,864,865; 5,903,650; 5,905,860; 5,910,803 and other Patents Pending.

Novell, Inc.
122 East 1700 South
Provo, UT 84606
U.S.A.

www.novell.com

LDAP Libraries for C
September 2001
No Part Number Assigned

Online Documentation: To access the online documentation for this and other Novell developer products, and to get updates, see developer.novell.com/ndk. To access online documentation for Novell products, see www.novell.com/documentation.

Novell Trademarks

AppNotes is a registered trademark of Novell, Inc.

AppTester is a trademark of Novell, Inc., in the United States.

ArcNet 68 is a trademark of Novell, Inc.

ASM is a trademark of Novell, Inc.

BorderManager is a trademark of Novell, Inc.

BorderManager FastCache is a trademark of Novell, Inc.

BrainShare is a registered service mark of Novell, Inc., in the United States and other countries.

C3PO is a trademark of Novell, Inc.

Client 32 is a trademark of Novell, Inc.

ConsoleOne is a trademark of Novell, Inc.

Controlled Access Printer is a trademark of Novell, Inc.

Custom 3rd-Party Object is a trademark of Novell, Inc.

DENIM is a service mark of Novell, Inc.

DeveloperNet is a registered trademark of Novell, Inc. in the United States and other countries.

DeveloperNet 2000 is a trademark of Novell, Inc.

digitalme is a trademark of Novell, Inc.

DirXML is a trademark of Novell, Inc.

eDirectory is a trademark of Novell, Inc.

Full Service Directory is a trademark of Novell, Inc.

GroupWise is a registered trademark of Novell, Inc. in the United States and other countries.

Hardware Specific Module is a trademark of Novell, Inc.

HostPublisher is a trademark of Novell, Inc.

Hot Fix is a trademark of Novell, Inc.

HSM is a trademark of Novell, Inc.

iChain is a trademark of Novell, Inc.

instantme is a trademark of Novell, Inc.

Internetwork Packet Exchange is a trademark of Novell, Inc.

IPX is a trademark of Novell, Inc.

IPX/SPX is a trademark of Novell, Inc.

LANalyzer is a registered trademark of Novell, Inc. in the United States and other countries.

Link Support Layer is a trademark of Novell, Inc.

LSL is a trademark of Novell, Inc.

ManageWise is a registered trademark of Novell, Inc., in the United States and other countries.

Mirrored Server Link is a trademark of Novell, Inc.

MLI is a trademark of Novell, Inc.

MLID is a trademark of Novell, Inc.

MSL is a trademark of Novell, Inc.

Multiple Link Interface is a trademark of Novell, Inc.

Multiple Link Interface Driver is a trademark of Novell, Inc.

NControl is a trademark of Novell, Inc.

NCP is a trademark of Novell, Inc.

NDebug is a trademark of Novell, Inc.

NDPS is a registered trademark of Novell, Inc.

NDR is a trademark of Novell, Inc.

NDS is a registered trademark of Novell, Inc. in the United States and other countries.

NDS Administrator is a trademark of Novell, Inc.

NDS Manager is a trademark of Novell, Inc.

NE2000 is a trademark of Novell, Inc.

NetWare is a registered trademark of Novell, Inc. in the United States and other countries.

NetWare/IP is a trademark of Novell, Inc.

NetWare 386 is a trademark of Novell, Inc.

NetWare Aware is a trademark of Novell, Inc.

NetWare Connect is a registered trademark of Novell, Inc. in the United States.

NetWare Core Protocol is a trademark of Novell, Inc.

NetWare DOS Requester is a trademark of Novell, Inc.

NetWare Loadable Module is a trademark of Novell, Inc.

NetWare Management Portal is a trademark of Novell, Inc.

NetWare Management System is a trademark of Novell, Inc.

NetWare MHS is a registered trademark of Novell, Inc.

NetWare Name Service is a trademark of Novell, Inc.

NetWare Peripheral Architecture is a trademark of Novell, Inc.

NetWare Print Server is a trademark of Novell, Inc.

NetWare Requester is a trademark of Novell, Inc.

NetWare SFT and NetWare SFT III are trademarks of Novell, Inc.

NetWare SQL is a trademark of Novell, Inc.

NetWare Telephony Services is a trademark of Novell, Inc.

NetWare Tools is a trademark of Novell, Inc.

NetWare Upgrade is a service mark of Novell, Inc.

NIMS is a trademark of Novell, Inc.

NLM is a trademark of Novell, Inc.

NMAS is a trademark of Novell, Inc.

NMS is a trademark of Novell, Inc.

Novell is a registered trademark of Novell, Inc. in the United States and other countries.

Novell Application Launcher is a trademark of Novell, Inc.

Novell Authorized Reseller is a service mark of Novell, Inc.

Novell Authorized Service Center is a service mark of Novell, Inc.

Novell BorderManager is a trademark of Novell, Inc.

Novell Certificate Server is a trademark of Novell, Inc.

Novell Client is a trademark of Novell, Inc.

Novell Directory Services is a registered trademark of Novell, Inc.

Novell Distributed Print Services is a trademark of Novell, Inc.

Novell Embedded Systems Technology is a registered trademark of Novell, Inc., in the United States and other countries.

Novell HostPublisher is a trademark of Novell, Inc.

Novell Internet Caching System is a trademark of Novell, Inc.

Novell Labs is a trademark of Novell, Inc.

Novell SecretStore is a trademark of Novell, Inc.

Novell Security Attributes is a trademark of Novell, Inc.
Novell Storage Services is a trademark of Novell, Inc.
Novell Web Server is a trademark of Novell, Inc.
Novell, Yes, Tested & Approved logo is a trademark of Novell, Inc.
ODI is a trademark of Novell, Inc.
OnDemand is a service mark of Novell, Inc.
Open Data-Link Interface is a trademark of Novell, Inc.
Packet Burst is a trademark of Novell, Inc.
Personal NetWare is a trademark of Novell, Inc.
Printer Agent is a trademark of Novell, Inc.
Public Access Printer is a trademark of Novell, Inc.
QuickFinder is a trademark of Novell, Inc.
Red Box is a trademark of Novell, Inc.
Remote Console is a trademark of Novell, Inc.
RX-Net is a trademark of Novell, Inc.
Sequenced Packet Exchange is a trademark of Novell, Inc.
SFT, SFT III, and SFT NetWare are trademarks of Novell, Inc.
SMS is a trademark of Novell, Inc.
SMSTSA is a trademark of Novell, Inc.
SoftSolutions is a registered trademark of SoftSolutions Technology Corporation, a wholly owned subsidiary of Novell, Inc.
SPX is a trademark of Novell, Inc.
Storage Management Services is a trademark of Novell, Inc.
SVR4 is a trademark of Novell, Inc.
System V is a trademark of Novell, Inc.
Topology Specific Module is a trademark of Novell, Inc.
Transaction Tracking System is a trademark of Novell, Inc.
TSM is a trademark of Novell, Inc.
TTS is a trademark of Novell, Inc.
Universal Component System is a registered trademark of Novell, Inc.
ViewMAX is a trademark of Novell, Inc.
ZENworks is a trademark of Novell, Inc.

Third-Party Trademarks

All third-party trademarks are the property of their respective owners.
Java is a trademark or registered trademark of Sun Microsystems, Inc., in the United States and other countries.

Contents

Preface	15
1 Concepts	17
Getting Started	17
Dependencies	17
Platform Libraries and Header Files	18
Supported Platforms	19
Supported Compilers	20
Tutorials	20
Sample Code	20
LDAP Test Server	21
Setting up SSL Security	21
LDAP Server	21
LDAP Libraries	22
Using the LDAP Functions	23
Using Dynamic Memory with LDAP Functions	23
Selecting a Function for an LDAP Operation	26
Using Asynchronous or Synchronous Functions	26
Using Simple Bind with LDAP v3	27
LDAP URLs	27
Threads	28
Internationalization	29
Searching the Directory	30
Setting the Search Parameters and Search Constraints	31
Using Search Filters	35
Operational Attributes	37
Understanding Character Conversions	37
A Brief History of Character Encoding	37
UTF-8 Encoding	38
Using UTF-8 Strings in the LDAP SDK	39
The Wchar_t Type	39
Understanding Time Formats	40
Handling Referrals in LDAP v3	41
Controlling Automatic Referral Handling	41
Limiting Referral Hops	42
Creating a Rebind Process	42
Using the Rebind Process	43
Controls and Extensions	44
Controls	44
Extensions	45

Runtime Version of the Library Files	47
Windows (NT, 95, 98, 2000)	47
NetWare	48
Unix (Solaris and Linux)	48

2 Tasks 51

Establishing an SSL Connection	51
Reading the Root DSE	52
Adding an Entry	53
Modifying an Entry.	53
Modifying an Entry's Password	54
Extending the Schema	55

3 Standard LDAP Functions 57

ber_alloc_t	58
ber_bvdup	60
ber_bvecfree	61
ber_bvfree	62
ber_first_element	63
ber_flatten	65
ber_free	67
ber_init	69
ber_next_element	70
ber_peek_tag	72
ber_printf	73
ber_scanf	76
ber_skip_tag	79
ldap_abandon	80
ldap_abandon_ext	82
ldap_add	84
ldap_add_ext	86
ldap_add_ext_s	89
ldap_add_s	92
ldap_bind	94
ldap_bind_s	96
ldap_compare	98
ldap_compare_ext	100
ldap_compare_ext_s	103
ldap_compare_s	105
ldap_control_free	107
ldap_controls_free	108
ldap_count_entries	109
ldap_count_messages	111
ldap_count_references	113

8 LDAP Libraries for C

ldap_count_values	115
ldap_count_values_len	117
ldap_create_persistentsearch_control	119
ldap_create_sort_control	122
ldap_create_sort_keylist	124
ldap_create_vlv_control	126
ldap_delete	128
ldap_delete_ext.	130
ldap_delete_ext_s	132
ldap_delete_s.	134
ldap_dn2ufn	136
ldap_err2string	138
ldap_explode_dn	140
ldap_explode_rdn.	142
ldap_extended_operation.	144
ldap_extended_operation_s	146
ldap_first_attribute	148
ldap_first_entry	150
ldap_first_message.	152
ldap_first_reference	154
ldap_free_sort_keylist	156
ldap_free_urldesc.	157
ldap_get_dn	158
ldap_get_entry_controls	160
ldap_get_lderrno	162
ldap_get_option.	164
ldap_get_values	166
ldap_get_values_len	168
ldap_init.	170
ldap_is_ldap_url	172
ldap_is_ldaps_url.	173
ldap_memfree	174
ldap_modify.	175
ldap_modify_ext	177
ldap_modify_ext_s	179
ldap_modify_s	181
ldap_modrdn	183
ldap_modrdn_s.	185
ldap_modrdn2	187
ldap_modrdn2_s	189
ldap_msgfree	191
ldap_msgid	193
ldap_msgtype.	194
ldap_next_attribute	195

ldap_next_entry	197
ldap_next_message	199
ldap_next_reference	201
ldap_open	203
ldap_parse_entrychange_control	205
ldap_parse_extended_result.	208
ldap_parse_reference	210
ldap_parse_result	212
ldap_parse_sasl_bind_result	215
ldap_parse_sort_control	217
ldap_parse_vlv_control	220
ldap_perror.	223
ldap_rename	224
ldap_rename_s	227
ldap_result	230
ldap_result2error.	233
ldap_sasl_bind	235
ldap_sasl_bind_s	237
ldap_search	239
ldap_search_ext	242
ldap_search_ext_s.	246
ldap_search_s	250
ldap_search_st.	253
ldap_set_lderrno	256
ldap_set_option	258
ldap_set_rebind_proc	260
ldap_simple_bind	263
ldap_simple_bind_s	265
ldap_sort_entries	267
ldap_sort_strcasecmp	269
ldap_sort_values.	271
ldap_unbind, ldap_unbind_s.	273
ldap_unbind_ext, ldap_unbind_ext_s	274
ldap_url_parse	276
ldap_url_search	279
ldap_url_search_s	281
ldap_url_search_st.	283
ldap_value_free	285
ldap_value_free_len	286
ldapssl_client_init	287
ldapssl_client_deinit	289
ldapssl_init.	290
ldapssl_install_routines	292
ldapssl_add_trusted_cert	294

ldapssl_get_cert	296
ldapssl_get_cert_attribute	298
ldapssl_set_verify_mode	300
ldapssl_get_verify_mode	302
ldapssl_set_verify_callback.	304
4 LDAP Extension Functions	307
ldap_abort_naming_context_operation.	308
ldap_add_replica	310
ldap_change_replica_type	313
ldap_create_naming_context.	316
ldap_create_orphan_naming_context	319
ldap_get_context_identity_name	321
ldap_get_effective_privileges.	323
ldap_get_replication_filter	326
ldap_get_replica_info.	328
ldap_list_replicas	331
ldap_merge_naming_contexts	333
ldap_naming_context_entry_count.	335
ldap_nds_to_ldap_name	337
ldap_receive_all_updates	339
ldap_refresh_server	342
ldap_remove_orphan_naming_context.	344
ldap_remove_replica	346
ldap_request_naming_context_sync	349
ldap_request_schema_sync	352
ldap_send_all_updates.	354
ldap_set_replication_filter	356
ldap_trigger_back_process.	358
ldapx_memfree	360
5 UTF-8 Functions	361
UTF-8 / Wide Character Conversions	361
ldap_x_utf8_to_wc	362
ldap_x_utf8s_to_wcs	364
ldap_x_wc_to_utf8	366
ldap_x_wcs_to_utf8s	368
UTF-8 Utility Functions.	369
ldap_x_utf8_chars	370
ldap_x_utf8_charlen	371
ldap_x_utf8_next	372
ldap_x_utf8_prev	373
ldap_x_utf8_copy.	374
ldap_x_utf8_strchr	375

ldap_x_utf8_strspn	376
ldap_x_utf8_strcspn	377
ldap_x_utf8_strpbrk	378
ldap_x_utf8_strtok	379

6 Values 381

Object Access Control Rights	381
Attribute Access Control Rights	382
Certificate Attribute IDs	383
Inheritance Control Rights	384
Replica States	385
Replication Filters	387
Replica Types	388
Request Message Types	389
Result Message Types	390
Session Preference Options	392

7 Structures 397

BerElement	398
berval	399
LDAP	400
LDAPAPIFeatureInfo	401
LDAPAPIInfo	402
LDAPControl	404
LDAPMessage	405
LDAPMod	406
LDAPReplicaInfo	408
LDAPSORTKey	410
LDAPSSL_Cert	411
LDAPSSL_Cert_VValidity_Period	412
LDAPURLDesc	414
LDAPVLIInfo	416
timeval	418

8 LDAP Tools 421

Novell Import Convert Export Utility	422
Using the Command Line Interface	423
LDAP Bulk Update/Replication Protocol	438
Forward References	440
Using XML Rules	440
Improving the Speed of LDIF Imports	450
Other LDAP Utilities	451
ldapadd	452
ldapdelete	453
ldapmodify	455

12 LDAP Libraries for C

ldapmodrdn	456
ldapsearch	458
LDIF Examples	460
LDIF Formatting Conventions	461
Modifying the Attribute Values of an Entry	462
Adding Entries	465
Deleting Entries.	466
Renaming or Moving an Entry	466
Adding NDS Indexes	467
Modifying the Schema	470
Using Content Records.	474
A Source Code Contributors	477
Revision History	479

Preface

The LDAP Libraries for C enable you to write applications to access, manage, update, and search for information stored in NDS and other LDAP-aware directories.

LDAP (Lightweight Directory Access Protocol) is becoming an Internet standard for accessing directory information, allowing LDAP-enabled applications to access multiple directories. LDAP v3 supports such features as secure connections (through SSL and SASL), entry management, schema management, and LDAP controls and extensions for expanding the functionality of LDAP.

The LDAP Libraries for C are available for the following platforms:

- ♦ Windows (NT, 95, 98, and 2000)
- ♦ NetWare
- ♦ Unix (Solaris and Linux)

1

Concepts

This manual assumes that you have a basic understanding of NDS and NDS and LDAP integration. For more information on these topics, see

- ♦ *NDS Technical Overview.*
- ♦ *LDAP and NDS.*

Getting Started

The following sections cover a few basic requirements for getting set up and started with the LDAP Libraries for C:

- ♦ “Dependencies” on page 17
- ♦ “Platform Libraries and Header Files” on page 18
- ♦ “Supported Platforms” on page 19
- ♦ “Supported Compilers” on page 20
- ♦ “Tutorials” on page 20
- ♦ “Sample Code” on page 20

Dependencies

In addition to LDAP Libraries for C, you will need the following to take full advantage of the functionality offered in the libraries:

- ♦ **LDAP Server.** The libraries can be used to access any LDAP server and its directory. If you are using them to access an NDS directory, the LDAP server must be running on NDS 7.xx or higher to access LDAP v3 functionality. Other servers in the tree can be running earlier versions of NDS; only the LDAP server needs to be on NDS 7.xx or higher. For

information on the functionality available in various versions of NDS, see LDAP and NDS.

- ♦ **SSL.**To use SSL, the LDAP server and the LDAP client must be configured for SSL. For more information, see “Setting up SSL Security” on page 21.
- ♦ **LDAP Extensions for NDS.**To use the LDAP extensions for naming contexts (partition and replica management) and getting effective rights, the LDAP server must be running on NDS eDirectory 8.5. To obtain a copy, see Novell NDS Download site (<http://developer.novell.com/nds/edirectory/>).

Platform Libraries and Header Files

The LDAP Libraries for C includes the following header files.

Header File	Description
ldap.h	Contains the prototypes for all the standard LDAP functions
ldapx.h	Contains the prototypes for LDAP functions for extensions
ldapsl.h	Contains the prototypes for all of the LDAP SSL functions
ldaputf8.h	Contains the prototypes for all of the UTF-8 conversions routines

The following header files are included in the LDAP libraries for C, but are linked by the header files listed in the previous table:

- ♦ lber.h
- ♦ lber_types.h
- ♦ ldap_cdefs.h
- ♦ ldap_features.h

The LDAP Libraries for C have been compiled into the following libraries (UNIX platforms add a lib prefix to the library names):

Library	Platforms
ldapsdk.dll	Win32 platforms (Windows 95, Windows 98, Windows 2000, Windows NT server with SP 4 or newer, Windows NT workstation with SP 3 or SP 4)
ldapx.dll	
ldapssl.dll	
ldapsdk.nlm	NetWare 5 with SP4 and NDS 8.2x, NetWare 5.1 with NDS 8.3x
ldapx.nlm	
ldapssl.nlm	
libldapsdk.so	Solaris (2.6 and 2.7) and Linux (RedHat 6.1)
libldapx.so	
libldapssl.so	

Supported Platforms

The LDAP Libraries for C SDK enables application developers to write applications to access, manage, update and search for information stored in NDS and other LDAP-aware directories.

Server applications run on the same machine as the LDAP server. The libraries currently supports development on the following NDS server platforms:

- ♦ NetWare 5.1
- ♦ NetWare 5 with SP4 or higher
- ♦ Windows NT server 4.0 with SP 4
- ♦ Solaris 2.6, 2.7, and 2.8
- ♦ Linux (tested on RedHat 6.1 and 6.2)

Client applications remotely access directory information stored on an LDAP server. The libraries currently support development of such applications on the following client platforms:

- ♦ Windows NT workstation 4.0 with SP 3 and SP 4
- ♦ Windows 95
- ♦ Windows 98

- ♦ Windows 2000
- ♦ Solaris 2.6, 2.7, 2.8
- ♦ Linux (tested on RedHat 6.1 and 6.2)

The Novell LDAP server is currently available on NetWare 5.x, NDS for NT, NDS for Solaris, and NDS eDirectory (for NetWare, NT, Solaris, and Linux).

Supported Compilers

The libraries can be used with the following C compilers:

- ♦ Microsoft Visual Studio C++ versions 5 and 6 for Windows
- ♦ Borland C Compiler for Windows
- ♦ CodeWarrior from Metrowerks for NLM
- ♦ Watcom for NLM
- ♦ GCC 2.8.1 for Linux and Solaris

Tutorials

DeveloperNet University has developed an LDAP tutorial that creates a White Pages application. This application is similar to looking up information in the white pages of a phone book and allows users to browse and search for employees in a company, view their pictures, and obtain phone numbers, titles, and other information.

To access this tutorial, see Programming NDS with C LDAP (<http://developer.novell.com/servlet/devnet/education/nds/tutorials/whitepages/index.htm>).

Sample Code

To access LDAP sample code, check the following sites:

- ♦ For source code examples that use the standard functions for LDAP operations (such as search, add, modify, and delete), see the LDAP sample code for C (http://developer.novell.com/ndk/doc/samplecode/cldap_sample/index.htm).
- ♦ For source code examples that use the Novell LDAP extensions for LDAP naming context operations (such as add, modify, and delete replicas or splitting and joining naming contexts), see the LDAP sample

code for extensions (http://developer.novell.com/ndk/doc/samplecode/cldap_sample/index.htm).

- ♦ For source code and task examples that authenticate, search, and read, see the LDAP examples in the Library and Laboratory departments of DeveloperNet University (<http://developer.novell.com/education/index.html>).

LDAP Test Server

Novell has set up an LDAP server that you can access over the Internet to test your LDAP application. The server's name is www.nldap.com, and it listens on the default LDAP port (389). To use authenticated access, you must set up your own account in your own NDS container. Your account is limited to 1MB of disk storage.

To access this site, go to the NDS/LDAP Services Access Test Site (<http://www.nldap.com/NLDAP/>).

Setting up SSL Security

The LDAP Libraries for C are independent of Novell client software, and they perform their own authentication. For SSL authentication to work, the LDAP server must have a certificate to use with SSL, and the LDAP libraries must be configured to trust the LDAP server's certificate. Thus, two components must be set up to use SSL:

- ♦ “LDAP Server” on page 21
- ♦ “LDAP Libraries” on page 22

LDAP Server

In NDS 8 and higher, the LDAP server is installed and started automatically with NDS. By default, the LDAP server is set up to service anonymous binds. To enable secure connections over SSL, the LDAP server must be set up with a digital certificate from a Certificate Authority.

The steps for setting up SSL on the LDAP server are slightly different for each release of NDS. For specific information, see one of the following:

- ♦ NDS eDirectory (<http://www.novell.com/documentation/lg/ndsse/docui/index.html>).

- ♦ NetWare 5 for PKI (<http://www.novell.com/documentation/lg/nw5/ussecur/crmdsenu/data/h0000014.html>) and for the LDAP server (http://www.novell.com/documentation/lg/nw5/usnds/ldap_enu/data/h0000012.html)

LDAP Libraries

The LDAP libraries perform SSL server authentication and must be configured with a trusted root certificate in DER or B64 (PEM) format.

- 1 If using eDirectory, use ConsoleOne to export the certificate authority's trusted root certificate. The certificate authority is used to sign the LDAP Server's SSL certificate.

For step-by-step instructions for this procedure, see the administration guide that comes with NDS eDirectory (<http://www.novell.com/documentation/lg/ndsse/docui/index.html>).

- 2 Copy the certificate to the client workstation.
- 3 Use `ldapssl_client_init` or `ldapssl_add_trusted_cert`, passing in the certificate file created in step1, to configure the LDAP SSL library to trust certificates signed by the certificate authority.

Turning Off Server Verification

It is possible to configure the LDAP SSL library to not verify server certificates. This is done by using `ldapssl_set_verify_mode` and setting the mode to `LDAPSSL_VERIFY_NONE` (see `ssl_any.c` in the "Sample Code" on page 20). Turning off server verification means any LDAP server will be considered trusted and should only be used in controlled environments where encrypted communication between servers and clients is desired but server verification is not necessary.

Interactive Server Verification

Clients can register a callback function using `ldapssl_set_verify_callback` (page 304) that will be called during the SSL handshake if a server certificate is not trusted. After reviewing the server certificate, the callback function can accept or reject the connection. This allows clients to decide at runtime if a server is trusted.

Using the LDAP Functions

The following sections discuss some general concepts for using the functions to accomplish a task and some principles for selecting the appropriate function for the task. It covers the following topics:

- ♦ “Using Dynamic Memory with LDAP Functions” on page 23
- ♦ “Selecting a Function for an LDAP Operation” on page 26
- ♦ “Using Asynchronous or Synchronous Functions” on page 26
- ♦ “Using Simple Bind with LDAP v3” on page 27
- ♦ “Threads” on page 28
- ♦ “Internationalization” on page 29

Using Dynamic Memory with LDAP Functions

If your application allocates any memory for use with LDAP functions, that memory must be freed by your application. Do not free this memory using an LDAP function, for example `ldap_memfree`.

All memory allocated by LDAP functions must be freed by LDAP functions. The following table lists the LDAP functions which allocate memory and the LDAP function you should use to free the memory.

Functions That Allocate Memory	Parameter/Structure Member	Memory Free Functions
<code>ber_alloc_t</code>	<code>return(BerElement *)</code>	<code>ber_free</code>
<code>ber_bvdup</code>	<code>return(struct berval *)</code>	<code>ber_bvfree</code>
<code>ber_flatten</code>	<code>bvptr</code>	<code>ber_bvfree</code>
<code>ber_init</code>	<code>return(BerElement *)</code>	<code>ber_free</code>
<code>ber_scanf</code>	"a" format option	<code>ldap_memfree</code>
<code>ber_scanf</code>	"B" format option	<code>ldap_memfree</code>
<code>ber_scanf</code>	"O" format option	<code>ber_bvfree</code>
<code>ber_scanf</code>	"V" format option	<code>ber_bvecfree</code>
<code>ber_scanf</code>	"v" format option	<code>ldap_memfree</code>

Functions That Allocate Memory	Parameter/Structure Member	Memory Free Functions
ldap_create_sort_control	ctrlp	ldap_control_free
ldap_create_sort_keylist	sortKeyList	ldap_free_sort_keylist
ldap_create_vlv_control	ctrlp	ldap_control_free
ldap_dn2ufn	return(char *)	ldap_memfree
ldap_explode_dn	return(char **)	ldap_value_free
ldap_explode_rdn	return(char **)	ldap_value_free
ldap_extended_operation_s	retdatap	ber_bvfree
ldap_extended_operation_s	retoidp	ldap_memfree
ldap_first_attribute	return (char *)	ldap_memfree
ldap_first_attribute	BerElement	ber_free (xxx, 0)
ldap_get_context_identity_name	identity	ldapx_memfree
ldap_get_dn	return (char *)	ldap_memfree
ldap_get_entry_controls	LDAPControl	ldap_controls_free
ldap_get_option	LDAP_OPT_API_INFO / ldapai_extensions	ldap_value_free
ldap_get_option	LDAP_OPT_API_INFO/ ldapai_vendor_name	ldap_memfree
ldap_get_option	LDAP_OPT_MATCHED_DN / outvalue	ldap_memfree ldap_controls_free
ldap_get_values	return (char **)	ldap_value_free
ldap_get_values_len	return(struct berval **)	ldap_value_free_len
ldap_init	return(LDAP *)	ldap_unbind, ldap_unbind_s
ldap_list_replicas	replicaList	ldapx_memfree
ldap_nds_to_ldap_name	ldapName	ldapx_memfree
ldap_next_attribute	return (char *)	ldap_memfree

Functions That Allocate Memory	Parameter/Structure Member	Memory Free Functions
ldap_next_attribute	BerElement	ber_free (xxx, 0)
ldap_parse_extended_result	retdatap	ber_bvfree
ldap_parse_extended_result	retoidp	ldap_memfree
ldap_parse_reference	referralsp	ldap_value_free
ldap_parse_reference	serverctrlsp	ldap_controls_free
ldap_parse_result	errmsgsp	ldap_memfree
ldap_parse_result	matcheddn	ldap_memfree
ldap_parse_result	referralsp	ldap_value_free
ldap_parse_result	serverctrlsp	ldap_controls_free
ldap_parse_sasl_bind_result	servercredp	ber_bvfree
ldap_parse_sort_control	attribute	ldap_memfree
ldap_parse_vlv_control	contextp	ber_bvfree
ldap_result	res	ldap_msgfree
ldap_sasl_bind_s	servercredp	ber_bvfree
ldap_search_ext_s	res	ldap_msgfree
ldap_search_s	res	ldap_msgfree
ldap_search_st	res	ldap_msgfree
ldap_ssl_client_init		ldap_ssl_client_deinit
ldap_ssl_init	return(LDAP *)	ldap_unbind
ldap_url_parse	ludpp	ldap_free_urldesc
ldap_url_search_s	res	ldap_msgfree
ldap_url_search_st	res	ldap_msgfree

The following functions free the memory allocated to the res parameter if the freit parameter is set to true:

ldap_parse_extended_result
ldap_parse_reference
ldap_parse_result
ldap_parse_sasl_bind_result
ldap_result2error

Selecting a Function for an LDAP Operation

Most LDAP functions that perform operations (such as add, delete, modify) have four variations:

- ♦ LDAP v2 asynchronous. These take the format of *ldap_operation*, for example, *ldap_search*.
- ♦ LDAP v3 asynchronous. These take the format of *ldap_operation_ext*, for example, *ldap_search_ext*.
- ♦ LDAP v2 synchronous. These take the format of *ldap_operation_s*, for example, *ldap_search_s*.
- ♦ LDAP v3 synchronous. These take the format of *ldap_operation_ext_s*, for example, *ldap_search_ext_s*.

If you are developing a new application, you should use the LDAP v3 version of the functions. The LDAP library supports the LDAP v2 versions for backwards compatibility with earlier releases.

Using Asynchronous or Synchronous Functions

Blocking versus Non-Blocking. Synchronous functions block and do not return until the LDAP server has serviced the request and returned a result. Asynchronous functions return as soon as the LDAP client processes the request, and the application is then free to do other work. However, the application is responsible to use the returned message ID to check on the status of the operation.

Return Values. The synchronous functions return both client and server error codes. The asynchronous *ldap_*_ext* functions return only the client error codes. The subsequent results must be parsed for the server error codes. The asynchronous *ldap_** functions return a -1 for the client error codes, and the *ldap_get_option* function must be used to retrieve the client error codes from the LDAP session handle.

Using Simple Bind with LDAP v3

Usually when you are just getting started, your application does not require a secure connection. By default, the LDAP v2 functions set up an LDAP v2 session because the session handle is configured for an LDAP v2 session. To ensure that your application sets up an LDAP v3 session, call the following functions in the order specified. The following example uses `ldap_simple_bind`.

- 1 Call the `ldap_set_option` function with the `ld` parameter set to `NULL` and the `option` parameter set to `LDAP_OPT_PROTOCOL_VERSION`, and the `invalue` parameter set to `LDAP_VERSION3`.

This sets the value in the global session handle to LDAP v3 and all subsequent session handles assume these values.

- 2 Call the `ldap_init` function.
- 3 Call the `ldap_simple_bind` or `ldap_simple_bind_s` function.

If you are connecting to an Novell LDAP server, make sure the LDAP server has been configured to accept clear-text passwords.

When you are ready to set up a secure connection, see the `ldaps*_*` functions.

LDAP URLs

LDAP URLs provide a uniform method to access information on an LDAP server. Defined in RFC 2255, LDAP URLs begin with the prefix `LDAP://` or `LDAPS://`. The following provides the syntax and descriptions of an LDAP URL.

```
ldap[s]://<hostname>:<port>/  
<base_dn>?<attributes>?<scope>?<filter>?<extension>
```

Note that `ldaps` is a common enhancement used to denote SSL, and is not defined in an RFC.

Table 1 Field descriptions for an LDAP URL

URL Element	Default Value	Description
hostname	none	DNS name or IP address of the LDAP server.
port	389	Port of the LDAP server.
base_dn	root	Base DN for the LDAP operation.

URL Element	Default Value	Description
attributes	all attributes	A comma delimited list of attributes to return.
scope	base	Search scope.
filter	objectClass=*	Search filter.
extension	none	LDAP extended operations.

NOTE: An attribute list is required if you want to provide a scope (even if the attribute list is blank). To return all attributes within a specific scope you must include `<base_dn>??<scope>`.

Determining if a URL is an LDAP URL

To determine if a URL is a valid `ldap://` or `ldaps://` URL use one of the following functions:

- ♦ `ldap_is_ldap_url` (page 172)
- ♦ `ldap_is_ldaps_url` (page 173)

Both functions take a URL as the parameter and return 1 if the URL is a valid LDAP URL, and 0 if it is not valid.

Parsing an LDAP URL

The `ldap_url_parse` (page 276) function parses an LDAP URL and returns an `LDAPURLDesc` (page 414) structure to your application. You can then retrieve the individual parameters from the URL, or you can pass this URL to a search function.

Searching with an LDAP URL

The `ldap_url_search` (page 279) functions allow you to pass an `LDAPURLDesc` structure to perform an LDAP search.

Threads

All LDAP API calls using the same session handle (`ld`) should be made from the same thread. If not, the application must be responsible for synchronizing threads, such that no two LDAP calls using the same session handle from different threads are executing concurrently.

Internationalization

The LDAP libraries have been enabled for internationalization. However, the messages are currently available only in English. For cross-platform support, the English messages have been placed in the following files:

File Name	Platform
ldapsdk.msg	NetWare
ldapsdkmsg.dll	Windows (NT, 95, 98, 2000)
ldapsdk.mo	Solaris (2.6, 2.7, 2.8), Linux (RedHat 6.1, 6.2)

Depending upon the platform, the message file is installed in the following locations:

- ♦ On a NetWare server in the sys:\system\nls\4 directory.
- ♦ On an NDS for NT server in the winnt\system32\nls\english directory.
- ♦ On a Windows client in the Novell\ndk\cllapsdk\bin\win32\nls\english directory.
- ♦ On a Unix platform (Solaris or Linux) in the [install directory]/ldapcsdk/sdk/usr/lib/locale/C/LC_MESSAGES directory.

NetWare NLMs

If you wish to translate the messages to another language for the NetWare platform, you will need to use the internationalization tools included in the NLM User Interface Developer Components (<http://developer.novell.com/ndk/nwsnut.htm>). Use the ldapmsg.h file and the tools to create an errormsg.mdb file. Use the tools to translate the errormsg.mdb file. Use the translated file and the tools to create an ldapsdk.msg file.

Windows

If you wish to translate the messages to another language for the Windows platform, translate the errormsg.rc file. When you save the file, the resource.h file is created. Build the code to convert the errormsg.rc and resource.h files to an errormsg.dll file.

Unix

If you wish to translate the messages to another language for a Unix platform, complete the following steps:

- 1 Translate the messages in the `ldapsdk.po` file to the target language. This file is located in the `<install directory>/ldapcsdk/sdk/usr/lib/locale/C/LC_MESSAGES` directory. The following steps assume that French is the target language.
- 2 Use the `msgfmt` command to convert the `ldapsdk.po` file to an `ldapsdk.mo` file.
- 3 Create the directory for the messages. For French, the directory would be the following:

```
<install directory>/ldapcsdk/sdk/usr/lib/locale/fr/  
LC_MESSAGES
```

- 4 Copy the `ldapsdk.mo` file to the directory created in Step 3.
- 5 Export the following:

```
NLDAPSDK_ROOT=<install directory>
```

Searching the Directory

In LDAP, search functions are used for both searching and reading information from the directory. The LDAP Libraries for C provide the following asynchronous and synchronous functions:

Function	Description
<code>ldap_search</code> <code>ldap_search_s</code>	Uses a search filter and specified attributes to search the directory either synchronously or asynchronously.
<code>ldap_search_ext</code> <code>ldap_search_ext_s</code>	Use a search filter, specified attributes, time limit, and LDAP controls to search the directory either synchronously or asynchronously.
<code>ldap_search_st</code>	Uses a search filter, specified attributes, and a time limit to synchronously search the directory.

A search results can contain entry, references, or search result messages. The last message in the results is always a search result message. Each message

type has its own set of functions for reading the results. If you use the `ldap_first_message` and `ldap_next_message` functions, the following message types are returned:

- ♦ `LDAP_RES_SEARCH_ENTRY`

Returns the directory entries from the search.

- ♦ `LDAP_RES_SEARCH_REFERENCE`

Returns a sequence of one or more LDAP URLs. An `LDAP_RES_SEARCH_REFERENCE` is returned for each area not explored by this LDAP server during the search.

If the LDAP server is configured to follow referrals automatically, the LDAP server will never return `LDAP_RES_SEARCH_REFERENCE` to the application.

- ♦ `LDAP_RES_SEARCH_RESULT`

Returns

If you are only interested in the entry messages from the search, you can use the `ldap_first_entry` and `ldap_next_entry` to read just the entry results and to skip any other type of message.

If you are only interested in search references returned from the search, you can use the `ldap_first_reference` and `ldap_next_reference` functions to read the references and to skip any other type of message.

Setting the Search Parameters and Search Constraints

The search parameters set up the criteria for where the search begins, what entries to find, and what information to return with the matching entries. Search constraints determine how many entries to return and set time limits on looking for the entries. These same parameters and constraints can be set up to perform read operations.

Base. The base parameter specifies the container in the directory where the search begins. You can specify the root of the directory tree, or any container or branch of the directory tree. For quick searches, be as specific as you can because a branch search is always faster than a full search from the tree root.

Scope. The scope parameter specifies how deep to search. It allows three levels to be set:

- ♦ LDAP_SCOPE_BASE (0x00)—searches only the entry specified by the base parameter. If the base parameter is set to the entry and the scope parameter set to this level, the search becomes a read of this entry.
- ♦ LDAP_SCOPE_ONELEVEL (0x01)—searches the immediate subordinates of the entry specified by the base parameter.
- ♦ LDAP_SCOPE_SUBTREE (0x02)—searches the entire subtree starting with, and including, the entry specified by the base parameter. If the NDS server does not contain replicas for all the containers in the specified subtree, the server can automatically follow referrals to other servers. A session option, LDAP_OPT_REFERRALS, allows you to specify whether referrals are followed automatically or whether search references should be returned to where the additional entries are located. For more information, see “Session Preference Options” on page 392.

Filter. The search filter specifies what you are searching for. The following is a simple filter that searches for all entries with the last name of Smith.

```
"sn=Smith"
```

The default filter, if no filter is specified, is "objectClass=*". This filter searches every entry in the directory since the objectClass attribute is a required attribute of all entries in the directory.

These simple filters are strings with the following format:

```
attribute_name operator value
```

LDAP supports the following operators:

Operator	Description
=	Returns entries whose attribute value matches the specified value.
>=	Returns entries whose attribute value is equal to or greater than the specified value.
<=	Returns entries whose attribute value is equal to or less than the specified value
=*	Returns entries who have at least one value for the attribute.

Operator	Description
~=	Returns entries whose attribute value approximately matches the specified value. NDS does not support this operator and treats it the same as the equal (=) operator.

For information about the grammar required to create more complex search filters, see “Using Search Filters” on page 35.

Attributes. The attribute parameter specifies which attributes to return with each matching entry. The parameter accepts the following types of values:

- ♦ To return specific attributes, you pass a NULL-terminated array of attribute names in the parameter.
- ♦ To return only entry names (and no attributes), set the first, and only, string in the array to LDAP_NO_ATTRS.
- ♦ To return all attributes, set this parameter to NULL

Attributes Only. This parameter determines whether values are returned with the attributes specified in the attribute parameter. Set this parameter to zero (0) to return attributes and values. Set it to a non-zero value to return only attribute names and no values.

Time Limit. The time limit determines how long the server should wait for search results before returning to the client. The time limit is approximate because the client passes the value to the LDAP server with the search request and is dependent upon the LDAP server’s interpretation of the limit.

The `ldap_search_ext`, `ldap_search_ext_s`, and `ldap_search_st` functions allow you to specify the time limit with a timeout parameter. This parameter points to a `timeval` structure that specifies the maximum time to wait for the results of a search to complete. The structure specifies both the time the server waits for the operation to complete as well as the time the local function waits for the server to respond. If the timeout parameter is set to NULL, the client timeout is infinite and the server uses the timeout value specified in the LDAP_OPT_TIMELIMIT option.

The other search functions do not have a timeout parameter and use the LDAP_OPT_TIMELIMIT option. This option determines the number of seconds an LDAP server will spend on a search. A value of LDAP_NO_LIMIT (0) means no limit. The default is LDAP_NO_LIMIT.

To get the option’s current value, use `ldap_get_option` (page 164).

To set the option's value, use `ldap_set_option` (page 258).

Search Result Limits. This parameter or constraint determines how many entries are returned in a search results. Two functions, `ldap_search_ext` and `ldap_search_ext_s`, have a `sizelimit` parameter. To specify no limit, set this parameter to `LDAP_NO_LIMIT` (0). To use the value in the `LDAP_OPT_SIZELIMIT` option, set this parameter to -1.

The other search functions use the `LDAP_OPT_SIZELIMIT` option to determine how many entries are returned from a search. A value of `LDAP_NO_LIMIT` (0) means no limit. The default is `LDAP_NO_LIMIT`.

To get the current value, use `ldap_get_option` (page 164).

To set the value, use `ldap_set_option` (page 258).

Alias Dereferencing. The `LDAP_OPT_DEREF` option determines how aliases are handled during a search. It supports the following values:

- ♦ `LDAP_DEREF_NEVER` (0X00)
- ♦ `LDAP_DEREF_SEARCHING` (0x01)
- ♦ `LDAP_DEREF_FINDING` (0x02)
- ♦ `LDAP_DEREF_ALWAYS` (0x03)

The default is `LDAP_DEREF_NEVER`.

The `LDAP_DEREF_SEARCHING` flag indicates that aliases are dereferenced during the search but not when locating the base object of the search.

The `LDAP_DEREF_FINDING` flag indicates that aliases are dereferenced when locating the base object but not during the search.

The `LDAP_DEREF_ALWAYS` flag indicates that aliases are dereferenced when locating the base object and when finding entries.

The `LDAP_DEREF_NEVER` flag indicates that aliases are not dereferenced.

To get the current value, use `ldap_get_option` (page 164).

To set the value, use `ldap_set_option` (page 258).

Using Search Filters

The LDAP search filter grammar is specified in RFC 2254 and 2251. The grammar uses ABNF notation.

```
filter = " ( " filtercomp " ) "  
filtercomp = and / or /not /item  
  
and = "&" filterlist  
    filterlist = 1*filter  
  
or = "|" filterlist  
    filterlist = 1*filter  
  
not = "!" filterlist  
    filterlist = 1*filter  
  
item = simple/present/substring/extendible  
  
simple = attr filtertype value  
    attr = name | name;binary  
    filtertype = equal/approx/greater/less  
    value = data valid for the attribute's syntax  
  
equal = "="  
approx = "~="  
greater = ">="  
less = "<="  
  
present = attr "="  
    attr = name | name;binary  
  
substring = attr "=" [initial] any [final]  
    attr = name | name;binary  
    initial = value  
    any = "*" *(value "*")  
    final = value  
  
extendible = attr [":dn"] [":" matchingrule] ":value  
    / [":dn"] [":" matchingrule] ":" value  
    /matchingrule = name | OID
```

For additional options for the attr option, see Section 4.1.5 of RFC 2251.

For additional information on the value option, see Section 4.1.6 of RFC 2251.

IMPORTANT: NDS eDirectory does not support LDAP approximate (~=) matching or extensible matching rules.

Operators

Table 2 LDAP Filter Operators

Operator	Description
=	Used for presence and equality matching. To test if an attribute exists in the directory, use <code>attributename=*</code>. All entries that have the specified attribute will be returned. To test for equality, use <code>attributename=value</code>. All entries that have <code>attributename=value</code> are returned. For example, <code>cn=Kim Smith</code> would return entries with Kim Smith as the common name attribute. <code>cn=*</code> would return all entries that contained a <code>cn</code> attribute. The <code>=</code> operator can also be used with wildcards to find a substring, <code>cn=*ary*</code> would return <code>mary</code>, <code>hillary</code>, and <code>gary</code>.
>=	Used to return attributes that are greater than or equal to the specified value. For this to work, the syntax type of the attribute must have defined a mechanism to make this comparison. For example, <code>cn>=Kim Smith</code> would return all entries from Kim Smith to Z.
<=	Used to return attributes that are greater than or equal to the specified value. For this to work, the syntax type of the attribute must have defined a mechanism to make this comparison. For example, <code>cn>=Kim Smith</code> would return all entries from A to Kim Smith.
~=	Used for approximate matching. The algorithm used for approximate matching varies with different LDAP implementations.

The following boolean operators can be combined with the standard operators to form more complex filters. Note that boolean operator syntax is used different in search filters than in the C and Java programming languages, but the concepts are the same.

Table 3 LDAP Filter Boolean Operators

Boolean Operators	Description
&	And. For example, <code>(&(cn=Kim Smith)(telephonenumber=555-5555))</code> would return entries with common name of Kim Smith and a telephone number of 555-5555.

Boolean Operators	Description
	Or. For example, <code>((cn=Kim Smith)(cn=Kimberly Smith))</code> would return entries with common name Kim Smith or Kimberly Smith.
!	Not. For example, <code>!(cn=Kim Smith)</code> would return entries with any cn other than Kim Smith. Note that the <code>!</code> operator is unary.

Examples:

`(cn = Kim Smith)`. Returns entries with a common name of Kim Smith

`(&(cn=Kim Smith)(telephonenumber=555*)(emailaddress=*acme.com))`
Returns entries with a common name of Kim Smith, a telephone number that starts with 555, and an e-mail address that ends in acme.com

`!(cn = Chris Jones)` Returns entries that do not have a common name of Chris Jones.

`(&(objectClass=inetOrgPerson) (| (sn=Smith) (cn=Chris S*) ) )` Returns entries that are of type `inetOrgPerson` with a surname of Smith or a common name beginning with Chris S.

`(&(o=acme)(objectclass=Country)(!|(c=spain)(c=us))` Returns entries that are of type `Country` from the organization Acme, that are not Spain or us.

Operational Attributes

Operational attributes are not automatically returned in search results; they must be requested by name in the search operation. For a list of supported operational attributes in NDS eDirectory 8.5, see *LDAP Operational Attributes* in *LDAP and NDS*. The LDAP servers in previous releases of NDS do not support operational attributes.

Understanding Character Conversions

This section contains reference information on character encoding and a description of UTF-8, the encoding used by LDAPv3.

A Brief History of Character Encoding

In the early days of computing, 7-bit ASCII was the standard. The need for more characters drove the creation of a number of 8-bit Single Byte Character Sets (SBCS). ISO-8859 for example provided the 7-bit ASCII characters and many of the accented characters required for Western Europe.

Asian languages required much more than 256 characters. Multi-byte character sets were developed using a variable number of bytes per character, such as Shift-JIS or EUC-JP.

Other encodings appeared that were stateful. They used Shift-In/Shift-Out characters, or escape sequences to switch between encoding schemes.

In an attempt to bring order to this confusion, two separate standards organizations started work on a Universal Character Set (UCS) which would encode all the characters of all the major languages in the world. The two organizations ultimately agreed to maintain a consistent encoding, and the ISO-10646/Unicode standard became widespread. ISO-10646 officially supports a 31-bit code space (0 - 0x7FFFFFFF), while Unicode supports the 21-bit space (0 - 0x10FFFF) of over a million characters. So far no characters have been assigned beyond the 16-bit Basic Multi-Lingual Plane (BMP). While the code point value assigned to each character are well defined, there are different ways that the value may be encoded.

UCS-2 refers to the encoding where each character is a fixed 16-bit length, allowing access to the BMP.

UCS-4 or UTF-32 refers to an encoding where each character is a fixed 32-bit value, allowing direct access to the entire UCS.

UTF-16 is an encoding where a character is one or two 16-bit values, allowing access to the full Unicode code space 0 - 0x10FFFF.

UTF-8 Encoding

There are a few problems with using these UCS-2/4 or UTF-16 encodings.

- ◆ Since most characters used today are still from the 7-bit ASCII set, it takes almost twice as much space to use Unicode.
- ◆ It is incompatible with many current APIs.
- ◆ Byte order (big endian/little endian) is an issue.
- ◆ If data is being sent across a byte stream, and a byte is dropped, all the rest of the 16 bit Unicode characters will be out of sync and there's no way to sync up.

To address these problems, a byte-encoded form of Unicode was developed called Unicode Transformation Format 8-bit Encoding (UTF-8). This is just a simple algorithmic encoding of each 16-bit Unicode character into 1, 2, or 3

bytes. 4 bytes cover the entire Unicode 21-bit space, or 6 bytes to get the full 31-bit address space.

The greatest advantage is that the encoding for all 7-bit ASCII characters is identical in UTF-8. This solves the wasted space problem nicely, and provides a degree of compatibility with older systems. Byte order is not an issue since it's a byte stream.

The encoding of UTF-8 also allows unambiguous determination of the start of a character. By examining only the first byte, one can determine the number of bytes in the UTF-8 character sequence. Continuation bytes are easily recognizable, allowing one to detect a missing byte in a stream.

RFC2279 describes the UTF-8 encoding format in detail. Many other resources on the Web, including the Unicode Consortium website contain more information.

Using UTF-8 Strings in the LDAP SDK

In the LDAP version 2 specification, strings were limited to the T.61 character set, which is basically 7-bit US-ASCII minus several characters (such as tilde, caret, and curly braces). T.61 was a severe limitation to globalization and efforts to make LDAP a world-wide standard. In LDAP version 3, strings are to be encoded in UTF-8.

Because 7-bit ASCII characters are encoded identically in UTF-8, many applications continue to use local text strings with the LDAP C APIs. This works for ASCII characters, but will fail for extended 8-bit characters such as , (e accent) or multi-byte Asian characters.

The correct approach is to make sure all local strings are encoded into UTF-8 before using them in an LDAP API. Likewise strings returned from the APIs should be converted to local text if required, such as displaying them with `printf`.

Novell's LDAP C SDK provides routines for converting either local or Unicode strings into UTF-8 strings. Both single character and string versions of these routines are provided. Several string processing routines are also provided, such as UTF-8 versions of `strchr` and `strtok`, `next`, and `prev`.

The Wchar_t Type

Novell's SDK conversion routines use the `wchar_t` type. This type is 2 bytes on some machines and 4 bytes on other machines, so care must be taken if

wchar_t strings are transported to other systems. UTF-8 is the most portable way to transfer strings between systems.

Wchar_t strings will either be UCS-2 or UCS-4 encoded, depending on the size of wchar_t. The advantage to using wchar_t strings is that all the standard wide character string processing routines may be used, such as wcslen, wcsat, etc.

In summary, LDAP C applications which make the distinction between local and UTF-8 strings, and handle each properly, will be much easier to internationalize and move into the global marketplace.

Understanding Time Formats

Generalized Time Format. generalized time represents the values of year, month, day, hour, minutes, seconds and fractions of a second in any of three forms:

- ♦ Local time "YYYYMMDDHHMMSS.fff", where fff is optional and is fractions of a second
- ♦ Greenwich Mean Time (UTC) "YYYYMMDDHHMMSS.fffZ", Z indicates Greenwich Mean Time
- ♦ Difference between local and UTC time, "YYYYMMDDHHMMSS.fff+-HHMM", the +HHMM or -HHMM represents the time differential between the local and Greenwich Mean Times.

UTC Time Format. UTC format represents the values of year (2 digit), month, day, hour, minutes and optionally seconds.

- ♦ Local time "YYMMDDHHMMSS", where seconds (SS) is optional
- ♦ Greenwich Mean Time (UTC), "YYMMDDHHMMSSZ", seconds (SS) is optional and Z represents Greenwich Mean Time
- ♦ Difference between local and UTC time, "YYMMDDHHMMSS+-HHMM", seconds (SS) is optional and +HHMM or -HHMM represents the time differential between local and Greenwich Mean Times.

Handling Referrals in LDAP v3

NDS is a distributed and replicated database, and NDS servers usually contain only a portion of the directory. A search sent to an LDAP server on NDS has a high probability of returning partial data and referrals for the rest of the data.

In NDS, the LDAP server can be configured to return complete data and not return referrals. For possible configurations in NDS e-Directory, see the documentation for the LDAP Group Object (<http://www.novell.com/documentation/lg/ndsse/ndsseenu/data/fbabcieb.html>).

NDS does not return referrals on bind requests. If the requested entry is not on the local partitions of the LDAP server, NDS creates an external reference which allows the entry to bind to the server.

Controlling Automatic Referral Handling

The LDAP Libraries for C are initially set up to automatically follow referrals. This feature is controlled through the `LDAP_OPT_REFERRALS` option in the `ld` session handle.

- ◆ When set to ON (the default value), the libraries follow the referrals and perform an anonymous bind to the referred servers. In NDS, this bind is equivalent to the [Public] user and grants minimal rights to entries in the directory.

If you want your application to follow referrals but to perform a stronger authentication than an anonymous bind, you must supply a rebind process (see “Creating a Rebind Process” on page 42).

- ◆ When set to OFF, the libraries return `LDAP_REFERRAL` status on LDAP operations and continuation references on search operations as part of the search results. If the application wishes to process information from other servers, the application is responsible to follow the referrals or to retrieve the referrals from the search results and then to follow them.

To change the setting of the `LDAP_OPT_REFERRALS` option, call the `ldap_set_option` function with the option parameter set to `LDAP_OPT_REFERRALS` (see `ldap_set_option` (page 258)).

Limiting Referral Hops

Your application can specify the maximum number of referral hops the LDAP libraries will follow. For example, suppose you set the limit to two. Your application does a search, and the search refers you to the following:

Server 1 refers you to Server 2—Hop 1

Server 2 refers you to Server 3—Hop 2

Server 3 refers you to Server 4—Hop 3

The libraries will follow the referral through Server 3, but they will not continue to Server 4 because Server 4 exceeds the hop limit of 2. They return an result code of `LDAP_REFERRAL_LIMIT_EXCEEDED`.

To set the referral hop limit, call the `ldap_set_option` function with the option parameter set to `LDAP_OPT_REFERRAL_HOP_LIMIT` (see `ldap_set_option` (page 258)).

Creating a Rebind Process

The rebind function must use the following syntax.

```
int LIBCALL rebind_function (
    LDAP      *ld,
    const char *url,
    int        request,
    ber_int_t   msgid)
{
    /* the body must perform a synchronous bind */
}
```

The `ld` parameter must be used by the application to bind to the referred server if the application wants the libraries to follow the referral.

The `url` parameter points to the URL referral string received from the LDAP server. The LDAP application can use the `ldap_url_parse` (page 276) function to parse the string into its components.

The `request` parameter specifies the request operation that generated the referral. For possible values, see “Request Message Types” on page 389.

The `msgid` parameter specifies the message ID of the request generating the referral.

The libraries set all the parameters when they call the rebind function. The application should not attempt to free either the ld or the url structures in the rebind function.

The application is responsible for obtaining the required authentication information (user name, password, and certificates) associated with the ld and passing this information to the rebind function. The rebind function is responsible for performing the synchronous bind.

You must design your application to handle the possibility that the rebind process cannot bind to any of the referrals (for example, the servers are down or the authentication information is invalid). When this happens, the LDAP libraries return either

- ♦ results with referrals
- ♦ search results with search references

Using the Rebind Process

For the libraries to use a rebind process, the application must configure the ld to the following values:

- ♦ `LDAP_OPT_REFERRALS` option must be set to ON (the default value). For configuration information, see `ldap_set_option` (page 258).
- ♦ `LDAP_REBIND_PROC` must be set to the rebind function (see `ldap_set_rebind_proc` (page 260)).

When the ld has the proper settings, the referrals are processed according to the following algorithm.

1. The LDAP server sends a referral back to the libraries.
2. The libraries call the rebind function, setting the ld and url parameters.
3. The application supplies the logic for determining the type of bind.
For example, if the referral is to a server outside of a firewall, the application could decide to do an anonymous bind rather than a secure bind.
4. The application supplies the bind credentials associated with the ld (user name, password, and certificates) and with the bind method (such as simple, SSL, or SASL)
5. The libraries process the rebind function. If successful, the rebind function returns `LDAP_SUCCESS`.

If any other value is returned, the referral process stops and either LDAP_REFERRAL is returned as a result code for the original LDAP operation, or if a search operation, a search continuation is returned with the search results.

Controls and Extensions

Controls and Extensions were added to version 3 of the LDAP protocol. In version 2, there was no standard mechanism to extend the protocol, requiring developers to extend the protocol non-standard ways. In version 3, extensions and controls were defined to provide consistent expansion of the protocol.

NOTE: The NDS and LDAP Integration guide contains a good introduction to LDAP controls and extensions, and contains information and limitations you need to be aware of when using these controls with NDS. It is recommended that you read the LDAP Controls and the LDAP Extensions chapters in the NDS and LDAP Integration guide.

Controls

The following table contains a list of controls supported in the LDAP Libraries for C. For examples using these controls, see the LDAP Libraries for C “Sample Code” on page 20.

OID	Description
1.2.840.113556.1.4.473	Sever-side sort control request
1.2.840.113556.1.4.474	Server-side sort control response
2.16.840.1.113730.3.4.9	Virtual list view request
2.16.840.1.113730.3.4.10	Virtual list view response
2.16.840.1.113730.3.4.3	Persistent search
2.16.840.1.113730.3.4.7	Entry change notification

- ♦ **Server Side Sort.** Returns results from a search operation in sorted order. This can be used to off-load processing from the client, or if you cannot sort the results for some reason.

- ♦ **Vertical List View.** Works in conjunction with the Server Side Sort control to provide a dynamic view of a scrolling list. This works in conjunction with the Server Side Sort control.
- ♦ **Persistent Search & Entry Change Notification.** Provides a control to perform a continuous search, notifying the application of changes.

Extensions

The following table contains a list of extensions supported in the LDAP Libraries for C. For examples using these extensions, see the LDAP Libraries for C “Sample Code” on page 20.

OID	Name
2.16.840.1.113719.1.27.100.1	ndsToLdapResponse
2.16.840.1.113719.1.27.100.2	ndsToLdapRequest
2.16.840.1.113719.1.27.100.3	createNamingContextRequest
2.16.840.1.113719.1.27.100.4	createNamingContextResponse
2.16.840.1.113719.1.27.100.5	mergeNamingContextRequest
2.16.840.1.113719.1.27.100.6	mergeNamingContextResponse
2.16.840.1.113719.1.27.100.7	addReplicaRequest
2.16.840.1.113719.1.27.100.8	addReplicaResponse
2.16.840.1.113719.1.27.100.9	refreshLDAPServerRequest
2.16.840.1.113719.1.27.100.10	refreshLDAPServerResponse
2.16.840.1.113719.1.27.100.11	removeReplicaRequest
2.16.840.1.113719.1.27.100.12	removeReplicaResponse
2.16.840.1.113719.1.27.100.13	namingContextEntryCountRequest
2.16.840.1.113719.1.27.100.14	namingContextEntryCountResponse
2.16.840.1.113719.1.27.100.15	changeReplicaTypeRequest
2.16.840.1.113719.1.27.100.16	changeReplicaTypeResponse

OID	Name
2.16.840.1.113719.1.27.100.17	getReplicaInfoRequest
2.16.840.1.113719.1.27.100.18	getReplicaInfoResponse
2.16.840.1.113719.1.27.100.19	listReplicaRequest
2.16.840.1.113719.1.27.100.20	listReplicaResponse
2.16.840.1.113719.1.27.100.21	receiveAllUpdatesRequest
2.16.840.1.113719.1.27.100.22	receiveAllUpdatesResponse
2.16.840.1.113719.1.27.100.23	sendAllUpdatesRequest
2.16.840.1.113719.1.27.100.24	sendAllUpdatesResponse
2.16.840.1.113719.1.27.100.25	requestNamingContextSyncRequest
2.16.840.1.113719.1.27.100.26	requestNamingContextSyncResponse
2.16.840.1.113719.1.27.100.27	requestSchemaSyncRequest
2.16.840.1.113719.1.27.100.28	requestSchemaSyncResponse
2.16.840.1.113719.1.27.100.29	abortNamingContextOperationRequest
2.16.840.1.113719.1.27.100.30	abortNamingContextOperationResponse
2.16.840.1.113719.1.27.100.31	getContextIdentityNameRequest
2.16.840.1.113719.1.27.100.32	getContextIdentityNameResponse
2.16.840.1.113719.1.27.100.33	getEffectivePrivilegesRequest
2.16.840.1.113719.1.27.100.34	getEffectivePrivilegesResponse
2.16.840.1.113719.1.27.100.35	setReplicationFilterRequest
2.16.840.1.113719.1.27.100.36	setReplicationFilterResponse
2.16.840.1.113719.1.27.100.37	getReplicationFilterRequest
2.16.840.1.113719.1.27.100.38	getReplicationFilterResponse
2.16.840.1.113719.1.27.100.39	createOrphanNamingContextRequest
2.16.840.1.113719.1.27.100.40	createOrphanNamingContextResponse

OID	Name
2.16.840.1.113719.1.27.100.41	removeOrphanNamingContextRequest
2.16.840.1.113719.1.27.100.42	removeOrphanNamingContextResponse

Runtime Version of the Library Files

The Novell NDK license gives you permission to redistribute the LDAP Libraries for C with your application. These files are also shipped with NDS and the service packs. However, the NDK updates them more frequently, so you may have a newer version than the version shipping with NDS. In some instances, you may have older versions. If you select to redistribute the files, make sure your installation program does not overwrite newer versions.

The following sections provide some guidelines for the following platforms:

- ♦ “Windows (NT, 95, 98, 2000)” on page 47
- ♦ “NetWare” on page 48
- ♦ “Unix (Solaris and Linux)” on page 48

Windows (NT, 95, 98, 2000)

On the Windows platforms, you can copy the LDAP Libraries for C files to the same directory in which you install your program or to a directory that is part of the system’s path variable. Copy the non-debug version of the following library files to that directory:

```
ldapsdk.dll
ldapssl.dll
ldapx.dll
nssl.dll
```

You also need the message file. Copy the nls directory and all its subdirectories and files to the same directory you copied the library files, keeping the ldapsdkmsg.dll file in the same relative directory structure.

If your application uses any of the LDAP tools, these executables also need to be copied to the same directory as the library files. The ice utility requires the following files:

```
ice.cfg
```

ice.exe
ldaphdlr.dll
ldif.dll
sal.dll
zone.dll

NetWare

Copy the non-debug version of the following library files to the sys:\system directory with your application:

ldapsdk.nlm
ldapssl.nlm
ldapx.nlm
sas.nlm

You also need to copy the nls directory and its subdirectories the sys:\system directory, keeping the ldapsdk.msg file in the same relative directory structure.

If your application uses any of the LDAP tools, these nlms also need to be copied to the sys:\system directory. The ice utility requires the following files:

ice.cfg
ice.nlm
ldaphdlr.nlm
ldif.nlm
sal.nlm
zone.nlm

Unix (Solaris and Linux)

The library files and the application's binaries must be copied to a directory where the user has all access permissions. In the following descriptions, this directory is labelled the <install directory>. Copy the non-debug version of the following libraries files to the <install directory>/ldapcsdk/sdk/usr/lib directory:

libldapsdk.so
libldapssl.so
libldapx.so
libsasclient.so

Copy your application binaries to the <install directory>/ldapcsdk/sdk/usr/bin directory.

Copy the locale directory and its subdirectories to the <install directory>/ldapcsdk/sdk/usr/lib directory, keeping the ldapsdk.mo file in the same relative directory structure.

Export the following:

```
export LD_LIBRARY_PATH=<install directory>/ldapcsdk/sdk/usr/lib
export NLDASKD_ROOT=<install directory>
```


2 Tasks

This chapter provides step-by-step instructions for a few of the common tasks most LDAP applications perform. See DeveloperNet University (<http://developer.novell.com/servlet/devnet/education/labs.html>) for C LDAP tasks that

- ♦ Create an authenticated connection
- ♦ Create an NDS entry
- ♦ Read attribute values
- ♦ Read and write stream attribute values
- ♦ Search for attribute values
- ♦ Write attribute values

Establishing an SSL Connection

To establish an SSL connection, both the client and the LDAP server must be set up to use SSL. For instructions, see “Setting up SSL Security” on page 21.

To establish the SSL connection, call the following functions.

- 1** Initialize the SSL library by calling the `ldapssl_client_init` function.
- 2** Create an LDAP session handle (`ld`) by calling the `ldapssl_init` function.
- 3** Establish an authenticated SSL connection by calling the `ldap_simple_bind_s` function with a login distinguished name and password.
- 4** When you are finished with the connection, call the `ldap_unbind` function to free the memory associated with the `ld`.

- 5** To uninitialized the SSL library and free the associated memory, call the `ldapsl_client_deinit` function.

For sample code, see `sslbind.c` (http://developer.novell.com/ndk/doc/samplecode/cldap_sample/index.htm).

Reading the Root DSE

Reading the root DSE returns information about support of the following features of the LDAP server:

- ♦ LDAP versions (2 and 3)
- ♦ LDAP controls
- ♦ Schema name

With the schema name, you can then extend the schema or read its definitions. You must establish an LDAP v3 connection to read the DSE.

To read the DSE, call the following functions.

- 1** Set the LDAP version to LDAP v3 by calling the `ldap_set_option` function with the option parameter set to `LDAP_OPT_PROTOCOL_VERSION` and the invalue parameter set to `LDAP_VERSION3`.
- 2** Initialize a session and obtain an LDAP session handle (`ld`) by calling the `ldap_init` function.
- 3** Establish an authenticated connection by calling the `ldap_simple_bind_s` function.
- 4** Read the DSE by calling the `ldap_search_ext_s` function. Set the search base to `NULL`, the search filter to `objectclass=*`, and the scope to `LDAP_SCOPE_BASE`.
- 5** Obtain the DSE entry from the results by calling the `ldap_first_entry` function.
- 6** Obtain the first attribute by calling the `ldap_first_attribute` function.
- 7** Obtain the other attributes by calling the `ldap_next_attribute` function.
- 8** Obtain the values for the attributes by calling the `ldap_get_values` function.
- 9** Free the attributes and values by calling the `ldap_memfree` function.

- 10** Free the memory from the search results by calling the `ldap_msgfree` function.
- 11** When you are done with the session handle, call the `ldap_unbind` function to free the `ld` and the associated memory.

For sample code, see `getdse.c` (http://developer.novell.com/ndk/doc/samplecode/cldap_sample/index.htm).

Adding an Entry

To add an entry to the directory, the client must have create permissions to the container that will be the entry's parent container. Entries can be created programmatically or from an LDIF file. (For more information on using an LDIF file, see "Adding Entries" on page 465.)

To add an entry programmatically, complete the following steps.

- 1** Create an `LDAPMod` structure for each attribute that will be added with the entry.

You need a structure for each attribute. For example, an entry with a base class of `inetOrgPerson` requires `LDAPMod` structures for the following attributes: `cn`, `sn`, and `objectClass`. If you want the entry to log in to the directory, the entry also requires a structure for the `userPassword` attribute.

- 2** In each structure, set the modification operation to `LDAP_MOD_ADD` and the type to the name of the attribute. Add a NULL-terminated string of values for each attribute.
- 3** Add each structure to a NULL-terminated array of `LDAPMod` structures.
- 4** Set the `dn` for the entry.

The containers in the entry's `dn` must already exist in the directory.

- 5** Call the `ldap_add_ext_s` function to add the entry.

For sample code, see `addentry.c` (http://developer.novell.com/ndk/doc/samplecode/cldap_sample/index.htm).

Modifying an Entry

To modify an entry, the client must have write permissions to the attributes that are being modified.

- 1** Create an LDAPMod structure for each attribute that will be modified.
- 2** Set the modification operation, type, and value in each structure.

To add a value even when it may already exist, set the operation to LDAP_MOD_REPLACE.

To add a value and report an error if it already exists, set the operation to LDAP_MOD_ADD.

To delete an existing value, set the operation to LDAP_MOD_DELETE.
- 3** Add each structure to a NULL-terminated array of LDAPMod structures.
- 4** Call `ldap_modify_ext_s` to modify the specified entry.

For sample code, see (http://developer.novell.com/ndk/doc/samplecode/cldap_sample/index.htm).

Modifying an Entry's Password

NDS has a number of restricts that prevent password modification. The user can have insufficient rights for the following reasons:

- ♦ The user is not a supervisor of the entry.
- ♦ The flag that allows user to change the password is false.
- ♦ The password unique flag is true and the password supplied matches a previous password.
- ♦ A minimum length for the password has been set and the password is too short.
- ♦ The user did not supply the old password value with the new value in the same operation.

Passwords in NDS are stored as RSA public and private key pairs. The Novell LDAP server uses the `userPassword` attribute to generate these key pairs for an LDAP client.

- ♦ NDS 8.17 or higher is required for users to change their own passwords.
- ♦ NDS 7.xx is required for an administrator to change user passwords.

If the user has sufficient rights, the process is similar to modifying any attribute of an entry. For a user to change his or her own password, complete the following steps.

- 1** Create two LDAPMod structures for the `userPassword` attribute.

- 2** In the first LDAPMod structure, set the modification operation to LDAP_MOD_DELETE, the modification type to "userPassword", and the value to the current password.
- 3** In the second LDAPMod structure, set the modification operation to LDAP_MOD_ADD, the modification type to "userPassword", and the value to the new password.
- 4** Add the structures to a NULL-terminated array of LDAPMod structures.
- 5** Call ldap_modify_ext_s to modify the specified entry's password.

For sample code that allows a user to change his or her password, see modpass.c (http://developer.novell.com/ndk/doc/samplecode/cldap_sample/index.htm)

For sample code that allows an administrator to set a password, see setpass.c (http://developer.novell.com/ndk/doc/samplecode/cldap_sample/index.htm).

Extending the Schema

The NDS schema can be extended through LDAP programmatically using LDAP functions or using an LDIF file with a utility such as the “Novell Import Convert Export Utility” on page 422 and “ldapmodify” on page 455.

The following steps give a simple example how to programmatically extend the schema by creating an auxiliary class that uses existing attributes.

- 1** Create a NULL-terminated string that defines the OID, the class name, description, super class, class type, must attributes, and may attributes. RFC 2252 defines the format of the string.

It should look similar to the following definition for the TestAuxClass.

```
char    *auxClassDefVals[] = { "( 1.1.1.1.1.1111
    NAME 'TestAuxClass'
    DESC 'Useless ObjectClass for testing'
    SUP 'top'
    AUXILIARY
    MUST ( operator $ server )
    MAY ( status ) )"
    , NULL };
```

NOTE: You need to use a valid OID when extending the schema. To register and obtain a unique OID for your group of attribute and class extensions, see the Novell Developer Support Web site (<http://developer.novell.com/support/schreg2c.htm>)

- 2** Create an LDAPMod structure for the class.

- ♦ Set the `mod_op` to `LDAP_MOD_ADD`
 - ♦ Set the `mod_type` to "objectclasses"
 - ♦ Set the `mod_values` to the string (in the example above, to `auxClassDefVals`)
- 3** Add each structure to a NULL-terminated array of `LDAPMod` structures.
- 4** To add the class, call `ldap_modify_ext_s` with the parameters set to the following values:
- ♦ `dn` to "cn=schema" (the name of the schema is obtained by reading the root DSE; see "Reading the Root DSE" on page 52)
 - ♦ `mods` to the NULL-terminated array of `LDAPMod` structures you have created
 - ♦ `serverctrls` to NULL
 - ♦ `clientctrls` to NULL

3

Standard LDAP Functions

This chapter documents the standard LDAP functions defined by RFCs and Internet drafts maintained by IETF. For information on the LDAP extensions Novell provides to perform NDS partition and replica operations, see “LDAP Extension Functions” on page 307.

ber_alloc_t

Constructs and returns an empty BerElement.

Library: *ldapsdk.*

NDS Version: 7.xx or higher

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldap.h> or <lber.h>

BerElement *ber_alloc_t (
    int    options);
```

Parameters

options
(IN) Specifies the options used to create a BerElement.

Return Values

Returns a newly created BerElement on success; otherwise, returns a NULL pointer on failure.

Remarks

The options parameter specifies a bitwise OR of options to be used when encoding a new BerElement. You should always supply the following option:

LBER_USE_DER 0x01	Specifies that lengths will always be encoded in the minimum number of octets. However, this option does not cause values of sets to be rearranged in tag and byte order or for default values to be removed, so these options are not sufficient for generating DER output as defined in the X.509 and X.680 specifications. If you order set values and remove default values correctly, you can produce output according to the defined specifications.
----------------------	--

Unrecognized option bits are ignored.

Calls to the `ber_printf` function append bytes to the end of the `BerElement`.

Each `BerElement` structure allocated by the `ber_alloc_t` function should be freed by a call to the function.

See Also

`ber_free` (page 67)

ber_bvdup

Returns a copy of a berval structure.

Library: *ldapsdk.*

NDS Version: 7.xx or higher

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldap.h> or <lber.h>

struct berval *ber_bvdup (
    const struct berval *bv);
```

Parameters

bv

(IN) Points to a structure to return.

Return Values

Returns a pointer to a berval structure on success; otherwise, returns NULL on failure.

Remarks

The bv_val field in the returned berval structure points to a different area of memory than the original bv_val field of the bv parameter.

The berval structures created by the ber_bvdup function should be freed by a call to the ber_bvfree function.

See Also

ber_bvfree (page 62)

ber_bvecfree

Frees an array of returned berval structures.

Library: *ldapsdk.*

NDS Version: 7.xx or higher

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldap.h> or <lber.h>

void ber_bvecfree (
    struct berval **bv);
```

Parameters

bv

(IN) Points to the array of berval structures that are to be freed.

Remarks

Each structure in the array is freed by calling the ber_bvfree function, then the array itself is freed.

If the bv parameter is NULL, the ber_bvfree function does nothing.

ber_bvfree

Frees a returned berval structure.

Library: *ldapsdk.*

NDS Version: 7.xx or higher

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldap.h> or <lber.h>

void ber_bvfree (
    struct berval *bv);
```

Parameters

bv

(IN) Points to the berval structure to be freed.

Remarks

Both the bv_val string in the berval structure and the structure itself are freed.

If the bv parameter is NULL, this function does nothing.

ber_first_element

Positions the state of a BerElement to its first element and returns the type of the first element.

Library: *ldapsdk.*

NDS Version: 7.xx or higher

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldap.h> or <lber.h>

ber_tag_t ber_first_element (
    BerElement *ber,
    ber_len_t *lenPtr,
    char **opaquePtr);
```

Parameters

ber

(IN) Points to the first element in the constructed type.

lenPtr

(OUT) Is used for internal use only. Use this value in the subsequent call to the ber_next_element function.

opaquePtr

(OUT) Is used for internal use only. Use this value in the subsequent call to the ber_next_element function

Return Values

On success, returns a tag indicating the type of the first element. Returns LBER_DEFAULT if there are no elements.

Remarks

Use the ber_scanf function to obtain the value of the first element.

See Also

`ber_scanf` (page 76)

ber_flatten

Allocates a berval structure whose contents are a BER encoding of the specified BerElement.

Library: *ldapsdk.*

NDS Version: 7.xx or higher

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldap.h> or <lber.h>

int ber_flatten (
    BerElement      *ber,
    struct berval    **bvPtr);
```

Parameters

ber

(IN) Points to the encoded contents for a BerElement.

bvPtr

(OUT) Points to the returned berval structure.

Return Values

Returns zero on success; otherwise, returns -1 on failure.

Remarks

The berval structure should be freed by calling the ber_bvfree function.

The ber_flatten function returns -1 if all '{' and '}' format modifiers are not properly matched.

Ber_init and ber_flatten are opposite functions. Ber_init converts a berval to a BerElement, and ber_flatten converts a BerElement to a berval.

See Also

`ber_bvfree` (page 62), `ber_init` (page 69)

ber_free

Frees a BerElement structure allocated by the ber_alloc_t or the ber_init function.

Library: *ldapsdk.*

NDS Version: 7.xx or higher

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldap.h> or <lber.h>

void ber_free (
    BerElement  *ber,
    int         fbuf);
```

Parameters

ber

(IN) Points to a BerElement to be freed.

fbuf

(IN) Flag indicating if the internal buffer associated with the BerElement should also be freed.

1 frees the internal buffer, 0 does not free it.

Remarks

BerElements allocated by the library and returned to the application should be freed.

Note that when ldap_first_attribute returns a BerElement, it should be freed with ber_free(ber, 0). The internal buffer should not be freed since it points to the original searchResults.

If the ber parameter is NULL, the ber_free function does nothing.

See Also

`ber_alloc_t` (page 58), `ber_init` (page 69)

ber_init

Allocates and initializes a new BerElement structure with a copy of the data in the given berval structure.

Library: *ldapsdk.*

NDS Version: 7.xx or higher

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldap.h> or <lber.h>

BerElement *ber_init (
    const struct berval *bv);
```

Parameters

bv

(IN) Points to the berval structure with which to initialize the new BerElement.

Return Values

Returns a new BerElement with the specified data on success; otherwise, returns a NULL pointer on failure.

Remarks

BerElements allocated with the ber_init function should be freed by calling the ber_free function.

Ber_init and ber_flatten are opposite functions. Ber_init converts a berval to a BerElement, and ber_flatten converts a BerElement to a berval.

See Also

ber_free (page 67), ber_flatten (page 65)

ber_next_element

Positions the state of a BerElement to the next element and returns its type.

Library: *ldapsdk.*

NDS Version: 7.xx or higher

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldap.h> or <lber.h>

ber_tag_t ber_next_element (
    BerElement  *ber,
    ber_len_t   *lenPtr,
    char        *opaquePtr);
```

Parameters

ber

(IN) Points to a BerElement structure.

lenPtr

(OUT) Is used for internal use only. Points to the value returned by the ber_first_element function. On subsequent calls, points to the value returned by the ber_next_element function.

opaquePtr

(OUT) Is used for internal use only. Points to the value returned by the ber_first_element function. On subsequent calls, points to the value returned by the ber_next_element function.

Return Values

On success, returns a tag indicating the type of the next element. Returns LBER_DEFAULT if there are no further elements.

Remarks

Use the `ber_scanf` function to obtain the value of the element.

See Also

`ber_scanf` (page 76)

ber_peek_tag

Returns the tag and length of the next element in a BerElement.

Library: *ldapsdk.*

NDS Version: 7.xx or higher

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldap.h> or <lber.h>

ber_tag_t ber_peek_tag (
    BerElement  *ber,
    ber_len_t   *lenPtr);
```

Parameters

ber

(IN) Points to the BerElement.

lenPtr

(OUT) Points to the length of next element to be parsed.

Return Values

Returns the tag of the next element to be parsed on success; returns LBER_DEFAULT if there is no further data to be read.

Remarks

The decoding position within the ber parameter is not changed and will not affect the future use of the ber parameter.

ber_printf

encodes data items into a BerElement.

Library: *ldapsdk.*

NDS Version: 7.xx or higher

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldap.h> or <lber.h>

int ber_printf (
    BerElement *ber,
    const char *fmt,
    ...);
```

Parameters

ber

(IN) Points to a BerElement.

fmt

(IN) Points to a format string.

...

(IN) Specifies data values for each tag in the format string.

Return Values

Returns a nonnegative number on success; otherwise, returns -1 on failure.

Remarks

The ber_printf function encodes a BerElement in a similar manner as the sprintf function. However, the ber_printf function must keep state information in the ber parameter so that this function can be called subsequently to append information to the end of a BerElement.

Similar to the `sprintf` function, each character in the `fmt` parameter refers to an argument to the `ber_printf` function.

The `fmt` parameter can have the following characters.

b	Boolean	The next parameter is a <code>ber_int_t</code> , which contains either 0 for False or 0xff for True.
B	Bitstring	The next two parameters are a <code>char*</code> pointer to the start of the bitstring, followed by a <code>ber_len_t</code> , which contains the number of bits in the bitstring. A bitstring element is output in primitive form. If this character is not preceded by the 't' modifier, the 0x03U tag is used for the element.
e	Enumerated	The next parameter is a <code>ber_int_t</code> , which contains the enumerated value in the host's byte order. An enumerated element is output. If this character is not preceded by the 't' modifier, the 0x0AU tag is used for the element.
i	Integer	The next parameter is a <code>ber_int_t</code> , which contains the integer in the host's byte order. An integer element is output. If this character is not preceded by the 't' modifier, the 0x02U tag is used for the element.
n	NULL	No parameter is needed. An ASN.1 NULL element is output. If this character is not preceded by the 't' modifier, the 0x05U tag is used for the element.
o	Octet string	The next two parameters are a <code>char*</code> pointer, followed by a <code>ber_len_t</code> that contains the length of the string. The string can contain NULL bytes and do not have to be zero terminated. An octet string element is output in primitive form. If this character is not preceded by the 't' modifier, the 0x04U tag is used for the element.
O	Octet string	The next parameter is a pointer to a <code>berval</code> structure. If this character is not preceded by the 't' modifier, the 0x04U tag is used for the element.
s	Octet string	The next parameter is a <code>char*</code> pointer to a zero-terminated string. An octet string is output in primitive form and does not include the trailing '\0' (NULL) byte. If this character is not preceded by the 't' modifier, the 0x04U tag is used for the element.
t	Tag	The next parameter is a <code>ber_tag_t</code> , which specifies the tag to override the next element to be written to the <code>BerElement</code> .
v	Several octet strings	The next parameter is a <code>char**</code> , an array of <code>char*</code> pointers to zero-terminated strings. The last element in the array must be a NULL pointer. The octet strings do not include the trailing '\0' (NULL) byte. A construct similar to '{v}' is used to get an actual sequence of octet strings. The 't' modifier cannot be used with this character.
V	Several octet strings	A NULL-terminated array of <code>berval</code> structure pointers is supplied. Note that a construct similar to '{V}' is used to get an actual sequence of octet strings. The 't' modifier cannot be used with this character.

{	Begin sequence	No parameter is needed. If this character is not preceded by the 't' modifier, the 0x30U tag is used for the element.
}	End sequence	No parameter is needed. The 't' modifier cannot be used with this character.
[	Begin set	No parameter is needed. If this character is not preceded by the 't' modifier, the 0x31U tag is used for the element.
]	End set	No parameter is needed. The 't' modifier cannot be used with this character.

Each use of a '{' character should be matched with a '}' character, either later in the format string or in the format string of a subsequent call to `ber_printf` for that specific `BerElement`. The same rules applies to the '[' and ']' characters.

ber_scanf

Decodes a BerElement, similar to the sscanf function.

Library: *ldapsdk.*

NDS Version: 7.xx or higher

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldap.h> or <lber.h>

ber_tag_t ber_scanf (
    BerElement *ber,
    const char *fmt,
    ...);
```

Parameters

ber

(IN) Points to a BerElement returned by the ber_init function.

fmt

(IN) Points to the format modifiers to use when interpreting the BerElement bytes.

...

(OUT) Returns pointers to data values returned by the function.

Return Values

Returns a non-LBER_ERROR value on success; otherwise, returns LBER_ERROR on failure.

Remarks

The ber_scanf function keeps some of the state information with the ber parameter so that the ber_scanf function can be called iteratively to sequentially read from the BerElement.

The results of successfully calling the `ber_scanf` function are stored in additional parameters.

The `fmt` parameter can have the following values.

a	Octet string	A <code>char*</code> pointer must be supplied. Memory is allocated and filled with the contents of the octet string (zero-terminated). The pointer to the string is stored in the parameter. The returned value should be freed by calling the <code>ldap_memfree</code> function. The element tag must indicate the primitive form (constructed strings are not supported) but is otherwise ignored and discarded during the decoding. This character cannot be used with octet string that contain NULL bytes.
b	Boolean	A pointer to <code>ber_int_t</code> must be supplied. The stored value will be zero for FALSE or nonzero for TRUE. The element tag must indicate the primitive form but is otherwise ignored during the decoding.
B	Bitstring	A <code>char**</code> parameter must be supplied that will point to the allocated bits. This is followed by a <code>ber_len_t*</code> parameter that will point to the length (in bits) of the returned bitstring. The <code>ldap_memfree</code> function should be called to free the bitstring. The element tag must indicate the primitive form (constructed bitstrings are not supported) but is otherwise ignored during the decoding.
e	Enumerated	A pointer to <code>ber_int_t</code> must be supplied. The stored value will be in host byte order. The element tag must indicate the primitive form but is otherwise ignored during the decoding. The <code>ber_scanf</code> function returns an error if the enumerated value cannot be stored in a <code>ber_int_t</code> .
i	Integer	A pointer to <code>ber_int_t</code> must be supplied. The stored value will be in host byte order. The element tag must indicate the primitive form but is otherwise ignored during the decoding. The <code>ber_scanf</code> function returns an error if the integer cannot be stored in a <code>ber_int_t</code> .
L	Length	A pointer to a <code>ber_len_t</code> must be supplied. The length of the next element in bytes is returned.
n	NULL	No parameter is needed. The element is verified to have a zero-length value and is skipped. The tag is ignored.
o	Octet string	A <code>berval *</code> parameter must be supplied, pointing to an existing empty <code>berval</code> structure. The buffer inside the <code>berval</code> is allocated as required and should be freed with the <code>ldap_memfree</code> function when done.
O	Octet string	A <code>berval **</code> parameter must be supplied, which will point to an allocated <code>berval</code> structure that contains the octet string and its length upon return. The <code>ber_bvfree</code> function should be called to free the allocated memory. The element tag must indicate the primitive form (constructed strings are not supported) but is otherwise ignored during the decoding.

s	Octet string	A char * buffer must be supplied, point to an existing buffer. It must be followed by a ber_len_t * parameter. The object of this pointer contains the size of the buffer on input and is replaced with the size of the data written to the buffer on output.
t	Tag	A pointer to ber_tag_t must be supplied. The stored value will be the tag of the next element in the BerElement ber parameter and represented so it can be written using the 't' modifier of the ber_printf function. The decoding position within the ber parameter is not changed and can be used in the future.
v	Several octet strings	A char*** parameter must be supplied, which points to an allocated, NULL-terminated array of char* pointers that contain the octet strings upon return. NULL is stored if the sequence is empty. The ldap_memfree function should be called to free each element of the array and the array itself. The sequence tag and the octet string tags are ignored.
V	Several octet strings	A berval*** structure pointer must be supplied, which points to an allocated, NULL-terminated array of berval* structure pointers that contain the octet strings and their lengths upon return. NULL is stored if the sequence is empty. The ber_bvecfree function can be called to free the allocated memory. The sequence tag and the octet string tags are ignored.
x	Skip element	The next element is skipped. No parameter is needed.
{	Begin sequence	No parameter is needed. The initial sequence tag and length are skipped.
}	End sequence	No parameter is needed.
[	Begin set	No parameter is needed. The initial set tag and length are skipped.
]	End set	No parameter is needed.

ber_skip_tag

Skips the next element of a BerElement, returning its length and tag.

Library: *ldapsdk.*

NDS Version: 7.xx or higher

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldap.h> or <lber.h>
```

```
ber_tag_t ber_skip_tag (  
    BerElement *ber,  
    ber_len_t   *lenPtr);
```

Parameters

ber

(IN) Points to the BerElement.

lenPtr

(OUT) Points to the length of the skipped element.

Return Values

Returns the tag of the element that was skipped on success; otherwise, returns LBER_DEFAULT if there is no further data to be read.

ldap_abandon

Abandons an asynchronous LDAP operation already in progress. This function has been deprecated; LDAP v3 clients should use `ldap_abandon_ext` (page 82).

LDAP Version: v2 or higher

Library: *ldapsdk.*

NDS Version: 7.xx or higher

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldap.h>

int ldap_abandon (
    LDAP      *ld,
    int       msgid);
```

Parameters

`ld`

(IN) Points to the handle for the LDAP session.

`msgid`

(IN) Specifies the message ID of the asynchronous LDAP operation to abandon.

Return Values

0	Success
-1	Failure

Remarks

The `msgid` parameter must specify a message ID returned by an outstanding asynchronous LDAP operation, such as `ldap_search` or `ldap_modify`.

The `ldap_abandon` function checks to see if the results of the operation has already come in.

- ♦ If not, it sends an LDAP abandon operation to the LDAP server.
- ♦ If the results have already come in, the LDAP operation cannot be abandoned.

If the `ldap_abandon` function returns -1, use the `ldap_get_option` function with the option parameter set to `LDAP_OPT_RESULT_CODE` to retrieve the error code from the LDAP session handle.

See Also

`ldap_abandon_ext` (page 82)

ldap_abandon_ext

Abandons an asynchronous LDAP operation already in progress using LDAP client or server controls.

LDAP Version: v3

Library: *ldapsdk.*

NDS Version: 7.xx or higher

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldap.h>

int ldap_abandon_ext (
    LDAP      *ld,
    int       msgid,
    LDAPControl **serverctrls,
    LDAPControl **clientctrls);
```

Parameters

ld

(IN) Points to the handle for the LDAP session.

msgid

(IN) Specifies the message ID of the asynchronous LDAP operation to abandon.

serverctrls

(IN) Points to a list of LDAP server controls to use with the abandon operation. Use NULL to specify no server controls.

clientctrls

(IN) Points to an array of LDAPControl structures that list the client controls to use with the abandon operation. Use NULL to specify no client controls.

Return Values

0x00	LDAP_SUCCESS
Non-zero	Failure. For a complete list, see LDAP Return Codes.
0x51	LDAP_SERVER_DOWN
0x53	LDAP_ENCODING_ERROR
0x59	LDAP_PARAM_ERROR
0x5A	LDAP_NO_MEMORY

Remarks

The msgid parameter must specify a message ID returned by an outstanding asynchronous LDAP operation, such as ldap_search or ldap_modify.

The ldap_abandon function checks to see if the results of the operation has already come in.

- ♦ If not, it sends an LDAP abandon operation to the LDAP server.
- ♦ If the results have already come in, the LDAP operation cannot be abandoned.

NDS does not currently support any controls to use with an abandon operation.

See Also

ldap_abandon (page 80)

ldap_add

Asynchronously adds an entry to the directory. This function has been deprecated; LDAP v3 clients should use `ldap_add_ext` (page 86).

LDAP Version: v2 or higher

Library: *ldapsdk.*

NDS Version: 7.xx or higher

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldap.h>

int ldap_add (
    LDAP      *ld,
    const char *dn,
    LDAPMod   **attrs);
```

Parameters

`ld`

(IN) Points to the handle for the LDAP session.

`dn`

(IN) Points to the distinguished name of the entry to add, for example: "o=novell", "ou=provo", "cn=kim"

All components of the dn must exist except for the leaf component. The leaf component name must be unique within the container.

`attrs`

(IN) Points to a NULL terminated array of LDAPMod structures that contain the attributes and value to add with the entry. All mandatory attributes must have values or the operation fails.

Return Values

>0	Message ID of request
-1	Failure

Remarks

To obtain the results of the operation, call the `ldap_result` function with the returned message ID.

For a list of mandatory attributes for an entry see the LDAP server's schema. For NDS, see *NDS Schema Reference*.

See Also

`ldap_add_s` (page 92), `ldap_add_ext` (page 86), `ldap_add_ext_s` (page 89), `ldap_modify` (page 175)

ldap_add_ext

Asynchronously adds an entry to the directory using LDAP client or server controls.

LDAP Version: v3

Library: *ldapsdk.*

NDS Version: 7.xx or higher

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldap.h>

int ldap_add_ext (
    LDAP          *ld,
    const char    *dn,
    LDAPMod       **attrs,
    LDAPControl   **serverctrls,
    LDAPControl   **clientctrls,
    int           *msgidp);
```

Parameters

ld

(IN) Points to the handle for the LDAP session.

dn

(IN) Points to the distinguished name of the entry to add, for example: "o=novell", "ou=provo", "cn=kim"

All components of the dn must exist except for the leaf component. The leaf component name must be unique within the container.

attrs

(IN) Points to a NULL-terminated array of LDAPMod structures that contain the attributes and values to add with the entry. All mandatory attributes must have values or the operation fails.

serverctrls

(IN) Points to an array of LDAPControl structures that list the server controls to use with the add. Use NULL to specify no server controls.

clientctrls

(IN) Points to an array of LDAPControl structures that list the client controls to use with the add. Use NULL to specify no client controls.

msgidp

(OUT) Points to the message ID of the request when the add request succeeds.

Return Values

0x00	LDAP_SUCCESS
Non-zero	Failure. For a complete list, see LDAP Return Codes.
0x53	LDAP_ENCODING_ERROR
0x5A	LDAP_NO_MEMORY

Remarks

To obtain the results of the operation, call the ldap_result function with the returned message ID.

For a list of mandatory attributes for an entry see the LDAP server's schema. For NDS, see *NDS Schema Reference*.

If you are adding an entry that logs in to the directory, you need to set a value for the userPassword attribute. The userPassword attribute is not a mandatory attribute. However, if you create an entry without a userPassword attribute, the entry cannot log in.

NDS does not currently support any server-side controls to use with adding entries.

See Also

`ldap_add` (page 84), `ldap_add_ext_s` (page 89), `ldap_add_s` (page 92)

ldap_add_ext_s

Synchronously adds an entry to the directory using LDAP client or server controls.

LDAP Version: v3

Library: *ldapsdk.*

NDS Version: 7.xx or higher

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldap.h>

int ldap_add_ext_s (
    LDAP          *ld,
    const char     *dn,
    LDAPMod        **attrs,
    LDAPControl    **serverctrls,
    LDAPControl    **clientctrls);
```

Parameters

ld

(IN) Points to the handle for the LDAP session.

dn

(IN) Points to the distinguished name of the entry to add, for example: "o=novell", "ou=provo", "cn=kim"

All components of the dn must exist except for the leaf component. The leaf component name must be unique within the container.

attrs

(IN) Points to a NULL-terminated array of LDAPMod structures that contain the attributes and values to add with the entry. All mandatory attributes must have values or the operation fails.

serverctrls

(IN) Points to an array of LDAPControl structures that list the server controls to use with the add. Use NULL to specify no server controls.

clientctrls

(IN) Points to an array of LDAPControl structures that list the client controls to use with the add. Use NULL to specify no client controls.

msgidp

(OUT) Points to the message ID of the request when the add request succeeds.

Return Values

0x00	LDAP_SUCCESS
Non-zero	Failure. For a complete list, see LDAP Return Codes.
0x01	LDAP_OPERATIONS_ERROR
0x02	LDAP_PROTOCOL_ERROR
0x08	LDAP_STRONG_AUTH_REQUIRED
0x11	LDAP_UNDEFINED_TYPE
0x13	LDAP_CONSTRAINT_VIOLATION
0x14	LDAP_TYPE_OR_VALUE_EXISTS
0x15	LDAP_INVALID_SYNTAX
0x0A	LDAP_REFERRAL
0x0C	LDAP_UNAVAILABLE_CRITICAL_EXTENSION
0x0D	LDAP_CONFIDENTIALITY_REQUIRED
0x20	LDAP_NO_SUCH_OBJECT
0x22	LDAP_INVALID_DN_SYNTAX
0x32	LDAP_INSUFFICIENT_ACCESS

0x33	LDAP_BUSY
0x35	LDAP_UNWILLING_TO_PERFORM
0x36	LDAP_LOOP_DETECT
0x40	LDAP_NAMING_VIOLATION
0x41	LDAP_OBJECT_CLASS_VIOLATION
0x44	LDAP_ALREADY_EXISTS
0x50	LDAP_OTHER
0x53	LDAP_ENCODING_ERROR
0x5A	LDAP_NO_MEMORY

Remarks

For a list of mandatory attributes for an entry see the LDAP server's schema. For NDS, see *NDS Schema Reference*.

If you are adding an entry that logs in to the directory, you need to set a value for the userPassword attribute. The userPassword attribute is not a mandatory attribute. However, if you create an entry without a userPassword attribute, the entry cannot log in.

NDS does not currently support any server-side controls to use with adding entries.

For sample code, see addentry.c and addentry1.c (http://developer.novell.com/ndk/doc/samplecode/cldap_sample/index.htm).

See Also

ldap_add (page 84), ldap_add_s (page 92), ldap_add_ext (page 86),

ldap_add_s

Synchronously adds an entry to the directory.

LDAP Version: v2 or higher

Library: *ldapsdk.*

NDS Version: 7.xx or higher

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldap.h>

int ldap_add_s (
    LDAP      *ld,
    const char *dn,
    LDAPMod   **attrs);
```

Parameters

ld

(IN) Points to the handle for the LDAP session.

dn

(IN) Points to the distinguished name of the entry to add, for example:
"o=novell", "ou=provo", "cn=kim"

All components of the dn must exist except for the leaf component. The leaf component name must be unique within the container.

attrs

(IN) Points to an array of LDAPMod structures that contain the attributes and values to add with the entry. All mandatory attributes must have values or the operation fails.

Return Values

0x00

LDAP_SUCCESS

Non-zero	Failure. For a complete list, see LDAP Return Codes.
0x53	LDAP_ENCODING_ERROR
0x5A	LDAP_NO_MEMORY

Remarks

The `ldap_add_s` is an older function. LDAP v3 clients should use the `ldap_add_ext_s` function.

For a list of mandatory attributes for an entry see the LDAP server's schema. For NDS, see *NDS Schema Reference*.

See Also

`ldap_add` (page 84), `ldap_add_ext` (page 86), `ldap_add_ext_s` (page 89), `ldap_modify` (page 175)

ldap_bind

Asynchronously authenticates a specified entry to the directory. This function has been deprecated; use the `ldap_simple_bind` function.

LDAP Version: v2 or higher

Library: *ldapsdk.*

NDS Version: 7.xx or higher

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldap.h>

int ldap_bind (
    LDAP      *ld,
    const char *dn,
    const char *cred,
    int        method);
```

Parameters

`ld`

(IN) Points to the handle for the LDAP session which is returned by either the `ldap_open` or `ldap_init` function.

`dn`

(IN) Points to the distinguished name of the entry to use for authentication, for example: "o=novell", "ou=provo", "cn=kim"

`cred`

(IN) Points to the credentials to use for authentication

`method`

(IN) Specifies the authentication method. NDS supports the following methods:

- ◆ `LDAP_AUTH_NONE` (0x00)—no authentication
- ◆ `LDAP_AUTH_SIMPLE` (0x80)—context specific + primitive

Return Values

>0	Message ID of operation
-1	Failure

See Also

`ldap_simple_bind` (page 263), `ldap_unbind`, `ldap_unbind_s` (page 273),
`ldap_unbind_ext`, `ldap_unbind_ext_s` (page 274)

ldap_bind_s

Synchronously authenticates a specified entry to the directory. This function has been deprecated; use the `ldap_simple_bind_s` function.

LDAP Version: v2 or higher

Library: *ldapsdk.*

NDS Version: 7.xx or higher

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldap.h>

int ldap_bind_s (
    LDAP      *ld,
    const char *dn,
    const char *cred,
    int        method);
```

Parameters

`ld`

(IN) Points to the handle for the LDAP session which is returned by either the `ldap_open` or `ldap_init` function.

`dn`

(IN) Points to the distinguished name of the entry to use for authentication, for example: "o=novell", "ou=provo", "cn=kim"

`cred`

(IN) Points to the credentials to use for authentication

`method`

(IN) Specifies the authentication method. NDS supports the following methods:

- ◆ `LDAP_AUTH_NONE` (0x00)—no authentication
- ◆ `LDAP_AUTH_SIMPLE` (0x80)—context specific + primitive

Return Values

0x00	LDAP_SUCCESS
Non-zero	Failure. For a complete list, see LDAP Return Codes.
0x54	LDAP_DECODING_ERROR
0x56	LDAP_AUTH_UNKNOWN
0x59	LDAP_PARAM_ERROR
0x5A	LDAP_NO_MEMORY
0x5C	LDAP_NOT_SUPPORTED

See Also

[ldap_simple_bind](#) (page 263), [ldap_unbind](#), [ldap_unbind_s](#) (page 273),
[ldap_unbind_ext](#), [ldap_unbind_ext_s](#) (page 274)

ldap_compare

Asynchronously determines whether a specified entry contains a specified attribute value.

LDAP Version: v2 or higher

Library: *ldapsdk.*

NDS Version: 7.xx or higher

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldap.h>

int ldap_compare (
    LDAP      *ld,
    const char *dn,
    const char *attr,
    const char *value);
```

Parameters

ld

(IN) Points to the handle for the LDAP session.

dn

(IN) Points to the distinguished name of the entry whose attribute is being compared.

attr

(IN) Points to the name of the attribute to compare.

value

(IN) Points to a string value of the attribute to compare.

Return Values

>0	Message ID of operation
-1	Failure

Remarks

This function compares the specified value with the values in the entry's attribute. The results specify whether a match is found.

The `ldap_compare` function is an older function. LDAP v3 clients should use the `ldap_compare_ext` function.

The `ldap_compare` function can compare only attributes with string values. Use `ldap_compare_ext` to compare binary values.

To obtain the results of the operation, call the `ldap_result` function with the returned message ID.

Compare operations are faster than search operations. Whenever possible in your application, use a compare rather than a search operation.

See Also

`ldap_compare_ext` (page 100)

ldap_compare_ext

Asynchronously determines whether a specified entry contains a specified attribute value. LDAP client or server controls can be used with the compare.

LDAP Version: v3

Library: *ldapsdk.*

NDS Version: 7.xx or higher

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldap.h>

int ldap_compare_ext (
    LDAP          *ld,
    const char    *dn,
    const char    *attr,
    const struct berval *bvalue,
    LDAPControl   **serverctrls,
    LDAPControl   **clientctrls,
    int           *msgidp);
```

Parameters

ld

(IN) Points to the handle for the LDAP session.

dn

(IN) Points to the distinguished name of the entry whose attribute is being compared.

attr

(IN) Points to the name of the attribute to compare.

bvalue

(IN) Points to a berval structure that contains the attribute's value to compare with the entry's attribute value.

serverctrls

(IN) Points to an array of LDAPControl structures that list the server controls to use with the search. Use NULL to specify no server controls.

clientctrls

(IN) Points to an array of LDAPControl structures that list the client controls to use with the search. Use NULL to specify no client controls.

msgidp

(OUT) Points to the message ID of the request when the search request succeeds.

Return Values

0x00	LDAP_SUCCESS
Non-zero	Failure. For a complete list, see LDAP Return Codes.
0x53	LDAP_ENCODING_ERROR
0x5A	LDAP_NO_MEMORY

Remarks

This function compares the specified value with the values in the entry's attribute. The results specify whether a match is found.

The ldap_compare_ext function can be used to compare any type of data. For string data, you can use the ldap_compare function.

The data returned in msgidp is opaque to the caller. To obtain the results of the operation, call the ldap_result function with the returned message ID.

Compare operations are faster than search operations. Whenever possible in your application, use a compare rather than a search operation.

NDS does not currently support any server controls to use with compare operations.

See Also

[ldap_compare](#) (page 98)

ldap_compare_ext_s

Synchronously determines whether a specified entry contains a specified attribute value. LDAP client or server controls can be used with the compare.

LDAP Version: v3

Library: *ldapsdk.*

NDS Version: 7.xx or higher

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldap.h>

int ldap_compare_ext_s (
    LDAP          *ld,
    const char     *dn,
    const char     *attr,
    const struct berval *bvalue,
    LDAPControl    **serverctrls,
    LDAPControl    **clientctrls);
```

Parameters

ld

(IN) Points to the handle for the LDAP session.

dn

(IN) Points to the distinguished name of the entry whose attribute is being compared.

attr

(IN) Points to the name of the attribute to compare.

bvalue

(IN) Points to berval structure that contains the attribute's value to compare with the entry's attribute value.

serverctrls

(IN) Points to an array of LDAPControl structures that list the server controls to use with the search. Use NULL to specify no server controls.

clientctrls

(IN) Points to an array of LDAPControl structures that list the client controls to use with the search. Use NULL to specify no client controls.

Return Values

0x05	LDAP_COMPARE_TRUE: the entry contains the attribute value
0x06	LDAP_COMPARE_FALSE: the entry does not contain the attribute value.
Non-zero value other than 0x05 or 0x06	Failure. For a complete list, see LDAP Return Codes.
0x53	LDAP_ENCODING_ERROR
0x5A	LDAP_NO_MEMORY

Remarks

This function compares the specified value with the values in the entry's attribute and returns whether a match is found.

The ldap_compare_ext_s function can be used to compare any type of data. For string data, you can use ldap_compare_s.

Compare operations are faster than search operations. Whenever possible in your application, use a compare rather than a search operation.

NDS does not currently support any server controls to use with compare operations.

For sample code, see cpattrs.c (http://developer.novell.com/ndk/doc/samplecode/cldap_sample/index.htm).

See Also

ldap_compare_s (page 105)

ldap_compare_s

Synchronously determines whether a specified entry contains a specified attribute value.

LDAP Version: v2 or higher

Library: *ldapsdk.*

NDS Version: 7.xx or higher

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldap.h>

int ldap_compare_s (
    LDAP      *ld,
    const char *dn,
    const char *attr,
    const char *value);
```

Parameters

ld

(IN) Points to the handle for the LDAP session.

dn

(IN) Points to the distinguished name of the entry whose attribute is being compared.

attr

(IN) Points to the name of the attribute to compare.

value

(IN) Points to a string value of the attribute to compare.

Return Values

0x05	LDAP_COMPARE_TRUE: the entry contains the attribute value
0x06	LDAP_COMPARE_FALSE: the entry does not contain the attribute value.
Non-zero value other than 0x05 or 0x06	Failure. For a complete list, see LDAP Return Codes.
0x53	LDAP_ENCODING_ERROR
0x5A	LDAP_NO_MEMORY

Remarks

The `ldap_compare_s` function takes the attribute and its value and compares them to those found in the specified entry (dn).

The `ldap_compare_s` function is an older function. LDAP v3 clients should use the `ldap_compare_ext_s` function.

This function can compare only attributes with string values. Use `ldap_compare_ext_s` to compare binary values.

Compare operations are faster than search operations. Whenever possible in your application, use a compare rather than a search operation.

See Also

`ldap_compare_ext` (page 100)

ldap_control_free

Frees an LDAPControl structure.

Library: *ldapsdk.*

NDS Version: 7.xx or higher

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldap.h>

void ldap_control_free (
    LDAPControl *ctrl);
```

Parameters

ctrl

(IN) Points to the control structure to free.

Remarks

If you have created a control, you should call this function to free the structure when you are finished with the control.

See Also

ldap_controls_free (page 108)

ldap_controls_free

Frees an array of LDAPControl structures.

Library: *ldapsdk.*

NDS Version: 7.xx or higher

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
void ldap_controls_free (  
    LDAPControl  **ctrls);
```

Parameters

ctrls

(IN) Points to an array of control structures.

Remarks

You should call this function to free any arrays of controls that you create or that are returned to you by other functions such as `ldap_parse_result`.

See Also

`ldap_control_free` (page 107)

ldap_count_entries

Returns the number of LDAPMessage structures that are of the type LDAP_RES_SEARCH_ENTRY.

LDAP Version: v2 or higher

Library: *ldapsdk.*

NDS Version: 7.xx or higher

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldap.h>

int ldap_count_entries (
    LDAP      *ld,
    LDAPMessage *res);
```

Parameters

ld

(IN) Points to the handle for the LDAP session.

res

(IN) Points to the result message chain returned by the ldap_result function or a synchronous search function.

Return Values

>0	Number of entries
0	No more entries
-1	Failure

Remarks

The `ldap_count_entries` function can be used to count the number of message structures that remain in a chain. Messages are removed from the chain by calling one of the following functions:

- ♦ `ldap_first_message`
- ♦ `ldap_next_message`
- ♦ `ldap_first_entry`
- ♦ `ldap_next_entry`

This function counts from the current position of the pointer to the end of the chain.

- ♦ If you pass a pointer that points to the first message structure in the chain, it counts all the entries in the chain.
- ♦ If you pass a pointer that points to a structure in the middle of the chain, it counts the entries from that point to the end of the chain.

See Also

`ldap_first_entry` (page 150), `ldap_next_entry` (page 197), `ldap_search` (page 239), `ldap_search_ext` (page 242), `ldap_search_ext_s` (page 246), `ldap_search_s` (page 250), `ldap_search_st` (page 253)

ldap_count_messages

Returns the number of LDAPMessage structures of any type in an LDAP message chain.

LDAP Version: v3

Library: *ldapsdk.*

NDS Version: 7.xx or higher

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldap.h>

int ldap_count_messages (
    LDAP      *ld,
    LDAPMessage *res);
```

Parameters

ld

(IN) Points to the handle for the LDAP session.

res

(IN) Points to the result message chain returned by the ldap_result function or a synchronous search function.

Return Values

>0	Number of messages in the chain
0	No more messages
-1	Failure

Remarks

The `ldap_count_messages` function can be used to count the number of message structures that remain in a chain. The following functions are used to iterate through the chain:

- ♦ `ldap_first_message`
- ♦ `ldap_next_message`
- ♦ `ldap_first_reference`
- ♦ `ldap_next_reference`

This function counts from the current position of the pointer to the end of the chain.

- ♦ If you pass a pointer that points to the first message structure in the chain, it counts all the messages in the chain.
- ♦ If you pass a pointer that points to a structure in the middle of the chain, it counts the messages from that point to the end of the chain.

See Also

`ldap_first_message` (page 152), `ldap_next_message` (page 199), `ldap_search` (page 239), `ldap_search_ext` (page 242), `ldap_search_ext_s` (page 246), `ldap_search_s` (page 250), `ldap_search_st` (page 253)

ldap_count_references

Returns the number of LDAPMessage structures in an LDAP result message chain that are of type LDAP_RES_SEARCH_REFERENCE.

LDAP Version: v3

Library: *ldapsdk.*

NDS Version: 7.xx or higher

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldap.h>

int ldap_count_references (
    LDAP      *ld,
    LDAPMessage *res);
```

Parameters

ld

(IN) Points to the handle for the LDAP session.

res

(IN) Points to the result chain returned by the ldap_result function or a synchronous search function.

Return Values

>0	Number of references
0	No more references
-1	Failure

Remarks

This function counts from the current position of the pointer to the end of the chain.

- ♦ If you pass a pointer that points to the first message structure in the chain, it counts all the references in the chain.
- ♦ If you pass a pointer that points to a structure in the middle of the chain, it counts the references from that point to the end of the chain.

See Also

`ldap_first_reference` (page 154), `ldap_next_reference` (page 201), `ldap_search` (page 239), `ldap_search_ext` (page 242), `ldap_search_ext_s` (page 246), `ldap_search_s` (page 250), `ldap_search_st` (page 253)

ldap_count_values

Returns the number of strings in the array.

LDAP Version: v2 or higher

Library: *ldapsdk.*

NDS Version: 7.xx or higher

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldap.h>

int ldap_count_values (
    char    **vals);
```

Parameters

vals

(IN) Points to the array of values returned by the ldap_get_values or ldap_get_values_len function.

Return Values

>0	Number of values
-1	Failure

Remarks

The ldap_count_values function can be used for attributes that have character string values. If the array contains berval structures (binary data), use the ldap_count_values_len function.

See Also

`ldap_get_values_len` (page 168), `ldap_get_values` (page 166),
`ldap_count_values_len` (page 117), `ldap_value_free` (page 285),
`ldap_value_free_len` (page 286)

ldap_count_values_len

Returns the number of berval structures in the array.

LDAP Version: v2 or higher

Library: *ldapsdk.*

NDS Version: 7.xx or higher

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldap.h>

int ldap_count_values_len (
    struct berval    **vals);
```

Parameters

vals

(IN) Points to the array of values returned by the ldap_get_values or ldap_get_values_len function.

Return Values

>0	Number of values
-1	Failure

Remarks

The ldap_count_values_len function can be used to count the number of values for attributes that have binary data. Use ldap_count_values to count string attribute values.

The memory for the vals parameter is dynamically allocated. When you are done with the array, free the memory by calling the ldap_value_free_len function.

See Also

`ldap_get_values_len` (page 168), `ldap_get_values` (page 166),
`ldap_count_values` (page 115), `ldap_value_free` (page 285),
`ldap_value_free_len` (page 286)

ldap_create_persistentsearch_control

Creates and encodes a persistent search control.

LDAP Version: v3

Library: *ldapsdk.*

NDS Version: 8.xx or higher

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldap.h>

int ldap_create_persistentsearch_control (
    LDAP          *ld,
    int            changeTypes,
    int            changesOnly,
    int            returnEchgCtls,
    char          isCritical,
    LDAPControl    **ctrlp);
```

Parameters

ld

(IN) Points to the handle of the LDAP session.

changeTypes

(IN) an integer whose value is the bit-wise OR of the flag values corresponding to the changes types for which a the client wishes to be notified. Valid flags are as follows:

LDAP_CHANGETYPE_ADD	specifies that you want to be notified when entries are added to the directory
LDAP_CHANGETYPE_DELETE	specifies that you want to be notified when entries are deleted from the directory
LDAP_CHANGETYPE_MODIFY	specifies that you want to be notified when entries are modified.

LDAP_CHANGETYPE_MODDN	specifies that you want to be notified when entries are renamed.
LDAP_CHANGETYPE_ANY	specifies that you want to be notified when any of the above changes are made.

changesOnly

(IN) If non-zero, the initial search is only used to establish a result set on the server. No results are returned from this initial search. As changes are subsequently made to entries in the result set, the server returns the changed entries to the client. If zero, both the results of the initial search and entries that are subsequently changed are returned.

returnEntryChangeCtrl

(IN) If non-zero, an entry change notification control is included with each entry. If 0, entry change notification controls are not included with the entries returned from the server.

isCritical

(IN) Specifies whether or not the persistent search control is critical to the search operation. If non-zero, the control is critical to the search operation. If the server does not support persistent searches, the server will return the error
LDAP_UNAVAILABLE_CRITICAL_EXTENSION.

If 0, the control is not critical to the search operation. Even if the server does not support persistent searches, the search operation is still performed.]

ctrlp

(OUT) Points to a pointer for the LDAPControl structure which this function creates and which can be used in the search operation. When you are done with this control, its memory should be freed by calling the ldap_control_free function.

Return Values

0x00	LDAP_SUCCESS
------	--------------

0x53	LDAP_ENCODING_ERROR
0x59	LDAP_PARAM_ERROR
0x5A	LDAP_NO_MEMORY
Non-zero	Non-zero values indicate errors. See LDAP Return Codes.

Remarks

This API creates an LDAP persistent search control using the supplied parameters. The control can then be used in a call to `ldap_search_ext` to request that the server perform a persistent search. A persistent search allows the client to be notified when changes are made to entries that satisfy the specified search filter. When a persistent search is performed, the connection to the server remains open until the client abandons the search or unbinds from the server. The timeout parameters to the search are ignored.

For example code, see `searchPersist.c` (http://developer.novell.com/ndk/doc/samplecode/cldap_sample/index.htm).

See Also

`ldap_search_ext` (page 242), `ldap_parse_entrychange_control` (page 205)

ldap_create_sort_control

Creates and encodes a server-side sort control.

LDAP Version: v3

Library: *ldapsdk.*

NDS Version: 8.xx or higher

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldap.h>

int ldap_create_sort_control (
    LDAP          *ld,
    LDAPSORTKEY   **keyList,
    int           isCritical,
    LDAPCONTROL   **ctrlp);
```

Parameters

ld

(IN) Points to the handle of the LDAP session.

keyList

(IN) Points to a NULL-terminated array of pointers to LDAPSORTKEY structures which contain the attributes to match and the rules to use for matching.

isCritical

(IN) Specifies whether the control is required for the search operation:

- ◆ Non-zero specifies that the control is required.
- ◆ Zero specifies that the search operation can be performed without the control.

ctrlp

(OUT) Points to a pointer for the LDAPCONTROL structure which this function creates and which can be used in the search operation. When you

are done with this control, its memory should be freed by calling the `ldap_control_free` function.

Return Values

0x00	LDAP_SUCCESS
Non-zero	Failure. See LDAP Return Codes.

Remarks

The `ldap_create_sort_control` function creates a sort control that you can use as the server control parameter in the `ldap_search_ext` and the `ldap_search_ext_s` functions.

For example code, see `sortcntl.c` (http://developer.novell.com/ndk/doc/samplecode/cldap_sample/index.htm).

See Also

`ldap_parse_sort_control` (page 217), `ldap_control_free` (page 107), `ldap_controls_free` (page 108)

ldap_create_sort_keylist

Creates an a array of pointers to LDAPSortKey structures.

LDAP Version: v3

Library: *Idpsdk.*

NDS Version: 8.xx or higher

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldap.h>

int ldap_create_sort_keylist (
    LDAPSortKey    ***sortKeyList,
    char            *keyString);
```

Parameters

- sortKeyList
(OUT) Points to a NULL-terminated array of pointers to LDAPSortKey structures which contain the attributes to sort on and the rules to use for ordering.
- keyString
(IN) Points to a string representation of one or more sort keys, separated by spaces.

Return Values

0x00	LDAP_SUCCESS
Non-zero	Failure. See LDAP Return Codes.

Remarks

A key string uses the following format:

`[-]attribute[:ordering rule]`

The optional - indicates reverse sort order.

The attribute specifies an attribute in the LDAP server's schema.

The optional ordering rule is an OID (dotted string format) specifying the matching rule to use for sorting.

IMPORTANT: NDS currently supports only a single sort key, no ordering rules, and only forward sorting.

If the attribute corresponds to an existing index on the NDS server, performance is extremely good even with very large result sets. NDS 8 and NDS eDirectory have indexes for the following attributes:

sn (NDS name: Surname)

givenName (NDS name: Given Name)

cn (NDS name: CN)

uid (NDS name: uniqueID)

If you create a sort key for an attribute that does not have a defined index, one of the following happens:

- ♦ If the control is specified as critical, the function returns “No such attribute”.
- ♦ If the control is not marked critical, the control is ignored and the results are returned unsorted.

The `ldap_create_sort_keylist` function allocates memory for the `sortKeyList` array and this memory should be freed by calling the `ldap_free_sort_keylist` function.

For example code, see `sortcntl.c` (http://developer.novell.com/ndk/doc/samplecode/cldap_sample/index.htm).

See Also

`ldap_create_sort_control` (page 122), `ldap_free_sort_keylist` (page 156)

ldap_create_vlv_control

Creates and encodes a server-side virtual list view control to use with a search operation.

LDAP Version: v3

Library: *ldapsdk.*

NDS Version: 8.xx or higher

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldap.h>

int ldap_create_vlv_control (
    LDAP          *ld,
    LDAPVLVInfo   *vlvinfo,
    LDAPControl    **ctrlp);
```

Parameters

- ld
(IN) Points to the handle of the LDAP session.
- vlvinfo
(IN) Points to an LDAPVLVInfo structure that contains the information required to create a virtual list view control.
- ctrlp
(OUT) Points to the address of the LDAPControl structure that contains the virtual list view control created by this function. When this control is no longer in use, the memory should be freed by calling the ldap_control_free function.

Return Values

0x00	LDAP_SUCCESS
------	--------------

Remarks

The virtual list view control must be used with the server-side sort control. The virtual list view control has been assigned the following OID:

2.16.840.1.113730.3.4.9

For example code, see `vlvcntl.c` (http://developer.novell.com/ndk/doc/samplecode/cldap_sample/index.htm).

See Also

`ldap_parse_vlv_control` (page 220), `ldap_search_ext` (page 242), `ldap_search_ext_s` (page 246), `ldap_create_sort_control` (page 122), `ldap_control_free` (page 107), `ldap_controls_free` (page 108)

`LDAPVLVInfo` (page 416), `LDAPControl` (page 404)

ldap_delete

Asynchronously deletes the specified entry.

LDAP Version: v2 or higher

Library: *ldapsdk.*

NDS Version: 7.xx or higher

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldap.h>

int ldap_delete (
    LDAP      *ld,
    const char *dn);
```

Parameters

- ld
(IN) Points to the handle for the LDAP session.
- dn
(IN) Points to the distinguished name of the entry to delete.

Return Values

>0	Message ID of operation
-1	Failure

Remarks

The entry specified for the delete must be a leaf entry. If the entry has children, the delete will fail. LDAP does not support the deletion of a subtree in a single operation.

To obtain the results of the operation, call the `ldap_result` function with the returned message ID.

If `ldap_delete` returns -1, check the `LDAP_OPT_RESULT_CODE` option in the LDAP handle for the error code.

See Also

`ldap_delete_s` (page 134), `ldap_delete_ext` (page 130), `ldap_delete_ext_s` (page 132)

ldap_delete_ext

Asynchronously deletes the specified entry using LDAP client or server controls.

LDAP Version: v3

Library: *ldapsdk.*

NDS Version: 7.xx or higher

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldap.h>

int ldap_delete_ext (
    LDAP          *ld,
    const char    *dn,
    LDAPControl    **serverctrls,
    LDAPControl    **clientctrls,
    int            *msgidp);
```

Parameters

ld

(IN) Points to the handle for the LDAP session.

dn

(IN) Points to the distinguished name of the entry to delete.

serverctrls

(IN) Points to an array of LDAPControl structures that list the server controls to use with this delete. Use NULL to specify no server controls.

clientctrls

(IN) Points to an array of LDAPControl structures that list the client controls to use with this delete. Use NULL to specify no client controls.

msgidp

(OUT) Points to the integer value to set as the message ID of the request. When the delete request succeeds, use `ldap_result` with this value to retrieve the response.

Return Values

0x00	LDAP_SUCCESS
Non-zero	Failure. For a complete list, see LDAP Return Codes.
0x53	LDAP_ENCODING_ERROR
0x5A	LDAP_NO_MEMORY

Remarks

The entry specified for the delete must be a leaf entry. If the entry has children, the delete will fail. LDAP does not support the deletion of a subtree in a single operation.

To obtain the results of the operation, call the `ldap_result` function with the returned message ID.

NDS does not currently support any server-side controls for delete operations.

See Also

`ldap_delete` (page 128), `ldap_delete_ext_s` (page 132), `ldap_delete_s` (page 134)

ldap_delete_ext_s

Synchronously deletes the specified entry using LDAP client or server controls.

LDAP Version: v3

Library: *ldapsdk.*

NDS Version: 7.xx or higher

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldap.h>

int ldap_delete_ext_s (
    LDAP      *ld,
    const char *dn,
    LDAPControl **serverctrls,
    LDAPControl **clientctrls);
```

Parameters

ld

(IN) Points to the handle for the LDAP session.

dn

(IN) Points to the distinguished name of the entry to delete.

serverctrls

(IN) Points to an array of LDAPControl structures that list the server controls to use with this delete. Use NULL to specify no server controls.

clientctrls

(IN) Points to an array of LDAPControl structures that list the client controls to use with this delete. Use NULL to specify no client controls.

Return Values

0x00	LDAP_SUCCESS
Non-zero	Failure. For a complete list, see LDAP Return Codes.
0x53	LDAP_ENCODING_ERROR
0x5A	LDAP_NO_MEMORY

Remarks

The entry specified for the delete must be a leaf entry. If the entry has children, the delete will fail. LDAP does not support the deletion of a subtree in a single operation.

NDS does not currently support any server-side controls for delete operations.

For sample code, see `delentry.c` (http://developer.novell.com/ndk/doc/samplecode/cldap_sample/index.htm).

See Also

`ldap_delete` (page 128), `ldap_delete_s` (page 134), `ldap_delete_ext` (page 130)

ldap_delete_s

Synchronously deletes the specified entry.

LDAP Version: v2 or higher

Library: *ldapsdk.*

NDS Version: 7.xx or higher

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldap.h>

int ldap_delete_s (
    LDAP      *ld,
    const char *dn);
```

Parameters

- ld
(IN) Points to the handle for the LDAP session.
- dn
(IN) Points to the distinguished name of the entry to delete.

Return Values

0x00	LDAP_SUCCESS
Non-zero	Failure. For a complete list, see LDAP Return Codes.
0x53	LDAP_ENCODING_ERROR
0x5A	LDAP_NO_MEMORY

Remarks

The entry specified for the delete must be a leaf entry. If the entry has children, the delete will fail. LDAP does not support the deletion of a subtree in a single operation.

See Also

`ldap_delete` (page 128), `ldap_delete_ext_s` (page 132), `ldap_delete_ext` (page 130)

ldap_dn2ufn

Converts a distinguished name into the user friendly format.

LDAP Version: v2 or higher

Library: *ldapsdk.*

NDS Version: 7.xx or higher

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldap.h>

char *ldap_dn2ufn (
    const char    *dn);
```

Parameters

dn
(IN) Points to the distinguished name to be converted.

Return Values

>0	Pointer to the converted name
NULL	Failure

Remarks

The user friendly format is defined in RFC 1781. The format strips off the types and places a comma between the components of the name. Components which have commas in their names are placed in quotation marks.

The memory for the user friendly format is newly allocated and should be freed with a call to the ldap_memfree function.

See Also

`ldap_get_dn` (page 158), `ldap_explode_dn` (page 140), `ldap_explode_rdn` (page 142)

ldap_err2string

Converts a numeric LDAP error code into a character string.

LDAP Version: v2 or higher

Library: *ldapsdk.*

NDS Version: 7.xx or higher

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldap.h>

char *ldap_err2string (
    int    err);
```

Parameters

err

(IN) Specifies an LDAP error code returned by an LDAP function.

Return Values

>0	Pointer to a zero-terminated character string.
----	--

Remarks

The ldap_err2string function converts LDAP error codes returned by the following functions:

- ♦ ldap_parse_result
- ♦ ldap_parse_sasl_bind_result
- ♦ ldap_parse_extended_result
- ♦ synchronous LDAP operation functions

The LDAP error code is converted to a zero-terminated character string which describes the error.

The return value points to a string contained in static data. Be aware of the following:

- ♦ It should be used or copied before another call to `ldap_err2string` is made.
- ♦ The pointer should not be used to modify the original string.
- ♦ The string should not be freed by the application program.
- ♦ The string is returned in the local code page. It is not UTF-8 encoded.

The returned pointer can be used directly in a `printf` statement, for example:

```
err=ldap_search(...);  
if (err)  
    printf("Search error: %s\n", ldap_err2string(err));
```

See Also

`ldap_parse_result` (page 212), `ldap_parse_extended_result` (page 208),
`ldap_parse_sasl_bind_result` (page 215)

ldap_explode_dn

Breaks a distinguished name into its components.

LDAP Version: v2 or higher

Library: *ldapsdk.*

NDS Version: 7.xx or higher

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldap.h>

char **ldap_explode_dn (
    const char    *dn,
    int           notypes);
```

Parameters

- dn
- (IN) Points to the distinguished name to explode.
- notypes
- (IN) Specifies whether the name should include type information:
- ♦ If zero, type information is included, for example "cn=Kim".
 - ♦ If non-zero, type information is stripped, for example "cn=Kim" becomes "Kim".

Return Values

>0	Pointer to a character array of components
NULL	Failure

Remarks

The `ldap_explode_dn` function takes a dn returned by `ldap_get_dn` and returns a NULL-terminated character array of the components in the name. The components are returned in the order they appear in the dn and are with or without types as indicated by the `otypes` parameter.

When the array is no longer in use, free the memory by calling the `ldap_value_free` function.

See Also

`ldap_get_dn` (page 158), `ldap_explode_rdn` (page 142), `ldap_dn2ufn` (page 136)

ldap_explode_rdn

Breaks a relative distinguished name into its components.

Library: *ldapsdk.*

NDS Version: 7.xx or higher

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldap.h>

char **ldap_explode_rdn (
    const char    *rdn,
    int           notypes);
```

Parameters

rdn

(IN) Points to the relative distinguished name of the entry.

notypes

(IN) Specifies whether the name should include type information:

- ♦ If zero, type information is included, for example "cn=Kim".
- ♦ If non-zero, type information is stripped, for example "cn=Kim" becomes "Kim".

Return Values

>0	Pointer to a character array of components
NULL	Failure

Remarks

The `ldap_explode_rdn` returns a NULL-terminated character array with or without types as indicated by the `notypes` parameter. The components are returned in the order they appear in the `rdn`.

When the array is no longer in use, free the memory by calling the `ldap_value_free` function.

See Also

`ldap_get_dn` (page 158), `ldap_explode_dn` (page 140), `ldap_dn2ufn` (page 136)

ldap_extended_operation

Asynchronously passes extended LDAP operations to the LDAP server.

LDAP Version: v3

Library: *ldapsdk.*

NDS Version: 7.xx or higher

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldap.h>

int ldap_extended_operation (
    LDAP          *ld,
    const char     *requestoid,
    const struct berval *requestdata,
    LDAPControl    **serverctrls,
    LDAPControl    **clientctrls,
    int            *msgidp);
```

Parameters

ld

(IN) Points to the handle for the LDAP session.

requestoid

(IN) Points to the dotted-OID text string identifying the extended operation to perform.

requestdata

(IN) Points to the data required for the operation. If NULL, no data is sent to the server.

serverctrls

(IN) Points to an array of LDAPControl structures that list the server controls to use with this extended operation. Use NULL to specify no server controls.

clientctrls

(IN) Points to an array of LDAPControl structures that list the client controls to use with this extended operation. Use NULL to specify no client controls.

msgidp

(OUT) Points to the integer value to set as the message ID of the request. When the extended operation succeeds, the results are identified by this value.

Return Values

0	Success
Non-zero	Failure

Remarks

The data returned in the msgidp parameter is opaque to the caller. You must use the ldap_result and ldap_parse_extended_result functions to obtain the result, the OID, and the data.

The LDAP server must support the operation; otherwise an LDAP_NOT_SUPPORTED error is returned.

See Also

ldap_extended_operation_s (page 146)

ldap_extended_operation_s

Synchronously passes extended LDAP operations to the LDAP server.

LDAP Version: v3

Library: *ldapsdk.*

NDS Version: 7.xx or higher

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldap.h>

int ldap_extended_operation_s (
    LDAP          *ld,
    const char     *requestoid,
    const struct berval *requestdata,
    LDAPControl    **serverctrls,
    LDAPControl    **clientctrls,
    char           **retoidp,
    struct berval   **retdatap);
```

Parameters

ld

(IN) Points to the handle for the LDAP session.

requestoid

(IN) Points to the dotted-OID text string identifying the operation to perform.

requestdata

(IN) Points to the data required for the operation. If NULL, no data is sent to the server.

serverctrls

(IN) Points to an array of LDAPControl structures that list the server controls to use with this extended operation. Use NULL to specify no server controls.

clientctrls

(IN) Points to an array of LDAPControl structures that list the client controls to use with this extended operation. Use NULL to specify no client controls.

retoidp

(OUT) Points to a dotted-OID text string returned by the LDAP server. A NULL value means an OID is not returned. The memory used by the string should be freed with the ldap_memfree function.

retdatap

(OUT) Points to a pointer to a berval structure that contains the returned data. If no data is returned, the server set this to NULL. The memory used by this structure should be freed with the ber_bvfree function.

Return Values

0x00	LDAP_SUCCESS
Non-zero	Failure. For a complete list, see LDAP Return Codes.
0x53	LDAP_ENCODING_ERROR
0x59	LDAP_PARAM_ERROR
0x5A	LDAP_NO_MEMORY
0x5C	LDAP_NOT_SUPPORTED

See Also

ldap_extended_operation (page 144)

ldap_first_attribute

Returns the name of the first attribute in an entry.

LDAP Version: v2 or higher

Library: *ldapsdk.*

NDS Version: 7.xx or higher

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldap.h>

char *ldap_first_attribute (
    LDAP          *ld,
    LDAPMessage    *entry,
    BerElement     **ptr);
```

Parameters

- ld
(IN) Points to the handle for the LDAP session.
- entry
(IN) Points to the entry whose attributes are being read.
- ptr
(OUT) Returns a pointer to a BerElement allocated by the library. It is used internally to track the current position in the entry. This returned value is passed in subsequent calls to the ldap_next_attribute function. It should be freed by the application with a call to the ber_free (ptr, 0) function.

Return Values

NULL	No more attributes or failure
>0	Pointer to the name of the first attribute in an entry

Remarks

The `ldap_first_attribute` function returns a pointer to the first attribute of an entry returned by either the `ldap_first_entry` or the `ldap_next_entry` function.

If NULL is returned and the `ptr` parameter is not NULL, check the `LDAP_OPT_RESULT_CODE` option in the LDAP handle for the error code.

If NULL is returned and the `ptr` parameter is not NULL, all attributes have been retrieved.

The pointer to the name of the first attribute should be passed to the `ldap_get_values` function (or others of its type) to retrieve the attribute's values. When you are done with the name pointer, you must free it by calling the `ldap_memfree` function.

The `ptr` parameter should be used in subsequent calls to the `ldap_next_attribute` function to retrieve other attributes of the entry. When you are done with the `BerElement` structure and its value is non-NULL, you must free it by calling the `ber_free` function with the second parameter set to 0. If the `ptr` parameter is set to NULL, then the `ldap_first_attribute` function frees the memory.

See Also

`ldap_next_attribute` (page 195), `ldap_get_values` (page 166)

ldap_first_entry

Returns a pointer to the first entry of message type, LDAP_RES_SEARCH_ENTRY, from a search result chain.

LDAP Version: v2 or higher

Library: *ldapsdk.*

NDS Version: 7.xx or higher

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldap.h>

LDAPMessage *ldap_first_entry (
    LDAP      *ld,
    LDAPMessage *res);
```

Parameters

- ld
(IN) Points to the handle for the LDAP session.
- res
(IN) Points to the result chain returned by the ldap_result function or a synchronous search function.

Return Values

NULL	No more entries in the chain or failure
>0	Pointer to the next entry in the chain

Remarks

The ldap_first_entry function parses the results received from the ldap_result, the ldap_search_s, the ldap_search_ext_s, or the ldap_search_st functions.

If the `ldap_first_entry` function encounters an error, the function returns `NULL` and sets the `LDAP_OPT_RESULT_CODE` option in the LDAP session handle.

Use the `ldap_get_dn`, `ldap_first_attribute`, `ldap_get_values` functions to retrieve information about the entry.

Use the value returned by the `ldap_first_entry` function as the entry parameter for the `ldap_next_entry` function to retrieve the next entry.

See Also

`ldap_next_entry` (page 197), `ldap_count_entries` (page 109), `ldap_search` (page 239), `ldap_search_ext` (page 242), `ldap_search_ext_s` (page 246), `ldap_search_s` (page 250), `ldap_search_st` (page 253)

ldap_first_message

Returns a pointer to the first message type, LDAP_RES_SEARCH_ENTRY, LDAP_RES_SEARCH_RESULT, or LDAP_RES_SEARCH_REFERENCE in a result chain.

LDAP Version: v3

Library: *ldapsdk.*

NDS Version: 7.xx or higher

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldap.h>

LDAPMessage *ldap_first_message (
    LDAP      *ld,
    LDAPMessage *res);
```

Parameters

- ld
(IN) Points to the handle for the LDAP session.
- res
(IN) Points to the result chain returned by the ldap_result function or a synchronous search function.

Return Values

NULL	No more messages or failure
>0	Pointer to a message

Remarks

If `ldap_first_message` encounters an error, the function returns `NULL` and sets the `LDAP_OPT_RESULT_CODE` option in the LDAP session handle.

Use the `ldap_count_messages` function to determine the number of messages in the chain. Use the `ldap_next_message` function to retrieve subsequent messages.

See Also

`ldap_next_message` (page 199), `ldap_count_messages` (page 111), `ldap_search` (page 239), `ldap_search_ext` (page 242), `ldap_search_ext_s` (page 246), `ldap_search_s` (page 250), `ldap_search_st` (page 253)

ldap_first_reference

Returns a pointer to the first reference of message type, LDAP_RES_SEARCH_REFERENCE, in a search result chain.

LDAP Version: v3

Library: *ldapsdk.*

NDS Version: 7.xx or higher

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldap.h>

LDAPMessage *ldap_first_reference (
    LDAP      *ld,
    LDAPMessage *res);
```

Parameters

- ld
(IN) Points to the handle for the LDAP session.
- res
(IN) Points to the result chain returned by the ldap_result function or a synchronous search function.

Return Values

NULL	No more references in the chain or failure
>0	Pointer to the next reference in the chain.

Remarks

If the `ldap_first_reference` function encounters an error, the function returns `NULL` and sets the `LDAP_OPT_RESULT_CODE` option in the LDAP session handle.

See Also

`ldap_next_reference` (page 201), `ldap_count_references` (page 113), `ldap_parse_reference` (page 210), `ldap_search` (page 239), `ldap_search_ext` (page 242), `ldap_search_ext_s` (page 246), `ldap_search_s` (page 250), `ldap_search_st` (page 253)

ldap_free_sort_keylist

Frees the memory allocated by the ldap_create_sort_keylist function.

LDAP Version: v3

Library: *Idapsdk.*

NDS Version: 8.xx or higher

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldap.h>

void ldap_free_sort_keylist (
    LDAPSortKey    **sortKeyList);
```

Parameters

sortKeyList

(IN) Points to an array of pointers to LDAPSortKey structures that you want to free.

Remarks

The ldap_free_sort_keylist function frees the memory used by the LDAPSortKey structures, the memory referenced by the structures, and the array of pointers to the structures. The ldap_free_sort_keylist function should be called only if the memory was allocated by the ldap_create_sort_keylist function.

See Also

ldap_create_sort_keylist (page 124)

ldap_free_urldesc

Frees the memory allocated by the ldap_url_parse function.

LDAP Version: v3

Library: *Idapsdk.*

NDS Version: 7.xx or higher

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldap.h>

void ldap_free_urldesc (
    const char      *ludp);
```

Parameters

ludp

(IN) Points to the LDAPURLDesc structure that you want to free.

See Also

ldap_url_parse (page 276)

ldap_get_dn

Returns the distinguished name of an entry from a search result chain.

LDAP Version: v2 or higher

Library: *ldapsdk.*

NDS Version: 7.xx or higher

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldap.h>

char *ldap_get_dn (
    LDAP      *ld,
    LDAPMessage *entry);
```

Parameters

- ld
(IN) Points to the handle for the LDAP session.
- entry
(IN) Points to the entry returned by the ldap_first_entry or the ldap_next_entry function.

Return Values

>0	Pointer to the distinguished name of the entry
NULL	Failure to parse the name

Remarks

The ldap_get_dn function takes an entry returned by either the ldap_first_entry or ldap_next_entry function and returns a copy of the entry's

dn. It returns a pointer to this newly allocated memory. When you are finished with the name, free the memory with a call to the `ldap_memfree` function.

The distinguished name is returned in the UTF-8 string format as described in RFC 2253.

See Also

`ldap_explode_dn` (page 140), `ldap_explode_rdn` (page 142), `ldap_dn2ufn` (page 136)

ldap_get_entry_controls

Retrieves LDAP controls from an entry.

LDAP Version: v3

Library: *ldapsdk.*

NDS Version: 7.xx or higher

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldap.h>

int ldap_get_entry_controls (
    LDAP          *ld,
    LDAPMessage    *entry,
    LDAPControl    ***serverctrlsp);
```

Parameters

- ld
(IN) Points to the handle for the LDAP session.
- entry
(IN) Points to the entry, returned by the ldap_first_entry or the ldap_next_entry function, from which to extract controls.
- serverctrlsp
(OUT) Points to an array of LDAPControl structures copied out of the entry. If this parameter is set to NULL, no controls are returned.

Return Values

0x00	LDAP_SUCCESS
Non-zero	Failure. For a complete list, see LDAP Return Codes.
0x54	LDAP_DECODING_ERROR

0x59	LDAP_PARAM_ERROR
0x5A	LDAP_NO_MEMORY

Remarks

Each LDAPControl structure represents one LDAP v3 server control. When the array of LDAPControl structures is no longer in use, free the memory by calling the `ldap_controls_free` function.

See Also

`ldap_control_free` (page 107), `ldap_controls_free` (page 108)

ldap_get_lderrno

Returns error information about the last LDAP operation. This function has been deprecated; use the `ldap_get_option` function.

LDAP Version: v2 or higher

Library: *ldapsdk.*

NDS Version: 7.xx or higher

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldap.h>

int ldap_get_lderrno (
    LDAP      *ld,
    char      **matchedDN,
    char      **errmsg);
```

Parameters

`ld`

(IN) Points to the handle for the LDAP session.

`matchedDN`

(OUT) Points to the name of the lowest entry in the directory that was matched on the operation.

`errmsg`

(OUT) Points to a text string that is optionally returned by the server. This string includes additional details about the error and is not the standard string associated with the error code. Applications should not depend on format of this parameter or assume that it contains data.

Return Values

Returns the LDAP error code from the last operation. Use the `ldap_err2string` function to get the text string associated with this error code.

Remarks

The pointers returned in the function point directly into the LDAP structure.

NOTE: The application should not free these pointers. The pointers must not be used after another LDAP operation has been called. The pointers should not be used to modify the data.

The application should examine or copy the strings before calling another LDAP function.

The pointers are set after every LDAP operation which returns or parses an LDAP result message.

If information is not needed for either the matchedDN or the errmsg parameter, the parameter can be set to NULL.

This is not a standard IETF function. It has been added for compatibility with other LDAP vendors' libraries and should not be used in new applications. Use the `ldap_get_option` function with `LDAP_OPT_ERR_NUMBER`, `LDAP_OPT_MATCHED_DN`, and `LDAP_OPT_ERROR_STRING`.

See Also

`ldap_set_lderrno` (page 256), `ldap_get_option` (page 164)

ldap_get_option

Returns information about session preferences.

LDAP Version: v3

Library: *Idapsdk.*

NDS Version: 7.xx or higher

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldap.h>

int    ldap_get_option (
    LDAP    *ld,
    int      option,
    void     *outvalue);
```

Parameters

- ld
(IN) Contains the session handle. If this is NULL, the function returns information about global defaults.
- option
(IN) Contains the name of the option for which information is returned (see “Session Preference Options” on page 392).
- outvalue
(OUT) Returns a pointer to a buffer that contains the information about the specified *option*.

Return Values

0x00	LDAP_SUCCESS
-1	Failure

Remarks

The type of buffer pointed to by the outvalue parameter depends on the option requested. For details, see “Session Preference Options” on page 392.

See Also

`ldap_set_option` (page 258)

ldap_get_values

Returns string values of a specified attribute from an entry.

LDAP Version: v2 or higher

Library: *Idapsdk.*

NDS Version: 7.xx or higher

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldap.h>

char **ldap_get_values (
    LDAP      *ld,
    LDAPMessage *entry,
    const char *attr);
```

Parameters

- ld
(IN) Points to the handle for the LDAP session.
- entry
(IN) Points to the entry returned by the ldap_first_entry or the ldap_next_entry function.
- attr
(IN) Points to the attribute returned by the ldap_first_attribute function, the ldap_next_attribute function, or the name of an attribute in string format.

Return Values

>0	An array of attribute values
NULL	Failure or no values were found for the attribute

Remarks

The `ldap_get_values` function takes an entry and attribute and returns a NULL-terminated array of attribute string values. The memory for the array is dynamically allocated. When you are done with the array, free the memory by calling the `ldap_value_free` function.

The `ldap_get_values` function can be used to return only character string values. For binary data, use the `ldap_get_values_len` function.

See Also

`ldap_get_values_len` (page 168), `ldap_count_values` (page 115), `ldap_count_values_len` (page 117), `ldap_value_free` (page 285), `ldap_value_free_len` (page 286)

ldap_get_values_len

Returns binary values of a specified attribute from an entry.

LDAP Version: v2 or higher

Library: *Idapsdk.*

NDS Version: 7.xx or higher

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldap.h>

struct berval **ldap_get_values_len (
    LDAP          *ld,
    LDAPMessage    *entry,
    const char     *attr);
```

Parameters

- ld
(IN) Points to the handle for the LDAP session.
- entry
(IN) Points to the entry returned by the ldap_first_entry or the ldap_next_entry function.
- attr
(IN) Points to the attribute returned by the ldap_first_attribute function, the ldap_next_attribute function, or the name of an attribute in string format.

Return Values

>0	An array of values
Null	Failure or no values were found for the attribute

Remarks

The `ldap_get_values_len` function takes an entry and attribute and returns the attribute values in a NULL-terminated array of pointers to `LDAPValue` structures. The memory for the array is dynamically allocated. When you are done with the array, free the memory by calling the `ldap_value_free_len` function.

See Also

`ldap_get_values` (page 166), `ldap_count_values` (page 115),
`ldap_count_values_len` (page 117), `ldap_value_free` (page 285),
`ldap_value_free_len` (page 286)

ldap_init

Initializes an LDAP session associated with an LDAP server and returns a pointer to a session handle.

Library: *ldapsdk.*

NDS Version: 7.xx or higher

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldap.h>

LDAP *ldap_init (
    const char    *host,
    int           port);
```

Parameters

- host
- (IN) Contains the names of the available hosts, each separated by a space, or a list of IP addresses (in dot format) of the hosts, each separated by a space. If a port number is included with the name or the address, it is separated from them with a colon (:).
- port
- (IN) Contains the TCP port number to connect to. If a port number is included with the *host* parameter, this parameter is ignored.

Return Values

>0	Success; session handle
NULL	Unsuccessful

Remarks

If you connect to an LDAP v2 server, you must call an LDAP bind operation before performing any operations. If you connect to an LDAP v3 server, some operations can be performed before calling a bind operation.

The `ldap_init` function does not actually communicate with the LDAP server. Communication begins when the application binds or does some other operation.

The LDAP libraries first contact the first server listed in the host parameter. If they are unable to communicate with that server, they try the next server and then the next.

The session handle returned contains opaque data identifying the session. To get or set handle information, use `ldap_set_option` and `ldap_get_option`. For a list of the handle options, see “Session Preference Options” on page 392.

IMPORTANT: The `ldap_init` function allocates memory for the LDAP structure. This memory must be freed by calling `ldap_unbind` or `ldap_unbind_s` even when an LDAP bind function is not called or the LDAP bind function fails.

See Also

`ldap_get_option` (page 164), `ldap_set_option` (page 258), `ldap_open` (page 203)

ldap_is_ldap_url

Determines whether the URL is an LDAP URL.

LDAP Version: v3

Library: *ldapsdk.*

NDS Version: 7.xx or higher

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldap.h>

int ldap_is_ldap_url (
    const char    *url);
```

Parameters

url
(IN) Points to the URL that you want to check.

Return Values

1	URL is an LDAP URL
0	URL is not an LDAP URL

Remarks

An LDAP URL has the protocol set to ldap:// for simple authentication.

See Also

ldap_url_parse (page 276), ldap_is_ldaps_url (page 173)

ldap_is_ldaps_url

Determines whether the URL is an LDAPS URL.

LDAP Version: v3

Library: *ldap sdk.*

NDS Version: 7.xx or higher

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldap.h>

int ldap_is_ldaps_url (
    const char    *url);
```

Parameters

url

(IN) Points to the URL that you want to check.

Return Values

1	URL is an LDAPS URL
0	URL is not an LDAPS URL

Remarks

An LDAPS URL has the protocol set to ldaps:// for an encrypted SSL connection.

See Also

ldap_url_parse (page 276), ldap_is_ldap_url (page 172)

ldap_memfree

Frees memory allocated by a call to the LDAP libraries.

LDAP Version: v3

Library: *ldapsdk.*

NDS Version: 7.xx or higher

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldap.h>

void ldap_memfree (
    char    *mem);
```

Parameters

mem

(IN) Points to the memory to free. If this argument is NULL, the function does nothing.

Remarks

The ldap_memfree function is used to free memory the LDAP libraries allocated for names on calls to the ldap_first_attribute, ldap_next_attribute, and ldap_get_dn functions.

See Also

ldap_first_attribute (page 148), ldap_next_attribute (page 195), ldap_get_dn

ldap_modify

Asynchronously modifies the specified entry on the LDAP server.

LDAP Version: v2 or higher

Library: *ldapsdk.*

NDS Version: 7.xx or higher

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldap.h>

int ldap_modify (
    LDAP      *ld,
    const char *dn,
    LDAPMod   **mods);
```

Parameters

ld

(IN) Points to the handle for the LDAP session.

dn

(IN) Points to the distinguished name of the entry to modify.

mods

(IN) Points to a NULL-terminated array of pointers to the modifications to make to the entry. Each LDAPMod structure contains the modifications for one attribute.

Return Values

>0	Message ID of operation
-1	Failure

Remarks

To obtain the results of the operation, call the `ldap_result` function with the returned message ID.

If the `ldap_modify` function returns -1, check the `LDAP_OPT_RESULT_CODE` option in the LDAP handle for the error code.

To free the memory used by the `LDAPMod` structures, call the `ldap_mods_free` function.

Use the `ldap_rename` or `ldap_rename_s` function to modify the entry's name.

See Also

`ldap_modify_s` (page 181), `ldap_modify_ext` (page 177), `ldap_modify_ext_s` (page 179), `ldap_rename` (page 224), `ldap_rename_s` (page 227)

ldap_modify_ext

Asynchronously modifies specified attributes of an entry on an LDAP server, using LDAP client or server controls.

LDAP Version: v3

Library: *ldapsdk.*

NDS Version: 7.xx or higher

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldap.h>

int ldap_modify_ext (
    LDAP          *ld,
    const char    *dn,
    LDAPMod       **mods,
    LDAPControl    **serverctrls,
    LDAPControl    **clientctrls,
    int           *msgidp);
```

Parameters

ld

(IN) Points to the handle for the LDAP session.

dn

(IN) Points to the distinguished name of the entry to modify.

mods

(IN) Points to a NULL-terminated array of pointers to the modifications to make to the entry. Each LDAPMod structure contains the modifications for one attribute.

serverctrls

(IN) Points to an array of LDAPControl structures that list the server controls to use with the modify operation. Use NULL to specify no server controls.

clientctrls

(IN) Points to an array of LDAPControl structures that list the client controls to use with the modify operation. Use NULL to specify no client controls.

msgidp

(OUT) Points to the message ID of the request when the search request succeeds.

Return Values

0x00	LDAP_SUCCESS
Non-zero	Failure. For a complete list, see LDAP Return Codes.
0x53	LDAP_ENCODING_ERROR
0x5A	LDAP_NO_MEMORY

Remarks

If successful, the message ID of the operation is placed in the msgidp parameter. To obtain the results of the operation, call the ldap_result function using the message ID returned in the msgidp parameter.

Use the ldap_rename or ldap_rename_s function to modify the entry's name.

NDS does not currently support any server-side controls to use with modify operations.

See Also

ldap_modify (page 175), ldap_modify_s (page 181), ldap_modify_ext_s (page 179)

ldap_modify_ext_s

Synchronously modifies specified attributes of an entry on an LDAP server, using LDAP client or server controls.

LDAP Version: v3

Library: *ldapsdk.*

NDS Version: 7.xx or higher

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldap.h>

int ldap_modify_ext_s (
    LDAP          *ld,
    const char    *dn,
    LDAPMod       **mods,
    LDAPControl    **serverctrls,
    LDAPControl    **clientctrls);
```

Parameters

ld

(IN) Points to the handle for the LDAP session.

dn

(IN) Points to the distinguished name of the entry to modify.

mods

(IN) Points to a NULL-terminated array of pointers to the modifications to make to the entry. Each LDAPMod structure contains the modifications for one attribute.

serverctrls

(IN) Points to an array of LDAPControl structures that list the server controls to use with the modify operation. Use NULL to specify no server controls.

clientctrls

(IN) Points to an array of LDAPControl structures that list the client controls to use with the modify operation. Use NULL to specify no client controls.

Return Values

0x00	LDAP_SUCCESS
Non-zero	Failure. For a complete list, see LDAP Return Codes.
0x53	LDAP_ENCODING_ERROR
0x5A	LDAP_NO_MEMORY

Remarks

Use the ldap_rename or ldap_rename_s function to modify the entry's name.

NDS does not currently support any server-side controls to use with modify operations.

For sample code, see modattrs.c (http://developer.novell.com/ndk/doc/samplecode/cldap_sample/index.htm).

See Also

ldap_modify (page 175), ldap_modify_s (page 181), ldap_modify_ext (page 177)

ldap_modify_s

Synchronously modifies the specified entry on an LDAP server.

LDAP Version: v2 or higher

Library: *ldapsdk.*

NDS Version: 7.xx or higher

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldap.h>

int ldap_modify_s (
    LDAP      *ld,
    const char *dn,
    LDAPMod   **mods);
```

Parameters

ld

(IN) Points to the handle for the LDAP session.

dn

(IN) Points to the distinguished name of the entry to modify.

mods

(IN) Points to a NULL-terminated array of pointers to the modifications to make to the entry. Each LDAPMod structure contains the modifications for one attribute.

Return Values

0x00	LDAP_SUCCESS
Non-zero	Failure. For a complete list, see LDAP Return Codes.
0x53	LDAP_ENCODING_ERROR

Remarks

Use the `ldap_rename` or `ldap_rename_s` function to modify the entry's name.

See Also

`ldap_modify` (page 175), `ldap_modify_ext` (page 177), `ldap_modify_ext_s` (page 179)

ldap_modrdn

Asynchronously modifies the relative distinguished name of a specified entry. This function has been deprecated; use the `ldap_rename` function.

LDAP Version: v2 or higher

Library: *ldapsdk.*

NDS Version: 7.xx or higher

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldap.h>

int ldap_modrdn (
    LDAP      *ld,
    const char *dn,
    const char *newrdn);
```

Parameters

`ld`

(IN) Points to the handle for the LDAP session.

`dn`

(IN) Points to the distinguished name of the entry to modify.

`newrdn`

(IN) Points to the new relative distinguished name for the entry. The entry's parent must remain the same.

Return Values

>0	Message ID of operation
-1	Failure

Remarks

The `ldap_modrdn` function replaces the old `rdn` with the value of the new `rdn`.

The `ldap_modrdn` function has been replaced by the `ldap_rename` function. Unless you need this older function for backwards compatibility, use the newer `ldap_rename` function.

See Also

`ldap_rename` (page 224), `ldap_rename_s` (page 227)

ldap_modrdn_s

Synchronously modifies the relative distinguished name of a specified entry. This function has been deprecated; use the `ldap_rename_s` function.

LDAP Version: v2 or higher

Library: *ldapsdk.*

NDS Version: 7.xx or higher

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldap.h>

int ldap_modrdn_s (
    LDAP      *ld,
    const char *dn,
    const char *newrdn);
```

Parameters

`ld`

(IN) Points to the handle for the LDAP session.

`dn`

(IN) Points to the distinguished name of the entry to modify.

`newrdn`

(IN) Points to the new relative distinguished name for the entry. The entry's parent must remain the same.

Return Values

0x00	LDAP_SUCCESS
Non-zero	Failure. For a complete list, see LDAP Return Codes.

Remarks

The `ldap_modrdn_s` function replaces the old `rdn` with the value of the new `rdn`.

The `ldap_modrdn_s` function has been replaced by the `ldap_rename` function. Unless you need this older function for backwards compatibility, use the newer `ldap_rename_s` function.

See Also

`ldap_rename` (page 224), `ldap_rename_s` (page 227)

ldap_modrdn2

Asynchronously modifies the relative distinguished name of the specified entry. This function has been deprecated; use the `ldap_rename` function.

Library: *ldapsdk.*

NDS Version: 7.xx or higher

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldap.h>

int ldap_modrdn2 (
    LDAP          *ld,
    const char    *dn,
    const char    *newrdn,
    int           deleteoldrdn);
```

Parameters

`ld`

(IN) Points to the handle for the LDAP session.

`dn`

(IN) Points to the distinguished name of the entry to modify.

`newrdn`

(IN) Points to the new relative distinguished name for the entry. The entry's parent must remain the same.

`deleteoldrdn`

(IN) Specifies whether the old RDN should be retained or deleted.

- ◆ Zero indicates that the old RDN should be retained. If you choose this option, the attribute will contain both names (the old and the new).
- ◆ Non-zero indicates that the old RDN should be deleted.

Return Values

>0	Message ID of operation
-1	Failure

Remarks

The `ldap_modrdn2` function has been replaced by the `ldap_rename` function. Unless you need this older function for backwards compatibility, use the newer `ldap_rename` function.

See Also

`ldap_rename` (page 224), `ldap_rename_s` (page 227)

ldap_modrdn2_s

Synchronously modifies the relative distinguished name of the specified entry. This function has been deprecated; use the `ldap_rename_s` function.

Library: *ldapsdk.*

NDS Version: 7.xx or higher

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldap.h>

int ldap_modrdn2_s (
    LDAP          *ld,
    const char     *dn,
    const char     *newrdn,
    int            deleteoldrdn);
```

Parameters

`ld`

(IN) Points to the handle for the LDAP session.

`dn`

(IN) Points to the distinguished name of the entry to modify.

`newrdn`

(IN) Points to the new relative distinguished name for the entry. The entry's parent must remain the same.

`deleteoldrdn`

(IN) Specifies whether the old RDN should be retained or deleted.

- ◆ Zero indicates that the old RDN should be retained. If you choose this option, the attribute will contain both names (the old and the new).
- ◆ Non-zero indicates that the old RDN should be deleted.

Return Values

0x00	LDAP_SUCCESS
Non-zero	Failure. For a complete list, see LDAP Return Codes.

Remarks

The `ldap_modrdn2_s` function has been replaced by the `ldap_rename` function. Unless you need this older function for backwards compatibility, use the newer `ldap_rename_s` function.

See Also

`ldap_rename` (page 224), `ldap_rename_s` (page 227)

ldap_msgfree

Frees each message in the result chain pointed to by the res parameter and returns the type of the last message in the chain.

LDAP Version: v2 or higher

Library: *ldapsdk.*

NDS Version: 7.xx or higher

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldap.h>

int ldap_msgfree (
    LDAPMessage *res);
```

Parameters

res

(IN) Points to the message chain to free. If res is set to NULL, nothing is done.

Return Values

>0x60	Success
0x00	Nothing was done.

Remarks

The ldap_msgfree function is used to free the memory allocated by the ldap_result, ldap_search_s, ldap_search_st, and ldap_search_ext_s functions and returns the type of the last message in the chain.

For a list of possible message types returned by this function, see “Result Message Types” on page 390.

See Also

`ldap_msgid` (page 193), `ldap_msgtype` (page 194)

ldap_msgid

Returns the message ID associated with the res parameter.

Library: *ldapsdk.*

NDS Version: 7.xx or higher

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldap.h>

int ldap_msgid (
    LDAPMessage *res);
```

Parameters

res

(IN) Points to a message chain returned by the ldap_result, ldap_search_s, ldap_search_st, or ldap_search_ext_s function.

Return Values

>0	The message ID
-1	Failure

See Also

ldap_msgfree (page 191), ldap_msgtype (page 194)

ldap_msgtype

Returns the type of message associated with the res parameter.

Library: *ldapsdk.*

NDS Version: 7.xx or higher

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldap.h>

int ldap_msgtype (
    LDAPMessage *res);
```

Parameters

res
(IN) Points to a message chain returned by the ldap_result, ldap_search_s, ldap_search_st, or ldap_search_ext_s function.

Return Values

>0x60	Type of message
-1	Failure

Remarks

For a list of possible types, see “Result Message Types” on page 390.

See Also

ldap_msgfree (page 191), ldap_msgid (page 193)

ldap_next_attribute

Returns the name of the next attribute in an entry.

LDAP Version: v2 or higher

Library: *Idapsdk.*

NDS Version: 7.xx or higher

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldap.h>

char *ldap_next_attribute (
    LDAP      *ld,
    LDAPMessage *entry,
    BerElement *ptr);
```

Parameters

ld

(IN) Points to the handle for the LDAP session.

entry

(IN) Points to the entry whose attributes are being read.

ptr

(IN/OUT) Points to a value used internally to track the current position in the entry. For the first call, use the value returned by the ldap_first_attribute function. In subsequent calls, use the value returned by the previous ldap_next_attribute call. When the application is done with the ptr, it should free the BerElement by calling the ber_free (ptr, 0) function.

Return Values

NULL	No more attributes or failure
------	-------------------------------

Remarks

The `ldap_next_attribute` function returns a pointer to the next attribute of an entry returned by either the `ldap_first_entry` or the `ldap_next_entry` function.

If `NULL` is returned and the `ptr` parameter is not `NULL`, check the `LDAP_OPT_RESULT_CODE` option in the LDAP handle for the error code.

If `NULL` is returned and the `ptr` parameter is `NULL`, all attributes have been retrieved.

The pointer to the name of the attribute should be passed to the `ldap_get_values` function (or others of its type) to retrieve the attribute's values. When you are done with this pointer, you must free it by calling the `ldap_memfree` function.

The `ptr` parameter should be used in subsequent calls to the `ldap_next_attribute` function to retrieve other attributes of the entry. When you are done with the `BerElement` structure and its value is non-`NULL`, you must free it by calling the `ber_free` function with the second parameter set to 0. If the `ptr` parameter is set to `NULL`, then the `ldap_next_attribute` function frees the memory.

See Also

`ldap_first_attribute` (page 148), `ldap_get_values` (page 166)

ldap_next_entry

Returns a pointer to the next entry of message type, LDAP_RES_SEARCH_ENTRY, in chain of LDAPMessage structures.

LDAP Version: v2 or higher

Library: *ldapsdk.*

NDS Version: 7.xx or higher

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldap.h>

LDAPMessage *ldap_next_entry (
    LDAP      *ld,
    LDAPMessage *entry);
```

Parameters

ld

(IN) Points to the handle for the LDAP session.

entry

(IN/OUT) Points to the next LDAPMessage structure in the chain. On the first call, this is the value returned by the ldap_first_entry function. On subsequent calls, it is the value returned by the ldap_next_entry function.

Return Values

NULL	No more entries in the chain or failure
>0	Pointer to the next entry in the chain

Remarks

If the `ldap_next_entry` function encounters an error, the function returns `NULL` and sets the `LDAP_OPT_RESULT_CODE` option in the LDAP session handle.

Use the `ldap_get_dn`, `ldap_first_attribute`, `ldap_get_values` functions to retrieve information about the entry.

See Also

`ldap_first_entry` (page 150), `ldap_count_entries` (page 109), `ldap_search` (page 239), `ldap_search_ext` (page 242), `ldap_search_ext_s` (page 246), `ldap_search_s` (page 250), `ldap_search_st` (page 253)

ldap_next_message

Returns a pointer to the next message of message type, LDAP_RES_SEARCH_ENTRY, LDAP_RES_SEARCH_RESULT, or LDAP_RES_SEARCH_REFERENCE, in chain of LDAPMessage structures.

LDAP Version: v3

Library: *ldapsdk.*

NDS Version: 7.xx or higher

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldap.h>

LDAPMessage *ldap_next_message (
    LDAP      *ld,
    LDAPMessage *msg);
```

Parameters

ld

(IN) Points to the handle for the LDAP session.

msg

(IN/OUT) Points to the LDAPMessage structure returned by a previous call. On the first call, this is the value returned by the ldap_first_message function. On subsequent calls, it is the value returned by the ldap_next_message function.

Return Values

NULL	No more messages or failure.
>0	Pointer to the next message in the chain

Remarks

If the `ldap_next_message` function encounters an error, the function returns `NULL` and sets the `LDAP_OPT_RESULT_CODE` option in the LDAP session handle.

See Also

`ldap_first_message` (page 152), `ldap_count_messages` (page 111), `ldap_msgid` (page 193), `ldap_msgtype` (page 194), `ldap_search` (page 239), `ldap_search_ext` (page 242), `ldap_search_ext_s` (page 246), `ldap_search_s` (page 250), `ldap_search_st` (page 253)

ldap_next_reference

Returns a pointer to the next reference of message type, LDAP_RES_SEARCH_REFERENCE, in chain of LDAPMessage structures.

LDAP Version: v3

Library: *ldapsdk.*

NDS Version: 7.xx or higher

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldap.h>

LDAPMessage *ldap_next_reference (
    LDAP      *ld,
    LDAPMessage *ref);
```

Parameters

ld

(IN) Points to the handle for the LDAP session.

ref

(IN/OUT) Points to the next LDAPMessage structure in the search result chain. On the first call, this is the value returned by the ldap_first_reference function. On subsequent calls, it is the value returned by the ldap_next_reference function.

Return Values

NULL	No more references in the chain or failure
>0	Pointer to the next reference in the chain.

Remarks

If the `ldap_next_reference` function encounters an error, the function returns `NULL` and sets the `LDAP_OPT_RESULT_CODE` option in the LDAP session handle.

See Also

`ldap_first_reference` (page 154), `ldap_count_references` (page 113), `ldap_parse_reference` (page 210), `ldap_search` (page 239), `ldap_search_ext` (page 242), `ldap_search_ext_s` (page 246), `ldap_search_s` (page 250), `ldap_search_st` (page 253)

ldap_open

Initializes the LDAP library and opens a connection to the LDAP server. This function has been deprecated; use the `ldap_init` function.

LDAP Version: v2 or higher

Library: *ldapsdk.*

NDS Version: 7.xx or higher

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldap.h>

LDAP *ldap_open (
    const char    *hostname,
    int           portno);
```

Parameters

hostname

(IN) Contains the names of the available hosts, each separated by a space, or a list of IP addresses (in dot format) of the hosts, each separated by a space. If a port number is included with the name or the address, it is separated from them with a colon (:), for example hostname:port.

portno

(IN) Contains the TCP port number to connect to. If a port number is included with the hostname parameter, this parameter is ignored.

Return Values

>0	Pointer to a session handle
NULL	Failure to establish a session

Remarks

The port number assigned to LDAP is 389.

If the connection is established to an LDAP v2 server, an LDAP bind function must be called before any other operations can be performed.

See Also

`ldap_init` (page 170)

ldap_parse_entrychange_control

Decodes the information returned from a search operation that used a persistent search control.

LDAP Version: v3

Library: *ldapsdk.*

NDS Version: 8.xx or higher

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldap.h>

int ldap_parse_entrychange_control (
    LDAP          *ld,
    LDAPControl    **ctrls,
    int            *changeType,
    char           **prevDN,
    int            *hasChangeNum,
    long           *changeNum) ;
```

Parameters

ld

(IN) Points to the handle of the LDAP session.

ctrls

(IN) A pointer to an array of pointers to controls returned by the server. The controls are obtained by calling the ldap_get_entry_controls() function on an entry returned by the server.

changeType

(OUT) A pointer to an integer specifying the type of change made to the entry. Valid flags are as follows:

LDAP_CHANGETYPE_ADD	Specifies that the entry was added to the directory.
---------------------	--

LDAP_CHANGETYPE_DELETE	Specifies that the entry was deleted from the directory.
LDAP_CHANGETYPE_MODIFY	Specifies that the entry was modified.
LDAP_CHANGETYPE_MODDN	Specifies that the DN or RDN of the entry was changed (a modify RDN or modify DN operation was performed).

prevDN

(OUT) A pointer to the previous DN of the entry, if the changetypes argument is LDAP_CHANGETYPE_MODDN. (If the changetypes argument has a different value, this argument is set to NULL.)

When done, you should free this by calling the ldap_memfree function. This parameter is optional and can be set to NULL.

hasChangeNum

(OUT) A pointer to an integer specifying whether or not the change number is included in the control. A non-zero value indicates that the change number is included and is available as the changeNum argument. Zero indicates that the change number is not included. This parameter and the changeNum parameter must either both be NULL or both be non-NULL.

changeNum

(OUT) A pointer to the change number identifying the change made to the entry. This parameter and the hasChangeNum parameter must either both be NULL or both be non-NULL. Change numbers are typically only returned by servers that support a change log.

Return Values

0x00	LDAP_SUCCESS
0x53	LDAP_DECODING_ERROR
0x5A	LDAP_NO_MEMORY
Non-zero	Non-zero values indicate errors. See LDAP Return Codes.

Remarks

The `ldap_parse_entrychange_control` function examines the controls returned with an entry as a result of a persistent search operation. If an entry change control is present, the control is parsed and its elements' values are retrieved. This function should be called after an entry is returned to the client as a result of a persistent search operation. An entry's controls are retrieved by calling the `ldap_get_entry_controls` function.

For example code, see `searchPersist.c` (http://developer.novell.com/ndk/doc/samplecode/cldap_sample/index.htm).

See Also

`ldap_search_ext` (page 242), `ldap_get_entry_controls` (page 160),
`ldap_create_persistentsearch_control` (page 119)

ldap_parse_extended_result

Retrieves data from an LDAPMessage that contains data from an extended operation.

LDAP Version: v3

Library: *ldapsdk.*

NDS Version: 7.xx or higher

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldap.h>

int ldap_parse_extended_result (
    LDAP          *ld,
    LDAPMessage    *res,
    char          **retoidp,
    struct berval  **retdatap,
    int           freeit);
```

Parameters

ld

(IN) Points to the handle for the LDAP session.

res

(IN) Points to an LDAPMessage containing the results of an LDAP extended operation.

retoidp

(OUT) Points to the dotted-OID text string that represents the name of the extended operation. Pass in NULL to ignore this field. When you are finished, you must free this string by calling the ldap_memfree function.

retdatap

(OUT) Points to a berval structure that contains data from the extended operation response.

freeit

(IN) Specifies whether the resources allocated by the res parameter are freed.

- ♦ Zero indicates that the resources used by the res parameter are not freed automatically. When you are done with the res parameter, you need to call the ldap_msgfree function to free the memory.
- ♦ Non-zero indicates that memory is freed after the function extracts the information.

Return Values

0x00	LDAP_SUCCESS
Non-zero	Failure. For a complete list, see LDAP Return Codes.
0x54	LDAP_DECODING_ERROR
0x59	LDAP_PARAM_ERROR
0x5A	LDAP_NO_MEMORY
0x5C	LDAP_NOT_SUPPORTED

Remarks

After calling the ldap_extended_operation and ldap_result functions, use ldap_parse_extended_result to parse the extended information returned by the LDAP server.

See Also

ldap_err2string (page 138), ldap_parse_result (page 212),
ldap_parse_sasl_bind_result (page 215)

ldap_parse_reference

Extracts URLs and controls from an LDAPMessage structure of type LDAP_RES_SEARCH_REFERENCE.

LDAP Version: v3

Library: *ldapsdk.*

NDS Version: 7.xx or higher

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldap.h>

int ldap_parse_reference (
    LDAP          *ld,
    LDAPMessage    *ref,
    char          ***referralsp,
    LDAPControl    ***serverctrlsp,
    int           freeit);
```

Parameters

ld

(IN) Points to the handle for the LDAP session.

ref

(IN) Points to the reference to parse which was returned by the ldap_result, ldap_first_reference, ldap_next_reference, ldap_first_message, or ldap_next_message function.

referralsp

(OUT) Points to a NULL-terminated array of strings which contains zero or more alternate LDAP server URLs where the request can be sent. Pass in NULL to ignore this parameter. When you are finished, free the referrals array by calling the ldap_value_free function.

serverctrlsp

(OUT) Points to a NULL-terminated array of LDAPControl structures which are returned by the LDAP server and which list the controls the

LDAP server supports. When you are finished, free the control array by calling the `ldap_controls_free` function. Pass in `NULL` to ignore this parameter.

`freit`

(IN) Specifies whether the resources specified by the `ref` parameter are freed.

- ◆ Zero indicates that the resources specified by the `res` parameter are not freed automatically. When you are done with the `LDAPMessage` structure, you must call the `ldap_msgfree` function to free the memory.
- ◆ Non-zero indicates that memory is freed by the `ldap_parse_reference` function after it extracts the information.

Return Values

0x00	LDAP_SUCCESS
Non-zero	Failure. For a complete list, see LDAP Return Codes.
0x54	LDAP_DECODING_ERROR
0x59	LDAP_PARAM_ERROR

See Also

`ldap_first_reference` (page 154), `ldap_next_reference` (page 201)

ldap_parse_result

Extracts error, referral, and server control information from an LDAPMessage structure.

LDAP Version: v3

Library: *ldapsdk.*

NDS Version: 7.xx or higher

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldap.h>

int ldap_parse_result (
    LDAP          *ld,
    LDAPMessage    *res,
    int            *errcodep,
    char           **matcheddn,
    char           **errmsgp,
    char           ***referralsp,
    LDAPControl    ***serverctrlsp,
    int            freeit);
```

Parameters

ld

(IN) Points to the handle for the LDAP session.

res

(IN) Points to an LDAPMessage containing the results of an LDAP operation.

errcodep

(OUT) Points to the LDAP error code that specifies the results of the last LDAP operation.

matcheddn

(OUT) Points to a string that specifies how much of the name in the request was recognized. Pass in NULL to ignore this parameter. When

you are finished, you must free the matched DN string by calling the `ldap_memfree` function.

`errmsgp`

(OUT) Points to the error message string that is associated with the error code. Pass in `NULL` to ignore this parameter. When you are finished, you must free the error message string by calling the `ldap_memfree` function.

`referralsp`

(OUT) Points to a `NULL`-terminated array of strings which contains zero or more alternate LDAP server URLs where the request can be sent. Pass in `NULL` to ignore this parameter. When you are finished, you must free the referrals array by calling the `ldap_value_free` function.

`serverctrlsp`

(OUT) Points to a `NULL`-terminated array of `LDAPControl` structures which are returned by the LDAP server and which list the controls the LDAP server supports. When you are finished, you must free the control array by calling the `ldap_controls_free` function.

`freit`

(IN) Specifies whether the resources specified by the `res` parameter are freed.

- ♦ Zero indicates that the resources specified by the `res` parameter are not freed automatically. When you are done with the `LDAPMessage` structure, you must call the `ldap_msgfree` function to free the memory.
- ♦ Non-zero indicates that memory is freed by the `ldap_parse_result` function after it extracts the information.

Return Values

0x00	LDAP_SUCCESS
0x5E	LDAP_NO_RESULTS_RETURNED
0x5F	LDAP_MORE_RESULTS_TO_RETURN
Non-zero	Failure. For a complete list, see LDAP Return Codes.

0x54	LDAP_DECODING_ERROR
0x59	LDAP_PARAM_ERROR

Remarks

Upon successful completion, the `ldap_result` function returns the type of the first result returned in the `res` parameter. The type will be one of the following constants:

`LDAP_RES_BIND`
`LDAP_RES_SEARCH_ENTRY`
`LDAP_RES_SEARCH_REFERENCE`
`LDAP_RES_SEARCH_RESULT`
`LDAP_RES_MODIFY`
`LDAP_RES_ADD`
`LDAP_RES_DELETE`
`LDAP_RES_MODDN`
`LDAP_RES_COMPARE`
`LDAP_RES_EXTENDED`

The `ldap_parse_result` function cannot be used to parse `LDAP_RES_SEARCH_ENTRY` or `LDAP_RES_SEARCH_REFERENCE` messages. Use `ldap_first_entry` to parse entries. Use `ldap_parse_reference` to parse references.

If a chain of messages is passed to this function, the function operates only on the first message in the result chain that is not of type `LDAP_RES_SEARCH_ENTRY` or `LDAP_RES_SEARCH_REFERENCE`. Use the `ldap_first_message` and `ldap_next_message` functions to step through a chain of messages.

If the result message contains data from an extended operation, use the `ldap_parse_extended_result` function to retrieve additional information.

See Also

[ldap_err2string](#) (page 138), [ldap_parse_extended_result](#) (page 208),
[ldap_parse_sasl_bind_result](#) (page 215)

ldap_parse_sasl_bind_result

Extracts SASL bind information from an LDAPMessage structure.

Library: *ldapsdk.*

NDS Version: 7.xx or higher

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldap.h>

int ldap_parse_sasl_bind_result (
    LDAP          *ld,
    LDAPMessage    *res,
    struct berval  **servercredp,
    int            freeit);
```

Parameters

ld

(IN) Points to the handle for the LDAP session.

res

(IN) Points to an LDAPMessage containing the results of an LDAP operation.

servercredp

(OUT) Points to the credentials passed back by the LDAP server to use for mutual authentication. When done with the structure, free the memory by calling the ber_bvfree function.

freeit

(IN) Specifies whether the resources allocated by the res parameter are freed.

- ♦ Zero indicates that the resources used by the res parameter are not freed automatically. When you are done with the res parameter, you need to call the ldap_msgfree function to free the memory.

- ♦ Non-zero indicates that memory is freed after the function extracts the information.

Return Values

0x00	LDAP_SUCCESS
Non-zero	Failure. For a complete list, see LDAP Return Codes.
0x54	LDAP_DECODING_ERROR
0x59	LDAP_PARAM_ERROR
0x5A	LDAP_NO_MEMORY
0x5C	LDAP_NOT_SUPPORTED

Remarks

After calling the `ldap_sasl_bind` and the `ldap_result` functions, use the `ldap_parse_sasl_bind_result` to obtain the SASL bind information.

See Also

`ldap_err2string` (page 138), `ldap_parse_extended_result` (page 208), `ldap_parse_result` (page 212)

ldap_parse_sort_control

Decodes the information returned from a search operation that used a server-side sort control.

LDAP Version: v3

Library: *ldapsdk.*

NDS Version: 8.xx or higher

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldap.h>

int ldap_parse_sort_control (
    LDAP          *ld,
    LDAPControl    **ctrls,
    unsigned long  *returnCode,
    char           **attribute);
```

Parameters

ld

(IN) Points to the handle of the LDAP session.

ctrls

(IN) Points to the address of a NULL-terminated array of LDAPControl structures, usually obtained by a call to the ldap_parse_result function.

returnCode

(OUT) Points to the sort control result code. This parameter must not be NULL. See Remarks for a list of possible return codes.

attribute

(OUT) If the sort operation fails, the server may return a string that indicates the first attribute in the sortKey list that caused the failure. If this parameter is NULL, no string is returned. If a string is returned, the memory should be freed by calling the ldap_memfree function.

Return Values

0x00	LDAP_SUCCESS
Non-zero	Failure. See LDAP Return Codes.

Remarks

The `returnCode` parameter returns one of the following values.

Return Value	Description
success (0)	Server returned sorted results.
operationsError (1)	Server had an internal failure.
timeLimitExceeded (3)	Server reached the time limit before the sorting was completed.
strongAuthRequired (8)	Server refused to return sorted results over an insecure protocol.
adminLimitExceeded (11)	The results contain too many matching entries for the server to sort.
noSuchAttribute (16)	Server does not recognized an attribute type in the sort key.
inappropriateMatching (18)	Server does not recognized the matching rule in the sort key, or the matching rule is inappropriate for the attribute type.
insufficientAccessRights (50)	Server refused to return sorted results to this client.
busy (51)	Server is too busy to process.
unwillingToPerform (53)	Server is unable to sort the results.
other (80)	An error occurred.

For example code, see `sortcntl.c` and `vlvcntl.c` (http://developer.novell.com/ndk/doc/samplecode/cldap_sample/index.htm).

See Also

`ldap_create_sort_control` (page 122)

ldap_parse_vlv_control

Decodes the information returned from a search operation that used a VLV (virtual list view) control.

LDAP Version: v3

Library: *ldapsdk.*

NDS Version: 8.xx or higher

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldap.h>

int ldap_parse_vlv_control (
    LDAP          *ld,
    LDAPControl    **ctrlp,
    unsigned long  *target_posp,
    unsigned long  *list_countp,
    struct berval  **contextp,
    int            *errcodep);
```

Parameters

ld

(IN) Points to the handle of the LDAP session.

ctrlp

(IN) Points to a NULL-terminated array of LDAPControl structures, typically obtained by calling the ldap_parse_result function.

target_posp

(OUT) Points to the list index of the target entry. If this parameter is NULL, the target position is not returned. The index returned is an approximation of the position of the target entry. It is not guaranteed to be exact.

list_countp

(OUT) Points to the server's estimate of the size of the list. If this parameter is NULL, the size is not returned.

contextp

(OUT) Points to the address of a berval structure that contains a server-generated context identifier if server returns one. If server does not return a context identifier, the server returns a NULL in this parameter. If this parameter is set to NULL, the context identifier is not returned.

You should use this returned context in the next call to create a VLV control.

When the berval structure is no longer needed, you should free the memory by calling the ber_bvfree function.

errcodep

(OUT) Points to the result code returned by the server. If this parameter is NULL, the result code is not returned. See Remarks for a list of possible return codes.

Return Values

0x00	LDAP_SUCCESS
Non-zero	Failure. See LDAP Return Codes.
0x5D	LDAP_CONTROL_NOT_FOUND

Remarks

The errcodep parameter returns one of the following values.

Return Value	Description
success (0)	Server returned VLV results.
operationsError (1)	Server had an internal failure.
timeLimitExceeded (3)	Server reached the time limit before the virtual list view was completed.
adminLimitExceeded (11)	The results contain too many matching entries for the server to place in a virtual list view.

Return Value	Description
insufficientAccessRights (50)	Server refused to return sorted results to this client.
busy (51)	Server is too busy to process.
unwillingToPerform (53)	Server is unable to sort the results.
sortControlMissing (60)	The sort control for the virtual list view is missing.
offsetRangeError (61)	The offset is set to less than zero.
other (80)	An error occurred.

For example code, see `vlvcntl.c` (http://developer.novell.com/ndk/doc/samplecode/cldap_sample/index.htm).

See Also

`berval` (page 399), `LDAPControl` (page 404)

ldap_perror

Prints a specified message and the current LDAP error message to standard error. This function has been deprecated; use the `ldap_err2string` function.

LDAP Version: v2 or higher

Library: *ldapsdk.*

NDS Version: 7.xx or higher

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldap.h>

void ldap_perror (
    LDAP      *ld,
    const char *msg);
```

Parameters

`ld`

(IN) Points to the handle for the LDAP session.

`msg`

(IN) Points to the message that is displayed before the LDAP error message.

See Also

`ldap_err2string` (page 138), `ldap_parse_result` (page 212)

ldap_rename

Asynchronously renames the specified entry.

LDAP Version: v3

Library: *ldapsdk.*

NDS Version: 7.xx or higher

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldap.h>

int ldap_rename (
    LDAP          *ld,
    const char    *dn,
    const char    *newrdn,
    const char    *newparent,
    int           deleteoldrdn,
    LDAPControl   **serverctrls,
    LDAPControl   **clientctrls,
    int           *msgidp);
```

Parameters

ld

(IN) Points to the handle for the LDAP session.

dn

(IN) Points to the distinguished name of the entry that is being renamed.

newrdn

(IN) Points to the new relative distinguished name to give the entry.

newparent

(IN) Points to the distinguished name of the entry's new parent. If this parameter is NULL, only the RDN is changed. The root DN is specified by passing a zero length string, "".

This function can be used with LDAP v2 servers if the newparent parameter is NULL. LDAP v2 does not allow the entry to be moved to a new parent.

deleteoldrdn

(IN) Specifies whether the old RDN should be retained or deleted.

- ◆ Zero indicates that the old RDN should be retained. If you choose this option, the attribute will contain both names (the old and the new).
- ◆ Non-zero indicates that the old RDN should be deleted.

serverctrls

(IN) Points to an array of LDAPControl structures that list the server controls to use with the rename. Use NULL to specify no server controls.

clientctrls

(IN) Points to an array of LDAPControl structures that list the client controls to use with the rename. Use NULL to specify no client controls.

msgidp

(OUT) Points to the message ID of the request when the rename request succeeds.

Return Values

0x00	LDAP_SUCCESS
Non-zero	Failure. For a complete list, see LDAP Return Codes.
0x53	LDAP_ENCODING_ERROR
0x5A	LDAP_NO_MEMORY
0x5C	LDAP_NOT_SUPPORTED

Remarks

The ldap_rename function changes the leaf component of an entry's distinguished name and optionally moves the entry to a new parent container.

To obtain the results of the operation, call the `ldap_result` function using the message ID in the `msgidp` parameter.

NDS does not currently support any server-side controls to use with renaming an entry.

See Also

`ldap_rename_s` (page 227)

ldap_rename_s

Synchronously renames the specified entry.

LDAP Version: v3

Library: *ldapsdk.*

NDS Version: 7.xx or higher

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldap.h>

int ldap_rename_s (
    LDAP          *ld,
    const char    *dn,
    const char    *newrdn,
    const char    *newparent,
    int           deleteoldrdn,
    LDAPControl   **serverctrls,
    LDAPControl   **clientctrls);
```

Parameters

ld

(IN) Points to the handle for the LDAP session.

dn

(IN) Points to the distinguished name of the entry that is being renamed.

newrdn

(IN) Points to the new relative distinguished name to give the entry.

newparent

(IN) Points to the distinguished name of the entry's new parent. If this parameter is NULL, only the RDN is changed. The root DN is specified by passing a zero length string, "".

This function can be used with LDAP v2 servers if the newparent parameter is NULL. LDAP v2 does not allow the entry to be moved to a new parent.

deleteoldrdn

(IN) Specifies whether the old RDN should be retained or deleted.

- ♦ Zero indicates that the old RDN should be retained. If you choose this option, the attribute will contain both names (the old and the new).
- ♦ Non-zero indicates that the old RDN should be deleted.

serverctrls

(IN) Points to an array of LDAPControl structures that list the server controls to use with the rename. Use NULL to specify no server controls.

clientctrls

(IN) Points to an array of LDAPControl structures that list the client controls to use with the rename. Use NULL to specify no client controls.

Return Values

0x00	LDAP_SUCCESS
Non-zero	Failure. For a complete list, see LDAP Return Codes.
0x53	LDAP_ENCODING_ERROR
0x5A	LDAP_NO_MEMORY
0x5C	LDAP_NOT_SUPPORTED

Remarks

NDS does not currently support any server-side controls to use with renaming an entry.

For sample code, see renamerdn.c (http://developer.novell.com/ndk/doc/samplecode/cldap_sample/index.htm).

See Also

`ldap_rename` (page 224)

ldap_result

Obtains results from a previous asynchronously initiated operation.

LDAP Version: v2 or higher

Library: *ldapsdk.*

NDS Version: 7.xx or higher

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldap.h>

int ldap_result (
    LDAP          *ld,
    int           msgid,
    int           all,
    struct timeval *timeout,
    LDAPMessage   **res);
```

Parameters

ld

(IN) Points to the handle for the LDAP session.

msgid

(IN) Specifies the message ID returned in the LDAP operation's msgidp parameter. Accepts the following special flags:

- ◆ LDAP_RES_UNSOLICITED (0) indicates that an unsolicited result is requested.
- ◆ LDAP_RES_ANY (-1) indicates that any result is to be returned.

all

(IN) Specifies how many messages are retrieved in a single call to the ldap_result function. Uses one of the following flags:

- ◆ LDAP_MSG_ONE (0x00) indicates that messages are retrieved one at a time.

- ◆ LDAP_MSG_ALL (0x01) indicates that all results of the search must be received before returning with all the messages in a single chain.
- ◆ LDAP_MSG_RECEIVED (0x02) indicates that all messages received so far must be returned in a result chain.

timeout

(IN) Points to a timeval structure that specifies how long to wait for the results to be returned.

- ◆ To block until the results are available, pass a NULL value.
- ◆ To cause continuous polling, set the tv_sec field in the timeval structure to zero seconds.

res

(OUT) Points to the results of the search. If no results are returned, this parameter is set to NULL.

Return Values

0x00	Timeout expired
-1	Failure

Remarks

Only asynchronous search operations can contain more than one message.

Upon successful completion, the ldap_result function returns the type of message. For a list of possible types, see “Result Message Types” on page 390.

If the ldap_result function returns a -1, use the ldap_get_option function with the option parameter set to LDAP_OPT_RESULT_CODE to retrieve the error code from the LDAP session handle.

The ldap_result function allocates memory for the res parameter. When you are done with it, free the memory by calling the ldap_msgfree function.

See Also

`ldap_msgfree` (page 191), `ldap_msgid` (page 193), `ldap_msgtype` (page 194)

ldap_result2error

Converts the result message into a numeric LDAP error code. This function has been deprecated. Use the `ldap_parse_result` function.

LDAP Version: v2 or higher

Library: *ldapsdk.*

NDS Version: 7.xx or higher

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldap.h>

int ldap_result2error (
    LDAP          *ld,
    LDAPMessage    *res,
    int            freeit);
```

Parameters

`ld`

(IN) Points to the handle for the LDAP session.

`res`

(IN) Points to an LDAPMessage containing the results of an `ldap_result` or `ldap_search_s` operation.

`freeit`

(IN) Specifies whether the resources allocated by the `res` parameter are freed.

- ◆ Zero indicates that the resources used by the `res` parameter are not freed automatically. When you are done with the `res` parameter, you need to call the `ldap_msgfree` function to free the memory.
- ◆ Non-zero indicates that memory is freed after the function extracts the information

Return Values

>0

LDAP error code. See LDAP Return Codes.

Remarks

The `ldap_result2error` function does the following:

- ♦ Converts the result message into a numeric LDAP error code.
- ♦ Parses the result message and puts the matched distinguished name in the `LDAP_OPT_MATCHED_DN` option of the LDAP session handle.
- ♦ Parses the result message and puts the error code in the `LDAP_OPT_ERROR_STRING` option of the LDAP session handle.

All synchronous operation routines call the `ldap_result2error` function before returning, ensuring that the options are set correctly.

See Also

`ldap_parse_result` (page 212), `ldap_parse_extended_result` (page 208),
`ldap_perror` (page 223)

ldap_sasl_bind

Asynchronously authenticates the specified client to the LDAP server using a Simple Authentication Security Layer (SASL).

LDAP Version: v3

Library: *ldapsdk.*

NDS Version: 7.xx or higher

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldap.h>

int ldap_sasl_bind (
    LDAP          *ld,
    const char    *dn,
    const char    *mechanism,
    const struct berval *cred,
    LDAPControl   **serverctrls,
    LDAPControl   **clientctrls,
    int           *msgidp);
```

Parameters

ld

(IN) Points to the handle for the LDAP session.

dn

(IN) Points to the distinguished name of the entry who is authenticating.

mechanism

(IN) Points to the method to use for authentication, either

- ◆ LDAP_SASL_SIMPLE (NULL) for simple authentication
- ◆ A dotted-string representation of the OID identifying the SASL method

cred

(IN) Points to the credentials with which to authenticate.

serverctrls

(IN) Points to a list of server controls. Use NULL to specify no server controls.

clientctrls

(IN) Points to a list of client controls. Use NULL to specify no client controls.

msgidp

(OUT) Points to the message ID of the request when the bind request succeeds.

Return Values

>0	Message ID of operation
-1	Failure

Remarks

The `ldap_sasl_bind` function is an asynchronous function and does not return the results directly. To obtain the results, call the `ldap_parse_result` function using the message ID in the `msgidp` parameter.

If you want the function to return the results directly, use the `ldap_sasl_bind_s` function.

If the `ldap_sasl_bind` function returns -1, check the `LDAP_OPT_RESULT_CODE` option in the LDAP handle for the error code.

The NDS directory does not currently support SASL authentication.

See Also

`ldap_sasl_bind_s` (page 237), `ldap_parse_sasl_bind_result` (page 215)

ldap_sasl_bind_s

Synchronously authenticates the specified client to the LDAP server using a Simple Authentication Security Layer (SASL).

LDAP Version: v3

Library: *ldapsdk.*

NDS Version: 7.xx or higher

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldap.h>

int ldap_sasl_bind_s (
    LDAP          *ld,
    const char     *dn,
    const char     *mechanism,
    const struct berval *cred,
    LDAPControl    **serverctrls,
    LDAPControl    **clientctrls,
    struct berval  **servercredp);
```

Parameters

ld

(IN) Points to the handle for the LDAP session.

dn

(IN) Points to the distinguished name of the entry who is authenticating.

mechanism

(IN) Points to the method to use for authentication, either

- ◆ LDAP_SASL_SIMPLE (NULL) for simple authentication
- ◆ A dotted-string representation of the OID identifying the SASL method

cred

(IN) Points to the credentials with which to authenticate.

serverctrls

(IN) Points to a list of server controls. Use NULL to specify no server controls.

clientctrls

(IN) Points to a list of client controls. Use NULL to specify no client controls.

servercredp

(OUT) Points to the credentials passed back by the server for mutual authentication. The berval structure must be freed by calling the `ber_bvfree` function. To ignore this parameter, set it to NULL.

Return Values

0x00	LDAP_SUCCESS
Non-zero	Failure. For a complete list, see LDAP Return Codes.
0x54	LDAP_DECODING_ERROR
0x59	LDAP_PARAM_ERROR
0x5A	LDAP_NO_MEMORY
0x5C	LDAP_NOT_SUPPORTED

Remarks

The NDS directory does not currently support either SASL authentication or mutual authentication.

See Also

`ldap_sasl_bind` (page 235)

ldap_search

Asynchronously searches the directory.

LDAP Version: v2 or higher

Library: *ldapsdk.*

NDS Version: 7.xx or higher

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldap.h>

int ldap_search (
    LDAP      *ld,
    const char *base,
    int        scope,
    const char *filter,
    char       **attrs,
    int        attrsonly);
```

Parameters

ld

(IN) Points to the handle for the LDAP session.

base

(IN) Points to the distinguished name of the entry from which to start the search.

scope

(IN) Specifies the scope of the search and uses one of the following flags:

- ◆ LDAP_SCOPE_BASE (0x00)—searches the entry specified by the base parameter.
- ◆ LDAP_SCOPE_ONELEVEL (0x01)—searches the immediate subordinates of the entry specified by the base parameter.
- ◆ LDAP_SCOPE_SUBTREE (0x02)—searches the entire subtree starting with the entry specified by the base parameter.

filter

(IN) Points to a search filter.

If NULL is passed, a default filter ("objectclass=*") is used, a filter which matches all entries in the directory. Using a NULL filter is not recommended for subtree searches on trees that potentially have hundreds of thousands of entries.

Simple filters take the form of strings: attribute name=attribute value. For more complex filters, see “Using Search Filters” on page 35.

attrs

(IN) Points to a NULL-terminated array of strings indicating which attributes to return with each matching entry. To return only entry names (and no attributes), set the first, and only string in the array, to LDAP_NO_ATTRS. To return all attributes, set this parameter to NULL.

For example, to return the cn, surname, and givenName attributes, declare attrs as: `char* attrs[]={ "cn", "surname", "givenName", NULL};`

attrsonly

(IN) Specifies whether to return just attributes or attributes and values.

- ◆ Zero—return both attributes and values
- ◆ Non-zero—return only attributes

Return Values

>0	Message ID of operation
-1	Failure

Remarks

The ldap_search function is an older function which does not allow you to specify LDAP controls.

The LDAP_OPT_DEREF option in the LDAP session handle affects how aliases are handled during the search.

- ♦ The LDAP_DEREF_FINDING value means aliases are dereferenced when locating the base object but not during the search.
- ♦ The LDAP_DEREF_SEARCHING value means aliases are dereferenced during the search but not when locating the base object of the search.

To obtain the results of the operation, call the `ldap_result` function using the message ID returned to the `ldap_search` function.

If the function returns a -1, use the `ldap_get_option` function with the option parameter set to `LDAP_OPT_RESULT_CODE` to retrieve the error code from the LDAP session handle.

See Also

`ldap_search_ext` (page 242), `ldap_search_st` (page 253)

ldap_search_ext

Asynchronously searches the directory using LDAP client or server controls.

LDAP Version: v3

Library: *ldapsdk.*

NDS Version: 7.xx or higher

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldap.h>

int ldap_search_ext (
    LDAP          *ld,
    const char    *base,
    int           scope,
    const char    *filter,
    char          **attrs,
    int           attrsonly,
    LDAPControl   **serverctrls,
    LDAPControl   **clientctrls,
    struct timeval *timeout,
    int           sizelimit,
    int           *msgidp);
```

Parameters

ld

(IN) Points to the handle for the LDAP session.

base

(IN) Points to the distinguished name of the entry from which to start the search.

scope

(IN) Specifies the scope of the search and uses one of the following flags:

- ◆ LDAP_SCOPE_BASE (0x00)—searches the entry specified by the base parameter.

- ♦ LDAP_SCOPE_ONELEVEL (0x01)—searches the immediate subordinates of the entry specified by the base parameter.
- ♦ LDAP_SCOPE_SUBTREE (0x02)—searches the entire subtree starting with the entry specified by the base parameter.

filter

(IN) Points to a search filter.

If NULL is passed, a default filter ("objectclass=*") is used, a filter which matches all entries in the directory. Using a NULL filter is not recommended for subtree searches on trees that potentially have hundreds of thousands of entries.

Simple filters take the form of strings: attribute name=attribute value. For more complex filters, see “Using Search Filters” on page 35.

attrs

(IN) Points to a NULL-terminated array of strings indicating which attributes to return with each matching entry. To return only entry names (and no attributes), set the first, and only string in the array, to LDAP_NO_ATTRS. To return all attributes, set this parameter to NULL.

For example, to return the cn, surname, and givenName attributes, declare attrs as: **char* attrs[]={ "cn", "surname", "givenName", NULL};**

attrsonly

(IN) Specifies whether to return just attributes or attributes and values.

- ♦ Zero—return both attributes and values
- ♦ Non-zero—return only attributes

serverctrls

(IN) Points to an array of LDAPControl structures that list the server controls to use with the search. Use NULL to specify no server controls.

clientctrls

(IN) Points to an array of LDAPControl structures that list the client controls to use with the search. Use NULL to specify no client controls.

timeout

(IN) Points to a `timeval` structure that specifies the maximum time to wait for the results of a search to complete. It specifies both the time the server waits for the operation to complete as well as the time the local function waits for the server to respond. If the timeout parameter is set to `NULL`, the client timeout is infinite and the server uses the timeout value stored in the session handle option, `LDAP_OPT_TIMELIMIT` (whose default value is no timeout). For more information about possible values, see `timeval` (page 418).

sizelimit

(IN) Specifies the maximum number of entries to return.

- ♦ To specify no limit, pass `LDAP_NO_LIMIT` (0).
- ♦ To use the current value in the LDAP session handle (the `LDAP_OPT_SIZELIMIT` option), pass `LDAP_DEFAULT_SIZELIMIT` (-1).

msgidp

(OUT) Points to the message ID of the request if the search request succeeds.

Return Values

0x00	<code>LDAP_SUCCESS</code>
Non-zero	Failure. For a complete list, see LDAP Return Codes.
0x53	<code>LDAP_ENCODING_ERROR</code>
0x55	<code>LDAP_TIMEOUT</code>
0x57	<code>LDAP_FILTER_ERROR</code>

Remarks

The `LDAP_OPT_DEREF` option in the LDAP session handle affects how aliases are handled during the search.

- ♦ The `LDAP_DEREF_FINDING` value means aliases are dereferenced when locating the base object but not during the search.

- ♦ The LDAP_DEREF_SEARCHING value means aliases are dereferenced during the search but not when locating the base object of the search.

NDS supports two server controls:

- ♦ Server-side sorting—1.2.840.113556.1.4.473
- ♦ Virtual list views—2.16.840.1.113730.3.4.9

To obtain the results of the operation, call the `ldap_result` function using the message ID returned to the `ldap_search_ext` function.

For sample code, see `searchas.c` (http://developer.novell.com/ndk/doc/samplecode/cldap_sample/index.htm).

See Also

`ldap_search` (page 239), `ldap_search_st` (page 253)

ldap_search_ext_s

Synchronously searches the directory using LDAP client or server controls.

LDAP Version: v3

Library: *ldapsdk.*

NDS Version: 7.xx or higher

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldap.h>

int ldap_search_ext_s (
    LDAP          *ld,
    const char    *base,
    int           scope,
    const char    *filter,
    char          **attrs,
    int           attrsonly,
    LDAPControl   **serverctrls,
    LDAPControl   **clientctrls,
    struct timeval *timeout,
    int           sizelimit,
    LDAPMessage   **res);
```

Parameters

ld

(IN) Points to the handle for the LDAP session.

base

(IN) Points to the distinguished name of the entry from which to start the search.

scope

(IN) Specifies the scope of the search and uses one of the following flags:

- ◆ LDAP_SCOPE_BASE (0x00)—searches the entry specified by the base parameter.

- ♦ LDAP_SCOPE_ONELEVEL (0x01)—searches the immediate subordinates of the entry specified by the base parameter.
- ♦ LDAP_SCOPE_SUBTREE (0x02)—searches the entire subtree starting with the entry specified by the base parameter.

filter

(IN) Points to a search filter.

If NULL is passed, a default filter ("objectclass=*") is used, a filter which matches all entries in the directory. Using a NULL filter is not recommended for subtree searches on trees that potentially have hundreds of thousands of entries.

Simple filters take the form of strings: attribute name=attribute value. For more complex filters, see “Using Search Filters” on page 35.

attrs

(IN) Points to a NULL-terminated array of strings indicating which attributes to return with each matching entry. To return only entry names (and no attributes), set the first, and only string in the array, to LDAP_NO_ATTRS. To return all attributes, set this parameter to NULL.

For example, to return the cn, surname, and givenName attributes, declare attrs as: **char* attrs[]={ "cn", "surname", "givenName", NULL};**

attrsonly

(IN) Specifies whether to return just attributes or attributes and values.

- ♦ Zero—return both attributes and values
- ♦ Non-zero—return only attributes

serverctrls

(IN) Points to an array of LDAPControl structures that list the server controls to use with the search. Use NULL to specify no server controls.

clientctrls

(IN) Points to an array of LDAPControl structures that list the client controls to use with the search. Use NULL to specify no client controls.

timeout

(IN) Points to a `timeval` structure that specifies the maximum time to wait for the results of a search to complete. It specifies both the time the server waits for the operation to complete as well as the time the local function waits for the server to respond. If the timeout parameter is set to `NULL`, the client timeout is infinite and the server uses the timeout value stored in the session handle option, `LDAP_OPT_TIMELIMIT` (whose default value is no timeout). For more information about possible values, see `timeval` (page 418).

sizelimit

(IN) Specifies the maximum number of entries to return.

- ♦ To specify no limit, pass `LDAP_NO_LIMIT` (0).
- ♦ To use the current value in the LDAP session handle (the `LDAP_OPT_SIZELIMIT` option), pass `LDAP_DEFAULT_SIZELIMIT` (-1).

res

(OUT) Returns a pointer to an array of result messages if the search succeeds or `NULL` if no results are returned.

Return Values

0x00	<code>LDAP_SUCCESS</code>
Non-zero	Failure. For a complete list, see LDAP Return Codes.
0x53	<code>LDAP_ENCODING_ERROR</code>
0x55	<code>LDAP_TIMEOUT</code>
0x57	<code>LDAP_FILTER_ERROR</code>

Remarks

The `LDAP_OPT_DEREF` option in the LDAP session handle affects how aliases are handled during the search.

- ♦ The `LDAP_DEREF_FINDING` value means aliases are dereferenced when locating the base object but not during the search.

- ♦ The LDAP_DEREF_SEARCHING value means aliases are dereferenced during the search but not when locating the base object of the search.

NDS supports two server controls:

- ♦ Server-side sorting—1.2.840.113556.1.4.473
- ♦ Virtual list views—2.16.840.1.113730.3.4.9

You must use the `ldap_result` and the `ldap_parse_result` functions to retrieve the results of the search.

For sample code, see `search.c` and `searchmsg.c` (http://developer.novell.com/ndk/doc/samplecode/cldap_sample/index.htm).

See Also

`ldap_search` (page 239), `ldap_search_s` (page 250), `ldap_search_st` (page 253), `ldap_search_ext` (page 242)

ldap_search_s

Synchronously searches the directory.

LDAP Version: v2 or higher

Library: *ldapsdk.*

NDS Version: 7.xx or higher

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldap.h>

int ldap_search_s (
    LDAP          *ld,
    const char    *base,
    int            scope,
    const char    *filter,
    char          **attrs,
    int            attrsonly,
    LDAPMessage   **res);
```

Parameters

ld

(IN) Points to the handle for the LDAP session.

base

(IN) Points to the distinguished name of the entry from which to start the search.

scope

(IN) Specifies the scope of the search and uses one of the following flags:

- ◆ LDAP_SCOPE_BASE (0x00)—searches the entry specified by the base parameter.
- ◆ LDAP_SCOPE_ONELEVEL (0x01)—searches the immediate subordinates of the entry specified by the base parameter.

- ♦ `LDAP_SCOPE_SUBTREE (0x02)`—searches the entire subtree starting with the entry specified by the base parameter.

filter

(IN) Points to a search filter.

If `NULL` is passed, a default filter ("`objectclass=*`") is used, a filter which matches all entries in the directory. Using a `NULL` filter is not recommended for subtree searches on trees that potentially have hundreds of thousands of entries.

Simple filters take the form of strings: attribute name=attribute value. For more complex filters, see “Using Search Filters” on page 35.

attrs

(IN) Points to a `NULL`-terminated array of strings indicating which attributes to return with each matching entry. To return only entry names (and no attributes), set the first, and only string in the array, to `LDAP_NO_ATTRS`. To return all attributes, set this parameter to `NULL`.

For example, to return the `cn`, `surname`, and `givenName` attributes, declare `attrs` as: `char* attrs[]={ "cn", "surname", "givenName", NULL};`

attrsonly

(IN) Specifies whether to return just attributes or attributes and values.

- ♦ Zero—return both attributes and values
- ♦ Non-zero—return only attributes

res

(OUT) Returns a pointer to an array of result messages if the search succeeds or `NULL` if no results are returned.

Return Values

0x00	<code>LDAP_SUCCESS</code>
Non-zero	Failure. For a complete list, see <code>LDAP Return Codes</code> .
0x53	<code>LDAP_ENCODING_ERROR</code>

Remarks

The `ldap_search` function is an older function which does not allow you to specify LDAP controls.

The `LDAP_OPT_DEREF` option in the LDAP session handle affects how aliases are handled during the search.

- ♦ The `LDAP_DEREF_FINDING` value means aliases are dereferenced when locating the base object but not during the search.
- ♦ The `LDAP_DEREF_SEARCHING` value means aliases are dereferenced during the search but not when locating the base object of the search.

See Also

`ldap_search_ext` (page 242), `ldap_search_ext_s` (page 246), `ldap_search_st` (page 253)

ldap_search_st

Synchronously searches the directory within a specified time limit.

LDAP Version: v2 or higher

Library: *Idapsdk.*

NDS Version: 7.xx or higher

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldap.h>

int ldap_search_st (
    LDAP          *ld,
    const char    *base,
    int           scope,
    const char    *filter,
    char          **attrs,
    int           attrsonly,
    struct timeval *timeout,
    LDAPMessage   **res);
```

Parameters

ld

(IN) Points to the handle for the LDAP session.

base

(IN) Points to the distinguished name of the entry from which to start the search.

scope

(IN) Specifies the scope of the search and uses one of the following flags:

- ◆ LDAP_SCOPE_BASE (0x00)—searches the entry specified by the base parameter.
- ◆ LDAP_SCOPE_ONELEVEL (0x01)—searches the immediate subordinates of the entry specified by the base parameter.

- ♦ `LDAP_SCOPE_SUBTREE (0x02)`—searches the entire subtree starting with the entry specified by the base parameter.

filter

(IN) Points to a search filter.

If `NULL` is passed, a default filter ("`objectclass=*`") is used, a filter which matches all entries in the directory. Using a `NULL` filter is not recommended for subtree searches on trees that potentially have hundreds of thousands of entries.

Simple filters take the form of strings: attribute name=attribute value. For more complex filters, see “Using Search Filters” on page 35.

attrs

(IN) Points to a `NULL`-terminated array of strings indicating which attributes to return with each matching entry. To return only entry names (and no attributes), set the first, and only string in the array, to `LDAP_NO_ATTRS`. To return all attributes, set this parameter to `NULL`.

For example, to return the `cn`, `surname`, and `givenName` attributes, declare `attrs` as: **`char* attrs[]={ "cn", "surname", "givenName", NULL};`**

attrsonly

(IN) Specifies whether to return just attributes or attributes and values.

- ♦ Zero—return both attributes and values
- ♦ Non-zero—return only attributes

timeout

(IN) Points to a `timeval` structure that specifies the maximum time to wait for the results of a search to complete. The structure specifies both the time the server waits for the operation to complete as well as the time the local function waits for the server to respond. If the `timeout` parameter is set to `NULL`, the client timeout is infinite and the server uses the timeout value stored in the session handle option, `LDAP_OPT_TIMELIMIT` (whose default value is no timeout). For more information about possible values, see `timeval` (page 418).

res

(OUT) Returns a pointer to an array of result messages if the search succeeds or `NULL` if no results are returned.

Return Values

0x00	LDAP_SUCCESS
Non-zero	Failure. For a complete list, see LDAP Return Codes.
0x53	LDAP_ENCODING_ERROR
0x55	LDAP_TIMEOUT
0x57	LDAP_FILTER_ERROR

Remarks

The LDAP_OPT_DEREF option in the LDAP session handle affects how aliases are handled during the search.

- ♦ The LDAP_DEREF_FINDING value means aliases are dereferenced when locating the base object but not during the search.
- ♦ The LDAP_DEREF_SEARCHING value means aliases are dereferenced during the search but not when locating the base object of the search.

The results in the res parameter are opaque to the caller. You must call ldap_parse_result to read the results.

See Also

ldap_search (page 239), ldap_search_ext (page 242), ldap_search_ext_s (page 246), ldap_search_s (page 250)

ldap_set_lderrno

Sets error information in an LDAP structure. This function has been deprecated; use the `ldap_set_option` function.

LDAP Version: v2 or higher

Library: *ldapsdk.*

NDS Version: 7.xx or higher

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldap.h>

int    ldap_set_lderrno (
    LDAP    *ld,
    int      errnum,
    char     *matchedDN,
    char     *errmsg);
```

Parameters

`ld`

(IN) Points to the handle for the LDAP session.

`errnum`

(IN) Specifies the LDAP error number to set.

`matchedDN`

(IN) Points to the name of the lowest entry in the directory that was matched on the search operation. May be NULL.

`errmsg`

(IN) Points to a text string that contains information from the LDAP server about this error. May be NULL.

Return Values

Always returns `LDAP_SUCCESS`.

Remarks

The `ldap_set_lderrno` function can be used to add or modify information about an error in an LDAP handle. This information can be retrieved in a subsequent call to the `ldap_get_lderrno` function.

The LDAP libraries make a copy of the string before storing it in the LDAP handle, so you do not need to preserve the original string after the call.

NOTE: This is not a standard IETF function. It has been added for compatibility with other LDAP vendors' libraries and should not be used in new applications. Use the `ldap_set_option` function with `LDAP_OPT_ERR_NUMBER`, `LDAP_OPT_MATCHED_DN`, and `LDAP_OPT_ERROR_STRING`.

See Also

`ldap_get_lderrno` (page 162), `ldap_set_option` (page 258)

ldap_set_option

Sets the value of session-wide parameters.

Library: *ldapsdk.*

NDS Version: 7.xx or higher

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldap.h>

int ldap_set_option (
    LDAP          *ld,
    int           option,
    LDAP_CONST void *invalue);
```

Parameters

- ld
(IN) Points to the session handle. If this is NULL, the function accesses the global defaults.
- option
(IN) Specifies the name of the option which is being set (see “Session Preference Options” on page 392).
- invalue
(IN) Points to the value to which the specified *option* is set.

Return Values

0x00	LDAP_SUCCESS
-1	Failure

Remarks

The `ldap_init` function returns the value for the `ld` parameter. If you use the `ldap_set_option` function before calling `ldap_init` and use `NULL` for the `ld` parameter, the values are set globally and copied to all LDAP session handles you create afterwards. If the `ldap_set_option` function is called after the `ldap_init` function, one of the following occurs:

- ♦ If the `ld` parameter is `NULL`, the values are set globally but do not affect the values in currently created LDAP session handles.
- ♦ If the `ld` parameter is set to the value returned by the `ldap_init` function, the values are set for only that LDAP session handle.

The following examples illustrate how to globally set two of the options.

```
/* Don't chase referrals */
rc = ldap_set_option( NULL, LDAP_OPT_REFERRALS, LDAP_OPT_OFF );

/* Set LDAP version 3 */
int version = LDAP_VERSION3;
rc = ldap_set_option( NULL, LDAP_OPT_PROTOCOL_VERSION, &version );
```

See Also

`ldap_get_option` (page 164)

ldap_set_rebind_proc

Sets the process that is used to bind when following referrals.

Library: *ldapsdk.*

NDS Version: 7.xx or higher

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldap.h>

int ldap_set_rebind_proc (
    LDAP      *ld,
    LDAP_REBIND_PROC  *ldap_proc);
```

Parameters

- ld
(IN) Points to the session handle. If this is NULL, the function sets the rebind process globally.
- ldap_proc
(IN) Specifies the rebind function.

Return Values

0x00	LDAP_SUCCESS
Non-zero	Failure. For a complete list, see LDAP Return Codes.

Remarks

The ldap_set_rebind_proc function sets the process to use for binding when an operation returns a referral. The LDAP_OPT_REFERRALS option in the ld must be set to ON for the libraries to use the rebind function. Use the ldap_set_option function to set the value.

The `rebind` function has the following syntax.

```
int LIBCALL rebind_function (
    LDAP      *ld,
    const char *url,
    int        request,
    ber_int_t   msgid)
{
    /* the body must perform a synchronous bind */
}
```

The `ld` parameter must be used by the application when binding to the referred server if the application wants the libraries to follow the referral.

The `url` parameter points to the URL referral string received from the LDAP server. The LDAP application can use the `ldap_url_parse` function to parse the string into its components.

The `request` parameter specifies the request operation that generated the referral. For possible values, see “Request Message Types” on page 389.

The `msgid` parameter specifies the message ID of the request generating the referral.

The LDAP libraries set all the parameters when they call the `rebind` function. The application should not attempt to free either the `ld` or the `url` structures in the `rebind` function.

Your application must supply to the `rebind` function the required authentication information such as user name, password, and certificates. The `rebind` function must use a synchronous bind method.

- ♦ If an anonymous bind is sufficient for your application, then you do not need to provide a `rebind` process. The LDAP libraries with the `LDAP_OPT_REFERRALS` option set to ON (default value) will automatically follow referrals using an anonymous bind.
- ♦ If your application needs stronger authentication than an anonymous bind, you need to provide a `rebind` process for that authentication method. The bind method must be synchronous.

For sample code, see `rebind.c` (http://developer.novell.com/ndk/doc/samplecode/cldap_sample/index.htm).

See Also

`ldap_set_option` (page 258), `ldap_url_parse` (page 276)

ldap_simple_bind

Asynchronously authenticates an entry to the directory.

LDAP Version: v2 or higher

Library: *Idapsdk.*

NDS Version: 7.xx or higher

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldap.h>

int ldap_simple_bind (
    LDAP      *ld,
    char      *dn,
    char      *passwd);
```

Parameters

ld

(IN) Points to the handle for the LDAP session.

dn

(IN) Points to the name of the entry to use for authentication. For an anonymous authentication, set this parameter to NULL.

passwd

(IN) Points to the entry's password which will be compared to the entry's userPassword attribute. For an anonymous authentication, set this parameter to NULL.

Return Values

>0	Message ID of operation
-1	Failure

Remarks

To obtain the results of the operation, call the `ldap_result` function using the message ID returned by the `ldap_simple_bind` function.

If the function returns a -1, use the `ldap_get_option` function with the option parameter set to `LDAP_OPT_RESULT_CODE` to retrieve the error code from the LDAP session handle.

By default, NDS does not accept clear text passwords. Make sure that the parameter for encrypted passwords is set to allow unencrypted passwords.

An anonymous bind to an NDS directory allows clients to access whatever the [Public] user has been granted access to. By default, this is just enough to allow the user to find an NDS server, match a distinguished name, and authenticate.

See Also

`ldap_bind` (page 94), `ldap_simple_bind_s` (page 265), `ldap_unbind`, `ldap_unbind_s` (page 273), `ldap_unbind_ext`, `ldap_unbind_ext_s` (page 274)

ldap_simple_bind_s

Synchronously authenticates the specified client to the LDAP server using a distinguished name and password.

LDAP Version: v2 or higher

Library: *ldapsdk.*

NDS Version: 7.xx or higher

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldap.h>

int ldap_simple_bind_s (
    LDAP      *ld,
    const char *dn,
    const char *passwd);
```

Parameters

ld

(IN) Points to the handle for the LDAP session.

dn

(IN) Points to the distinguished name of the entry who is authenticating. For an anonymous authentication, set this parameter to NULL.

passwd

(IN) Points to the client's password. For an anonymous authentication, set this parameter to NULL.

Return Values

0x00	LDAP_SUCCESS
Non-zero	Failure. For a complete list, see LDAP Return Codes.

0x54	LDAP_DECODING_ERROR
0x59	LDAP_PARAM_ERROR
0x5A	LDAP_NO_MEMORY
0x5C	LDAP_NOT_SUPPORTED

Remarks

By default, NDS does not accept clear text passwords. Make sure that the parameter for encrypted passwords is set to allow unencrypted passwords.

An anonymous bind to an NDS directory allows clients to access whatever the [Public] user has been granted access to. By default, this is just enough to allow the user to find an NDS server, match a distinguished name, and authenticate.

For sample code, see `bind.c` (http://developer.novell.com/ndk/doc/samplecode/cldap_sample/index.htm).

See Also

`ldap_simple_bind` (page 263), `ldap_unbind`, `ldap_unbind_s` (page 273), `ldap_bind` (page 94), `ldap_bind_s` (page 96)

ldap_sort_entries

Sorts a chain of entries, returned by an LDAP search operation, using either the entries' DN or a specified attribute.

LDAP Version: v2 or higher

Library: *ldapsdk.*

NDS Version: 7.xx or higher

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldap.h>

int ldap_sort_entries (
    LDAP          *ld,
    LDAPMessage   **res,
    char          *attr,
    int           (*cmp) (const void *, const void *));
```

Parameters

ld

(IN) Points to the handle of the LDAP session.

res

(IN) Points to an LDAPMessage containing the results returned by the ldap_result or ldap_search_s function.

attr

(IN) Points to name of the attribute to use for sorting. Pass in NULL to sort by distinguished name.

cmp

(IN) Points to a function to use for sorting. This function returns an int and has two void pointers for parameters.

Return Values

0	LDAP_SUCCESS
-1	Failure.

Remarks

If the function returns failure, use `ldap_get_option` (page 164) to check the `LDAP_OPT_RESULT_CODE` option in the LDAP handle for the error code.

See Also

`ldap_result` (page 230), `ldap_search_s` (page 250), `ldap_sort_strcasecmp` (page 269)

ldap_sort_strcasecmp

Compares two strings, ignoring any differences in upper and lower case characters between the strings.

LDAP Version: v2 or higher

Library: *ldapsdk.*

NDS Version: 7.xx or higher

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldap.h>

int ldap_sort_strcasecmp (
    const void    *a,
    const void    *b);
```

Parameters

a

(IN) Points to the address of the pointer for first string to use in the compare.

b

(IN) Points to the address of the pointer for second string to use in the compare.

Return Values

0	String a is equal to string b.
>0	String a is greater than string b.
<0	String a is less than string b.

See Also

`ldap_sort_values` (page 271), `ldap_sort_entries` (page 267)

ldap_sort_values

Sorts an array of values retrieved from an ldap_get_values function.

LDAP Version: v2 or higher

Library: *ldapsdk.*

NDS Version: 7.xx or higher

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldap.h>

int ldap_sort_values (
    LDAP      *ld,
    char      **vals,
    int        (*cmp) (const void *, const void *));
```

Parameters

- ld
(IN) Points to the handle of the LDAP session.
- vals
(IN) Points to the array of values to sort.
- cmp
(IN) Points to the function to use for sorting. This function returns an int and has two void pointers for parameters. The ldap_sort_strcasecmp function can be used for this parameter to compare ASCII strings.

Return Values

0x00	LDAP_SUCCESS
Non-zero	Failure. See LDAP Return Codes.

See Also

`ldap_get_values` (page 166), `ldap_sort_strcasecmp` (page 269)

ldap_unbind, ldap_unbind_s

Unbinds from the directory, closes the connection, and frees resources associated with the session. Functionally, there are no differences between `ldap_unbind` and `ldap_unbind_s`.

LDAP Version: v2 or higher

Library: *ldapsdk.*

NDS Version: 7.xx or higher

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldap.h>

int ldap_unbind[_s] (
    LDAP *ld);
```

Parameters

`ld`

(IN) Points to the handle of the LDAP session that is to be unbound.

Return Values

0x00	LDAP_SUCCESS
Non-zero	Failure. See LDAP Return Codes.

Remarks

After the call to `ldap_unbind[_s]`, the session handle (`ld`) is invalid.

Note that there are no functional differences between the four unbind functions.

See Also

`ldap_unbind_ext`, `ldap_unbind_ext_s` (page 274)

ldap_unbind_ext, ldap_unbind_ext_s

Unbinds from the directory, closes the connection, and frees resources associated with the session. Functionally, there are no differences between ldap_unbind_ext and ldap_unbind_ext_s.

LDAP Version: v3

Library: *ldapsdk.*

NDS Version: 7.xx or higher

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldap.h>

int ldap_unbind_ext[_s] (
    LDAP          *ld,
    LDAPControl   **serverctrls,
    LDAPControl   **clientctrls);
```

Parameters

- ld
(IN) Points to the handle of the LDAP session that is to be unbound.
- serverctrls
(IN) Points to a list of server controls. Use NULL to specify no server controls.
- clientctrls
(IN) Points to a list of client controls. Use NULL to specify no client controls.

Return Values

0x00	LDAP_SUCCESS
Non-zero	Failure. See LDAP Return Codes.

Remarks

After the call to `ldap_unbind_ext[_s]`, the session handle (`ld`) is invalid.

`Ldap_unbind_ext` allows controls to be specified with the operation. NDS does not currently support any server-side controls to use with an unbind operation.

Note that there are no functional differences between the four unbind functions.

See Also

`ldap_unbind`, `ldap_unbind_s` (page 273)

ldap_url_parse

Parses the specified URL into its components.

LDAP Version: v3

Library: *ldapsdk.*

NDS Version: 7.xx or higher

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldap.h>

int ldap_url_parse (
    const char      *url,
    LDAPURLDesc     **ludpp) ;
```

Parameters

- url
(IN) Points to the URL that you want to parse.
- ludpp
(OUT) Points to an LDAPURLDesc structure that contains the components of the URL.

Return Values

This function does not return a standard LDAP error code. It returns one of the following:

0x00	LDAP_URL_SUCCESS
Non-zero	Failure
0x01	LDAP_URL_ERR_MEM—can't allocate memory space
0x02	LDAP_URL_ERR_PARAM—invalid parameter
0x03	LDAP_URL_ERR_NOTLDAP—URL doesn't begin with "ldap[s]://"

0x04	LDAP_URL_ERR_BADENCLOSURE—URL is missing trailing ">"
0x05	LDAP_URL_ERR_BADURL—invalid URL
0x06	LDAP_URL_ERR_BADHOST—host port is invalid
0x07	LDAP_URL_ERR_BADATTRS—invalid or missing attributes
0x08	LDAP_URL_ERR_BADSCOPE—invalid or missing scope string
0x09	LDAP_URL_ERR_BADFILTER—invalid or missing filter
0x0A	LDAP_URL_ERR_BADEXTS—invalid or missing extensions

Remarks

Since this function does not return a standard LDAP error code, you should not call `ldap_err2string` to parse the return code.

An LDAP URL has the following format:

```
ldap[s]://<hostname>:<port>/<base_dn>?<attributes>?<scope>?<filter>?<extensions>
```

<code>ldap://</code>	Specifies a clear-text connection.
<code>ldaps://</code>	Specifies an SSL connection
<code><port></code>	Specifies the port number. Defaults to zero if unspecified. Port 0 causes the appropriate port (389 for clear-text and 636 for SSL) to be selected when the connection is made.
<code><base_dn></code>	Specifies the distinguished name of an entry in the directory where the search begins. Defaults to an empty string which starts the search at the top level of the directory.
<code><attributes></code>	Specifies a comma-separated list of attributes to return. If missing, all attributes are returned.

<scope>	Specifies the scope of the search: - base—search just base entry - one—search the immediate subordinates of the base entry - sub—search the entire subtree of the base entry Defaults to base.
<filter>	Specifies a search filter. If empty, defaults to (objectclass=*).
<extensions>	Specifies a comma-separated list of extension in one of the following formats: [!]type=value [!]type Extensions prefixed with "!" are considered critical extensions.

The following examples illustrate this URL format:

- ♦ ldap://acme.com/ou=sales,o=acme?sn,telephoneNumber?sub?(objectclass=inetOrgPerson)?ext1=value1,ext2=value2
- ♦ ldaps://1.2.3.4:636/o=novell??one

See Also

ldap_free_urldesc (page 157)

ldap_url_search

Uses the specified URL to perform an asynchronous search operation.

LDAP Version: v3

Library: *ldapsdk.*

NDS Version: 7.xx or higher

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldap.h>

int ldap_url_search (
    LDAP          *ld,
    const char    *url,
    int           attrsonly);
```

Parameters

ld

(IN) Points to the handle for the LDAP session.

url

(IN) Points to the URL to use in the search operation.

attrsonly

(IN) Specifies whether attribute values are returned with the specified attributes.

- ♦ 0—return attributes and values
- ♦ 1—return only attributes

Return Values

Returns the message ID of the search operation.

Remarks

To check the results of the operation, use the `ldap_result` or the `ldap_result2error` function.

See Also

`ldap_free_urldesc` (page 157), `ldap_url_search_s` (page 281),
`ldap_url_search_st` (page 283)

ldap_url_search_s

Uses the specified URL to perform a synchronous search operation.

LDAP Version: v3

Library: *ldapsdk.*

NDS Version: 7.xx or higher

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldap.h>

int ldap_url_search_s (
    LDAP          *ld,
    const char    *url,
    int           attrsonly,
    LDAPMessage   **res);
```

Parameters

ld

(IN) Points to the handle for the LDAP session.

url

(IN) Points to the URL to use in the search operation.

attrsonly

(IN) Specifies whether attribute values are returned with the specified attributes.

- ♦ 0—return attributes and values
- ♦ 1—return only attributes

res

(OUT) Returns a pointer to an array of result messages if the search succeeds or NULL if no results are returned.

Return Values

0x00	LDAP_SUCCESS
Non-zero	Failure. For a complete list, see LDAP Return Codes.

See Also

ldap_free_urldesc (page 157), ldap_url_search (page 279), ldap_url_search_st (page 283)

LDAPMessage (page 405)

ldap_url_search_st

Uses the specified URL to perform a synchronous search operation that includes a specified time limit.

LDAP Version: v3

Library: *ldapsdk.*

NDS Version: 7.xx or higher

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldap.h>

int ldap_url_search_st (
    LDAP          *ld,
    const char    *url,
    int            attrsonly,
    struct timeval *timeout,
    LDAPMessage   **res);
```

Parameters

ld

(IN) Points to the handle for the LDAP session.

url

(IN) Points to the URL to use in the search operation.

attrsonly

(IN) Specifies whether attribute values are returned with the specified attributes.

- ♦ 0—return attributes and values
- ♦ 1—return only attributes

timeout

(IN) Points to a timeval structure that specifies the maximum time to wait for the results of a search to complete. It specifies both the time the server waits for the operation to complete as well as the time the local function

waits for the server to respond. If the timeout parameter is set to NULL, the client timeout is infinite and the server uses the timeout value stored in the session handle option, LDAP_OPT_TIMELIMIT (whose default value is no timeout). For more information about possible values, see timeval (page 418).

res

(OUT) Returns a pointer to an array of result messages if the search succeeds or NULL if no results are returned.

Return Values

Returns the message ID of the search operation.

Remarks

To check the results of the operation, use the ldap_result or the ldap_result2error function.

See Also

ldap_free_urldesc (page 157), ldap_url_search (page 279), ldap_url_search_s (page 281)

timeval (page 418), LDAPMessage (page 405)

ldap_value_free

Frees the memory allocated for an array of string values.

LDAP Version: v2 or higher

Library: *ldapsdk.*

NDS Version: 7.xx or higher

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldap.h>

void ldap_value_free (
    char    **vals);
```

Parameters

vals

(IN) Points to the array of values returned by the ldap_get_values function.

Remarks

The memory for each value is freed as well as the array.

If NULL is passed for the vals parameter, this function does nothing.

See Also

ldap_get_values (page 166), ldap_count_values (page 115),
ldap_value_free_len (page 286)

ldap_value_free_len

Frees the memory allocated for an array of berval structures.

LDAP Version: v2 or higher

Library: *Idapsdk.*

NDS Version: 7.xx or higher

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldap.h>

void ldap_value_free_len (
    struct berval    **vals);
```

Parameters

vals

(IN) Points to the array of values returned by the ldap_get_values_len function.

Remarks

The memory for each berval structure is freed as well as the array.

If NULL is passed for the vals parameter, this function does nothing.

See Also

ldap_get_values_len (page 168), ldap_count_values_len (page 117),
ldap_value_free (page 285)

ldapssl_client_init

Initializes the SSL (Secure Socket Layer) library.

LDAP Version: v3

Library: *ldapssl.*

NDS Version: 7.xx or higher

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldap_ssl.h>

int ldapssl_client_init (
    const char    *certFile,
    void          *reserved);
```

Parameters

certFile

(IN) Points to the trusted root certificate file, a fully-qualified file path and the file must contain a DER encoded certificate.

reserved

(IN) Not currently used. Pass a NULL.

Return Values

0	Success
-1	Failure

Remarks

The LDAP SSL library provides SSL server authentication. In order to verify the server, the library needs to be configured with a trusted root certificate.

The `certFile` parameter is the fully qualified path of a file containing a trusted root certificate DER encoded.

It is also possible to pass `NULL` in the `certFile` parameter and use `ldapssl_add_trusted_cert` to add trusted root certificates to the LDAP SSL library. The API `ldapssl_add_trusted_cert` accepts DER and B64 (PEM) encoded certificates.

If the SSL handshake fails, the LDAP library returns an `LDAP_SERVER_DOWN` error. The handshake can fail because the server is down or because SSL has not been set up correctly on the client or LDAP server.

When you are finished with the SSL library, you should call the `ldapssl_client_deinit` function.

For sample code, see `sslbind.c` (http://developer.novell.com/ndk/doc/samplecode/cldap_sample/index.htm).

See Also

`ldapssl_client_deinit` (page 289), `ldapssl_init` (page 290),
`ldapssl_install_routines` (page 292)

ldapssl_client_deinit

Deinitializes the SSL library.

LDAP Version: v3

Library: *ldapssl.*

NDS Version: 7.xx or higher

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldap_ssl.h>

int ldapssl_client_deinit (
    void);
```

Return Values

0x00	LDAP_SUCCESS
Non-zero	Failure. See LDAP Return Codes.

Remarks

This function must be called after you are finished using the SSL library. Before calling this function, all SSL LDAP session handles must be closed using the ldap_unbind function.

For sample code, see sslbind.c (http://developer.novell.com/ndk/doc/samplecode/cldap_sample/index.htm).

See Also

ldap_unbind, ldap_unbind_s (page 273), ldapssl_client_init (page 287)

ldapssl_init

Creates an LDAP session handle that is SSL enabled.

LDAP Version: v3

Library: *ldapssl.*

NDS Version: 7.xx or higher

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldap_ssl.h>

LDAP * ldapssl_init (
    const char    *host,
    int           port,
    int           secure);
```

Parameters

host

(IN) Contains the names of the available hosts, each separated by a space, or a list of IP addresses (in dot format) of the hosts, each separated by a space. If a port number is included with the name or the address, it is separated from them with a colon (:).

port

(IN) Contains the TCP port number to connect to, which for an SSL connection is the SSL port number of the LDAP server. If a port number is included with the *host* parameter, this parameter is ignored.

secure

(IN) Specifies whether the connection is established over SSL.

- ◆ Zero—do not establish the connection over SSL (which makes this function essentially the same as the `ldap_init` function)
- ◆ Non-zero—establish the connection over SSL

Return Values

>0	Pointer to an LDAP structure
-1	Failure

Remarks

Before calling this function, you must first call the `ldapssl_client_init` function which initializes the SSL library.

Calling the `ldapssl_init` function is equivalent to calling the `ldap_init` function followed by the `ldapssl_install_routines` function.

For sample code, see `sslbind.c` (http://developer.novell.com/ndk/doc/samplecode/cldap_sample/index.htm).

See Also

`ldapssl_client_init` (page 287)

ldapssl_install_routines

Enables an existing, but new, LDAP session handle for SSL.

LDAP Version: v3

Library: *ldapssl.*

NDS Version: 7.xx or higher

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldap_ssl.h>

int ldapssl_install_routines (
    LDAP      *ld);
```

Parameters

ld
(IN) Points to the handle of the LDAP session.

Return Values

0	Success
-1	Failure

Remarks

To use this function, you must call the following LDAP function in the specified order:

- ♦ ldapssl_client_init which initializes the SSL library
- ♦ ldap_init which creates the session handle
- ♦ ldapssl_install_routines which enables the session handle for SSL

Behavior is unpredictable when other LDAP functions are called between the ldap_init function and the ldapssl_install_routines function.

The preferred method is to use the `ldapssl_init` function.

See Also

`ldapssl_client_init` (page 287), `ldapssl_init` (page 290), `ldap_init` (page 170)

ldapssl_add_trusted_cert

Adds certificates to the list of trusted certificates.

LDAP Version: v3

Library: *ldapssl.*

NDS Version: 7.xx or higher

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldap_ssl.h>

int ldapssl_add_trusted_cert (
    void    *cert,
    int     type);
```

Parameters

cert

(IN) Points to the trusted root certificate to add.

type

(IN) Certificate type. This must be one of the following values:

- ♦ LDAPSSL_CERT_FILETYPE_B64
- ♦ LDAPSSL_CERT_FILETYPE_DER
- ♦ LDAPSSL_CERT_BUFFTYPE_B64
- ♦ LDAPSSL_CERT_BUFFTYPE_DER

Return Values

0	Success
-1	Failure

Remarks

This function can be called repeatedly to build a group of trusted certificates. It supports certificates encoded as DER and B64 (PEM) formats.

When one of the "FILETYPE" types is specified (see the type parameter), the cert parameter must be a pointer to a character array containing the fully qualified filename of the file containing the certificate. When one of the "BUFFTYPE" types are specified, the cert parameter must be a pointer an LDAPSSL_Cert (page 411) structure.

For sample code, see sslbind.c, sslbind_interactive.c (http://developer.novell.com/ndk/doc/samplecode/cldap_sample/index.htm).

See Also

ldapssl_client_init (page 287), ldapssl_client_deinit (page 289), ldapssl_init (page 290), ldapssl_install_routines (page 292)

ldapssl_get_cert

Returns a certificate encoded in the requested format.

LDAP Version: v3

Library: *ldapssl.*

NDS Version: 7.xx or higher

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldap_ssl.h>

int ldapssl_get_cert (
    void          *certHandle,
    int           type,
    LDAPSSL_Cert  *cert);
```

Parameters

- certHandle
- (IN) certificate handle received by a verify callback function.
- type
- (IN) Desired certificate encoding. This must be one of the following values:
- ◆ LDAPSSL_CERT_BUFFTYPE_B64
 - ◆ LDAPSSL_CERT_BUFFTYPE_DER
- cert
- (I/O) Pointer to an LDAPSSL_Cert (page 411) structure.

Return Values

0	Success
-1	Failure

Remarks

Applications use `ldapssl_get_cert` to retrieve the certificate from the certificate handle passed to the `ldapssl_set_verify_callback` (page 304) function.

The `certHandle` parameter is the certificate handle (`void *`) received by the `verify` callback routine.

An `LDAPSSL_Cert` (page 411) structure contains two elements, `length` and `data`. The `data` element is a pointer to a buffer allocated by the application and `length` is the size of the buffer. To determine the correct size for the buffer, applications can pass in an `LDAPSSL_Cert` structure with the `data` element set to `NULL` and the `length` element will be updated with the appropriate size. The appropriate memory can then be allocated and `ldapssl_get_cert` can be called again with the `LDAPSSL_Cert` data element set to the allocated memory.

Applications can use `ldapssl_get_cert` to retrieve the certificate information as a buffer and use it as desired. One possibility is to add it to the list of trusted certificates using `ldapssl_add_trusted_cert` (page 294). After adding the certificate to the list of trusted certificates, the `verify` callback routine will no longer be called if the certificate is received when establishing future SSL connections.

For sample code, see `sslbind_interactive.c` (http://developer.novell.com/ndk/doc/samplecode/cldap_sample/index.htm).

See Also

`ldapssl_set_verify_callback` (page 304), `ldapssl_add_trusted_cert` (page 294)

ldapssl_get_cert_attribute

Returns requested certificate information.

LDAP Version: v3

Library: *ldapssl.*

NDS Version: 7.xx or higher

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldap_ssl.h>

int ldapssl_get_cert_attribute (
    void          *certHandle,
    unsigned long  attrID,
    void          *value,
    int            *length);
```

Parameters

- certHandle
(IN) Certificate handle received by a verify callback function.
- attrID
(IN) Certificate information to return. See Table 5, “Certificate Attribute IDs,” on page 383.
- value
(OUT) Pointer to memory appropriate for the information requested.
- length
(I/O) Pointer to length of value parameter memory.

Return Values

0	Success
---	---------

Remarks

This function is used to query information about a server certificate received by the verify callback routine.

The certHandle parameter is the certificate handle (void *) received by the verify callback routine.

The attrID parameter specifies the information to retrieve, and the value parameter points to memory appropriate for the information. For specific attrID(s) and data types see Table 5, “Certificate Attribute IDs,” on page 383.

The length parameter is both an input and an output. On input, length is the size of the memory pointed to by the value parameter. On output, length is updated to reflect the actual size of the information copied.

In order to allocate memory, applications can pass in a NULL for the value parameter and the length parameter will be updated with the appropriate size, but no data will be copied. Applications can then allocate the appropriate memory and call ldapssl_get_cert_attribute again to retrieve the information.

For sample code, see sslbind_interactive.c (http://developer.novell.com/ndk/doc/samplecode/cldap_sample/index.htm).

See Also

ldapssl_set_verify_callback (page 304)

ldapssl_set_verify_mode

Sets the server certificate verification mode used when establishing an SSL connection.

LDAP Version: v3

Library: *ldapssl.*

NDS Version: 7.xx or higher

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldap_ssl.h>

int ldapssl_set_verify_mode (
    int mode);
```

Parameters

- mode
- (IN) Server certificate verify mode. This must be set to one of the following values:
- ♦ LDAPSSL_VERIFY_SERVER
 - ♦ LDAPSSL_VERIFY_NONE

Return Values

0	Success
-1	Failure

Remarks

The default mode is server verification (LDAPSSL_VERIFY_SERVER). Turning off server verification (LDAPSSL_VERIFY_NONE), means any LDAP server will be considered trusted and should only be used in controlled

environments where encrypted communication between servers and clients is desired, but server verification is not necessary.

For sample code, see `ssl_any.c` (http://developer.novell.com/ndk/doc/samplecode/cldap_sample/index.htm).

See Also

`ldapssl_get_verify_mode` (page 302)

ldapssl_get_verify_mode

Returns the current server certificate verification mode that is used when establishing an SSL connection.

LDAP Version: v3

Library: *ldapssl.*

NDS Version: 7.xx or higher

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldap_ssl.h>

int ldapssl_get_verify_mode (
    int *mode);
```

Parameters

mode

(OUT) Current server certificate verify mode. This will be one of the following values:

- ♦ LDAPSSL_VERIFY_SERVER
- ♦ LDAPSSL_VERIFY_NONE

Return Values

0	Success
-1	Failure

Remarks

The default mode is server verification (LDAPSSL_VERIFY_SERVER). Turning off server verification (LDAPSSL_VERIFY_NONE), means any LDAP server will be considered trusted and should only be used in controlled

environments where encrypted communication between servers and clients is desired, but server verification is not necessary.

For sample code, see `ssl_any.c` (http://developer.novell.com/ndk/doc/samplecode/cldap_sample/index.htm).

See Also

`ldapssl_set_verify_mode` (page 300)

ldapssl_set_verify_callback

Sets the routine to be called during SSL connection establishment if the server certificate received is not trusted.

LDAP Version: v3

Library: *ldapssl.*

NDS Version: 7.xx or higher

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldap_ssl.h>

int ldapssl_set_verify_callback (
    int (LIBCALL *certVerifyFunc)(void*) );
```

Parameters

certVerifyFunc

(IN) Callback routine, called during SSL connection establishment if the server certificate received is not trusted.

Return Values

0	Success
-1	Failure

Remarks

The certVerifyFunc must be a pointer to a function that takes one parameter (a void *) and returns an int.

If an untrusted server certificate is received while establishing an SSL connection, the callback routine is called with a handle to the certificate (void*).

This handle can be passed into `ldapssl_get_cert_attribute` (page 298) to query specific certificate information.

In order to accept the server certificate and continue the SSL connection, the callback routine should return `LDAPSSL_CERT_ACCEPT`. To reject the server certificate and abort the connection the callback routine should return `LDAPSSL_CERT_REJECT`.

For sample code, see `sslbind_interactive.c` (http://developer.novell.com/ndk/doc/samplecode/cldap_sample/index.htm).

See Also

`ldapssl_get_cert_attribute` (page 298), `ldapssl_get_cert` (page 296)

4

LDAP Extension Functions

This chapter describes the Novell LDAP extensions for naming contexts and replicas. These extensions allow access to the following directory features:

- ♦ Naming contexts: split, join, get number of entries, abort operation
- ♦ Replicas: add, remove, change type, list on server, return information
- ♦ Replica synchronization: to a specified server, to all replicas, at a specified time
- ♦ Schema synchronization
- ♦ Get effective NDS rights for attributes
- ♦ Get DN of logged in caller
- ♦ Restart the LDAP server

All of the naming context and replica functions are synchronous functions. If the naming context or replica is in a state that makes the requested operation possible, NDS responds to the client with a successful completion code. NDS then completes the operation in the background since some operations on large trees can take hours to complete. Clients can check on the status of the operation by calling the `ldap_get_replica_info` function.

All of these functions require LDAP extensions on the LDAP server.

NOTE: LDAP distinguished names are UTF-8 encoded.

ldap_abort_naming_context_operation

Aborts the last partition operation on the specified partition.

LDAP Version: v3

Library: *ldapx.*

NDS Version: 8.5

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldapx.h>

int ldap_abort_naming_context_operation (
    LDAP      *ld,
    char      *dn,
    int       flags);
```

Parameters

- ld
(IN) Points to the handle for the LDAP session.
- dn
(IN) Points to the distinguished name, in LDAP format, of partition whose current operation should be aborted.
- flags
(IN) Set to zero. Not currently used.

Return Values

0x00	LDAP_SUCCESS or no partition operation is pending.
0x01	LDAP_OPERATIONS_ERROR: A string is returned with this error code that indicates the source of the error.
0x53	LDAP_ENCODING_ERROR

0x5A	LDAP_NO_MEMORY
0x5C	LDAP_NOT_SUPPORTED
Non-zero	Non-zero values indicate errors. See LDAP Return Codes.

Remarks

In NDS, partition operations include

- ♦ Adding, changing, and removing replicas
- ♦ Joining and splitting partitions (creating and merging naming contexts)

At any given time, only one partition operation is pending. If a partition operation is not pending when this function is called, the function returns LDAP_SUCCESS.

For sample code, see abortpo.c (http://developer.novell.com/ndk/doc/samplecode/cldap_sample/index.htm).

Packet Format

The requestName is set to the OID (2.16.840.1.113719.1.27.100.29) and the requestValue is a BER encoding of the following:

```
RequestBer
  flags      INTEGER
  dn         LDAPDN
```

The responseName is set to the OID (2.16.840.1.113719.1.27.100.30) and there is no responseValue.

```
ResponseBer
  NULL
```

See Also

ldap_add_replica (page 310)
 ldap_change_replica_type (page 313)
 ldap_create_naming_context (page 316)
 ldap_merge_naming_contexts (page 333)
 ldap_remove_replica (page 346)

ldap_add_replica

Adds a replica to the specified directory server.

LDAP Version: v3

Library: *ldapx.*

NDS Version: 8.5

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldapx.h>

int ldap_add_replica (
    LDAP          *ld,
    char          *dn,
    char          *serverDN,
    LDAP_REPLICA_TYPE replicaType,
    int           flags);
```

Parameters

ld

(IN) Points to the handle for the LDAP session.

dn

(IN) Points to the distinguished name, in LDAP format, of the replica's partition root.

serverDN

(IN) Points to the distinguished name, in LDAP format, of the server on which a new replica is to be added.

replicaType

(IN) Specifies the type of replica to add (see “Replica Types” on page 388).

flags

(IN) Specifies whether all the servers in the replica ring must be up before proceeding. When set to zero, the status of the servers is not checked. When set to LDAP_ENSURE_SERVERS_UP, all the servers must be up for the operation to proceed.

Return Values

0x00	LDAP_SUCCESS
0x01	LDAP_OPERATIONS_ERROR: A string is returned with this error code that indicates the source of the error.
0x53	LDAP_ENCODING_ERROR
0x5A	LDAP_NO_MEMORY
0x5C	LDAP_NOT_SUPPORTED
Non-zero	Non-zero values indicate errors. See LDAP Return Codes.

Remarks

This operation is performed on the server with the master replica of the replica that is being added.

The caller must have supervisor rights to the master replica.

For sample code, see addrepl.c (http://developer.novell.com/ndk/doc/samplecode/cldap_sample/index.htm).

Packet Format

The requestName is set to the OID (2.16.840.1.113719.1.27.100.7) and the requestValue is a BER encoding of the following:

```
RequestBer
  flags      INTEGER
  replicaType INTEGER
  serverName LDAPDN
  dn         LDAPDN
```

The responseName is set to the OID (2.16.840.1.113719.1.27.100.8) and there is no responseValue.

```
ResponseBer
  NULL
```

See Also

[ldap_change_replica_type](#) (page 313)

[ldap_remove_replica](#) (page 346)

[ldap_abort_naming_context_operation](#) (page 308)

ldap_change_replica_type

Changes the type of the specified replica on the specified directory server.

LDAP Version: v3

Library: *ldapx.*

NDS Version: 8.5

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldapx.h>

int ldap_change_replica_type (
    LDAP          *ld,
    char          *dn,
    char          *serverDN,
    LDAP_REPLICA_TYPE  replicaType,
    int           flags);
```

Parameters

ld

(IN) Points to the handle for the LDAP session.

dn

(IN) Points to the distinguished name, in LDAP format, of the replica's partition root.

serverDN

(IN) Points to the distinguished name, in LDAP format, of the server on which the replica resides.

replicaType

(IN) Specifies the new type for the replica (see “Replica Types” on page 388).

flags

(IN) Specifies whether all the servers in the replica ring must be up before proceeding. When set to zero, the status of the servers is not checked. When set to LDAP_ENSURE_SERVERS_UP, all the servers must be up for the operation to proceed.

Return Values

0x00	LDAP_SUCCESS
0x01	LDAP_OPERATIONS_ERROR: A string is returned with this error code that indicates the source of the error.
0x53	LDAP_ENCODING_ERROR
0x5A	LDAP_NO_MEMORY
0x5C	LDAP_NOT_SUPPORTED
Non-zero	Non-zero values indicate errors. See LDAP Return Codes.

Remarks

This operation is performed on the server with the master replica of the replica that is being changed.

The replica type of the master replica cannot be changed by directly calling this function. The master's replica type can only be changed as a side effect of using this function to change another replica to the master replica. When this happens, the old master automatically becomes a secondary replica.

The caller must have supervisor rights to the master replica.

For sample code, see chgrepl.c (http://developer.novell.com/ndk/doc/samplecode/cldap_sample/index.htm).

Packet Format

The requestName is set to the OID (2.16.840.1.113719.1.27.100.15) and the requestValue is a BER encoding of the following:

```
RequestBer
    flags          INTEGER
    replicaType    INTEGER
    serverName     LDAPDN
    dn             LDAPDN
```

The responseName is set to the OID (2.16.840.1.113719.1.27.100.16) and there is no responseValue.

```
ResponseBer
    NULL
```

See Also

[ldap_add_replica](#) (page 310)

[ldap_remove_replica](#) (page 346)

[ldap_abort_naming_context_operation](#) (page 308)

ldap_create_naming_context

Creates a new LDAP naming context.

LDAP Version: v3

Library: *ldapx.*

NDS Version: 8.5

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldapx.h>

int ldap_create_naming_context (
    LDAP      *ld,
    char      *dn,
    int       flags);
```

Parameters

ld

(IN) Points to the handle for the LDAP session.

dn

(IN) Specifies the distinguished name, in LDAP format, of the child container where the new naming context is created.

flags

(IN) Specifies whether all the servers in the replica ring must be up before proceeding. When set to zero, the status of the servers is not checked. When set to LDAP_ENSURE_SERVERS_UP, all the servers must be up for the operation to proceed.

Return Values

0x00

LDAP_SUCCESS

0x01	LDAP_OPERATIONS_ERROR: A string is returned with this error code that indicates the source of the error.
0x53	LDAP_ENCODING_ERROR
0x5A	LDAP_NO_MEMORY
0x5C	LDAP_NOT_SUPPORTED
Non-zero	Non-zero values indicate errors. See LDAP Return Codes.

Remarks

In NDS terminology, creating a naming context splits a partition into a parent partition and a child partition at the child container specified in the call.

This operation is performed on the server with the master replica of the parent replica. The server with the parent's master replica must be running or this operation fails.

The caller must have supervisor rights to the parent's master replica.

For sample code, see `splitpart.c` (http://developer.novell.com/ndk/doc/samplecode/cldap_sample/index.htm).

Packet Format

The `requestName` is set to the OID (2.16.840.1.113719.1.27.100.3) and the `requestValue` is a BER encoding of the following:

```
RequestBer
  flags      INTEGER
  dn         LDAPDN
```

The `responseName` is set to the OID (2.16.840.1.113719.1.27.100.4) and there is no `responseValue`.

```
ResponseBer
  NULL
```

See Also

`ldap_merge_naming_contexts` (page 333)

ldap_create_orphan_naming_context

Creates an orphan naming context on the specified server.

LDAP Version: v3

Library: *ldapx.*

NDS Version: 8.5

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldapx.h>

int ldap_create_orphan_naming_context (
    LDAP      *ld,
    char      *serverDN,
    char      *contextName) ;
```

Parameters

ld

(IN) Points to the handle for the LDAP session.

serverDN

(IN) Points to the distinguished name, in LDAP format, of the server that will hold the orphan naming context.

contextName

(IN) Points to the distinguished name for the naming context, for example, "dc=test, dc=germany, dc=provo, dc=novell, dc=com, t=dns".

Return Values

0x00	LDAP_SUCCESS
0x53	LDAP_ENCODING_ERROR
0x5A	LDAP_NO_MEMORY

0x5C	LDAP_NOT_SUPPORTED
Non-zero	Non-zero values indicate errors. See LDAP Return Codes.

Remarks

In NDS terminology, creating an orphan naming context creates an orphan partition.

Packet Format

The requestName is set to the OID (2.16.840.1.113719.1.27.100.39) and the requestValue is a BER encoding of the following:

```
RequestBer
  serverDN      LDAPDN
  contextName   LDAPDN
```

The responseName is set to the OID (2.16.840.1.113719.1.27.100.40) and there is no responseValue.

```
ResponseBer
  NULL
```

See Also

[ldap_create_naming_context](#) (page 316)

[ldap_abort_naming_context_operation](#) (page 308)

ldap_get_context_identity_name

Returns the distinguished name of the client associated with the LDAP connection.

LDAP Version: v3

Library: *ldapx.*

NDS Version: 8.5

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldapx.h>

int ldap_get_context_identity_name (
    LDAP      *ld,
    char      **identity);
```

Parameters

ld

(IN) Points to the handle for the LDAP session.

identity

(OUT) Points to the distinguished name, in LDAP format, of the client.

Return Values

0x00	LDAP_SUCCESS
0x01	LDAP_OPERATIONS_ERROR: A string is returned with this error code that indicates the source of the error.
0x53	LDAP_ENCODING_ERROR
0x5A	LDAP_NO_MEMORY
0x5C	LDAP_NOT_SUPPORTED

Remarks

If the connection is not authenticated and is using an anonymous bind, the function returns an empty string.

The function allocates the memory for the identity parameter, and the caller is responsible for freeing it with the `ldapx_memfree` function.

The first field in the structure contains the length of the name, and the second field contains the name.

For sample code, see `getidname.c` (http://developer.novell.com/ndk/doc/samplecode/cldap_sample/index.htm).

Packet Format

The `requestName` is set to the OID (2.16.840.1.113719.1.27.100.31) and the `requestValue` has no value.

```
RequestBer
    NULL
```

The `responseName` is set to the OID (2.16.840.1.113719.1.27.100.32) and the `responseValue` is a BER encoding of the following:

```
ResponseBer
    identity    OCTET STRING
```

ldap_get_effective_privileges

Returns the effective rights of the specified entry to the specified attribute.

LDAP Version: v3

Library: *ldapx.*

NDS Version: 8.5

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldapx.h>

int ldap_get_effective_privileges (
    LDAP      *ld,
    char       *dn,
    char       *trusteeDN,
    char       *attrName,
    int        *privileges);
```

Parameters

ld

(IN) Points to the handle for the LDAP session.

dn

(IN) Points to the distinguished name, in LDAP format, of the entry that contains the attribute in question.

trusteeDN

(IN) Points to the distinguished name, in LDAP format, of the trustee whose rights are being returned, or you can specify [Public] or [Self].

attrName

(IN) Points to attribute whose rights are being returned or you can specify [Entry Rights] or [All Attribute Rights].

privileges

(OUT) Points to bitmask of the trustee's effective rights (see "Object Access Control Rights" on page 381 and "Attribute Access Control Rights" on page 382).

Return Values

0x00	LDAP_SUCCESS
0x01	LDAP_OPERATIONS_ERROR: A string is returned with this error code that indicates the source of the error.
0x53	LDAP_ENCODING_ERROR
0x5A	LDAP_NO_MEMORY
0x5C	LDAP_NOT_SUPPORTED
Non-zero	Non-zero values indicate errors. See LDAP Return Codes.

Remarks

To understand the difference between the *dn* and the *trusteeDN* arguments, suppose that an entry named Kim has a telephone number attribute, and a client named Tom wants to know if he has rights to the attribute. In this case,

- ♦ *dn* points to the distinguished name of Kim
- ♦ *trusteeDN* points to the distinguished name of Tom
- ♦ *attrName* points to Telephone Number
- ♦ *privileges* points to the rights Tom has to Kim's Telephone Number attribute

For sample code, see `getpriv.c` (http://developer.novell.com/ndk/doc/samplecode/cldap_sample/index.htm).

Packet Format

The `requestName` is set to the OID (2.16.840.1.113719.1.27.100.33) and the `requestValue` is a BER encoding of the following:

```
RequestBer
  dn          LDAPDN
  trusteeDN   LDAPDN
  attrName    OCTET STRING
```

The responseName is set to the OID (2.16.840.1.113719.1.27.100.34) and the responseValue is a BER encoding of the following:

```
ResponseBer
  privileges  INTEGER
```

ldap_get_replication_filter

Gets the replication filter defined for an NDS server.

LDAP Version: v3

Library: *ldapx.*

NDS Version: 8.5, SP1

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldapx.h>

int ldap_get_replication_filter (
    LDAP      *ld,
    char      *serverDN,
    char      **filter);
```

Parameters

- ld
(IN) Points to the handle for the LDAP session.
- serverDN
(IN) Points to the distinguished name of the server from which to read the replication filter.
- filter
(OUT) Points to the address of the filter. For the format, see “Replication Filters” on page 387. The returned filter must be freed by calling the ldapx_memfree function.

Return Values

0x00	LDAP_SUCCESS
0x01	LDAP_OPERATIONS_ERROR: A string is returned with this error code that indicates the source of the error.

0x53	LDAP_ENCODING_ERROR
0x59	LDAP_PARAM_ERROR
0x5A	LDAP_NO_MEMORY
0x5C	LDAP_NOT_SUPPORTED
Non-zero	Non-zero values indicate errors. See LDAP Return Codes.

Remarks

An NDS server can only have one replication filter, and that filter is used for all filtered replicas on the server.

NDS eDirectory 8.5 and above supports filtered replicas. The extension to get and set the replication filter is not available until NDS 8.5 SP1.

Packet Format

The requestName is set to the OID (2.16.840.1.113719.1.27.100.37) and the requestValue is a BER encoding of the following:

```
RequestBer
    serverName    LDAPDN
```

The responseName is set to the OID (2.16.840.1.113719.1.27.100.38) and the responseValue is a BER encoding of the following:

```
ResponseBer
    SEQUENCE of SEQUENCE {
        classname    OCTET STRING
        SEQUENCE of ATTRIBUTES
    }
where
    ATTRIBUTES:: OCTET STRING
```

See Also

[ldap_set_replication_filter](#) (page 356)

ldap_get_replica_info

Returns information about the specified replica on the specified directory server.

LDAP Version: v3

Library: *ldapx.*

NDS Version: 8.5

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldapx.h>

int ldap_get_replica_info (
    LDAP          *ld,
    char          *dn,
    char          *serverDN,
    struct LDAPReplicaInfo *partitionInfo);
```

Parameters

ld

(IN) Points to the handle for the LDAP session.

dn

(IN) Points to the distinguished name, in LDAP format, of the replica from which information is being requested.

serverDN

(IN) Points to the distinguished name, in LDAP format, of the server containing the replica.

partitionInfo

(OUT) Points to a LDAPReplicaInfo (page 408) structure that returns with the replica information.

Return Values

0x00	LDAP_SUCCESS
0x01	LDAP_OPERATIONS_ERROR: A string is returned with this error code that indicates the source of the error.
0x53	LDAP_ENCODING_ERROR
0x5A	LDAP_NO_MEMORY
0x5C	LDAP_NOT_SUPPORTED
Non-zero	Non-zero values indicate errors. See LDAP Return Codes.

Remarks

The specified server holding the replica must be running or an error is returned.

For sample code, see `getrinfo.c` (http://developer.novell.com/ndk/doc/samplecode/cldap_sample/index.htm).

Packet Format

The `requestName` is set to the OID (2.16.840.1.113719.1.27.100.17) and the `requestValue` is a BER encoding of the following:

```
RequestBer
  serverName      LDAPDN
  dn              LDAPDN
```

The `responseName` is set to the OID (2.16.840.1.113719.1.27.100.18) and the `responseValue` is a BER encoding of the following:

```
ResponseBer
  partitionID      INTEGER
  replicaState     INTEGER
  modificationTime INTEGER
  purgeTime        INTEGER
  localPartitionID INTEGER
  partitionDN      LDAPDN
  replicaType      INTEGER
```

flags

INTEGER

See Also

[ldap_list_replicas](#) (page 331)

ldap_list_replicas

Lists all the replicas on the specified directory server.

LDAP Version: v3

Library: *ldapx.*

NDS Version: 8.5

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldapx.h>

int ldap_list_replicas (
    LDAP      *ld,
    char      *serverDN,
    char      ***replicaList);
```

Parameters

ld

(IN) Points to the handle for the LDAP session.

serverDN

(IN) Points to the distinguished name, in LDAP format, of the server whose replicas are being listed.

replicaList

(OUT) Points to a list of replicas.

Return Values

0x00	LDAP_SUCCESS
0x01	LDAP_OPERATIONS_ERROR: A string is returned with this error code that indicates the source of the error.
0x53	LDAP_ENCODING_ERROR

0x5A	LDAP_NO_MEMORY
0x5C	LDAP_NOT_SUPPORTED
Non-zero	Non-zero values indicate errors. See LDAP Return Codes.

Remarks

The function allocates the memory for replicaList, but the caller is responsible for freeing the memory with the ldapx_memfree function.

This function returns all replicas including subordinate references. The replicaList argument points to a null terminated array of strings. Each string in the array contains the distinguished name of a replica's partition root.

For sample code, see listrepl.c (http://developer.novell.com/ndk/doc/samplecode/cldap_sample/index.htm).

Packet Format

The requestName is set to the OID (2.16.840.1.113719.1.27.100.19) and the requestValue is a BER encoding of the following:

```
RequestBer
    serverDN      LDAPDN
```

The responseName is set to the OID (2.16.840.1.113719.1.27.100.20) and the responseValue is a BER encoding of the following:

```
ResponseBer
    replicaList    SEQUENCE OF OCTET STRINGS
```

See Also

ldap_get_replica_info (page 328)

ldap_merge_naming_contexts

Joins a parent and child naming context.

LDAP Version: v3

Library: *ldapx.*

NDS Version: 8.5

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldapx.h>

int ldap_merge_naming_contexts (
    LDAP      *ld,
    char      *dn,
    int       flags);
```

Parameters

ld

(IN) Points to the handle for the LDAP session.

dn

(IN) Specifies the distinguished name, in LDAP format, of the child naming context's root that is to be joined to its parent.

flags

(IN) Specifies whether all the servers in the replica ring must be up before proceeding. When set to zero, the status of the servers is not checked. When set to LDAP_ENSURE_SERVERS_UP, all the servers must be up for the operation to proceed.

Return Values

0x00	LDAP_SUCCESS
------	--------------

0x01	LDAP_OPERATIONS_ERROR: A string is returned with this error code that indicates the source of the error.
0x53	LDAP_ENCODING_ERROR
0x5A	LDAP_NO_MEMORY
0x5C	LDAP_NOT_SUPPORTED
Non-zero	Non-zero values indicate errors. See LDAP Return Codes.

Remarks

This operation is performed on the server containing the master replica of the parent naming context. The caller must have supervisor rights to the child's master replica and the parent's master replica.

For sample code, see `joinpart.c` (http://developer.novell.com/ndk/doc/samplecode/cldap_sample/index.htm).

Packet Format

The `requestName` is set to the OID (2.16.840.1.113719.1.27.100.5) and the `requestValue` is a BER encoding of the following:

```
RequestBer
  flags      INTEGER
  dn         LDAPDN
```

The `responseName` is set to the OID (2.16.840.1.113719.1.27.100.6) and the `requestValue` has no value.

```
ResponseBer
  NULL
```

See Also

`ldap_create_naming_context` (page 316)
`ldap_abort_naming_context_operation` (page 308)

ldap_naming_context_entry_count

Returns the number of entries in the specified naming context.

LDAP Version: v3

Library: *ldapx.*

NDS Version: 8.5

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldapx.h>

int ldap_naming_context_entry_count (
    LDAP          *ld,
    char          *dn,
    unsigned long  *count);
```

Parameters

ld

(IN) Points to the handle for the LDAP session.

dn

(IN) Points to the distinguished name, in LDAP format, of an entry in the naming context whose entries are to be counted.

count

(OUT) Points to the address where the count is returned.

Return Values

0x00	LDAP_SUCCESS
0x01	LDAP_OPERATIONS_ERROR: A string is returned with this error code that indicates the source of the error.
0x53	LDAP_ENCODING_ERROR

0x5A	LDAP_NO_MEMORY
0x5C	LDAP_NOT_SUPPORTED
Non-zero	Non-zero values indicate errors. See LDAP Return Codes.

Remarks

This function stops at the boundary of the naming context, or in NDS terms, at the partition boundary. It does not cross the boundary and count the entries in child naming contexts or partitions.

If this function is called immediately after creating a new naming context, the count will be inaccurate until the naming context moves from the new state (LDAP_RS_NEW_REPLICA) to the on state (LDAP_RS_ON).

For sample code, see `getcount.c` (http://developer.novell.com/ndk/doc/samplecode/cldap_sample/index.htm).

Packet Format

The `requestName` is set to the OID (2.16.840.1.113719.1.27.100.13) and the `requestValue` is a BER encoding of the following:

```
RequestBer
  dn      LDAPDN
```

The `responseName` is set to the OID (2.16.840.1.113719.1.27.100.14) and the `responseValue` is a BER encoding of the following:

```
ResponseBer
  count   INTEGER
```

See Also

`ldap_get_replica_info` (page 328)

ldap_nds_to_ldap_name

Converts a typeless, distinguished NDS name into LDAP format.

LDAP Version: v3

Library: *ldapx.*

NDS Version: 8.5

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldapx.h>

int ldap_nds_to_ldap_name (
    LDAP          *ld,
    unsigned short *ndsName,
    char          **ldapName);
```

Parameters

ld

(IN) Points to the handle for the LDAP session.

ndsName

(IN) Points to the NDS distinguished name in typeless, dotted format that includes the tree name (for example: ksmith.provo.novell.novell_inc). The string must be a Unicode string. If the object is in a DNS rooted tree, com.dns must be included as the tree name (for example: ksmith.provo.novell.com.dns).

ldapName

(OUT) Points to the entry's distinguished name in LDAP format, for example, "cn=ksmith, ou=provo, o=novell"

Return Values

0x00	LDAP_SUCCESS
------	--------------

0x01	LDAP_OPERATIONS_ERROR: A string is returned with this error code that indicates the source of the error.
0x53	LDAP_ENCODING_ERROR
0x5A	LDAP_NO_MEMORY
0x5C	LDAP_NOT_SUPPORTED
Non-zero	Non-zero values indicate errors. See LDAP Return Codes.

Remarks

This function is provided for legacy NDS applications and utilities that expect the entry names to be entered in NDS typeless, dotted format.

It is provided for convenience, but should be used sparingly or the application's performance will be affected. This is not a local function. The function makes a call to the LDAP server to find the types.

The function allocates memory for the `ldapName` parameter, and the caller is responsible for freeing it with the `ldapx_memfree` function.

Packet Format

The `requestName` is set to the OID (2.16.840.1.113719.1.27.100.1) and the `requestValue` is a BER encoding of the following:

```
RequestBer
  dn          LDAPDN
```

The `responseName` is set to the OID (2.16.840.1.113719.1.27.100.2) and the `responseValue` is a BER encoding of the following:

```
ResponseBer
  ldapName    OCTET STRING
```

ldap_receive_all_updates

Requests that a specified replica on a specified server receive all updates.

LDAP Version: v3

Library: *ldapx.*

NDS Version: 8.5

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldapx.h>

int ldap_receive_all_updates (
    LDAP      *ld,
    char      *partitionRoot,
    char      *toServerDN,
    char      *fromServerDN);
```

Parameters

ld

(IN) Points to the handle for the LDAP session.

partitionRoot

(IN) Points to the distinguished name, in LDAP format, of the replica that receives the updates.

toServerDN

(IN) Points to the distinguished name, in LDAP format, of the server holding the replica to be updated.

fromServerDN

(IN) Points to distinguished name, in LDAP format, of the server from which the updates are sent. Not currently used.

Return Values

0x00	LDAP_SUCCESS
0x01	LDAP_OPERATIONS_ERROR: A string is returned with this error code that indicates the source of the error.
0x53	LDAP_ENCODING_ERROR
0x5A	LDAP_NO_MEMORY
0x5C	LDAP_NOT_SUPPORTED
Non-zero	Non-zero values indicate errors. See LDAP Return Codes.

Remarks

A replica’s distinguished name is the distinguished name of the replica’s root container, called the partition root in NDS.

In NDS 7.x and above, updates can come from any server that holds a replica of the partition; therefore, NDS does not currently use the fromServerDN parameter to specify which server should send the updates.

Each ld is associated with a particular server. NDS uses the ld rather than the toServerDN parameter to specify the server with the replica that needs updating.

For sample code, see recvupd.c (http://developer.novell.com/ndk/doc/samplecode/cldap_sample/index.htm).

Packet Format

The requestName is set to the OID (2.16.840.1.113719.1.27.100.21) and the requestValue is a BER encoding of the following:

```
RequestBer
  partitionRoot    LDAPDN
  toServerDN       LDAPDN
  fromServerDN     LDAPDN
```

The responseName is set to the OID (2.16.840.1.113719.1.27.100.22) and the responseValue has no value.

ResponseBer
NULL

See Also

[ldap_send_all_updates](#) (page 354)

ldap_refresh_server

Restarts the LDAP server associated with the specified session handle.

LDAP Version: v3

Library: *ldapx.*

NDS Version: 8.5

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldapx.h>

int ldap_refresh_server (
    LDAP *ld);
```

Parameters

ld
(IN) Points to the handle for the LDAP session.

Return Values

0x00	LDAP_SUCCESS
0x01	LDAP_OPERATIONS_ERROR: A string is returned with this error code that indicates the source of the error.
0x32	LDAP_INSUFFICIENT_ACCESS
Non-zero	Non-zero values indicate errors. See LDAP Return Codes.

Remarks

Each ld is associated with a particular LDAP server. By specifying the ld, you specify the LDAP server that is restarted, and you use the authentication credentials established for that ld.

To restart the LDAP server, the client must have write permissions to the extensionInfo attribute of the LDAP server object.

Packet Format

The requestName is set to the OID (2.16.840.1.113719.1.27.100.9) and the requestValue has no value:

```
RequestBer
  NULL
```

The responseName is set to the OID (2.16.840.1.113719.1.27.100.10) and the responseValue has no value:.

```
ResponseBer
  NULL
```

ldap_remove_orphan_naming_context

Removes the specified orphan naming context from the specified server.

LDAP Version: v3

Library: *ldapx.*

NDS Version: 8.5

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldapx.h>

int ldap_remove_orphan_naming_context (
    LDAP      *ld,
    char      *serverDN,
    char      *contextName) ;
```

Parameters

- ld
(IN) Points to the handle for the LDAP session.
- serverDN
(IN) Points to the distinguished name of the server holding the orphan naming context to remove.
- contextName
(IN) Points to the distinguished name of the orphan naming context to remove.

Return Values

0x00	LDAP_SUCCESS
0x53	LDAP_ENCODING_ERROR

0x5A	LDAP_NO_MEMORY
0x5C	LDAP_NOT_SUPPORTED
Non-zero	Non-zero values indicate errors. See LDAP Return Codes.

Remarks

This function fails if the server does not hold the specified naming context.

Packet Format

The requestName is set to the OID (2.16.840.1.113719.1.27.100.41) and the requestValue is a BER encoding of the following:

```
RequestBer
    serverDN      LDAPDN
    contextName   LDAPDN
```

The responseName is set to the OID (2.16.840.1.113719.1.27.100.42) and the responseValue has no value.

```
ResponseBer
    NULL
```

See Also

[ldap_create_orphan_naming_context](#) (page 319)
[ldap_abort_naming_context_operation](#) (page 308)

ldap_remove_replica

Removes a replica from the specified directory server.

LDAP Version: v3

Library: *ldapx.*

NDS Version: 8.5

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldapx.h>

int ldap_remove_replica (
    LDAP    *ld,
    char     *dn,
    char     *serverDN,
    int      flags);
```

Parameters

ld

(IN) Points to the handle for the LDAP session.

dn

(IN) Points to the distinguished name, in LDAP format, of the replica to be removed.

serverDN

(IN) Points to the distinguished name, in LDAP format, of the server holding the replica to be removed.

flags

(IN) Specifies whether all the servers in the replica ring must be up before proceeding. When set to zero, the status of the servers is not checked. When set to LDAP_ENSURE_SERVERS_UP, all the servers must be up for the operation to proceed.

Return Values

0x00	LDAP_SUCCESS
0x01	LDAP_OPERATIONS_ERROR: A string is returned with this error code that indicates the source of the error.
0x53	LDAP_ENCODING_ERROR
0x5A	LDAP_NO_MEMORY
0x5C	LDAP_NOT_SUPPORTED
Non-zero	Non-zero values indicate errors. See LDAP Return Codes.

Remarks

A replica's distinguished name is the distinguished name of the replica's root container, called the partition root in NDS.

The caller must have supervisor rights to the master replica.

This function can remove all the replicas of a partition except the master replica. If the master replica is the only replica left, it is removed by merging the child partition with its parent partition.

For sample code, see `remrepl.c` (http://developer.novell.com/ndk/doc/samplecode/cldap_sample/index.htm).

Packet Format

The `requestName` is set to the OID (2.16.840.1.113719.1.27.100.11) and the `requestValue` is a BER encoding of the following:

```
RequestBer
  flags          INTEGER
  serverName     LDAPDN
  dn             LDAPDN
```

The `responseName` is set to the OID (2.16.840.1.113719.1.27.100.12) and the `responseValue` has no value.

ResponseBer
NULL

See Also

[ldap_add_replica](#) (page 310)
[ldap_abort_naming_context_operation](#) (page 308)
[ldap_merge_naming_contexts](#) (page 333)

ldap_request_naming_context_sync

Schedules synchronization of the specified partition after the specified delay.

LDAP Version: v3

Library: *ldapx.*

NDS Version: 8.5

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldapx.h>

int ldap_request_naming_context_sync (
    LDAP      *ld,
    char      *serverDN,
    char      *partitionRoot,
    int       delay);
```

Parameters

ld

(IN) Points to the handle for the LDAP session.

serverDN

(IN) Points to distinguished name, in LDAP format, of the server containing the partition.

partitionRoot

(IN) Points to the distinguished name, in LDAP format, of the partition root to synchronize.

delay

(IN) Specifies the time, in seconds, to delay before synchronization starts.

Return Values

0x00	LDAP_SUCCESS
0x01	LDAP_OPERATIONS_ERROR: A string is returned with this error code that indicates the source of the error.
0x53	LDAP_ENCODING_ERROR
0x5A	LDAP_NO_MEMORY
0x5C	LDAP_NOT_SUPPORTED
Non-zero	Non-zero values indicate errors. See LDAP Return Codes.

Remarks

In NDS, this function causes the server to examine its Transitive Vector attribute, and the server initiates synchronization with those servers whose replica update time is older than the local time stamp.

For sample code, see parsync.c (http://developer.novell.com/ndk/doc/samplecode/cldap_sample/index.htm).

Packet Format

The requestName is set to the OID (2.16.840.1.113719.1.27.100.25) and the requestValue is a BER encoding of the following:

```
RequestBer
  serverName      LDAPDN
  partitionRoot   LDAPDN
  delay           INTEGER
```

The responseName is set to the OID (2.16.840.1.113719.1.27.100.26) and the responseValue has no value.

```
ResponseBer
  NULL
```

See Also

`ldap_receive_all_updates` (page 339)

`ldap_send_all_updates` (page 354)

ldap_request_schema_sync

Schedules synchronization of the schema after the specified delay.

LDAP Version: v3

Library: *ldapx.*

NDS Version: 8.5

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldapx.h>

int ldap_request_schema_sync (
    LDAP      *ld,
    char      *serverDN,
    int        delay);
```

Parameters

- ld
(IN) Points to the handle for the LDAP session.
- serverDN
(IN) Points to the distinguished name, in LDAP format, of the server.
- delay
(IN) Specifies the time, in seconds, to delay before synchronization starts.

Return Values

0x00	LDAP_SUCCESS
0x01	LDAP_OPERATIONS_ERROR: A string is returned with this error code that indicates the source of the error.
0x53	LDAP_ENCODING_ERROR
0x5A	LDAP_NO_MEMORY

0x5C	LDAP_NOT_SUPPORTED
Non-zero	Non-zero values indicate errors. See LDAP Return Codes.

Remarks

For sample code, see `schsync.c` (http://developer.novell.com/ndk/doc/samplecode/cldap_sample/index.htm).

Packet Format

The `requestName` is set to the OID (2.16.840.1.113719.1.27.100.27) and the `requestValue` is a BER encoding of the following:

```
RequestBer
  serverName      LDAPDN
  delay           INTEGER
```

The `responseName` is set to the OID (2.16.840.1.113719.1.27.100.28) and the `responseValue` has no value.

```
ResponseBer
  NULL
```

See Also

`ldap_request_naming_context_sync` (page 349)

ldap_send_all_updates

Requests that a specified server send all updates to the replica ring.

LDAP Version: v3

Library: *ldapx.*

NDS Version: 8.5

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldapx.h>

int ldap_send_all_updates (
    LDAP      *ld,
    char      *partitionRoot,
    char      *origServerDN);
```

Parameters

- ld
(IN) Points to the handle for the LDAP session.
- partitionRoot
(IN) Points to the distinguished name, in LDAP format, of the replica that contains the updates.
- origServerDN
(IN) Points to the distinguished name, in LDAP format, of the server that sends the updates. Not currently used.

Return Values

0x00	LDAP_SUCCESS
0x01	LDAP_OPERATIONS_ERROR: A string is returned with this error code that indicates the source of the error.
0x53	LDAP_ENCODING_ERROR

0x5A	LDAP_NO_MEMORY
0x5C	LDAP_NOT_SUPPORTED
Non-zero	Non-zero values indicate errors. See LDAP Return Codes.

Remarks

A replica's distinguished name is the distinguished name of the replica's root container, called the partition root in NDS.

In NDS 7.x and higher, any server containing a replica can send updates. Since each ld has a server associated with it, NDS uses the ld to specify the originating server rather than the origServerDN parameter.

For sample code, see sendupd.c (http://developer.novell.com/ndk/doc/samplecode/cldap_sample/index.htm).

Packet Format

The requestName is set to the OID (2.16.840.1.113719.1.27.100.23) and the requestValue is a BER encoding of the following:

```
RequestBer
    partitionRoot    LDAPDN
    origServerDN     LDAPDN
```

The responseName is set to the OID (2.16.840.1.113719.1.27.100.24) and the responseValue has no value.

```
ResponseBer
    NULL
```

See Also

ldap_receive_all_updates (page 339)

ldap_set_replication_filter

Sets the attribute and class filter for an NDS filtered replica.

LDAP Version: v3

Library: *ldapx.*

NDS Version: 8.5, SP1

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldapx.h>

int ldap_set_replication_filter (
    LDAP      *ld,
    char      *serverDN,
    char      *filter);
```

Parameters

- ld
(IN) Points to the handle for the LDAP session.
- serverDN
(IN) Points to the distinguished name of the server that holds the filtered replicas.
- filter
(IN) Points to the filter that identifies the classes and attributes that are allowed in filtered replicas on the server. For the format, see “Replication Filters” on page 387.

Return Values

0x00	LDAP_SUCCESS
0x53	LDAP_ENCODING_ERROR

0x59	LDAP_PARAM_ERROR
0x5A	LDAP_NO_MEMORY
0x5C	LDAP_NOT_SUPPORTED
Non-zero	Non-zero values indicate errors. See LDAP Return Codes.

Remarks

An NDS server can only have one replication filter, and that filter is used for all filtered replicas on the server.

NDS eDirectory 8.5 and above supports filtered replicas. The extension to get and set the replication filter is not available until NDS 8.5 SP1.

Packet Format

The requestName is set to the OID (2.16.840.1.113719.1.27.100.35) and the requestValue is a BER encoding of the following:

```
RequestBer
  serverName      LDAPDN
  SEQUENCE of SEQUENCE {
    classname     OCTET STRING
    SEQUENCE of ATTRIBUTES
  }
```

```
where
  ATTRIBUTES:: OCTET STRING
```

The responseName is set to the OID (2.16.840.1.113719.1.27.100.36) and the responseValue has no value.

```
ResponseBer
  NULL
```

See Also

[ldap_get_replication_filter](#) (page 326)

ldap_trigger_back_process

Triggers a background process.

LDAP Version: v3

Library: *ldapx.*

NDS Version: 8.5, SP1

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldapx.h>

LIBLDAP_F(int) ldap_trigger_back_process (
    LDAP      *ld,
    int       processID);
```

Parameters

- ld
 - (IN) Points to the handle for the LDAP session.
- processID
 - (IN) Flag determining the background process to trigger. This flag will be one of the following:
 - LDAP_BK_PROCESS_BKLINKER 1
 - LDAP_BK_PROCESS_JANITOR 2
 - LDAP_BK_PROCESS_LIMBER 3
 - LDAP_BK_PROCESS_SKULKER 4
 - LDAP_BK_PROCESS_SCHEMA_SYNC 5
 - LDAP_BK_PROCESS_PART_PURGE 6

Return Values

0x00	LDAP_SUCCESS
0x53	LDAP_ENCODING_ERROR

0x59	LDAP_PARAM_ERROR
0x5A	LDAP_NO_MEMORY
0x5C	LDAP_NOT_SUPPORTED
Non-zero	Non-zero values indicate errors. See LDAP Return Codes.

Remarks

NDS background processes run automatically at periodic intervals to keep different replicas in an NDS tree synchronized. On rare occasions, it may be desirable to initiate one or more of these processes manually rather than waiting for the next scheduled execution.

Back Linker (Reference Checker). Keeps the "backlink" attribute of objects synchronized between servers. This attribute keeps track of external references to the object and also maintains DRLs (Dynamic Reference Links, "Used By" attribute).

Janitor. Cleans up bindery information, refreshes server status, and updates inherited ACLs.

Limber. Maintains tree connectivity, making sure tree names are consistent among servers. Also caches information from the NCPServer object and index information. After creating a new index, an application may want to kick off the limber process to cause it to start creating the index immediately.

Skulker. Replicates and synchronizes directory data among replicas. A developer may want to initiate this process to start data synchronization immediately.

Schema Sync. Replicates and synchronizes the schema.

Partition Purge. Purges deleted entries and deleted values that have been synchronized with all replicas.

ldapx_memfree

Frees memory allocated by the LDAP extension library.

LDAP Version: v3

Library: *ldapx.*

NDS Version: 8.5

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldapx.h>

void ldapx_memfree (
    void    *mem);
```

Parameters

mem

(IN) Points to the memory to free.

Remarks

This function should be used to free any memory allocated by the LDAP extension library.

The request and reply packets have the following formats.

See Also

[ldap_get_context_identity_name](#) (page 321)

[ldap_list_replicas](#) (page 331)

[ldap_nds_to_ldap_name](#) (page 337)

5

UTF-8 Functions

Directory data in the LDAPv3 API is sent and received in UTF-8 format. For a discussion of the relationship between UTF-8, local, multi-byte, wide character, and unicode, see “Understanding Character Conversions” on page 37.

To simplify the use of UTF-8 character sets, the LDAP SDK contains functions to provide developers a standard, cross-platform method to work with UTF-8 strings.

Functions to convert data between wide character and UTF-8 formats are grouped in the following categories:

- ♦ “UTF-8 / Wide Character Conversions” on page 361.

In addition, the LDAP SDK contains a number of utility functions for working with UTF-8 strings. They are contained in the following category:

- ♦ “UTF-8 Utility Functions” on page 369.

UTF-8 / Wide Character Conversions

The UTF-8 / wide conversion routines follow the pattern of the ANSI C conversion routines. These routines use the `wchar_t` type, which is two bytes on some systems and four bytes on others. The advantage to using the `wchar_t` type is that all the standard wide-character functions may be used on these strings, such as **wcslen**, **wscat**, etc.

ldap_x_utf8_to_wc

Convert a single UTF-8 encoded character to a wide character.

Library: *ldapsdk.*

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldap_utf8.h>

int ldap_x_utf8_to_wc (
    wchar_t      *wchar,
    const char   *utf8char);
```

Parameters

wchar

(OUT) Points to a wide character to receive the converted character code.

utf8char

(IN) Address of the UTF8 sequence of bytes.

Return Values

If successful, the function returns the length in bytes of the UTF-8 input character.

If *utf8char* is NULL or points to an empty string, the function returns 1 and a NULL is written to *wchar*.

If *utf8char* contains an invalid UTF-8 sequence -1 is returned.

Example

```
char utchr_in[] = { 0xE2U, 0x98U, 0xA0U };
wchar_t wc_out;
int n;

/* Convert a UTF-8 character to a wide character.
   Returns wc_out = 0x2620.
```

```
    Returns n = 3.  (Byte length of utchr_in)
*/
n = ldap_x_utf8_to_wc(&wc_out, utchr_in);
```

ldap_x_utf8s_to_wcs

Convert a UTF-8 string to a wide character string.

Library: *ldapsdk.*

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldap_utf8.h>

int ldap_x_utf8s_to_wcs (
    wchar_t      *wcstr,
    const char   *utf8str,
    size_t       count);
```

Parameters

wcstr

(OUT) Points to a wide char buffer to receive the converted wide char string. The output string will be null-terminated if there is space for it in the buffer.

utf8char

(IN) Address of the null-terminated UTF-8 string to convert.

count

(IN) The number of UTF-8 characters to convert, or equivalently, the size of the output buffer in wide characters.

Return Values

If successful, the function returns the number of wide characters written to *wcstr*, excluding the null termination character, if any.

If *wcstr* is NULL, the function returns the number of wide characters required to contain the converted string, excluding the null-termination character.

If an invalid UTF-8 sequence is encountered, the function returns -1.

If the return value equals count, there was not enough space to fit the string and the null terminator in the buffer. As much of the string as will fit is written to the buffer, but it is not null-terminated.

Example

```
#define WCSTRLEN    10
char utstr_in[] = { 0xE2U, 0x98U, 0xA0U, 'a', 'b', 'c', 0 };
wchar_t wctr_out[WCSTRLEN];
int n;
/* Convert a UTF-8 string to a wide char string.
   Returns wctr_out = { 0x2620, 'a', 'b', 'c', 0 }
   Returns n = 4.  ( # wide chars written, excl null )
   If n==WCSTRLEN, the string could not completely fit in the given buffer.
*/
n = ldap_x_utf8s_to_wcs(wctr_out, utstr_in, WCSTRLEN);
```

ldap_x_wc_to_utf8

Convert a single wide character to a UTF-8 sequence.

Library: *ldapsdk.*

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldap_utf8.h>

int ldap_x_wc_to_utf8 (
    char      *utf8char,
    wchar_t   wchar,
    size_t    count);
```

Parameters

utf8char

(OUT) Points to a byte array to receive the converted UTF-8 string.

wchar

(IN) The wide character to be converted.

count

(IN) The maximum number of bytes to write to the output buffer.

Normally set this to LDAP_MAX_UTF8_LEN, which is defined as 3 or 6 depending on the size of wchar_t. A partial character will not be written.

Return Values

If successful, the function returns the length in bytes of the converted UTF-8 output character.

If *wchar* is NULL, the function returns 1 and 0 is written to *utf8char*.

If *wchar* cannot be converted to a UTF-8 character, the function returns -1.

If the converted character will not fit in count bytes, 0 is returned.

Example

```
wchar_t wc_in = 0x2620;  
char utchr_out[LDAP_MAX_UTF8_LEN];    /* Either 3 or 6 bytes */  
int n;  
  
/* Convert a wide character to a UTF-8 character.  
   Returns utchr_out[] = { 0xE2, 0x98, 0xA0 }  
   Returns n = 3.  (Byte length of utchr_out)  
*/  
n = ldap_x_wc_to_utf8(utchr_out, wc_in, LDAP_MAX_UTF8_LEN);
```

ldap_x_wcs_to_utf8s

Convert a wide character string to a UTF-8 string.

Library: *ldapsdk.*

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldap_utf8.h>

int ldap_x_wcs_to_utf8s (
    char            *utf8str,
    const wchar_t   *wcstr,
    size_t          count);
```

Parameters

utf8str

(OUT) Points to a byte array to receive the converted UTF-8 string. The output string will be null terminated if there is space for it in the buffer.

wcstr

(IN) Address of the null-terminated wide char string to convert.

count

(IN) The size of the output buffer in bytes.

Return Values

If successful, the function returns the number of bytes written to *utf8str*, excluding the null termination character, if any.

If *utf8str* is NULL, the function returns the number of bytes required to contain the converted string, excluding the null-termination character. The *count* parameter is ignored.

If the function encounters a wide character that cannot be mapped to a UTF-8 sequence, the function returns -1.

If the return value equals *count*, there was not enough space to fit the string and the null terminator in the buffer. As much of the string as will fit is written to the buffer, but it is not null-terminated. A partial character will not be written.

Example

```
#define UTFSTRLEN    20
wchar_t wctr_in[] = { 0x2620, 'a', 'b', 'c', 0 };
char utstr_out[UTFSTRLEN];
int n;

/* Convert a wide char string to a UTF-8 string.
   Returns utstr = { 0xE2, 0x98, 0xA0, 'a', 'b', 'c', 0 }
   Returns n = 6. ( # bytes written, excl null )
   If n==UTFSTRLEN, the string could not completely fit in the given buffer.
*/
n = ldap_x_wcs_to_utf8s(utstr_out, wctr_in, UTFSTRLEN);
```

UTF-8 Utility Functions

The LDAP SDK contains a number of utility functions for working with UTF-8 strings. They are as follows:

- ♦ `ldap_x_utf8_chars` (page 370) Return the number of UTF-8 characters (not bytes) in a null-terminated UTF-8 string.
- ♦ `ldap_x_utf8_charlen` (page 371) Return the number of bytes in a UTF-8 character.
- ♦ `ldap_x_utf8_next` (page 372) Find the next character in a UTF-8 string.
- ♦ `ldap_x_utf8_prev` (page 373) Find the previous character in a UTF-8 string.
- ♦ `ldap_x_utf8_copy` (page 374) Copy one UTF-8 character.
- ♦ `ldap_x_utf8_strchr` (page 375) Find a character in a string.
- ♦ `ldap_x_utf8_strspn` (page 376) Find the first substring.
- ♦ `ldap_x_utf8_strcspn` (page 377) Find a substring in a string.
- ♦ `ldap_x_utf8_strpbrk` (page 378) Find first occurrence of a character from one string in another string.
- ♦ `ldap_x_utf8_strtok` (page 379) Find next token in string.

ldap_x_utf8_chars

Return the number of UTF-8 characters (not bytes) in a null-terminated UTF-8 string.

Library: *ldapsdk.*

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldap_utf8.h>

ber_len_t ldap_x_utf8_chars (
    const char *p);
```

Parameters

p

(IN) Contains the null-terminated UTF-8 string to count.

Return Values

Number of chars (not bytes) in *p*.

Example

```
/* String with 4 UTF-8 characters */
char utstr[] = { 0xe2U, 0x98U, 0xa0U, 'a', 'b', 'c', 0 };
int n = ldap_x_utf8_chars(utstr);    /* Returns 4 */
```

ldap_x_utf8_charlen

Return the number of bytes in a UTF-8 character.

Library: *ldapsdk.*

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldap_utf8.h>

int ldap_x_utf8_charlen (
    const char *p);
```

Parameters

p
(IN) Points to the UTF-8 character.

Return Values

Returns length in bytes of the UTF-8 character. (1-6). 0 is returned for an invalid character.

Remarks

The length of the character is determined by looking only at the first byte of the UTF-8 character.

Example

```
/* String starts with a 3-byte UTF-8 character. */
char utstr[] = { 0xe2U, 0x98U, 0xa0U, 'a', 'b', 'c', 0 };
int n = ldap_x_utf8_charlen(utstr);    /* Returns 3 */
n = ldap_x_utf8_charlen(utstr+4);    /* 'b' char. Returns 1 */
```

ldap_x_utf8_next

Find the next character in a UTF-8 string.

Library: *ldapsdk.*

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldap_utf8.h>

char* ldap_x_utf8_next (
    const char *p);
```

Parameters

p
(IN) Points to a UTF-8 string.

Return Values

Returns the address of the next UTF-8 character in the string.

Remarks

The function will step over NULLs just like any other character. The application must take care not to step beyond the end of the string.

Example

```
/* String starts with a 3-byte UTF-8 character. */
char utstr[] = { 0xe2U, 0x98U, 0xa0U, 'a', 'b', 'c', 0 };
char *p = ldap_x_utf8_next(utstr); /* p now points to the 'a' char */
p = ldap_x_utf8_next(p); /* p now points to the 'b' char */
```

ldap_x_utf8_prev

Find the previous character in a UTF-8 string.

Library: *ldapsdk.*

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldap_utf8.h>

char* ldap_x_utf8_prev (
    const char *p);
```

Parameters

p
(IN) Points to a UTF-8 string.

Return Values

Returns a pointer to the previous character in the string.

Remarks

The application must take care not to step beyond the beginning of the string.

Example

```
char utstr[] = { 0xe2U, 0x98U, 0xa0U, 'a', 'b', 'c', 0 };
char *p = ldap_x_utf8_prev(utstr+4);    /* p now points to the 'a' char */
p = ldap_x_utf8_prev(p);    /* p now points to the beginning char */
```

ldap_x_utf8_copy

Copy one UTF-8 character.

Library: *ldapsdk.*

NDS Version: 7.xx or higher

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldap_utf8.h>

int ldap_x_utf8_copy (
    char      *dst,
    const char *src);
```

Parameters

dst

(IN) Points to the output buffer.

src

(IN) Points to the UTF-8 character to copy.

Return Value

Number of bytes copied.

Example

```
char utstr[] = { 0xe2U, 0x98U, 0xa0U, 'a', 'b', 'c', 0 };
char dest[3];
int n = ldap_x_utf8_copy(dest, utstr); /* Copies 1st char. Returns 3. */
```

ldap_x_utf8_strchr

Find a character in a string.

Library: *ldapsdk.*

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldap_utf8.h>

char * ldap_x_utf8_strchr (
    const char *str,
    const char *chr);
```

Parameters

str

(IN) Null-terminated UTF-8 string to search.

chr

(IN) Points to the UTF-8 character to be located.

Return Values

Returns the first occurrence of *chr* in *str*, or NULL if not found.

Example

```
char utstr[] = { 'a', 'b', 0xe2U, 0x98U, 0xa0U, 'x', 'y', 0 };
char chr[] = { 0xe2U, 0x98U, 0xa0U };
char *p = ldap_x_utf8_strchr(utstr, chr);    /* Returns utstr+2 */
```

ldap_x_utf8_strspn

Find the first substring.

Library: *ldapsdk.*

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldap_utf8.h>

ber_len_t ldap_x_utf8_strspn (
    const char *str,
    const char *set);
```

Parameters

str

(IN) Null-terminated UTF-8 string to search.

set

(IN) Null-terminated character set.

Return Values

Returns the length of the substring in *str* that consists entirely of characters in *set*.

Remarks

This function returns the number of bytes, not characters.

Example

```
char utstr[] = { 'a', 'b', 0xe2U, 0x98U, 0xa0U, 'x', 'y', 0 };
char set[] = { 'b', 0xe2U, 0x98U, 0xa0U, 'a', 0 };
int n = ldap_x_utf8_strspn(utstr, set); /* Returns 5 */
```


ldap_x_utf8_strcspn

Find a substring in a string.

Library: *ldapsdk.*

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldap_utf8.h>

ber_len_t ldap_x_utf8_strcspn (
    const char *str,
    const char *set);
```

Parameters

str

(IN) Null-terminated UTF-8 string to search.

set

(IN) Null-terminated character set.

Return Values

Returns the length of the initial segment of *str* that consists entirely of characters not in *set*.

Remarks

This function returns the number of bytes, not characters.

Example

```
char utstr[] = { 'a', 'b', 0xe2U, 0x98U, 0xa0U, 'x', 'y', 0 };
char set[] = { 'x', 0xe2U, 0x98U, 0xa0U, 0 };
int n = ldap_x_utf8_strcspn(utstr, set);    /* Returns 2 */
```

ldap_x_utf8_strpbrk

Find first occurrence of a character from one string in another string.

Library: *ldapsdk.*

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldap_utf8.h>

char * ldap_x_utf8_strpbrk (
    const char *str,
    const char *set);
```

Parameters

str

(IN) Null-terminated UTF-8 string to search.

set

(IN) Null-terminated character set.

Return Values

Returns a pointer to the first occurrence of any character from *set* in *str*.

Example

```
char utstr[] = { 'a', 'b', 0xe2U, 0x98U, 0xa0U, 'x', 'y', 0 };
char set[] = { 'x', 0xe2U, 0x98U, 0xa0U, 0 };
char *p = ldap_x_utf8_strpbrk(utstr, set);    /* Returns utstr+2 */
```

ldap_x_utf8_strtok

Find next token in string.

Library: *ldapsdk.*

NDS Version: 7.xx or higher

Platform: NLM, Windows (NT, 95, 98, 2000), Solaris, Linux

Syntax

```
#include <ldap_utf8.h>

char * ldap_x_utf8_strtok (
    char      *str,
    const char *sep,
    char      **last);
```

Parameters

str

(IN) UTF-8 string to parse. On subsequent calls to this function this parameter should be NULL.

sep

(IN) Set of separator characters.

last

(IN/OUT) After each function call, returns a pointer to the token following the separator character. This value should be passed in to the next function call.

Return Values

Returns a pointer to the next token found in *str*.

Remarks

Parses a string into tokens based on a set of delimiters. When a delimiter is encountered, it is replaced by a NULL, allowing the token to be processed as a null-terminated string.

On the first call, a value is returned in *last*. On subsequent calls, NULL should be given for *str*. *Last* is updated after each call.

Example

```
char utstr[] = { 'a', 0xe2U, 0x98U, 0xa0U, 'b', 0xe2U, 0x98U, 0xa0U, 'c', 0 };
char delims[] = { 0xe2U, 0x98U, 0xa0U, 0 };
char *last;
char *p;

p = ldap_x_utf8_strtok(utstr, delims, &last);    /* p points to "a" */
p = ldap_x_utf8_strtok(NULL, delims, &last);    /* p points to "b" */
p = ldap_x_utf8_strtok(NULL, delims, &last);    /* p points to "c" */
```

6 Values

This chapter defines the flags used by the LDAP functions.

Object Access Control Rights

Table 4 Object Rights

Flag Name	C Value	Description
DS_ENTRY_BROWSE	0x00000001L	Allows a trustee to discover objects in the NDS tree.
DS_ENTRY_ADD	0x00000002L	Allows a trustee to create child objects (new objects that are subordinate to the object in the tree).
DS_ENTRY_DELETE	0x00000004L	Allows a trustee to delete an object. This right does not allow a trustee to delete a container object that has subordinate objects.
DS_ENTRY_RENAME	0x00000008L	Allows a trustee to rename the object.
DS_ENTRY_SUPERVISOR	0x00000010L	Gives a trustee all rights to an object and its attributes.
DS_ENTRY_INHERIT_CTL	0x00000040L	Allows a trustee to inherit the rights granted in the ACL and exercise them on subordinate objects.

Attribute Access Control Rights

Flag Name	C Value	Description
LDAP_DS_ATTR_COMPARE	0x00000001L	Allows a trustee to compare a value with an attribute's value. This allows the trustee to see if the attribute contains the value without having rights to see the value.
LDAP_DS_ATTR_READ	0x00000002L	Allows a trustee to read an attribute value. This right confers the Compare right.
LDAP_DS_ATTR_WRITE	0x00000004L	Allows a trustee to add, delete, or modify an attribute value. This right also gives the trustee the Self (Add or Delete Self) right.
LDAP_DS_ATTR_SELF	0x00000008L	Allows a trustee to add or delete its name as an attribute value on those attributes that take object names as their values.
LDAP_DS_ATTR_SUPERVISOR	0x00000020L	Gives a trustee all rights to the object's attributes.
LDAP_DS_ATTR_INHERIT_CTL	0x00000040L	Allows a trustee to inherit the rights granted in the ACL and exercise these attribute rights on subordinate objects.

Certificate Attribute IDs

Table 5 Certificate Attribute IDs

Attribute ID	Data Type	Description
LDAPSSL_CERT_ATTR_ISSUER	char *	A pointer to a character array containing the certificate issuer name. The issuer is the distinguished name of the certificate authority that issued the certificate. The length returned is the length of the string not including the NULL termination character.
LDAPSSL_CERT_ATTR_SUBJECT	char *	A pointer to a character array containing the certificate subject name. The subject is the distinguished name of the entity that owns the certificate. The length returned is the length of the string not including the NULL termination character.
LDAPSSL_CERT_ATTR_VALIDITY_PERIOD	LDAPSSL_Cert_VValidity_Period *	A pointer to a LDAPSSL_Cert_VValidity_Period structure. The validity period structure contains a not valid after and a not valid before timestamp which defines the period during which the certificate should be considered valid. The timestamps can be a universal time string or a generalized time string (see LDAPSSL_Cert_VValidity_Period (page 412)).

Inheritance Control Rights

The bit settings for the Inheritance Control rights use values that ensure compatibility with NetWare 4.x.

Table 6 Inheritance Control Settings

NetWare Version	Object Right DS_ENTRY_INHERIT_CTL	[All Attributes Rights] DS_ATTR_INHERIT_CTL	Specific Attribute DS_ATTR_INHERIT_CTL
NetWare 4.x	NetWare 4.x does not support this functionality. Inheritance of object rights is always supported. NetWare 4.x requires this bit to be set to 0.	NetWare 4.x does not support this functionality. Inheritance of rights to [All Attributes Rights] is always supported. NetWare 4.x requires this bit to be set to 0.	NetWare 4.x does not support this functionality. Inheritance of ACLs to specific attributes is always blocked. NetWare 4.x requires this bit to be set to 0.
NetWare 5.x	NetWare 5.x supports this right. Set this bit to 0 (zero) to allow the inheritance of the rights in the ACL. Set this bit to 1 (one) to block the inheritance of the ACL rights.	NetWare 5.x supports this right. Set this bit to 0 (zero) to allow the inheritance of the rights granted to [All Attributes Rights]. Set this bit to 1 (one) to block the inheritance of the ACL rights.	NetWare 5.x supports this right. Set this bit to 1 (one) to allow the inheritance of the rights granted to the specific attribute. Set this bit to 0 to block the inheritance of the ACL rights.

Replica States

The replica states indicate the current state of the replica. For more information, see *Replica Transition States (NDS Technical Overview)*.

NOTE: These values are not continuous.

Flag Name	C Value	Meaning
LDAP_RS_ON	0	Indicates that the replica is fully functioning and capable of responding to NDS™ requests.
LDAP_RS_NEW_REPLICA	1	Indicates that a new replica has been added but has not received a full download of information from ♦ The master replica if NDS 6.x and lower ♦ Another replica if NDS 7.x and higher
LDAP_RS_DYING_REPLICA	2	Indicates that a replica of the partition is being deleted. In NDS 6.x and lower, the replica stays in this state until it synchronizes with another replica. In NDS 7.x and higher, indicates that the request has been received.
LDAP_RS_LOCKED	3	Indicates that the replica is locked. The move partition operation uses this state to lock the parent partition of the child partition that is moving.
LDAP_RS_TRANSITION_ON	6	Indicates that a new replica has finished receiving its download from the master replica and is now receiving synchronization updates from the other replicas. Used only in NDS 6.x and lower.
LDAP_RS_DEAD_REPLICA	7	Indicates that the dying replica needs to synchronize with another replica before being converted to an external reference, if a root replica, or to a subordinate reference, if a nonroot replica. Used only in NDS 7.x and higher.
LDAP_RS_BEGIN_ADD	8	Indicates that subordinate references of the new replica are being added. Used only in NDS 7.x and higher.
LDAP_RS_MASTER_START	11	Indicates that a partition is receiving a new master replica. The replica that will be the new master replica is set to this state.

Flag Name	C Value	Meaning
LDAP_RS_MASTER_DONE	12	Indicates that a partition has a new master replica. When the new master is set to this state, it knows it is now the master and changes its replica type to master and the old master to Read/Write.
LDAP_RS_SS_0	48	Indicates that a partition is going to split into two partitions. In this state, other replicas of the partition are informed of the pending split operation.
LDAP_RS_SS_1	49	Indicates that the split partition operation has started. When the split is finished, the state will change to RS_ON.
LDAP_RS_JS_0	64	Indicates that two partitions are in the process of joining into one partition. In this state, the replicas that are affected are informed of the join operation. The master replica of the parent and child partitions are first set to this state and then all the replicas of the parent and child. New replicas are added where needed.
LDAP_RS_JS_1	65	Indicates that two partitions are in the process of joining into one partition. This state indicates that the join operation is waiting for the new replicas to synchronize and move to the RS_ON state.
LDAP_RS_JS_2	66	Indicates that two partitions are in the process of joining into one partition. This state indicates that all the new replicas are in the RS_ON state and that the rest of the work can be completed.

Replication Filters

NDS eDirectory 8.5 and above support filtered replicas. Previous versions of NDS do not support filtered replicas.

A single replication filter is set for an NDS server, and all replicas that reside on that specified server conform to that particular filter. The filter parameter (for the `ldap_get_replication_filter` and `ldap_set_replication_filter` functions) is a UTF string that comprises a sequence of object class names and attribute names delimited by the dollar (\$) sign. The filter follows these rules:

1. Each class name and each attribute name is terminated by a \$ sign.
2. Each sequence of a class with its attribute names is terminated by a \$ sign.
3. The filter is terminated with a \$ sign.

The following sample filter selects three attributes from the user class and one attribute from the `groupOfUniqueNames` class for the filter.

```
"user$cn$surname$mail$$groupOfUniqueNames$memeber$$$"
```

A single \$ sign in a filter is used for two special cases:

- ♦ It resets the filter.
- ♦ It represents the absence of a filter on the server.

Replica Types

The replica types identify the type of replica and are defined in the `REPLICA_TYPE` typedef enumeration in the `ldapx.h` file. Replica type determines the types of client operations that can be performed on the replica.

Flag Name	C Value	Meaning
<code>LDAP_RT_MASTER</code>	0	Identifies this replica as the master replica of the partition. Entries can be modified; partition operations can be performed.
<code>LDAP_RT_SECONDARY</code>	1	Identifies this replica as a secondary replica of the partition. Secondary replicas are Read/Write replicas and entries can be modified.
<code>LDAP_RT_READONLY</code>	2	Identifies the replica as a Read-Only replica. Only the NDS synchronization processes can modify the information on this replica.
<code>LDAP_RT_SUBREF</code>	3	Identifies the replica as a subordinate reference. NDS automatically adds these replicas to a server when the server does not contain replicas of all child partitions. Only NDS can modify information on this replica.
<code>LDAP_RT_SPARSE_WRITE</code>	4	Identifies the replica as a Read/Write replica with sparse data. It was been configured to contain only specified object types and attributes.
<code>LDAP_RT_SPARSE_READ</code>	5	Identifies the replica as a Read-Only replica with sparse data. It was been configured to contain only specified object types and attributes.
<code>LDAP_RT_COUNT</code>	6	Identifies the total number of replica types that have been defined.

Request Message Types

Type	Description
LDAP_REQ_DELETE (0x4A)	Indicates a delete operation.
LDAP_REQ_UNBIND (0x42)	Indicates an unbind operation.
LDAP_REQ_ABANDON (0x50)	Indicates a request to abandon an operation.
LDAP_REQ_BIND (0x60)	Indicates a bind operation.
LDAP_REQ_SEARCH (0x63)	Indicates a search operation.
LDAP_REQ_MODIFY (0x66)	Indicates a modify operation.
LDAP_REQ_ADD (0x68)	Indicates an add operation.
LDAP_REQ_RENAME	See LDAP_REQ_MODRDN.
LDAP_REQ_MODDN	See LDAP_REQ_MODRDN.
LDAP_REQ_MODRDN (0x6C)	Indicates a modify RDN operation.
LDAP_REQ_COMPARE (0x6E)	Indicates a compare operation.
LDAP_REQ_EXTENDED (0x77)	Indicates an extended operation

Result Message Types

Type	Description
LDAP_RES_BIND (0x61)	Indicates that the LDAPMessage structure contains the results of a bind operation.
LDAP_RES_SEARCH_ENTRY (0x64)	Indicates that the LDAPMessage structure contains information about an entry which was found during a search operation.
LDAP_RES_SEARCH_RESULT (0x65)	Indicates that the LDAPMessage structure contains the results of a search operation
LDAP_RES_MODIFY (0x67)	Indicates that the LDAPMessage structure contains the results of a modify operation.
LDAP_RES_ADD (0x69)	Indicates that the LDAPMessage structure contains the results of an add operation.
LDAP_RES_DELETE (0x6B)	Indicates that the LDAPMessage structure contains the results of a delete operation.
LDAP_RES_RENAME (0x6D)	Indicates that the LDAPMessage structure contains the results of a rename operation.
LDAP_RES_MODDN (0x6D)	Indicates that the LDAPMessage structure contains the results of a modify DN operation.
LDAP_RES_MODRDN (0x6D)	Indicates that the LDAPMessage structure contains the results of a modify RDN operation.
LDAP_RES_COMPARE (0x6F)	Indicates that the LDAPMessage structure contains the results of a compare operation.

Type	Description
LDAP_RES_SEARCH_REFERENCE (0x73)	Indicates that the LDAPMessage structure contains a referral to another LDAP server which was found during a search operation.
LDAP_RES_EXTENDED (0x78)	Indicates that the LDAPMessage structure contains the results of an extended operation

Session Preference Options

These flags are used by the `ldap_get_option` and `ldap_set_option` functions.

Option	Value	Description
LDAP_OPT_API_INFO	0x0000	Retrieves basic information about the API implementation. It cannot be used to set information. It includes the API version, minimum LDAP version, maximum LDAP version, vendor name, and vendor version. If the <code>ldap_get_option</code> function returns ♦ The vendor name, the application must free the memory by calling the <code>ldap_memfree</code> function. ♦ Some ldap extensions, the application must free the memory by calling the <code>ldap_value_free</code> function. Input data type: Not supported; output data type: <code>LDAPAPIInfo *</code>;

Option	Value	Description
LDAP_OPT_DEREF	0x0002	Determines how aliases are handled during a search. Supports the following values: LDAP_DEREF_NEVER (0x00) LDAP_DEREF_SEARCHING (0x01) LDAP_DEREF_FINDING (0x02) LDAP_DEREF_ALWAYS (0x03) The LDAP_DEREF_SEARCHING flag indicates that aliases are dereferenced during the search but not when locating the base object of the search. The LDAP_DEREF_FINDING flag indicates that aliases are dereferenced when locating the base object but not during the search. The LDAP_DEREF_ALWAYS flag indicates that aliases are dereferenced when locating the base object and when finding entries. The LDAP_DEREF_NEVER flag indicates that aliases are not dereferenced. The default is LDAP_DEREF_NEVER. Input and output data type: int *
LDAP_OPT_SIZELIMIT	0x0003	Determines how many entries are returned from a search. A value of LDAP_NO_LIMIT (0) means no limit. The default is LDAP_NO_LIMIT. Input and output data type: int *
LDAP_OPT_TIMELIMIT	0x0004	Determines the number of seconds an LDAP server will spend on a search. A value of LDAP_NO_LIMIT (0) means no limit. This value is passed to the LDAP server in the search request. The default is LDAP_NO_LIMIT. Input and output data type: int *

Option	Value	Description
LDAP_OPT_REFERRALS	0x0008	Determines whether the LDAP libraries automatically follow referrals. It can be set to one of the following values: LDAP_OPT_ON (void*) 1 LDAP_OPT_OFF (void*) 0 The default is ON. Input data type: void*; output data type: int*
LDAP_OPT_RESTART	0x0009	Determines whether LDAP I/O operations automatically restart if they abort prematurely. It can be set to one of the following values: LDAP_OPT_ON (void*) 1 LDAP_OPT_OFF (void*) 0 The default is OFF Input data type: void*; output data type: int*
LDAP_OPT_PROTOCOL_VERSION	0x0011	Specifies the version of the LDAP protocol used when communication with the LDAP server. It can be set to one of the following values: LDAP_VERSION2 (2) LDAP_VERSION3 (3) If no version is set, the default is LDAP_VERSION2. Input and output data type: int *
LDAP_OPT_SERVER_CONTROLS	0x0012	Specifies a default list of LDAP server controls that are sent with each request. Input data type, LDAPControl **; output data type: LDAPControl ***
LDAP_OPT_CLIENT_CONTROLS	0x0013	Specifies a default list of client controls that affect the LDAP session. Input data type, LDAPControl **; output data type: LDAPControl ***

Option	Value	Description
LDAP_OPT_API_FEATURE_INFO	0x0015	Specifies version information about an LDAP API extended feature. Input data type: Not supported; output data type: LDAPAPIFeatureInfo *
LDAP_OPT_HOST_NAME	0x0030	Specifies the host name or a list of hosts for the primary LDAP server. Input data type: char *; output data type: char **
LDAP_OPT_RESULT_CODE	0x0031	Specifies the code of the most recently returned LDAP error that occurred on this session. Input and output data type: int *
LDAP_OPT_ERROR_STRING	0x0032	Contains the message that returned with the most recent LDAP error that occurred on this session. Input data type: char *; output data type: char **
LDAP_OPT_MATCHED_DN	0x0033	Contains the matched DN value returned with the most recent LDAP error that occurred on this session. Input data type: char *; output data type: char **

Option	Value	Description
LDAP_OPT_DEBUG_LEVEL	0x5001	Contains the debug level. Uses the following values: 0x0001 LDAP_DEBUG_TRACE 0x0002 LDAP_DEBUG_PACKETS 0x0004 LDAP_DEBUG_ARGS 0x0008 LDAP_DEBUG_CONNS 0x0010 LDAP_DEBUG_BER 0x0020 LDAP_DEBUG_FILTER 0x0040 LDAP_DEBUG_CONFIG 0x0080 LDAP_DEBUG_ACL 0x0100 LDAP_DEBUG_STATS 0x0200 LDAP_DEBUG_STATS2 0x0400 LDAP_DEBUG_SHELL 0x0800 LDAP_DEBUG_PARSE 0x8000 LDAP_DEBUG_NONE -1 LDAP_DEBUG_ANY
Input and output data type: int*		

7

Structures

This chapter describes the structures used by the LDAP functions.

BerElement

Contains an opaque data structure for data encoded with BER (Basic Encoding Rules).

Remarks

The LDAP libraries provide functions for creating and manipulating the data within the BerElement structure, but clients do not need access to the fields in the structure.

For example, the `ldap_first_attribute` function creates a BerElement structure that tracks the current position in the entry.

The `ber_init` function converts a BerElement structure to a berval structure, and the `ber_flatten` function converts a berval structure to a BerElement structure.

berval

Contains binary data that is encoded with simplified BER (Basic Encoding Rules).

Structure

```
typedef struct berval {  
    unsigned long    bv_len;  
    char             *bv_val;  
};
```

Fields

bv_len

Specifies the length of the data.

bv_val

Points to the encoded data.

Remarks

The `ber_init` function converts a `BerElement` structure to a `berval` structure, and the `ber_flatten` function converts a `berval` structure to a `BerElement` structure.

LDAP

Contains an opaque data structure for LDAP session handle information.

Remarks

All LDAP operation functions require the client to use an LDAP structure with the request. The LDAP structure contains session specific data about the connection to the LDAP server.

The LDAP library does not allow the client to directly manipulate the data in this session handle. Instead, it provides the following functions for various tasks.

Task	Function
Create	ldap_init or ldap_open
View settings	ldap_get_option
Modify settings	ldap_set_option
Delete	ldap_unbind, ldap_unbind_s, ldap_unbind_ext

For a list of the options that can be viewed or set, see “Session Preference Options” on page 392.

LDAPAPIFeatureInfo

Contains version information about the LDAP API extended features.

Structure

```
typedef struct ldap_apifeature_info {  
    int    ldapaif_info_version;  
    char   *ldapaif_name;  
    int    ldapaif_version;  
} LDAPAPIFeatureInfo;
```

Fields

ldapaif_info_version

Specifies the version of the LDAPAPIFeatureInfo structure.

ldapaif_name

Points to the name of the supported feature.

ldapaif_info_version

Specifies the revision of the supported feature.

Remarks

LDAPAPIInfo

Contains information about the vendor's implementation of the LDAP API.

Structure

```
typedef struct ldapapiinfo {  
    int      ldapai_info_version;  
    int      ldapai_api_version;  
    int      ldapai_protocol_version;  
    char     **ldapai_extensions;  
    char     *ldapai_vendor_name;  
    int      ldapai_vendor_version;  
} LDAPAPIInfo;
```

Fields

`ldapai_info_version`

Specifies the version of the LDAPAPIInfo structure.

`ldapai_api_version`

Specifies the revision of the API supported.

`ldapai_protocol_version`

Specifies the highest LDAP version supported by the LDAP library.

`ldapai_extensions`

Points to a NULL-terminated array of character strings that names the vendor's LDAP extensions. If no API extensions are supported, this field is set to NULL. The application is responsible for freeing this memory by calling the `ldap_value_free` function.

`ldapai_vendor_name`

Points to the vendor's name. The application is responsible for freeing this memory by calling the `ldap_memfree` function.

`ldapai_vendor_version`

Specifies the vendor's version of the LDAP libraries.

Remarks

To retrieve more information about an extension (the `ldapai_extensions` field), call the `ldap_get_option` function with the option parameter set to `LDAP_OPT_API_FEATURE_INFO`.

LDAPControl

Contains data about an LDAP control.

Structure

```
typedef struct ldapcontrol {  
    char            *ldctl_oid;  
    struct berval    ldctl_value;  
    char            ldctl_iscritical;  
} LDAPControl;
```

Fields

`ldctl_oid`

Points to the string object identifier (OID) assigned to the control.

`ldctl_value`

Specifies a `berval` (page 399) structure that contains the data, if any, associated with the control. The `ldctl_value` field can contain no data.

- ♦ To indicate a zero-length value, set `ldctl_value.bv_len` to zero and `ldctl_value.bv_val` to a zero-length string.
- ♦ To indicate that no data is associated with the control, set `ldctl_value.bv_val` to `NULL`.

`ldctl_iscritical`

Specifies whether the control is critical to the operation.

- ♦ If this field is non-zero, the operation fails if the LDAP server doesn't recognize the control.
- ♦ If this field is set to zero, the LDAP can continue the operation when it doesn't recognize the control.

Remarks

LDAPMessage

Contains an opaque data structure for the results of an asynchronous LDAP operation or a search operation.

Remarks

The following functions create either an LDAPMessage structure or an array of LDAPMessage structure.

- ♦ Search functions: ldap_search_ext_s, ldap_search_ext, ldap_search, ldap_search_s, and ldap_search_st
- ♦ Asynchronous operations that require ldap_result to read the results such as ldap_add and ldap_add_ext, ldap_compare and ldap_compare_ext, ldap_delete and ldap_delete_ext, ldap_modify and ldap_modify_ext

Use the ldap_msgfree function to free the LDAPMessage structure.

LDAPMod

Contains the modifications to make to one attribute of an entry.

Structure

```
typedef union mod_vals_u {
    char          **modv_strvals;
    struct berval  *mod_bvals;
} mod_vals_u_t;

typedef struct ldapmod {
    int            mod_op;
    char           *mod_type;
    mod_vals_u_t   mod_vals;
#define mod_values  mod_vals.modv_strvals
#define mod_bvalues mod_vals.modv_bvals
} LDAPMod;
```

Fields

`mod_op`

Specifies the type of modification operation.

- ◆ `LDAP_MOD_ADD (0x0000)`—adds the value, adding the attribute if no values currently exist.
- ◆ `LDAP_MOD_DELETE (0x0001)`—deletes the specified values, removing the attribute if no values remain.
- ◆ `LDAP_MOD_REPLACE (0x0002)`— replaces the current values with the specified values, adding the attribute if no values currently exist and removing the attribute if the specified value's field is `NULL`.
- ◆ `LDAP_MOD_BVALUES (0x0080)`—specifies binary values. If the `mod_vals` structure contains binary values, this flag should be ORed to one of the other flags to specify a binary modification. If this flag is not ORed, the default is to assume string value modifications.

`mod_type`

Points to the name of attribute to modify.

`modv_strvals`

Points to a NULL-terminated array of string values for the attribute. This field cannot contain values if the `modv_bvals` field contains values.

`modv_bvals`

Points to a NULL-terminated array of `berval` structures which are used to modify an attribute's binary values. This field cannot contain values if the `modv_strvals` field contains values.

Remarks

If `mod_op` is set to an operation flag with `LDAP_MOD_BVALUES` ORed to it, the `modv_strvals` should be empty. If `LDAP_MOD_BVALUES` is not ORed to the operation flag, the `modv_strvals` should contain the values for the modification operation.

Either the attribute contains string or binary values. Select the one that matches the attribute's syntax.

LDAPReplicaInfo

Contains information about a replica.

Service: LDAP

Defined In: ldapx.h

Structure

```
typedef struct ldapreplicainfo {
    unsigned long    rootID;
    unsigned long    state;
    unsigned long    modificationTime;
    unsigned long    purgeTime;
    unsigned long    localReplicaID;
    char             namingContextDN[2*256];
    LDAP_REPLICA_TYPE replicaType;
    unsigned long    flags;
} LDAPReplicaInfo;
```

Fields

rootID

Contains the entry ID of the naming context's root object on the local server.

state

Contains the current state of the replica (see "Replica States" on page 385).

modificationTime

Contains the time and the event ID for the most recent modification to the replica.

purgeTime

Contains the time at which all data has been synchronized. Data scheduled for deletion, that predates this time, can now be deleted.

localReplicaID

Contains the local server's identifier for the replica.

namingContextDN

Contains the distinguished name of the naming context.

replicaType

Contains the replica's type (see "Replica Types" on page 388).

flags

Indicates whether the replica is busy performing an operation or if the replica is a boundary. Uses the following flags:

LDAP_DS_FLAG_BUSY (0x01)

LDAP_DS_FLAG_BOUNDARY (0x02)

LDAPSortKey

Contains information about a sort key.

Structure

```
typedef struct ldapsortkey {  
    char    attributeType;  
    char    *orderingRule;  
    int     reverseOrder;  
} LDAPSortKey;
```

Fields

attributeType

Points to the name of the attribute to use for sorting.

orderingRule

Points to the OID of the ordering rule to use for the sorting. NDS does not support ordering rules.

reverseOrder

Specifies whether to sort results in reverse order:

- ♦ Non-zero indicates that the results are sorted in reverse order (large to small). NDS does not support this type of sort.
- ♦ Zero indicates that the results are sorted in forward order (small to large)

LDAPSSL_Cert

Contains SSL certificate information.

Structure

```
typedef struct LDAPSSL_Cert {  
    unsigned long    length;  
    void             *data;  
} LDAPSSL_Cert, *pLDAPSSL_Cert;
```

Fields

length

The length of the memory pointed to by data.

data

Points to memory allocated by the application for the certificate information.

Remarks

The LDAPSSL_Cert structure is used by `ldapssl_get_cert` (page 296). After retrieving the certificate, the structure can be passed into `ldapssl_add_trusted_cert` (page 294) to add the certificate to the list of trusted certificates.

LDAPSSL_Cert_VValidity_Period

Contains the earliest and latest times that a certificate is valid.

Structure

```
typedef struct LDAPSSL_Cert_VValidity_Period {  
    char    notBeforeTime[40];  
    int     notBeforeType;  
    char    notAfterTime[40];  
    int     notAfterType;  
} LDAPSSL_Cert_VValidity_Period,  
*pLDAPSSL_Cert_VValidity_Period;
```

Fields

notBeforeTime

A string representation of the first time that the certificate should be considered valid.

notBeforeType

The type of the notBeforeTime parameter. The time can be represented as universal time string or a generalized time string.
(LDAPSSL_CERT_UTC_TIME or LDAPSSL_CERT_GEN_TIME).

notAfterTime

A string representation of the expiration time of the certificate.

notAfterType

The type of the notAfterTime parameter. The time can be represented as universal time string or generalized time string.
(LDAPSSL_CERT_UTC_TIME or LDAPSSL_CERT_GEN_TIME)

Remarks

Generalized Time Format. generalized time represents the values of year, month, day, hour, minutes, seconds and fractions of a second in any of three forms:

- ♦ Local time "YYYYMMDDHHMMSS.fff", where fff is optional and is fractions of a second

- ♦ Greenwich Mean Time (UTC) "YYYYMMDDHHMMSS.fffZ", Z indicates Greenwich Mean Time
- ♦ Difference between local and UTC time, "YYYYMMDDHHMMSS.fff+-HHMM", the +HHMM or -HHMM represents the time differential between the local and Greenwich Mean Times.

UTC Time Format. UTC format represents the values of year (2 digit), month, day, hour, minutes and optionally seconds.

- ♦ Local time "YYMMDDHHMMSS", where seconds (SS) is optional
- ♦ Greenwich Mean Time (UTC), "YYMMDDHHMMSSZ", seconds (SS) is optional and Z represents Greenwich Mean Time
- ♦ Difference between local and UTC time, "YYMMDDHHMMSS+-HHMM", seconds (SS) is optional and +HHMM or -HHMM represents the time differential between local and Greenwich Mean Times.

LDAPURLDesc

Contains URL information and the parameters for the search operation.

Structure

```
typedef struct ldap_url_desc {
    struct ldap_url_desc  *lud_next;
    char    *lud_scheme;
    char    *lud_host;
    int     lud_port;
    char    *lud_dn;
    char    **lud_attrs;
    int     lud_scope;
    char    *lud_filter;
    char    **lud_exts;
} LDAPURLDesc;
```

Fields

`lud_next`

Points to the next URL.

`lud_scheme`

Specifies the URL scheme (either ldap or ldaps).

`lud_host`

Points to the name of the host as a dotted IP address or DNS format.

`lud_port`

Specifies the port from the URL.

`lud_dn`

Points to the distinguished name of the base entry from the URL.

`lud_attrs`

Points to a NULL-terminated list of attributes specified in the URL.

`lud_scope`

Specifies the scope in the URL and uses one of the following flags.

- ♦ LDAP_SCOPE_BASE (0x00)—searches the entry specified by the base parameter.
- ♦ LDAP_SCOPE_ONELEVEL (0x01)—searches the entry specified by the base parameter and one level beneath that entry.
- ♦ LDAP_SCOPE_SUBTREE (0x02)—searches the entire subtree starting with the entry specified by the base parameter.

`lud_filter`

Points to the search filter specified in the URL.

If NULL is passed, a default filter ("objectclass=*") is used.

`lud_exts`

Points to a NULL-terminated list of the extensions specified in the URL.

LDAPVLVInfo

Contains state information associated with a series of virtual list view interactions between a client and an LDAP server.

Structure

```
typedef struct ldapvlvinfo {
    int                ldvlv_version;
    unsigned long      ldvlv_before_count;
    unsigned long      ldvlv_after_count;
    unsigned long      ldvlv_offset;
    unsigned long      ldvlv_count;
    struct berval       *ldvlv_attrvalue;
    struct berval       *ldvlv_context;
    void               *ldvlv_extradata;
} LDAPVLVInfo;
```

Fields

`ldvlv_version`

Specifies the version of this structure, which is currently 1.

`ldvlv_before_count`

Specifies the number of entries before the target entry that the client wants the server to return.

`ldvlv_after_count`

Specifies the number of entries after the target entry that the client wants the server to return.

`ldvlv_offset`

Specifies the target entry's position in the list. This parameter is used in connection with the `ldvlv_count` field, but is used only if the `ldvlv_attrvalue` field is NULL.

`ldvlv_count`

Specifies the total number of entries in the list. This parameter is used in connection with the `ldvlv_offset` field, but is used only if the `ldvlv_attrvalue` field is NULL. The following values have special consequences:

- ♦ If the `ldvly_count` field is set to 0, the Novell LDAP server returns the entry specified by the `ldvly_offset` parameter and sets this field to the number of entries currently in the list.
- ♦ If the value of the `ldvly_count` field does not match the current number of entries in the list, the Novell LDAP server assumes that the `ldvly_offset` parameter is relative to `ldvly_count`. For example, if the list contains 10,000 entries and you specify the count as 500 and the offset as 250, the middle entry of the list is returned which, in this case, is entry number 5,000.

`ldvly_attrvalue`

Points to the attribute value that the target entry's attribute is equal to or greater than. This can be used as a typedown value. For example, if the value specified is `abc`, the target entry will be the first entry in the list with `abc`, or if no `abc` entries exist, the first entry with `abd`. If this field is `NULL`, the `ldvly_offset` and `ldvly_count` fields are used to select the target entry.

`ldvly_context`

Points to server-specific data. On the first call, set this field to `NULL`. The server returns data that helps the server track who you are and where you are in the list. The context obtained from calling the `ldap_parse_vlv_control` function should be used as the context in the next `ldap_create_vlv_control` call.

`ldvly_extradata`

Reserved for application specific data. The virtual list view control does not use this field.

timeval

Contains timeout values for search requests.

Structure

```
typedef struct timeval {
    long    tv_sec;
    long    tv_usec;
};
```

Fields

- tv_sec
- Specifies the number of seconds for the time interval component.
- tv_usec
- Specifies the number of microseconds for the time interval component.

Remarks

These fields are used to determine the timeout value for both the LDAP server and the LDAP client libraries:

- ♦ If the server timeout expires before the server finishes the search operation, the server returns LDAP_TIMELIMIT_EXCEEDED to the application.
- ♦ If the client timeout expires before the server returns, the client returns LDAP_TIMEOUT to the application and sends an ldap_abandon to the server.

These fields have the following meanings for the timeout value for the LDAP server.

Field Values	Description
tv_sec=0; tv_usec>0	Sends a timeout value of one second to the server.
tv_sec>0; tv_usec>=0	Sends the tv_sec value to the server. The server ignores the tv_usec field.

The fields have the following meaning for a client timeout value.

Field Values	Description
tv_sec>=0; tv_usec>=0	Waits the time specified by the combination of the tv_sec and tv_usec fields.

The following table shows potential values for the fields and the timeout value that is computed for the server and the client.

Field Values	Server Timeout Value	Client Timeout Value
tv_sec=0; tv_usec=1	1 second	1 microsecond
tv_sec=1; tv_usec=500000	1 second	1.5 seconds
tv_sec=2; tv_usec=0	2 seconds	2 seconds

Only one of the fields can be set to zero. When both the tv_sec and tv_usec fields are set to zero, LDAP returns LDAP_PARAM_ERROR.

8

LDAP Tools

A number of tools have been developed to import entries from a file to an LDAP directory, to modify the entries in a directory from a file, and to export the entries to a file. These tools also support schema modifications by adding attribute and class definitions from a file.

Two types of tools are included in this SDK:

- ♦ Novell Import Convert Export utility developed by Novell to perform these tasks as quickly as possible using LDIF files as well as another LDAP server.
- ♦ LDAP utilities which are similar to utilities developed by other LDAP SDKs that perform some of these tasks using an LDIF file.

Novell recommends using the Novell Import Convert Export utility because it has the following advantages:

- ♦ **Asynchronous Requests.** The LDAP utilities use synchronous processing to ensure that all records are processed in the order they appear in the file. The LDAP server is idle from the time it sends its response to one request until it receives the next request. For a small file, this delay is not significant, but when you are importing a file with a million records, the delay becomes significant. The Novell Import Convert Export utility uses the LDAP Bulk Update/Replication Protocol (LBURP), which allows it to send asynchronous requests that contain more than one update operation while still guaranteeing that the records are processed in order. For more information, see “LDAP Bulk Update/Replication Protocol” on page 438.
- ♦ **Simple Passwords.** The Novell Import Convert Export utility has an option for NDS e-Directory 8.5 that allows simple passwords to be stored securely in the directory until the user logs in for the first time. At that point, the public and private key pairs are generated. Storing simple

passwords is a quick process whereas generating key pairs is a CPU-intensive process. For a small file, generating the key pairs as the users are added is not significant. However for a large file, this option greatly increases the speed with which the password information can be loaded.

- ♦ **Server to Server Migration.** The Novell Import Convert Export utility supports the migration of LDAP data from one LDAP server to another LDAP server. Traditional LDAP utilities only support importing data from an LDIF file to an LDAP server. With these utilities, you would have to first export the data to an LDIF file and then import the data to the other server.
- ♦ **Forward Referencing.** Since NDS enforces referential integrity, container entries must be created before user entries that belong to the container. If you have an LDIF file which has been exported from an LDAP server that does not enforce referential integrity, the records may not be in the right order for NDS to process and will thus fail. The Novell Import Convert Export utility has an option that creates a forward reference for such a container. This forward reference is then replaced with a normal entry when the record that adds the container is processed. For more information, see “Forward References” on page 440.

Novell Import Convert Export Utility

The Novell Import Convert Export utility lets you

- ♦ Import and export LDIF files
- ♦ Import and export comma-delimited data files
- ♦ Migrate data between LDAP servers

The Novell Import Convert Export is compatible with NDS 8.5 or higher. The utility includes two LDAP extension modules that are automatically loaded when the LDAP server starts, and a client utility that runs as a snap-in to ConsoleOne™. The utility replaces both the BULKLOAD and ZoneImport utilities included with previous versions of NDS.

You can use the utility as either a command line utility or a graphical utility. The comma-delimited data handler is a command line utility only. For information about the graphical utility, see “Using the NDS Import Convert Export Wizard” in the *Administration Guide* for NDS eDirectory 8.5.

This section describes the following features of the command line utility:

- ♦ “Using the Command Line Interface” on page 423

- ♦ “LDAP Bulk Update/Replication Protocol” on page 438
- ♦ “Forward References” on page 440
- ♦ “Using XML Rules” on page 440
- ♦ “Improving the Speed of LDIF Imports” on page 450

Using the Command Line Interface

You can use the command line version of the utility to perform the following:

- ♦ LDIF imports
- ♦ LDIF exports
- ♦ Comma-delimited data imports
- ♦ Comma-delimited data exports
- ♦ Data migration between LDAP servers
- ♦ Schema Compare and Update

The utility is installed as part of ConsoleOne. A Win32 version (ICE.EXE), a NetWare version (ICE.NLM), Solaris version (ICE), and Linux version (ICE) are included in the installation.

The utility manages a collection of handlers that read or write data in a variety of formats. Source handlers read data, while destination handlers write data. A single executable module can be both a source and a destination handler. The utility receives data from a source handler, processes the data, then passes the data to a destination handler.

For example, if you want to import LDIF data into an LDAP directory, the utility uses an LDIF source handler to read an LDIF file and an LDAP destination handler to send the data to the LDAP directory server.

See “LDIF Examples” on page 460 for more information about LDIF files.

Syntax

The utility is launched with the following command:

```
ice <general options>
  -S[LDIF | LDAP | DELIM] <source options>
  -D[LDIF | LDAP | DELIM] <destination options>
```

or when using the schema cache:

```
ice -C <general options>
    -S[LDIF | LDAP | DELIM] <source options>
    -D[LDIF | LDAP | DELIM] <destination options>
```

Note that when performing an update using the schema cache, an LDIF file is not a valid destination.

General options are optional and must come before any source or destination options. The -S (source) and -D (destination) handler sections can be placed in any order. For more information on these options, see

- ♦ “General Options” on page 424
- ♦ “Source Handler Options” on page 425
- ♦ “Destination Handler Options” on page 431
- ♦ “Schema Cache” on page 434

General Options

General options affect the overall processing of the utility.

Option	Description
-C	Specifies that you are using the schema cache to perform schema compare and update. For more information see “Schema Cache” on page 434.
-l <log file>	Specifies a filename where output messages (including error messages) are logged. If this option is not used, error message are sent to ICE.LOG.
-o	Overwrites an existing log file. If this flag is not set, messages are appended to the log file instead.
-e <ldif error log file>	Specifies a filename where entries that fail are output in LDIF format. This file can be examined, modified to correct the errors, then reapplied to the directory.
-p <URL>	Specifies the location of an XML placement rule to be used by the utility. Placement rules let you change the placement of an entry. For more information, see “Using XML Rules” on page 440.

Option	Description
-c <URL>	Specifies the location of an XML creation rule to be used by the utility. Creation rules let you supply missing information that might be needed to allow an entry to be created successfully on import. For more information, see “Using XML Rules” on page 440.
-s <URL>	Specifies the location of an XML schema mapping rule to be used by the utility. You can use a schema mapping rule to map a schema element on a source server to a different but equivalent schema element on a destination server. For more information, see “Using XML Rules” on page 440.
-b (NetWare Only)	Specifies

Source Handler Options

The source handler option (-S) determines the source of the import data. Only one of the following can be specified on the command line.

Option	Description
-SLDIF	Specifies that the source is an LDIF file. For a list of supported LDIF options, see “LDIF Source Handler Options” on page 425.
-SLDAP	Specifies that the source is an LDAP server. For a list of supported LDAP options, see “LDAP Source Handler Options” on page 426
-SDELIM	Specifies that the source is a comma-delimited data file. For a list of supported DELIM options, see “DELIM Source Handler Options” on page 429.

LDIF Source Handler Options

The LDIF source handler reads data from an LDIF file then sends it to the destination handler.

Option	Description
-f <LDIF file>	Specifies a filename containing LDIF records read by the LDIF source handler and sent to the destination handler.
-a	If the records in the LDIF file are content records (that is, they contain no changetypes), they will be treated as records with a change type of add.
-c	Prevents the LDIF source handler from stopping on errors. This includes errors on parsing LDIF and errors sent back from the destination handler. When this option is set and an error occurs, the LDIF source handler reports the error, finds the next record in the LDIF file, and goes on.
-n	Does not perform update operations, but prints what would be done. When this option is set, the LDIF source handler parses the LDIF file but does not send any records to the destination handler.
-v	Enables the verbose mode of the handler.

LDAP Source Handler Options

The LDAP source handler reads data from an LDAP server by sending a search request to the server. It then sends the search entries it receives from the search operation to the destination handler.

Option	Description
-s <server name>	Specifies the DNS name or IP address of the LDAP server to which the handler binds and sends a search request. If this option is omitted, it defaults to the local host (127.0.0.1).
-p <port>	Specifies the integer port number of the LDAP server specified by the -s option. If this option is omitted, it defaults to 389.

Option	Description
-d <DN>	Specifies the distinguished name of the entry that should be used when binding to the server. If this option is omitted, the handler performs an anonymous bind.
-w <password>	Specifies the password attribute of the entry specified by the -d option.
-F <search filter>	Specifies an RFC 2254 compliant search filter. If this option is omitted, the search filter defaults to objectclass=*. For more information on the format of the search filter, see LDAP Search Filters in <i>LDAP and NDS</i> .
-n	Specifies not to perform the search, but to show what search would be performed.
-a <attribute list>	Specifies a comma-separated list of attributes to retrieve as part of the search. In addition to an attribute list, the option supports the following values: 1.1—Get no attributes *—Get all non-operational attributes An empty list gets all non-operational attributes. Operational attributes must be listed by name. To get all standard attributes and a few operational attributes, use the * with a list of the names of the operational attributes. If this option is omitted, the attribute list defaults to the empty list.
-o <attribute list>	Specifies a comma-separated list of attributes to be omitted from the search results received from the LDAP server before they are sent to the destination handler. This option is very useful in cases where you want to use a wildcard with the -a option to get all attributes of some class and then remove a few of them from the search results before passing the data on to the destination handler. For example, -a* -otelephoneNumber searches for all attributes and then filters the telephoneNumber from the results.

Option	Description
-R	Disables the automatic following of referrals.
-e <value>	Specifies which debugging flags should be enabled in the LDAP client SDK. (See the SDK for supported flags.)
-b <base DN>	Specifies the distinguished name of the entry where the search request begins. If this option is omitted, the base DN defaults to ""(empty string) and the tree root is used as the base DN for the search.
-c <search scope>	Specifies the scope of the search request. The option supports the following values: ♦ One—search only the immediate subordinates of the base object. ♦ Base—search only the base object. ♦ Sub—search the base object and all of its subordinates. If this option is omitted, the search scope defaults to one.
-r <deref aliases>	Specifies the way aliases should be dereferenced during the search operation. The option supports the following values: ♦ Never—aliases are never dereferenced when locating the base object or searching. ♦ Always—aliases are always dereferenced when locating the base object and searching. ♦ Search—aliases are dereferenced when searching subordinates of the base object but not when locating the base object. ♦ Find—aliases are dereferenced when locating the base object but not when search the subordinates of the base object. If this option is omitted, the alias dereferencing behavior defaults to never.
-l <time limit>	Specifies, in seconds, the time limit for the search.

Option	Description
-z <size limit>	Specifies the maximum number of entries to be returned by the search.
-V <version>	Specifies the LDAP protocol version to be used for the connection. It must be 2 or 3. If this option is omitted, the default is 3.
-v	Enables verbose mode of the handler.
-L <filename>	Specifies a file in DER format containing a server key to use for SSL authentication.
-A	Retrieves attribute names only. Attribute values are not returned by the search operation.

DELIM Source Handler Options

The DELIM source handler reads data from a comma-delimited data file then sends it to the destination handler.

Option	Description
-f <filename>	Specifies a filename containing comma-delimited records read by the DELIM source handler and sent to the destination handler.
-F <value>	Specifies a filename containing the attribute data order for the file specified by -f. If this option is not specified, you must enter this information directly using -t. See “Performing a Comma-Delimited Import” on page 437 for more information.
-t <value>	Comma-delimited list of attributes specifying the attribute data order for the file specified by -f. Either this option or -F must be specified. See “Performing a Comma-Delimited Import” on page 437 for more information.

Option	Description
-c	Prevents the DELIM source handler from stopping on errors. This includes errors on parsing comma-delimited data files and errors sent back from the destination handler. When this option is set and an error occurs, the DELIM source handler reports the error, finds the next record in the comma-delimited data file, and goes on.
-n <value>	Specifies the LDAP naming attribute for the new object. This attribute must be contained in the attribute data you specify using -F or -t.
-l <value>	Specifies the path to append the RDN to (such as o=myCompany). If you are passing the DN this value is not necessary.
-o <value>	Comma-delimited list of object classes (if none are contained in your input file), or additional object classes such as auxiliary classes. The default value is inetorgperson.
-i <value>	Comma-delimited list of columns to skip. This value is an integer specifying the number of the column to skip. For example, to skip the third and fifth columns you would pass -i 3,5.
-d <value>	Specifies the delimiter. The default delimiter is a comma (,). The following values are special case delimiters: [q] = quote (a single " as the delimiter) [t] = tab For example, to specify a tab as a delimiter, you would pass: -d [t]
-q <value>	Specifies the secondary delimiter. The default secondary delimiter is a single double quote ("). The following values are special case delimiters: [q] = quote (a single " as the delimiter) [t] = tab For example, to specify a tab as a delimiter, you would pass: -d [t]

Destination Handler Options

The destination handler options (-D) specify the destination of the export data. Only one of the following can be specified on the command line.

Option	Description
-DLdif	Specifies that the destination is an LDIF file. For a list of supported options, see “LDIF Destination Handler Options” on page 431.
-DLdap	Specifies that the destination is an LDAP server. For a list of supported options, see “LDAP Destination Handler Options” on page 431.
-DDELIM	Specifies that the destination is a comma-delimited file. For a list of supported options, see “DELIM Destination Handler Options” on page 433.

LDIF Destination Handler Options

The LDIF destination handler receives data from the source handler and writes it to an LDIF file.

Option	Description
-f <LDIF file>	Specifies the filename where LDIF records can be written.
-v	Enables verbose mode of the handler.

LDAP Destination Handler Options

The LDAP destination handler receives data from the source handler and sends it to an LDAP server in the form of update operations to be performed by the server.

Option	Description
-s <server name>	Specifies the DNS name or IP address of the LDAP server to which the handler binds and sends a search request. If this option is omitted, it defaults to the local host (127.0.0.1).
-p <port>	Specifies the integer port number of the LDAP server specified by the -s option. If this option is omitted, it defaults to 389.
-d <DN	Specifies the distinguished name of the entry that should be used when binding to the server.
-w <password>	Specifies the password attribute of the entry specified by the -d option.
-B	Disables the use of asynchronous LBURP requests for transferring update operations to the server. The handler uses standard synchronous LDAP update operation requests instead. See “LDAP Bulk Update/Replication Protocol” on page 438 for more information.
-F	Allows the creation of forward references. When an entry is going to be created before its parent exists, a placeholder called a forward reference is created for the entry's parent to allow the entry to be successfully created. If a later operation creates the parent, the forward reference is changed into a normal entry. For more information, see “Forward References” on page 440.
-I	Stores password values using the simple password method of Novell's Modular Authentication Service (NMAS). Passwords are kept in a secure location in the directory, but key pairs are not generated until they are actually needed for authentication between servers. This improves the speed with which an object that has password information can be loaded.
-e <value>	Specifies which debugging flags should be enabled in the LDAP client SDK (see the client SDK for supported values).

Option	Description
-V <version>	Specifies the LDAP protocol version to be used for the connection. It must be 2 or 3. If this option is omitted, the default is 3.
-v	Enables verbose mode of the handler
-L <filename>	Specifies a file in DER format containing a server key to use for SSL authentication.

DELIM Destination Handler Options

The DELIM destination handler receives data from the source handler and writes it to a comma-delimited data file.

Option	Description
-f <filename>	Specifies the filename where comma-delimited records can be written.
-v	Enables verbose mode of the handler.
-F <value>	Specifies a filename containing the attribute data order for the source data. If this option is not specified, you must enter this information directly using -t.
-t <value>	Comma-delimited list of attributes specifying the attribute data order for the source data. Either this option or -F must be specified.
-l <value>	Can be either RDN or DN. Specifies whether the driver should place the entire DN or just the RDN in the data. RDN is the default value.
-d <value>	Specifies the delimiter. The default delimiter is a comma (','). The following values are special case delimiters: [q] = quote (a single " as the delimiter) [t] = tab For example, to specify a tab as a delimiter, you would pass: -d [t]

Option	Description
-q <value>	Specifies the secondary delimiter. The default secondary delimiter is a single double quote ("). The following values are special case delimiters: [q] = quote (a single " as the delimiter) [t] = tab For example, to specify a tab as a delimiter, you would pass: -d [t]
-n <value>	Specifies a naming attribute to appended during import, e.g. cn.

Schema Cache

The Novell Import Convert Export Utility contains a schema cache that allows you to compare and update LDAP schema. The following samples provide details on how to use the schema compare and update.

The schema cache is triggered by passing -C to the ice engine. To perform a comparison, use:

```
ice -C -c [LDIF file for the compare results]
```

To perform an update, use:

```
ice -C -a
```

Compare Example 1

```
ice -C -ccompare.ldif -SLDAP -s192.168.0.1 -p389 -DLdap -s192.168.0.2 -p389
```

Schema 1: LDAP server 192.168.0.1, port 389

Schema 2: LDAP server 192.168.0.2, port 389

Source Handler: NONE

Destination Handler: LDIF -fcompare.ldif

If no search filter is specified, this will check the Root DSE of both servers to see where their schema is located. Currently if either one has multiple subschema subentries the compare will fail. If a subschema subentry is explicitly specified, it will use that subentry without checking the Root DSE. Compare.ldif will contain a list of LDIF records that if applied to the server 192.168.0.2 would add all of the schema definitions in the schema of the server at 192.168.0.1.

Compare Example 2

```
ice -C -ccompare.ldif -SLDAP -s192.168.0.1 -p389 -DLDIF -  
fschema2.ldif
```

Schema 1: LDAP server 192.168.0.1, port 389

Schema 2: schema2.ldif

Source Handler: NONE

Destination Handler: LDIF -fcompare.ldif

If no search filter is specified, this will check the Root DSE of the server to see where the schema is located. If there are multiple subschema subentries the compare will fail. If a subschema subentry is explicitly specified, it will use that subentry without checking the Root DSE. Compare.ldif will contain a list of LDIF records that if applied to a server with the schema of schema2.ldif would add all of the schema definitions in the schema of the server at 192.168.0.1.

Compare Example 3

```
ice -C -ccompare.ldif -SLDIF -fschema1.ldif -DLdap -  
s192.168.0.2 -p389
```

Schema 1: schema1.ldif

Schema 2: LDAP server 192.168.0.2, port 389

Source Handler: NONE

Destination Handler: LDIF -fcompare.ldif

If no search filter is specified, this will check the Root DSE of the server to see where the schema is located. If there are multiple subschema subentries the compare will fail. If a subschema subentry is explicitly specified, it will use that subentry without checking the Root DSE. Compare.ldif will contain a list of LDIF records that if applied to the server 192.168.0.2 would add all of the schema definitions in the schema of schema1.ldif.

Compare Example 4

```
ice -C -ccompare.ldif -SLDIF -fschema1.ldif -DLDIF -  
fschema2.ldif
```

Schema 1: schema1.ldif

Schema 2: schema2.ldif

Source Handler: NONE

Destination Handler: LDIF -fcompare.ldif

Compare.ldif will contain a list of LDIF records that if applied to the server with the schema in schema2.ldif would add all of the schema definitions in the schema of schema1.ldif.

Update Example 1

```
ice -C -a -SLDAP -s192.168.0.1 -p389 -DLDA -s192.168.0.2 -p389
```

Schema 1: LDAP server 192.168.0.1, port 389

Schema 2: LDAP server 192.168.0.2, port 389

Source Handler: NONE

Destination Handler: LDAP server 192.168.0.2, port 389

If no search filter is specified, this will check the Root DSE of both servers to see where their schema is located. Currently if either one has multiple subschema subentries the update will fail. If a subschema subentry is explicitly specified, it will use that subentry without checking the Root DSE. The result will be that the schema of the server at 192.168.0.2 will have all the definitions in the schema at 192.168.0.1.

Note that when performing an update using the schema cache, an LDIF file is not a valid destination.

Update Example 2

```
ice -C -a -SLDIF -fschema1.ldif -DLDA -s192.168.0.2 -p389
```

Schema 1: schema1.ldif

Schema 2: LDAP server 192.168.0.2, port 389

Source Handler: NONE

Destination Handler: LDAP server 192.168.0.2, port 389

If no search filter is specified, this will check the Root DSE of the server to see where the schema is located. Currently if either one has multiple subschema subentries the update will fail. If a subschema subentry is explicitly specified, it will use that subentry without checking the Root DSE. The result will be that the schema of the server at 192.168.0.2 will have all the definitions in the schema of schema1.ldif.

Note that when performing an update using the schema cache, an LDIF file is not a valid destination.

Sample Commands

Listed below are sample commands that can be used with the utility for the following functions:

- ♦ “Performing an LDIF Import” on page 437
- ♦ “Performing an LDIF Export” on page 437
- ♦ “Performing a Data Migration between LDAP Servers” on page 438

Performing an LDIF Import

To perform an LDIF import, use a command similar to the following:

```
ice -SLDIF -fc:\temp\chapter2.ldif -DLDA  
-sserver1.acme.com -p389 -dcn=admin,c=us -wsecret
```

This particular command line reads LDIF data from the c:\temp\chapter2.ldif file and sends it to the LDAP server server1.acme.com at port 389 using the identity cn=admin,c=us, and password “secret.”

Performing an LDIF Export

To perform an LDIF export, use a command similar to the following:

```
ice -SLDAP -sserver1.acme.com -p389 -dcn=admin,c=us  
-wpassword -lobjectClass=* -csub -DLDIF  
-c:\temp\server1.ldif
```

This command line performs a subtree search for all objects in the server server1.acme.com at port 389 using the identity cn=admin,c=us, and password “password” and outputs the data in LDIF format to the c:\tmp\server1.ldif file.

Performing a Comma-Delimited Import

To perform a comma-delimited import, use a command similar to the following:

```
ice -SDELIM -fc:\tmp\in.csv -Fc:\tmp\order.csv -ncn -  
lo=acme -DLDA  
-sserver1.acme.com -p389 -dcnadmin,c=us  
-wsecret
```

This command reads comma-delimited values from the c:\tmp\in.csv file and reads the attribute order from the c:\tmp\order.csv file. For each attribute entry in in.csv, the attribute type is specified in order.csv. For example, if in.csv contains the following:

```
pat,engineer,555-1212,pat@acme.com,"Acme, Inc."
```

Then order.csv would contain the following:

```
cn,title,phonenumber,emailaddress,company
```

The information in order.csv could be input directly using the -t option.

The data is then sent to the LDAP server server1.acme.com at port 389 using the identity cn=admin,c=us, and password “secret.”

We specified that cn should become the new DN for this object using the -n option, and we added this object to the organization container acme using the -l option.

Performing a Comma-Delimited Export

To perform a comma-delimited export, use a command similar to the following:

```
ice -SLDAP -sserver1.acme.com -p389 -dcn=admin,c=us  
-wpassword -lobjectClass=* -csub -DDELIM -  
fc:\tmp\server1.csv -Forder.csv
```

This command line performs a subtree search for all objects in the server server1.acme.com at port 389 using the identity cn=admin,c=us, and password “password” and outputs the data in comma-delimited format to the c:\tmp\server1.csv file.

Performing a Data Migration between LDAP Servers

To perform a data migration between LDAP servers, use a command similar to the following:

```
ice -SLDAP -sserver1.acme.com -p389 -dcn=admin,c=us  
-wpassword -lobjectClass=* -csub -DLLDAP  
-sserver2.acme.com -p389 -dcn=admin,c=us -wpassword
```

This particular command line performs a subtree search for all objects in the server server1.acme.com at port 389 using the identity cn=admin,c=us with password “password” and sends it to the LDAP server server1.acme.com at port 389 using the identity cn=admin,c=us and password “secret.”

LDAP Bulk Update/Replication Protocol

The utility uses the LDAP Bulk Update/Replication Protocol (LBURP) to send asynchronous requests to an LDAP server. This guarantees that the

requests are processed in the order specified by the protocol and not in an arbitrary order influenced by multiprocessor interactions or the operating system's scheduler.

LBURP also lets the utility send several update operations in a single request and receive the response for all of those update operations in a single response. This adds to the network efficiency of the protocol.

LBURP works as follows:

1. The client utility binds to an LDAP server.
2. The server sends a bind response to the client.
3. The client sends a start LBURP extended request to the server.
4. The server sends a start LBURP extended response to the client.
5. The client sends zero or more LBURP operation extended requests to the server.

These requests can be sent asynchronously. Each request contains a sequence number identifying the order of this request relative to other requests sent by the client over the same connection. Each request also contains one or more LDAP update operations.

6. The server processes each of the LBURP operation extended requests in the order specified by the sequence number and sends an LBURP operation extended response for each request.
7. After all of the updates have been sent to the server, the client sends an end LBURP extended request to the server.
8. The server sends an end LBURP extended response to the client.

The LBURP protocol lets the utility present data to the server as fast as the network connection between the two allows. This lets the server stay busy processing update operations 100 percent of the time because it never has to wait for the utility to give it more work to do.

The LBURP processor in NDS eDirectory also commits update operations to the database in groups to gain further efficiency in processing the update operations. LBURP can greatly improve the efficiency of your LDIF imports over a traditional synchronous approach.

To disable LBURP during an LDIF import, use the -B option with the destination handler.

IMPORTANT: Since LBURP is a relatively new protocol, NDS servers prior to version 8.5 (and most non-NDS servers) do not support it. If you are using the NDS Import and Export utility to import an LDIF file to one of these servers, you must disable the LBURP option for the LDIF import to work.

Forward References

In an LDIF file, the record to add an entry can come before the record to add its parent container. In NDS which guarantees referential integrity, this situation generates an error because the entry's parent does not exist. The -F option for the LDAP destination handler solves this problem. It enables the creation of a forward reference for the parent container. When the record to create the parent is processed, the forward reference is replaced with a normal entry.

It is possible, that after the entire LDIF file is processed, that a few forward references will remain because the LDIF file did not contain records to add them. Such forward references remain in the directory as unknown objects, and the entries below them remain and function as normal entries. You can either add these remaining forward references as entries or move the subordinate entries to another container.

To identify the unknown objects in your directory, you can

- ♦ Use ConsoleOne where unknown objects are represented by a round yellow icon with a question mark in the center.
- ♦ Use an LDAP search with the search filter set to `objectClass=unknown`.

Both of these methods display all entries that have an `objectClass` of `unknown`, not just the entries that are forward references. From these entries, you need to select the entries to add.

- ♦ When NDS processes an add for an unknown object that already exists as a forward reference, NDS transforms the existing forward reference entry into a normal entry.
- ♦ When NDS processes an add for an unknown object that isn't a forward reference, NDS returns an error of "object already exists."

Using XML Rules

The Novell Import Convert Export utility allows you to specify a set of rules that are applied to each source record before the record is sent to the

destination. These rules are specified in XML and solve the following problems when importing entries from one LDAP directory to another:

- ♦ Schema differences—two LDAP directories will seldom, if ever, have identical schemas. Schema mapping rules allow you to map an attribute or class definition in the source directory to an attribute or class definition in the destination directory. For example, since most NDS directories use User class for users, you would probably want to map the inetOrgPerson class from the source directory to the User class in NDS destination directory. If the destination schema has no equivalent class or attribute, the schema can be extended to include new classes and attributes. (For more information, see “Modifying the Schema” on page 470.) For information on the format of these rules, see “Schema Mapping Rules” on page 442.
- ♦ Hierarchical differences—two LDAP directories will seldom, if ever, have the same parent containers. As you move entries from one directory to another, you may not want the entries to be created in the same containers in the destination directory. Placement rules allow you to take entries from the source container and place these entries in another container in the destination directory. For example, if two departments (sales1 and sales2) have been combined to one department (sales), you can create a rule that places all the entries from sales1 and sales2 containers in the sales container on the destination server. For information on the format of these rules, see “Placement Rules” on page 446.
- ♦ Missing information—two LDAP directories will seldom, if ever, have the same rules for creating new entries. For example, if you are importing entries from a directory that only requires a value for the cn attribute for users to a directory that requires a value for both the cn and the sn attributes, the adds for records which do not have a sn value will fail. Create rules allow you to supply default values for such attributes so that the adds will not fail. For information on the format of these rules, see “Create Rules” on page 443.

You enable rule processing with the -p, -c, and -s general options (see “General Options” on page 424). Each of these options must be followed by a URL with the following format:

```
file://<[path/]filename>
```

The file must be on the local file system.

The rules use the same XML format as the DirXML product.

Schema Mapping Rules

The <attr-name-map> element is the top level element for the schema mapping rules. Mapping rules determine how the import schema interacts with the export schema. They associate specified import class definitions and attributes with corresponding definitions in the export schema.

Mapping rules can be set up for attribute names or class names.

- ♦ For an attribute mapping, the rule must specify that it is an attribute mapping, a name space (nds-name is the tag for the source name), the name in the NDS name space, and then the other name space (app-name is the tag for the destination name) and the name in that name space. It can specify that the mapping applies to a specific class, or it can be applied to all classes with the attribute.
- ♦ For a class mapping, the rule must specify that it is a class mapping rule, a name space (NDS or the application), the name in that name space, and then the other name space and the name in that name space.

The following is the formal DTD definition of schema mapping rules.

```
<!ELEMENT attr-name-map (attr-name | class-name)*>

<!ELEMENT attr-name (nds-name, app-name)>
<!ATTLIST attr-name
            class-name      CDATA      #IMPLIED>

<!ELEMENT class-name (nds-name, app-name)>

<!ELEMENT nds-name (#PCDATA)>

<!ELEMENT app-name (#PCDATA)>
```

You can have multiple mapping elements in the file. Each element is processed in the order that it appears in the file. If you map the same class or attribute more than once, the first mapping takes precedence.

The following samples illustrate how to create a schema mapping rule.

Schema Rule 1. The following rule maps the source's surname attribute to the destination's sn attribute for the inetOrgPerson class.

```
<attr-name-map>
  <attr-name class-name="inetOrgPerson">
    <nds-name>surname</nds-name>
    <app-name>sn</app-name>
```

```

    </attr-name>
</attr-name-map>

```

Schema Rule 2. The following rule maps the source's inetOrgPerson class definition to the destination's User class definition.

```

<attr-name-map>
  <class-name>
    <nds-name>inetOrgPerson</nds-name>
    <app-name>User</app-name>
  </class-name>
</attr-name-map>

```

Schema Rule 3. The following example contains two rules. The first rule maps the source's Surname attribute to the destination's sn attribute for all classes that use these attributes. The second rule maps the source's inetOrgPerson class definition to the destination's User class definition.

```

<attr-name-map>
  <attr-name>
    <nds-name>surname</nds-name>
    <app-name>sn</app-name>
  </attr-name>
  <class-name>
    <nds-name>inetOrgPerson</nds-name>
    <app-name>User</app-name>
  </class-name>
</attr-name-map>

```

Sample Command. If the schema rules are saved to an sr1.xml file, the following command instructs the utility to use the rules while processing the 1entry.ldf file and to send the results to a destination file, outt1.ldf.

```

ice -o -sfile://sr1.xml -SLDIF -f1entry.ldf -c -DLDIF
    -foutt1.ldf

```

Create Rules

Create rules specify the conditions for creating a new entry in the destination directory. They support the following elements:

- ♦ **Required attributes.** Specifies that an add record must have values for all of the required attributes, or the add fails. The rule can supply a default value for a required attribute. If a record does not have a value for the attribute, the entry is given the default value. If the record has a value, the record value is used.

- ♦ **Matching attributes.** Specifies that an add record must have the specific attributes and match the specified values, or the add fails.
- ♦ **Templates.** Specifies the distinguished name of a template object in the NDS directory. The Novell Import Convert Export utility does not currently support specifying templates in create rules.

The following is the formal DTD definition for create rules:

```
<!ELEMENT create-rules (create-rule)*>

<!ELEMENT create-rule (match-attr*,
                        required-attr*,
                        template?) >

<!ATTLIST create-rule
            class-name    CDATA    #IMPLIED
            description    CDATA    #IMPLIED>

<!ELEMENT match-attr (value)+ >
<!ATTLIST match-attr
            attr-name      CDATA    #REQUIRED>

<!ELEMENT required-attr (value)*>
<!ATTLIST required-attr
            attr-name      CDATA    #REQUIRED>

<!ELEMENT template EMPTY>
<!ATTLIST template
            template-dn    CDATA    #REQUIRED>
```

You can have multiple create rule elements in the file. Each rule is processed in the order that it appears in the file. If a record does not match any of the rules, that record is skipped and the skipping does not generate an error.

The following examples illustrate how to format create rules.

Create Rule 1. The following rule places three conditions on add records that belong to the inetOrgPerson class. These records must have givenName and surname attributes. They ought to have an L attribute, but if they don't, the create rule supplies a default value of Provo for them.

```
<create-rules>
  <create-rule class-name="inetOrgPerson">
    <required-attr attr-name="givenName"/>
    <required-attr attr-name="surname"/>
    <required-attr attr-name="L">
      <value>Provo</value>
```

```

        </required-attr>
    </create-rule>
</create-rules>

```

Create Rule 2. The following create rule places three conditions on all add records, regardless of their base class:

- ♦ The record must contain a givenName attribute. If it doesn't, the add fails.
- ♦ The record must contain a surname attribute. If it doesn't, the add fails.
- ♦ The record must contain an L attribute. If it doesn't, the attribute is set to a value of Provo.

```

<create-rules>
  <create-rule>
    <required-attr attr-name="givenName"/>
    <required-attr attr-name="Surname"/>
    <required-attr attr-name="L">
      <value>Provo</value>
    </required-attr>
  </create-rule>
</create-rules>

```

Create Rule 3. The following create rule places two conditions on all records, regardless of base class:

- ♦ It checks to see if the record has a uid attribute with a value of ratuid. If it doesn't, the add fails.
- ♦ It checks to see if the record has an L attribute. If it does not have this attribute, the L attribute is set to a value of Provo.

```

<create-rules>
  <create-rule>
    <match-attr attr-name="uid">
      <value>cn=ratuid</value>
    </match-attr>
    <required-attr attr-name="L">
      <value>Provo</value>
    </required-attr>
  </create-rule>
</create-rules>

```

Sample Command. If the create rules are saved to an cr1.xml file, the following command instructs the utility to use the rules while processing the 1entry.ldf file and to send the results to a destination file, outt1.ldf.

```

ice -o -cfile://cr1.xml -SLDIF -f1entry.ldf -c -DLDIF
    -foutt1.ldf

```

Placement Rules

Placement rules determine where an entry is created in the destination directory. They support the following conditions for determining whether the rule should be used to place an entry:

- ♦ **Match class.** If the rule contains any match class elements, an objectClass specified in the record must match the class-name attribute in the rule. If the match fails, the placement rule is not used for that record.
- ♦ **Match attribute.** If the rule contains any match attribute elements, the record must contain an attribute value for each of the attributes specified in the match attribute element. If the match fails, the placement rule is not used for that record.
- ♦ **Match path.** If the rule contains any match path elements, a portion of the record's dn must match the prefix specified in the match path element. If the match fails, the placement rule is not used for that record.

The last element in the rule specifies where to place the entry. The placement rule can use zero or more of the following:

- ♦ **PCDATA.** Uses parsed character data to specify the DN of a container for the entries.
- ♦ **Copy the name.** Specifies that the RDN of the old DN is used in the entry's new DN.
- ♦ **Copy the attribute.** Specifies the naming attribute to use in the entry's new DN. The specified naming attribute must be a valid naming attribute for the entry's base class.
- ♦ **Copy the path.** Specifies that the source DN should be used as the destination DN.
- ♦ **Copy the path suffix.** Specifies that the source DN, or a portion of its path, should be used as the destination DN. If a match-path element is specified, only the part of the old DN that matches the prefix attribute of the match-path element is used as part of the entry's DN.

The following is the formal DTD definition for the placement rule.

```
<!ELEMENT placement-rules (placement-rule*)>
<!ATTLIST placement-rules
    src-dn-format      (%dn-format;)      "slash"
    dest-dn-format     (%dn-format;)      "slash"
    src-dn-delims      CDATA              #IMPLIED
    dest-dn-delims     CDATA              #IMPLIED>
```

```

<!ELEMENT placement-rule (match-class*,
                           match-path*,
                           match-attr*,
                           placement)>

<!ATTLIST placement-rule
      description      CDATA          #IMPLIED>

<!ELEMENT match-class      EMPTY>
<!ATTLIST match-class
      class-name      CDATA          #REQUIRED>

<!ELEMENT match-path      EMPTY>
<!ATTLIST match-path
      prefix          CDATA          #REQUIRED>

<!ELEMENT match-attr      (value)+ >
<!ATTLIST match-attr
      attr-name      CDATA          #REQUIRED>

<!ELEMENT placement      (#PCDATA |
                           copy-name |
                           copy-attr |
                           copy-path |
                           copy-path-suffix)* >

```

You can have multiple placement-rule elements in the file. Each rule is processed in the order that it appears in the file. If a record does not match any of the rules, that record is skipped, and the skipping does not generate an error.

The following examples illustrate how to format placement rules. The `src-dn-format="ldap"` and `dest-dn-format="ldap"` attributes set the rule so that the name space for the dn in the source and destination is LDAP format.

NOTE: The Novell Import Convert Export utility only supports source and destination names in LDAP format.

Placement Example 1. The following placement rule requires that the record have a base class of `inetOrgPerson`. If the record matches this condition, the entry is placed immediately subordinate to the test container and left-most component of its source dn is used as part of its dn.

```

<placement-rules src-dn-format="ldap" dest-dn-format="ldap">
  <placement-rule>
    <match-class class-name="inetOrgPerson"></match-class>
    <placement>o=test<copy-name/></placement>
  </placement-rule>
</placement-rules>

```

```

    </placement-rule>
  </placement-rules>

```

With this rule, a record with a base class of inetOrgPerson and with the following dn

```
dn: cn=Kim Jones, ou=English, ou=Humanities, o=UofZ
```

would have the following dn in the destination directory

```
dn: cn=Kim Jones, o=test
```

Placement Example 2. The following placement rule requires that the record have an sn attribute. If the record matches this condition, the entry is placed immediately subordinate to the test container and left-most component of its source dn is used as part of its dn.

```

<placement-rules src-dn-format="ldap" dest-dn-format="ldap">
  <placement-rule>
    <match-attr attr-name="sn"></match-attr>
    <placement>o=test<copy-name/></placement>
  </placement-rule>
</placement-rules>

```

With this rule, a record with the following dn and sn attribute

```
dn: cn=Kim Jones, ou=English, ou=Humanities, o=UofZ
sn: Jones
```

would have the following dn in the destination directory

```
dn: cn=Kim Jones, o=test
```

Placement Example 3. The following placement rule require the record to have an sn attribute. If the record matches this condition, the entry is placed immediately subordinate to the test container and its sn attribute is used as part of its dn. The specified attribute in the copy-attr element must be a naming attribute of the entry's base class.

```

<placement-rules src-dn-format="ldap" dest-dn-format="ldap">
  <placement-rule>
    <match-attr attr-name="sn"></match-attr>
    <placement>o=test<copy-attr attr-name="sn"/></placement>
  </placement-rule>
</placement-rules>

```

With this rule, a record with the following dn and sn attribute


```
dn: cn=Kim Jones, ou=English, ou=Humanities, o=UofZ
sn: Jones
```

would have the following dn in the destination directory

```
dn: cn=Jones, o=test
```

Placement Sample 4. The following placement rule requires the record to have an sn attribute. If the record matches this condition, the source dn is used as the destination dn.

```
<placement-rules src-dn-format="ldap" dest-dn-format="ldap">
  <placement-rule>
    <match-attr attr-name="sn"></match-attr>
    <placement><copy-path/></placement>
  </placement-rule>
</placement-rules>
```

Placement Sample 5. The following placement rule requires the record to have an sn attribute. If the record matches this condition, the entry's entire DN is copied to the test container.

```
<placement-rules src-dn-format="ldap" dest-dn-format="ldap">
  <placement-rule>
    <match-attr attr-name="sn"></match-attr>
    <placement>o=test<copy-path-suffix/></placement>
  </placement-rule>
</placement-rules>
```

With this rule, a record with the following dn and sn attribute

```
dn: cn=Kim Jones, ou=English, ou=Humanities, o=UofZ
sn: Jones
```

would have the following dn in the destination directory

```
dn: cn=Kim Jones, ou=English, ou=Humanities, o=UofZ, o=test
```

Sample Command. If the placement rules are saved to an pr1.xml file, the following command instructs the utility to use the rules while processing the 1entry.ldf file and to send the results to a destination file, foutt1.ldf.

```
ice -o -pfile://pr1.xml -SLDIF -f1entry.ldf -c -DLDIF
-foutt1.ldf
```

Improving the Speed of LDIF Imports

In cases where you have thousands or even millions of records in a single LDIF file you are importing, consider the following:

- ♦ “Import Directly to a Server with a Read/Write Replica” on page 450
- ♦ “Use LBURP” on page 450
- ♦ “Configure the Database Cache” on page 450
- ♦ “Use Simple Passwords” on page 451
- ♦ “Use Indexes Appropriately” on page 451

Import Directly to a Server with a Read/Write Replica

If it's possible to do so, select a destination server for your LDIF import that has read/write replicas containing all the entries represented in the LDIF file. This will maximize network efficiency.

Avoid having the destination server chain to other NDS servers for updates. This can severely reduce performance. However, if some of the entries to be updated are only on NDS servers that are not running LDAP, you might have to allow chaining to import the LDIF file.

Use LBURP

The NDS Import Convert Export Wizard maximizes network and NDS server processing efficiency by using LBURP to transfer data between the wizard and the server. Using LBURP during an LDIF import greatly improves the speed of your LDIF import.

For more information on LBURP, see “LDAP Bulk Update/Replication Protocol” on page 438.

Configure the Database Cache

The amount of database cache available for use by NDS has a direct bearing on the speed of LDIF imports, especially as the total number of entries on the server increases. When doing an LDIF import, you might want to allocate the maximum memory possible to NDS during the import. After the import is completed and the server is handling an average load, you can restore your previous memory settings. This is particularly important if the import is the only activity taking place on the NDS server.

Use Simple Passwords

NDS uses public and private key pairs for authentication. Generating these keys is a very CPU-intensive process. With NDS eDirectory 8.5, you can choose to store passwords using the simple password feature of the Novell Modular Authentication Service (NMAS). When you do this, passwords are kept in a secure location in the directory, but key pairs are not generated until they are actually needed for authentication between servers. This greatly improves the speed with which an object that has password information can be loaded.

To enable simple passwords during an LDIF import, use the `-l` option with the destination handler.

If you choose to store passwords using simple passwords, you must use an NMAS-aware Novell[®] Client[™] to log in to the NDS tree and access traditional file and print services. LDAP applications binding with name and password will work seamlessly with the simple password feature.

Use Indexes Appropriately

Having unnecessary indexes can slow down your LDIF import because each defined index requires additional processing for each entry having attribute values stored in that index. You should make sure that you don't have unnecessary indexes before you do an LDIF import, and you might want to consider creating some of your indexes after you have finished loading the data and after you have reviewed predicate statistics to see where they are really needed.

Other LDAP Utilities

The LDAP command line utilities can be called by your LDAP application to delete entries, modify entries, add entries, extend the schema, modify relative distinguished names, move entries to new containers, or search the directory. They all accept a `-f <file>` option which allows the processing of an LDIF file to perform a sequence of operations. The LDAP Libraries for C include the following utilities:

- ♦ “`ldapadd`” on page 452
- ♦ “`ldapdelete`” on page 453
- ♦ “`ldapmodify`” on page 455
- ♦ “`ldapmodrdn`” on page 456

- ♦ “ldapsearch” on page 458

ldapadd

The ldapadd utility adds new entries. It has the following syntax:

```
ldapadd [options]
```

NOTE: On a NetWare server, the utility is called ladd.

If the -f option is specified, ldapadd reads the modifications from a file. If the -f option is not specified, ldapadd reads the modifications from stdin.

Replace [options] with one or more of the following:

Option	Description
-c	Enables continuous operation mode. Errors are reported, but ldapadd will continue. The default is to exit after reporting an error
-f <file>	Reads the add information from an LDIF file instead of from standard input. The maximum length of a record is 4096 lines. The record is assumed to be an LDIF record if the first line starts with a pound sign or a colon appears to the left of an equals sign.
-F	Forces the application to apply all changes regardless of the contents of input lines that begin with the following: <pre>replica:</pre> By default, these lines are compared against the LDAP server host and port in use to decide if a relog record should actually be applied.
Common Options	Description
-d <level>	Sets the LDAP debugging level to the specified level. The ldapmodify utility must be compiled with LDAP_DEBUG defined for this option to have any effect.
-D <binddn>	Specifies the dn to use in binding to the LDAP server. The dn should be a string-represented dn as defined in RFC 1779.

Option	Description
-e <file>	Specifies the certificate file to use with an SSL bind.
-h <host>	Specifies an alternate host on which the LDAP server is running.
-n	Shows what would be done, but doesn't actually modify entries. Useful for debugging in conjunction with -v (verbose mode) which writes many diagnostics to standard output.
-p <port>	Specifies an alternate TCP port where the LDAP server is listening.
-P	Specifies the LDAP version (2 or 3).
-v	Uses the verbose mode which writes many diagnostics to standard output.
-w <passwd>	Specifies the password to use for simple authentication.
-W	Prompts the user for the password for simple authentication. This is used instead of specifying the password on the command line

ldapdelete

The ldapdelete utility deletes the specified entry. It opens a connection to an LDAP server, binds, and then deletes. It has the following syntax:

```
ldapdelete [options] dn ...
```

NOTE: On a NetWare server, the utility is called ldelete.

The dn parameter is a list of distinguished names of the entries to be deleted. It interacts with the -f option in the following ways:

- ◆ If the -f option is missing from the command line and dn's are specified on the command line, the utility deletes the specified entries.
- ◆ If both dn and the -f option are in the command line, the utility reads the file for the dn's to delete and ignores any dn's in the command line.
- ◆ If both dn and the -f option are missing in the command line, the utility reads the dn from stdin.

Replace [options] with one of the following:

Option	Description
-c	Enables continuous operation mode. Errors are reported, but ldapdelete will continue with deletions. The default is to exit after reporting an error.
-f <file>	Reads a series of dn's from the specified file. Each dn should be on its own line.
-r	Delete recursively
Common Options	Description
-d <level>	Sets the LDAP debugging level to the specified level. The ldapdelete utility must be compiled with LDAP_DEBUG defined for this option to have any effect.
-D <binddn>	Specifies the dn to use in binding to the LDAP server. The dn should be a string-represented dn as defined in RFC 1779.
-e <file>	Specifies the certificate file to use with an SSL bind.
-h <host >	Specifies an alternate host on which the LDAP server is running
-n	Shows what would be done, but doesn't actually delete entries. Useful for debugging in conjunction with -v (verbose mode).
-p <port>	Specifies an alternate TCP port where the LDAP server is listening.
-P <version>	Specifies the LDAP protocol version: 2 or 3.
-v	Turns on verbose mode which writes diagnostics to standard output.
-w <passwd>	Specifies the password to use for simple authentication.
-W	Prompts for simple authentication. This is used instead of specifying the password on the command line.

ldapmodify

The ldapmodify utility modifies the attributes of an existing entry or adds new entries. It has the following syntax:

```
ldapmodify [options]
```

NOTE: On a NetWare server, the utility is called lmodify.

If the -f option is specified, ldapmodify reads the modifications from a file. If the -f option is not specified, ldapmodify reads the modifications from stdin.

Replace [options] with one or more of the following:

Option	Description
-a	Adds new entries. The default for ldapmodify is to modify existing entries. If invoked to add entries, this flag must be set. This option can be used with the -f <file> option to read the entries to add from a file. A changetype should not be specified in the file.
-c	Enables continuous operation mode. Errors are reported, but ldapmodify will continue with modifications. The default is to exit after reporting an error
-r	Replaces existing values by default.
-f <file>	Reads the entry modification information from a file instead of from standard input. The maximum length of a record is 4096 lines. The record is assumed to be an LDIF record if the first line starts with a pound sign or a colon appears to the left of an equals sign. Each record should include a changetype line specifying add, modify, replace, or delete. (See “LDIF Examples” on page 460.)
-F	Forces the application to apply all changes regardless of the contents of input lines that begin with the following: replica: By default, these lines are compared against the LDAP server host and port in use to decide if a replog record should actually be applied.
Common Options	Description

Option	Description
-d <level>	Sets the LDAP debugging level to the specified level. The ldapmodify utility must be compiled with LDAP_DEBUG defined for this option to have any effect.
-D <binddn>	Specifies the dn to use in binding to the LDAP server. The dn should be a string-represented dn as defined in RFC 1779.
-e <file>	Specifies the certificate file to use with an SSL bind.
-h <host>	Specifies an alternate host on which the LDAP server is running.
-n	Shows what would be done, but doesn't actually modify entries. Useful for debugging in conjunction with -v (verbose mode) which writes many diagnostics to standard output.
-p <port>	Specifies an alternate TCP port where the LDAP server is listening.
-P	Specifies the LDAP version (2 or 3).
-v	Uses the verbose mode which writes many diagnostics to standard output.
-w <passwd>	Specifies the password to use for simple authentication.
-W	Prompts the user for the password for simple authentication. This is used instead of specifying the password on the command line

ldapmodrdn

The ldapmodrdn modifies the relative distinguished name of an entry. It can also move the entry to a new container. It has the following syntax

```
ldapmodrdn [options] [-s newSuperior] [dn newrdn]
```

Dn is a full LDAP dn. Newrdn is a new relative distinguished name.

If given, newrdn will replace the RDN of the entry specified by DN. If not given, the list of modification is read from stdin or from the file specified by "-f file".

NOTE: On a NetWare server, the utility is called lmodrdn dn <newrdn>).

The following table describes these options and parameters.

Option	Description
-c	Enables continuous operation mode. Errors are reported, but <code>ldapmodrdn</code> will continue with modifications. The default is to exit after reporting an error.
-f <file>	Performs the sequences of modifications listed in the specified file. Each record consists of the distinguished name followed by the new rdn attribute on the following line. For example: <pre>dn1 rdn1 dn2 rdn2</pre> Blank lines are ignored. This option cannot be used with the <code>dn <newrdn></code> option.
-r	Specifies to remove the old rdn value from the entry. If not specified, defaults to keeping the old value.
-s <newSuperior>	Specifies the distinguished name of the container to which the entry is moving.
Common Options	Description
-d <level>	Sets the LDAP debugging level to the specified level.
-D <binddn>	Specifies the dn of the entry to use in a bind operation.
-e <file>	Specifies the certificate file to use with an SSL bind.
-h <host>	Specifies an alternate host on which the LDAP server is running.
-n	Shows what would be done, but doesn't actually rename the entry.
-p <port>	Specifies an alternate TCP on which the LDAP server is listening.
-P <version>	Specifies the LDAP version (2 or 3).

Option	Description
-v	Turns on verbose mode (sends diagnostics to standard output).
-w <passwd>	Specifies the password to use for simple authentication.
-W	Prompts the user for the password for bind operations.

ldapsearch

The ldapsearch utility searches the directory for specified attributes and object classes. It has the following syntax:

```
ldapsearch [options] filter [attribute list]
```

NOTE: On a NetWare server, the utility is called lsearch.

By default, ldapsearch will NOT follow referrals. To follow referrals, pass -C as an option. The ldapsearch utility now outputs in LDIF format, and binary attributes are printed in base64.

The filter must be an RFC-2254 compliant LDAP search filter. For more information, see LDAP Search Filters in *LDAP and NDS*.

The attribute list is a whitespace-separated list of the attributes to retrieve. The following values may replace the list:

- ♦ 1.1— no attributes
- ♦ *— all user attributes
- ♦ +— all operational attributes
- ♦ an empty list gets all non-operational attributes

Replace options with one or more of the following:

Option	Description
-a <deref>	Specifies how to handle the dereferencing of an alias. It uses the following values: ♦ Never—aliases are never dereferenced when locating the base object or searching. ♦ Always—aliases are always dereferenced when locating the base object and searching. ♦ Search—aliases are dereferenced when searching subordinates of the base object but not when locating the base object. ♦ Find—aliases are dereferenced when locating the base object but not when search the subordinates of the base object.
-A	Retrieves attribute names only (no values).
-b <basedn>	Specifies the dn of the base container where the search should begin
-C	Enable referral following.
-l <limit>	Specifies the time limit (in seconds) for the search.
-L	Prints entries in LDIF format (implies -B).
-LL	Prints entries in LDIF format without comments.
-LLL	Prints entries in LDIF format without comments and version.
-s <scope>	Specifies the extent of the scope and use one of the following scope values: ♦ base—search base object only ♦ one—search the immediate subordinates of the base object ♦ sub—search the base object and all of its subordinates
-S <attr>	Sorts the results by the specified attribute.
-t	Writes binary values to files in TMPDIR.
-tt	Writes all values to files in TMPDIR.

Option	Description
-T <path>	Write files to directory specified by path (default: "/tmp").
-u	Includes User Friendly entry names in the output.
-V	URL prefix for files.
-V <prefix>	Specifies the URL prefix for files (default: "file://tmp/").
-z <limit>	Specifies the maximum number of entries to return in the search.
Common Options	Description
-d <level>	Sets LDAP debugging to the specified level.
-D <binddn>	Specifies the dn of the entry to use in bind operations.
-e <file>	Specifies the certificate file to use with an SSL bind.
-f <file>	Performs the sequence of searches listed in the specified file. Each line specifies the parameters for one search.
-h <host>	Specifies an alternate host on which the LDAP server is running.
-n	Shows what would be done, but doesn't actually search the directory.
-p <port>	Specifies an alternate TCP port on which the LDAP server is listening.
-P <version>	Specifies the LDAP version (2 or 3).
-v	Turns on verbose mode (diagnostics to standard output).
-w <passwd>	Specifies the password to use for simple authentication
-W	Prompts the user for bind password.

LDIF Examples

RFC 2849 "The LDAP Data Interchange Format (LDIF)—Technical Specification" describes the LDIF file format. This section contains information on basic LDIF formatting conventions and provides examples on how to perform the following tasks using LDIF:

- ♦ “Modifying the Attribute Values of an Entry” on page 462
- ♦ “Adding Entries” on page 465
- ♦ “Renaming or Moving an Entry” on page 466
- ♦ “Adding NDS Indexes” on page 467
- ♦ “Modifying the Schema” on page 470
- ♦ “Using Content Records” on page 474

LDIF Formatting Conventions

For complete details, see RFC 2849 “The LDAP Data Interchange Format (LDIF)—Technical Specification.” An LDIF file consists of a series of records. Each LDIF file begins with a Version record and then is followed either by a series of Change records or Content records. Change and Content records cannot be in the same file.

This section describes the following general LDIF formatting conventions:

- ♦ Version record
- ♦ Comment specifier
- ♦ Record separator
- ♦ Line folding
- ♦ Valid string characters

Version Record. Each LDIF file should start with a Version record. If no Version record is specified, any application processing the file is allowed to assume the file is version 0. Both the Novell Import Convert Export utility and the LDAP utilities support Version records. A Version record must be the first non-comment line in the file. It has the following format.

```
version: 1
```

Zero or more spaces are allowed after the colon.

Comment Specifier. Lines beginning with the pound (#) sign are treated as comments and are ignored when processing the file.

Record Separator. Records are separated by one or more blank lines (created by either a linefeed or carriage return/linefeed pair). The last record in the file must be terminated with a record separator (one or more blank lines).

Line Folding. Each record consists of a series of fields, one field per line. However, sometimes the information required by a field does not fit on one line. To fold a line, insert a line separator (either a linefeed or carriage return/linefeed pair) followed by a space. The LDIF parser processes this sequence by concatenating the lines and discarding the leading space.

Valid String Characters. Every operation takes a distinguished name. A colon following the specifier indicates that the name is in safe UTF-8 format, for example,

```
dn: cn=aperson,o=anorg,c=any
```

Safe strings can begin with any character in the range 0x00 to 0x7F except for the following characters:

```
0x00 (NUL)
0x0A (LF)
0x0D (CR)
0x20 (space " ")
0x3A (colon ":")
0x3C ((less-than "<"))
```

Subsequent characters in a safe string can be any character in the range 0x00 to 0x7F except the following:

```
0x00 (NUL)
0x0A (LF)
0x0D (CR)
```

Two colons following the specifier indicates that the name is a Base64 encoded name. The following example Base64 encodes the name Kim (which really doesn't require encoding because all of its characters are safe):

```
givenName:: a2lt
```

Any name or value that contains a prohibited character must be Base64 encoded. For more information on Base64 encoding, see Section 5.2 of RFC 1521.

Modifying the Attribute Values of an Entry

You use an LDIF change record to modify attribute values. A modify change record uses one or more of the following keywords.

Keyword	Description
add	Indicates that the values that follow should be added to the attribute.
delete	Indicates that the values that follow should be deleted from the attribute. Beware of the following conditions: ♦ If no values follow, all values of the attribute are deleted. ♦ If the attribute has no values, the operation fails.
replace	Indicates that the values that follow should replace the existing attribute values. Beware of the following conditions: ♦ If no values follow, all values of the attribute are deleted. ♦ If the attribute has no values, the operation succeeds. ♦ If the attribute has any values, all values are deleted and replaced with the specified values.

Adding a Value. The following example adds a value of `aperson@anorg.dom` to the mail attribute.

```
version: 1

dn: cn=aperson,o=anorg,c=any
changetype: modify
add: mail
mail: aperson@anorg.dom
```

If the attribute is multi-valued, it adds the value. If the specified value already exists in the attribute, the operation fails.

Deleting a Value. The following example deletes the `aperson@anorg.dom` value from the mail attribute.

```
version: 1

dn: cn=aperson,o=anorg,c=any
changetype: modify
delete: mail
mail: aperson@anorg.dom
```

If the attribute does not contain the specified value, the operation fails. However, if the purpose was to ensure that the specified value did not exist in the attribute, the operation succeeds in achieving this result.

Deleting an Attribute. The following example deletes all values of the mail attribute.

```
version: 1

dn: cn=aperson,o=anorg,c=any
changetype: modify
delete: mail
```

If the attribute does not exist, the operation fails. However, if the purpose was to ensure that the attribute has no values, the operation succeeds in achieving this result.

Replacing a Value. The following example replaces the existing mail value with the specified value, aperson@anorg.dom.

```
version: 1

dn: cn=aperson,o=anorg,c=any
changetype: modify
replace: mail
mail: aperson@anorg.dom
```

If the attribute is multi-valued, all values are removed and the specified value is added. If the attribute doesn't exist, it is added with the specified value.

Making Multiple Modifications. You can specify multiple modifications in a single LDIF record. The following example adds a telephone number, deletes the fax telephone number attribute, and replaces the mail attribute.

A line containing only a hyphen (-) character is used to mark the end of an attribute entry.

```
version: 1

dn: cn=aperson,o=anorg,c=any
changetype: modify
add: telephonenumber
telephonenumber: +1 415 555 0002
-
delete: facsimileTelephoneNumber
-
```



```
replace: mail
mail: aperson@anorg.dom
-
```

The LDIF specification requires the ending hyphen (-) after the last attribute entry, but not all LDAP servers support it. The Novell LDAP server supports it.

Adding Entries

You use can LDIF change record to add entries. An add change record requires values for all of the mandatory attributes of the entry's base class. The objectClass attribute must be set to the base class and may also be set to the super classes of the base class.

Since NDS maintains referential integrity, an LDIF file with multiple add change records must first add the container entries and then the leaf entries that will reside in the container. Leaf entries cannot be added to containers that don't already exist. (See the forward referencing option in the Novell Import Convert Export utility if you have an existing file with invalid ordering.)

The following example adds two organizational units (a college and department) under the UofZ organization and then adds an inetOrgPerson.

```
version: 1

dn: ou=Humanities, o=UofZ
changetype: add
objectClass: top
objectClass: ndsLoginProperties
objectClass: organizationalUnit
telephoneNumber: +1 415 555 0000

dn: ou=English, ou=Humanities, o=UofZ
changetype: add
objectClass: top
objectClass: ndsLoginProperties
objectClass: organizationalUnit
telephoneNumber: +1 415 555 1111

dn: cn=Kim Jones, ou=English, ou=Humanities, o=UofZ
changetype: add
sn: Jones
givenName: Kim
objectClass: top
```

```
objectClass: ndsLoginProperties
objectClass: person
objectClass: organizationalPerson
objectClass: inetOrgPerson
telephoneNumber: +1 415 555 2222
mail: Kim.Jones@UofZ.edu
userPassword: Kim1234
```

Deleting Entries

You can use an LDIF change to delete entries. A delete change record requires two items: the distinguished name of the entry to delete and the delete changetype. Container entries cannot be deleted unless they are empty.

The following example deletes the entries created in the adding entries example. The entries are deleted in reverse order of adding so that the container entries are empty.

```
version: 1

dn: cn=Kim Jones, ou=English, ou=Humanities, o=UofZ
changetype: delete

dn: ou=English, ou=Humanities, o=UofZ
changetype: delete

dn: ou=Humanities, o=UofZ
changetype: delete
```

Renaming or Moving an Entry

You can use an LDIF change record to rename an entry, move it, or do both in one operation. Modifying the DN uses two keywords and moving an entry requires a third keyword. These keywords have the following meaning:

Keyword	Description
newrdn	Indicates that this line contains the new RDN for the entry.
deleteoldrdn	Indicates whether the old RDN value should be deleted: 0=save, 1=delete. When the old value is saved, it becomes an additional attribute value but is no longer part of the entry's distinguished name.

Keyword	Description
newsuperior	Indicates that this line contains the DN of the new container for the entry.

Renaming an Entry. The following example shows how to rename an entry and retain the old value.

```
version: 1

dn: cn=Kim Jones, ou=English, ou=Humanities, o=UofZ
changetype: moddn
newrdn: cn= Kim Jones-Smith
deleteoldrdn: 0
```

Moving an Entry. The following example shows how to move an entry from the English department to Spanish department.

```
version: 1

dn: cn=Kim Jones, ou=English, ou=Humanities, o=UofZ
changetype: moddn
newrdn: cn=Kim Jones
deleteoldrdn: 1
newsuperior: ou=Spanish, ou=Humanities, o=UofZ
```

Renaming and Moving an Entry. The following example shows how to rename and move an entry in the same operation.

```
version: 1

dn: cn=Kim Jones, ou=English, ou=Humanities, o=UofZ
changetype: moddn
newrdn: cn=Kim Jones-Smith
deleteoldrdn: 0
newsuperior: ou=Spanish, ou=Humanities, o=UofZ
```

Adding NDS Indexes

Indexes are used to sort the entries in the replicas of the local server according to specified attributes. They significantly increase search performance and allow the display of sorted results. NDS 8 and above ship with predefined

indexes for commonly used attributes. NDS eDirectory 8.5 allows you to specify additional attributes for indexing.

However, each additional index adds overhead to update operations. Before adding an index, you should check the Predicate Data on the server to see if any of the attributes your application is using would benefit from an index. For more information, see the *Administration Guide* for NDS eDirectory 8.5.

To specify an index in an LDIF file, you must supply values for following case ignore strings which are separated by a dollar (\$) sign.

Order	String	Description
1	Index version	Reserved for future use. In NDS eDirectory 8.5, this should always be set to zero (0).
2	Index name	Specifies the user-defined name for the index, such as "Family Name" or "Zip Code." The string should not contain the dollar (\$) sign.
3	Index state	Specifies the state of the index. When defining an index, this field should be set to 2 (online). NDS supports the following values: ♦ 0—suspended which indicates the index is not used in queries and is not updated. ♦ 1—bringing online which indicates the index is in the process of being created. ♦ 2—online which indicates the index is up and working. ♦ 3—pending creation which indicates the index has been defined and is waiting for the background process to run. The background process changes the state once the building begins.

Order	String	Description
4	Index rule	Specifies the type of matching: ♦ 0—Value matching which optimizes queries that involve the entire value or the first part of the value, for example, a query for all entries with a surname equal to Jensen or begin with Jen. ♦ 1—Presence matching which optimizes queries that involve only the presence of an attribute, for example, a query for all entries with a surname attribute. ♦ 2—Substring matching which optimizes queries that involve a match of a few characters, for example, a query for all entries with a surname containing “der”. This query returns entries with the surnames of Derington, Anderson, and Lauder.
5	Index type	Specifies who created the index. When defining an index, you must set this value to 0. NDS supports the following values: ♦ 0—user defined ♦ 1—added on attribute creation ♦ 2—required for operation ♦ 3—system index
6	Index value state	Specifies the source of the index. When defining an index, set this string to 1. NDS supports the following values: ♦ 0—uninitialized ♦ 1—added from server ♦ 2—added from local DIB ♦ 3—deleted from local DIB ♦ 4—modified from local DIB
7	Attribute name	Specifies the NDS name for the attribute. Many attributes in NDS have both an LDAP name and an NDS name. This string requires the NDS name.

The following example shows how to add an index for the mail attribute (which has an NDS name of EMail Address).

```
version: 1

dn: cn=Kim Jones, ou=English, ou=Humanities, o=UofZ
changetype: modify
add: indexDefinition
indexDefinition: 0$myEmailIndex$2$1$0$1$Email Address
```

Modifying the Schema

You can use LDIF files to add attribute and class definitions. NDS also allows you to modify existing schema definitions.

NOTE: Modifying existing definitions should be done with caution since you are changing an attribute or class that another application may be using.

You can modify the following features of existing class definitions:

- ♦ Containment rules. You can add classes to or delete classes from the definition's containment rules.
- ♦ Super class rules. You can add a new super class to an object class, but you cannot remove any defined super classes.
- ♦ ASN.1 IDs. You can modify the ASN.1 ID assigned to the class.
- ♦ Class flags. You can change a leaf class definition into a container class by turning on the container class flag. But once turned on, it cannot be removed.
- ♦ Optional attributes. You can add new optional attributes to or delete optional attributes from the class definition. This is not the preferred method. If possible, create an auxiliary class and use the auxiliary class to add the attributes to the entries created from the class definition.

You can modify the following features of existing attributes:

- ♦ On sized attributes, you can increase the upper boundary.
- ♦ You can set the following attribute flags:

```
X-NDS_NEVER_SYNC
X-NDS_NOT_SCHED_SYNC_IMMEDIATE
X-NDS_PUBLIC_READ
X-NDS_SERVER_READ
```

- ♦ You can clear the following attribute flags:

SINGLE-VALUE
 X-NDS_LOWER_BOUND
 X-NDS_NEVER_SYNC
 X-NDS_NOT_SCHED_SYNC_IMMEDIATE
 X-NDS_PUBLIC_READ
 X-NDS_SERVER_READ

Adding an Attribute Definition. To add an attribute definition, you modify the subschemaSubentry object which always has a DN of cn=schema in NDS. You obtain this name by reading the root DSE.

The following example adds five attributes that could be used for a student class definition. In the example, each structural element in attributeTypes uses line folding (new line plus one space) so that the various elements are more visible. (The OIDs are only sample ASN.1 IDs and have not been registered.)

```
version: 1

dn: cn=schema
changetype: modify
add: attributeTypes
attributeTypes: (
  1.20.300.400.500.4.1
  NAME 'advisor'
  DESC 'faculty advisor for the student'
  SYNTAX 1.3.6.1.4.1.1466.115.121.1.12
  SINGLE-VALUE
  X-NDS_PUBLIC_READ '1'
  X-NDS_NOT_SCHED_SYNC_IMMEDIATE '1')
attributeTypes: (
  1.20.300.400.500.4.2
  NAME 'major'
  DESC 'students major accepted by the department'
  SYNTAX 1.3.6.1.4.1.1466.115.121.1.15
  X-NDS_NOT_SCHED_SYNC_IMMEDIATE '1')
attributeTypes: (
  1.20.300.400.500.4.3
  NAME 'minor'
  DESC 'students minor accepted by the department'
  SYNTAX 1.3.6.1.4.1.1466.115.121.1.15
  X-NDS_NOT_SCHED_SYNC_IMMEDIATE '1')
attributeTypes: (
  1.20.300.400.500.4.4
  NAME 'department'
```

```

DESC 'department supervising the student'
SYNTAX 1.3.6.1.4.1.1466.115.121.1.15
SINGLE-VALUE
X-NDS_PUBLIC_READ '1'
X-NDS_NOT_SCHED_SYNC_IMMEDIATE '1'
)
attributeTypes: (
    1.20.300.400.500.4.5
    NAME 'studentID'
    DESC 'unique ID assigned to the student'
    SYNTAX 1.3.6.1.4.1.1466.115.121.1.36
    SINGLE-VALUE)

```

The values 1.20.300.400.500.4.1 through 1.20.300.400.500.4.5 specify the OID or ASN1 ID of the attribute. For information on obtaining an OID and a naming prefix that guarantees uniqueness in the NDS schema, see *Schema Extensions in NDS Schema Reference*.

The NAME element specifies the name of the attribute.

The DESC element describes the attribute.

The SYNTAX element specifies the type of data the attribute can contain. For a list of supported OIDs, see *Attribute Syntax Definitions in NDS Schema Reference*.

The SINGLE-VALUE flag specifies that the attribute is single valued. When not specified, the attribute defaults to multi-valued. For a description of the possible LDAP and X-NDS flags, see *Attribute Flags in LDAP and NDS*.

Adding a Structural Class Definition. To add a class definition, you modify the schema object, dn: cn=schema. You obtain this name by reading the root DSE.

NOTE: NDS maintains referential integrity, so any new attributes must be added in the file before they are added to the class. The following example uses attributes that need to be added to the schema before the class definition.

In the following example, each structural element in objectClasses uses line folding (new line plus one space) so that the various elements are more visible.

```

version: 1

dn: cn=schema
changetype: modify
objectClasses: (
    1.20.300.400.500.6.1

```



```

NAME 'student'
SUP 'person'
STRUCTURAL
MUST (cn $ studentID $ givenName $ sn)
MAY (advisor $ major $ minor $ department)
X-NDS_NAMING ('cn')
X-NDS_CONTAINMENT ('organization' 'organizationalUnit'
'domain' )
X-NDS_NOT_CONTAINER '1'
X-NDS_NONREMOVABLE '1')

```

The 1.20.300.400.500.6.1 value specifies the OID or ASN1 ID of the class. For information on obtaining an OID and a naming prefix that guarantees uniqueness in the NDS schema, see *Schema Extensions in NDS Schema Reference*.

The NAME element specifies the name of the object class.

The SUP element specifies the super class of the object class.

The STRUCTURAL element specifies that the class is an effective class. ABSTRACT specifies a non-effective class, and AUXILIARY specifies an auxiliary class.

The MUST element specifies the mandatory attributes.

The MAY element specifies the optional attributes.

For a description of the possible X-NDS flags, see *Object Class Flags in LDAP and NDS*.

NOTE: This example shows how to add a structural class. Since student is a class that might apply to more than one type of user in the directory (inetOrgPerson or User), this class might be more useful as an auxiliary class. See the next example.

Adding an Auxiliary Class. The following example creates two new attributes, creates an auxiliary class with these new attributes, and then adds an inetOrgPerson entry with the auxiliary class as an object class of the entry and with values for the auxiliary class attributes.

```
version: 1
```

```

# Add an attribute to track a bear's hair. The attribute is
# multi-valued, uses a case ignore string syntax,
# and has public read rights
# Values may include: long hair, short, curly, straight,
# none, black, and brown

```

```

# X-NDS_PUBLIC_READ '1' The 1 allows public read,
# 0 denies public read
dn: cn=schema
changetype: modify
add: attributeTypes
attributeTypes: ( 2.16.840.1.113719.1.186.4.10 NAME
'bearHair' SYNTAX 1.3.6.1.4.1.1466.115.121.1.15 X-
NDS_PUBLIC_READ '1' )

# add an attribute to store a bear's picture
dn: cn=schema
changetype: modify
add: attributeTypes
attributeTypes: ( 2.16.840.1.113719.1.186.4.11 NAME
'bearPicture' SYNTAX 1.3.6.1.4.1.1466.115.121.1.5 SINGLE-
VALUE )

# create an Auxiliary class for the bearfeatures
dn: cn=schema
changetype: modify
add: objectclasses
objectclasses: (2.16.840.1.113719.1.186.6.101 NAME
'bearFeatures' MAY (bearHair $ bearPicture) AUXILIARY)

# now create a user named booboo
dn: cn=booboo,o=jellystone
changetype: add
cn: booboo
sn: bear
givenName: booboo
bearHair: Short
bearHair: Brown
bearHair: Curly
bearPicture:< file:///c:/tmp/alien.jpg
objectClass: top
objectClass: person
objectClass: inetOrgPerson
objectClass: bearFeatures

```

Using Content Records

Content records are the same as change records except that they do not have a changetype specifiers. They are usually generated to list the current entries in a directory and the entry's attributes and values. Content records and change records cannot be placed in the same LDIF file.

Both the Novell Import Convert Export utility and the ldapmodify utility have options that allow content records to be used as add change records.

- ♦ The -a option of the LDIF source handler of the Novell Import Convert Export utility changes content records into change records with a changetype of add.
- ♦ The -a option of the ldapmodify utility changes content records into change records with a changetype of add.

The following examples of content records could be used by either utility to add these entries to the directory.

```
version: 1
```

```
dn: o=UofZ
objectClass: top
objectClass: ndsLoginProperties
objectClass: organization
```

```
dn: ou=Humanities, o=UofZ
objectClass: top
objectClass: ndsLoginProperties
objectClass: organizationalUnit
telephoneNumber: +1 415 555 0000
```

```
dn: ou=English, ou=Humanities, o=UofZ
objectClass: top
objectClass: ndsLoginProperties
objectClass: organizationalUnit
telephoneNumber: +1 415 555 1111
```

```
dn: cn=Kim Jones, ou=English, ou=Humanities, o=UofZ
sn: Jones
givenName: Kim
objectClass: top
objectClass: ndsLoginProperties
objectClass: person
objectClass: organizationalPerson
objectClass: inetOrgPerson
telephoneNumber: +1 415 555 2222
mail: Kim.Jones@UofZ.edu
userPassword: Kim1234
```


A

Source Code Contributors

Novell would like to acknowledge the following for contributing source code to the ldapsdk.* library:

- ♦ Copyright 1998, 1999 The OpenLDAP Foundation, Redwood City, CA
All rights reserved.
- ♦ Copyright © 1990, 1995 Regents of the University of Michigan. All
rights reserved.
- ♦ Copyright © 1987 Regents of the University of California
- ♦ Copyright 1991 by the Massachusetts Institute of Technology
- ♦ Copyright © 1994 Enrique Silvestre Mora, Universitat Jaume I, Spain.
- ♦ Copyright © 1992, 1993, 1994 Henry Spencer.
- ♦ Copyright © 1992, 1993, 1994 The Regents of the University of
California.
- ♦ Copyright © 1997, 1998, 1999 Computing Research Labs, New Mexico
State University
- ♦ Copyright © 1999 PADL Software Pty Ltd.

For the complete text of these copyright notices, view the source code from OpenLDAP (<http://www.openldap.org>).

Revision History

The following table lists all changes made to the LDAP Libraries for C documentation:

September 2001	Added the following: ♦ More information on search filters. ♦ More information on time formats. ♦ Interactive SSL APIs
June 2001	Added the following: ♦ Information on LDAP URLs. ♦ Updated the LDAPURLDesc (page 414) struct to maintain IETF conformance. ♦ Update the ldap_get_effective_privileges (page 323) function with new rights flags. ♦ Added documentation for the DELIM handler in the Novell Import Convert Export Utility. ♦ Updated the LDAP Utilities documentation. ♦ Removed Multi-byte functions. ♦ Updated the documentation for the LDAP unbind functions.

February 2001	Added the following: ♦ New functions—<code>ldap_set_replication_filter</code>, <code>ldap_get_replication_filter</code>, <code>ldap_create_orphan_naming_context</code>, and <code>ldap_remove_orphan_naming_context</code>. ♦ New UTF-8 conversion routines ♦ Updated the <code>LDAPMod</code> structure ♦ Replaced the <code>LDAP_OPT_ERROR_NUMBER</code> constant with <code>LDAP_OPT_RESULT_CODE</code> ♦ Updated the <code>fbuf</code> parameter description for the <code>ber_free</code> function.
September 2000	Added the following: ♦ Information to the <code>ldap_url_parse</code> function ♦ Runtime information ♦ XML rule information to the Novell Import Convert Export utility
July 2000	Added the following: ♦ Support for the Solaris and Linux platforms ♦ A task for changing a user's password ♦ An <code>ldapx_memfree</code> function to free memory allocated by the LDAP extension library ♦ <code>ldapadd</code> and Novell Import/Export utility information ♦ LDIF examples
May 2000	Added information about functions that allocate and free memory. Added the following new functions: <code>ldap_set_rebind_proc</code>, <code>ldap_is_ldap_url</code>, <code>ldap_is_ldaps_url</code>, <code>ldap_free_urldesc</code>, <code>ldap_url_search</code>, <code>ldap_url_search_s</code>, <code>ldap_url_search_st</code>, and <code>ldap_refresh_server</code>. Added the following structure: <code>LDAPURLDesc</code>.
